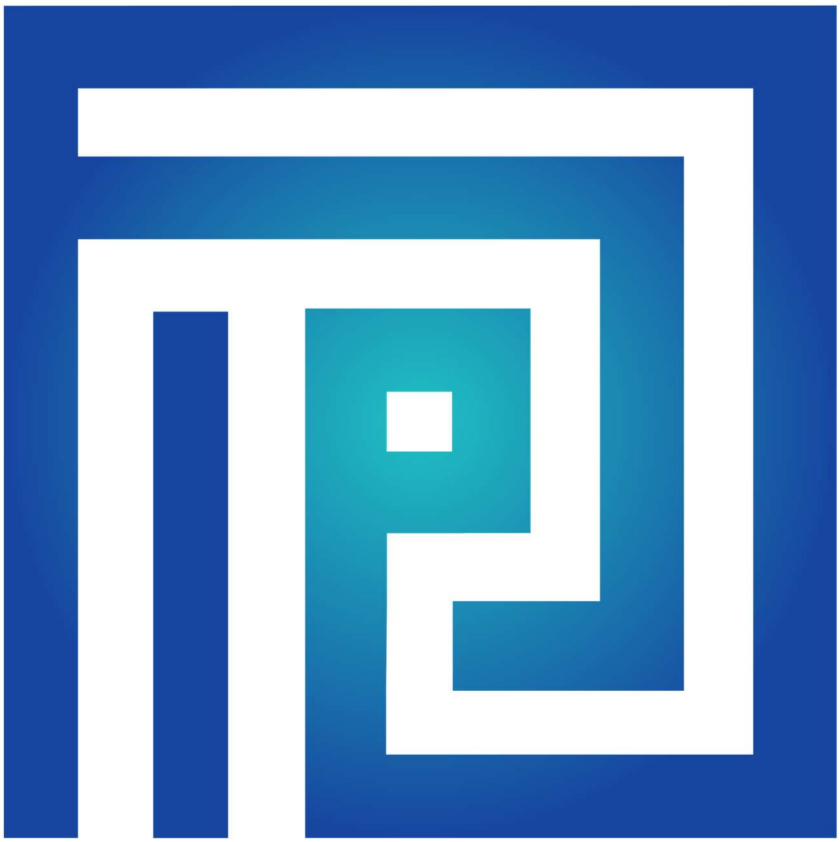
Plutus Pioneer Program

4th iteration - textbook



IOG

Plutus Pioneer Program 4th iteration – textbook

This book was prepared by the IOG Education team. The content is based on the 4rd Plutus Pioneer program that was presented by Lars Brünjes and his colleagues. All program code in this book is taken from the Plutus Pioneer program and is freely available at:

https://github.com/input-output-hk/plutus-pioneer-program/tree/fourth-iteration

All pictures in this book were created by IOG, if not otherwise noted.

All content in this textbook is provided for educational purposes only.

Table of Contents

Table of Figures

1 Plutus introduction

Plutus is the native smart contract language for Cardano. It is a Turing-complete language written mostly in Haskell, and Plutus smart contracts are effectively Haskell programs. By using Plutus, you can be confident in the correct execution of your smart contracts. It draws from modern language research to provide a safe, full-stack programming environment based on Haskell, the leading purely functional programming language. [1]

So, in order to understand the code in this book, you must understand the basics of the Haskell programming language. The Haskell project contains a list of learning resources as books and tutorials under its documentation section, from which some are free and some commercial. It can be found at Haskell's official web page:

- https://www.haskell.org/documentation/

IOG provides their own Haskell course that is tailored to the needs of learning Plutus and also Marlowe, a low code smart contract language for financial contracts and alternative to Plutus. The Haskell course can be found at the IOG GitHub page:

https://github.com/input-output-hk/haskell-course

If you would like to learn more about Plutus and its software development kit (SDK) in addition to what this book offers, you can check out the online documentation which can be found at:

- https://plutus.readthedocs.io/en/latest/
- https://plutus-apps.readthedocs.io/en/latest/

1.1 Setting up your development environment

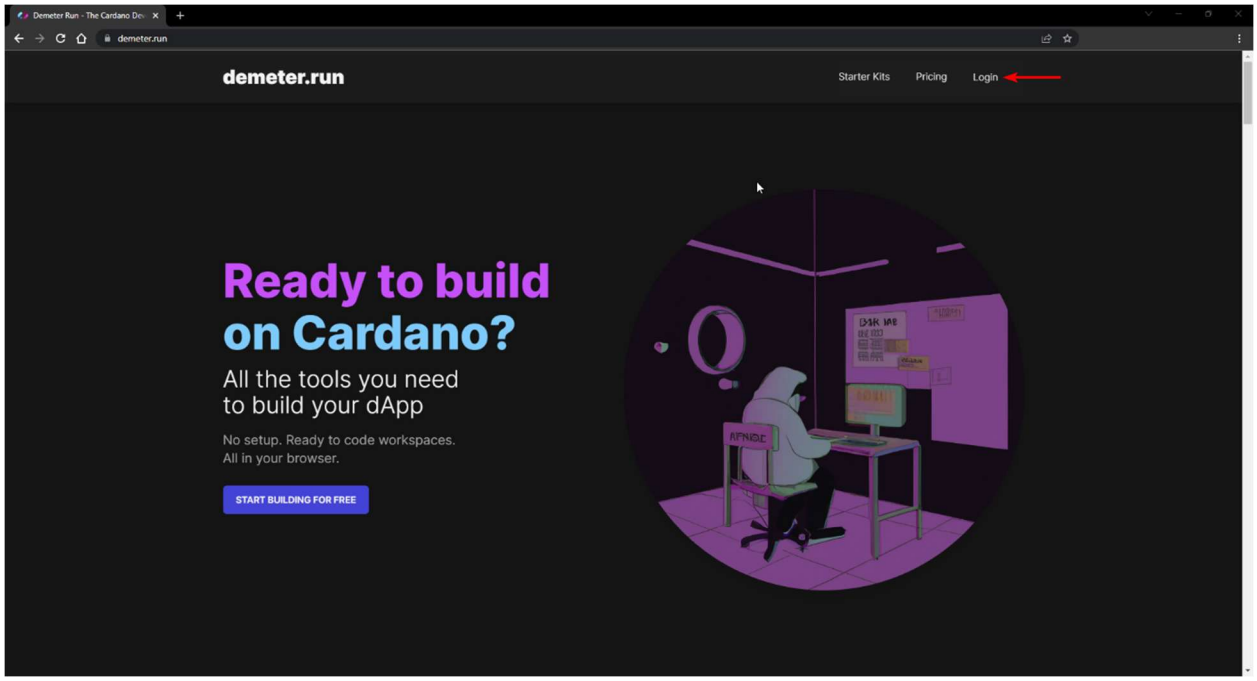
The installation instructions for setting your development environment are split into two parts. The first part is using the online platform *Demeter.run* and the second part is setting up a Docker container locally on your PC. The second instructions are general and can be used on Windows, Mac OS and Linux operating systems.

1.1.1 Using Demeter.run

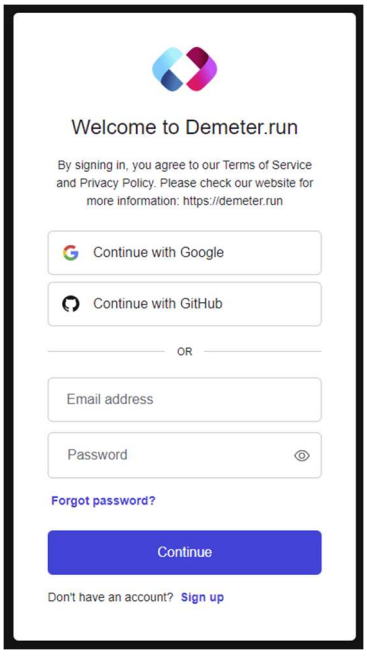
Demeter.run is a cloud-based environment that provides all the tools required for building and deploying Cardano applications. It can be found at https://demeter.run/. To use demeter.run, you need a web browser on any operating system and internet access. Follow the steps below to set up your demeter.run account and to use this development environment.

To setup a demeter.run account and start using it, follow these steps:

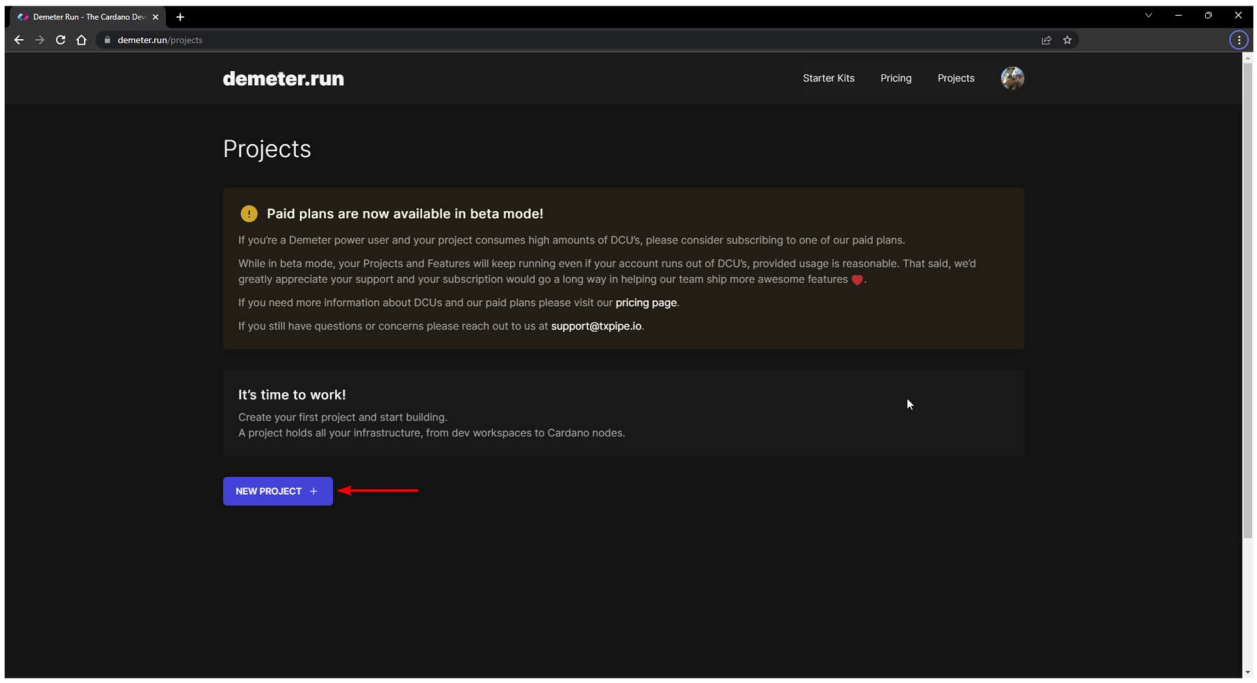
1. Open your browser and navigate to https://demeter.run/. You will see the home page of demeter.run as illustrated below.



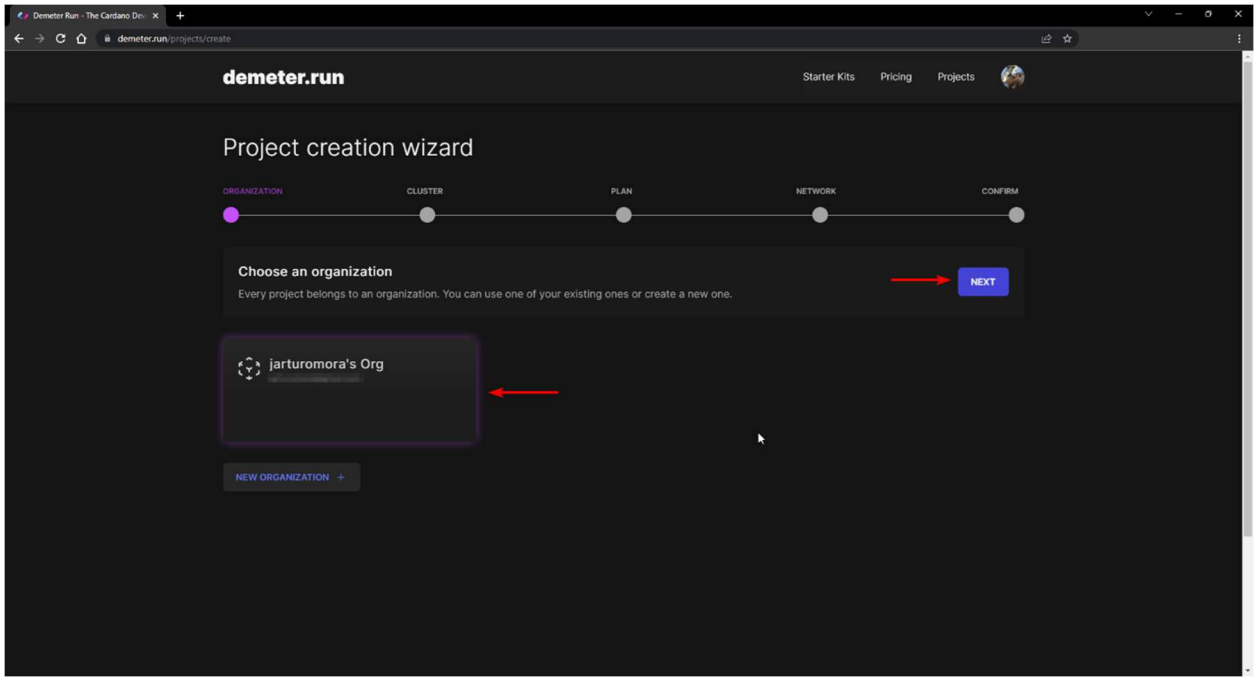
2. Click the **Login** option on the top navigation menu.
3. Next, you need to set up an account. You can choose to set a username and password, or you can sign in by using your Google or GitHub account. Select the method that best fits your preferences to continue.



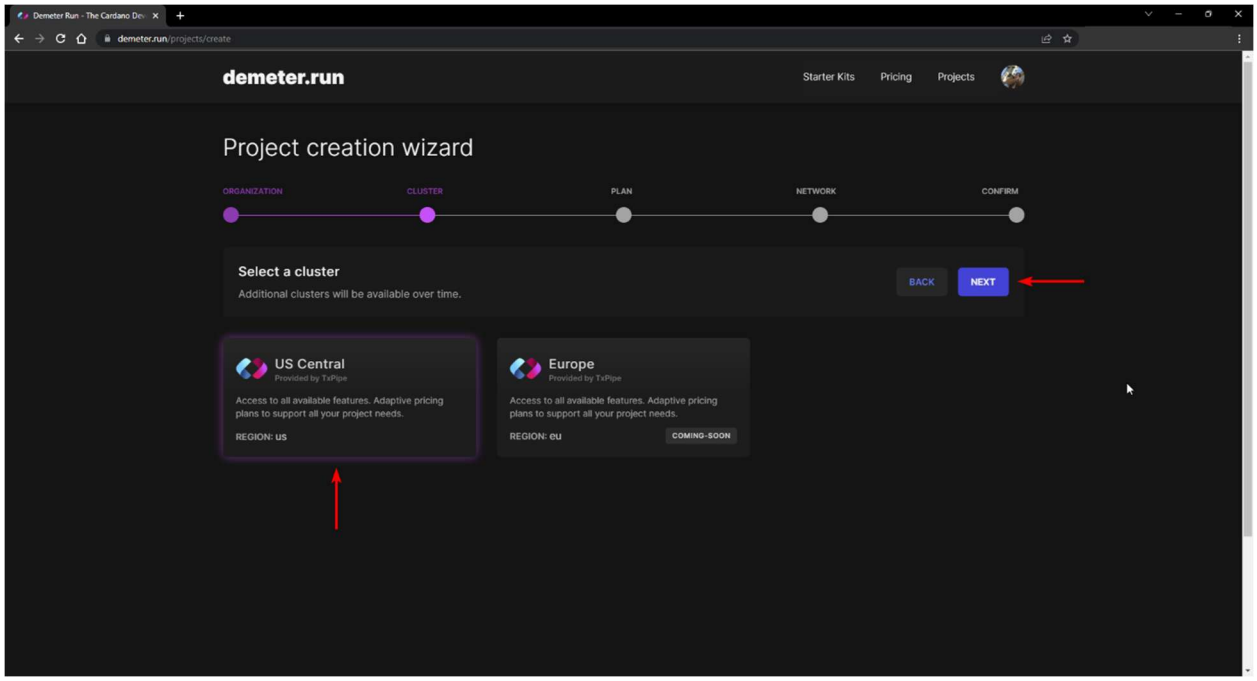
4. After signing in, you are now ready to use demeter.run. You will see the Projects page, as illustrated bellow.



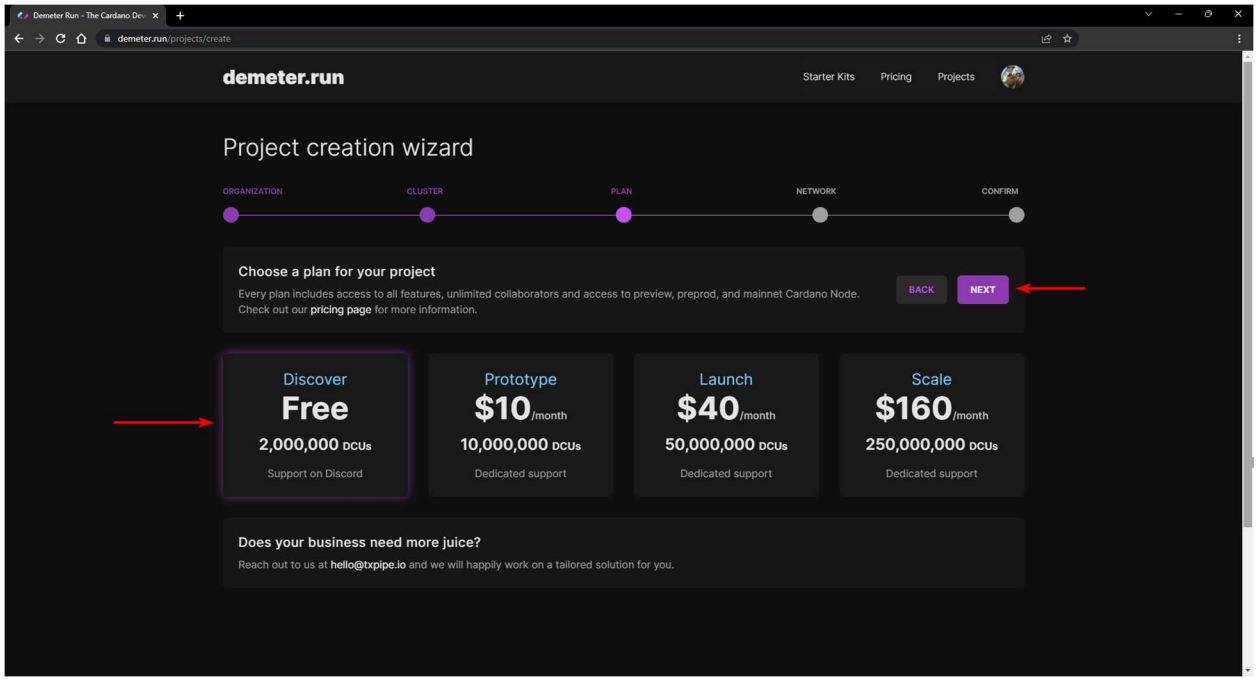
5. Click the **NEW PROJECT** button.
6. Next, you need to follow the required steps to set up a new project. First choose an organization where your project will reside. By default, an organization with your username exists. We will select the default organization for this demo, as shown in the image below, where the username is “jarturomora”.



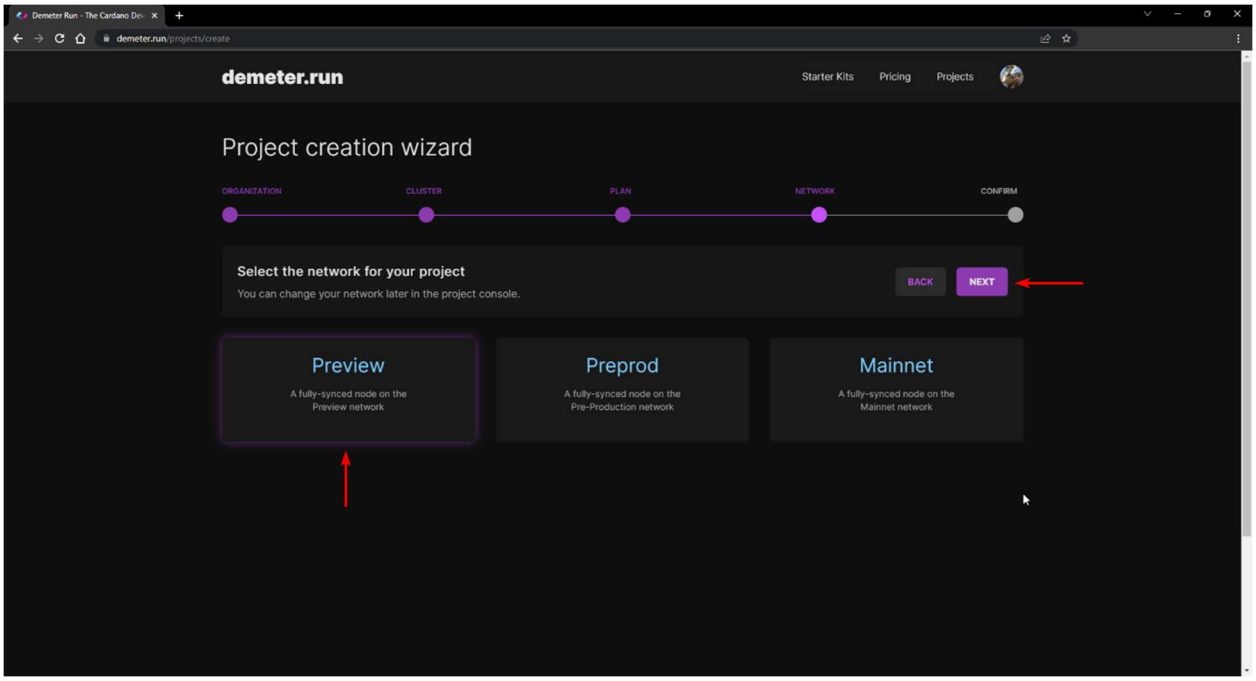
7. After choosing the organization, click the **NEXT** button.
8. Choose the location of the cluster that you will use. As for now, only a US-based cluster exists. Select the **US Central** cluster and click the **NEXT** button to continue, as illustrated below.



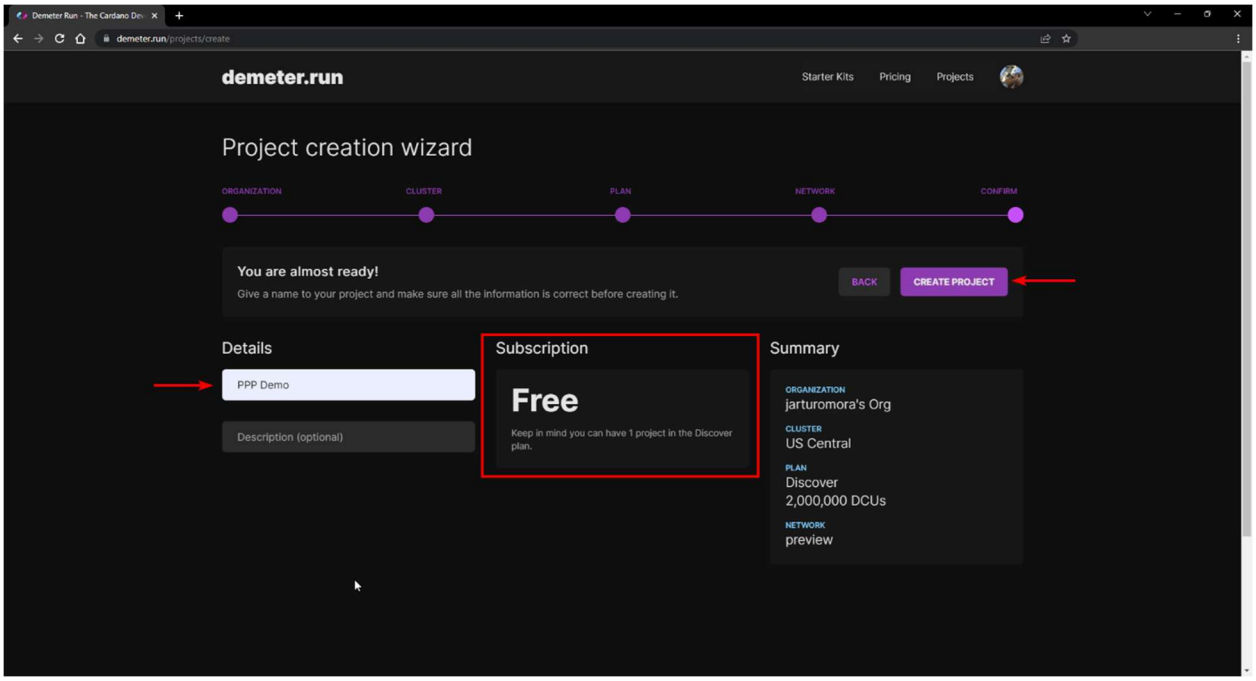
9. After choosing a cluster, the next step is to select a plan. For the purpose of the program, we will use the **Discover** plan that is available free of charge. Select the **Discover** plan and click on the **NEXT** button to continue, as illustrated below.



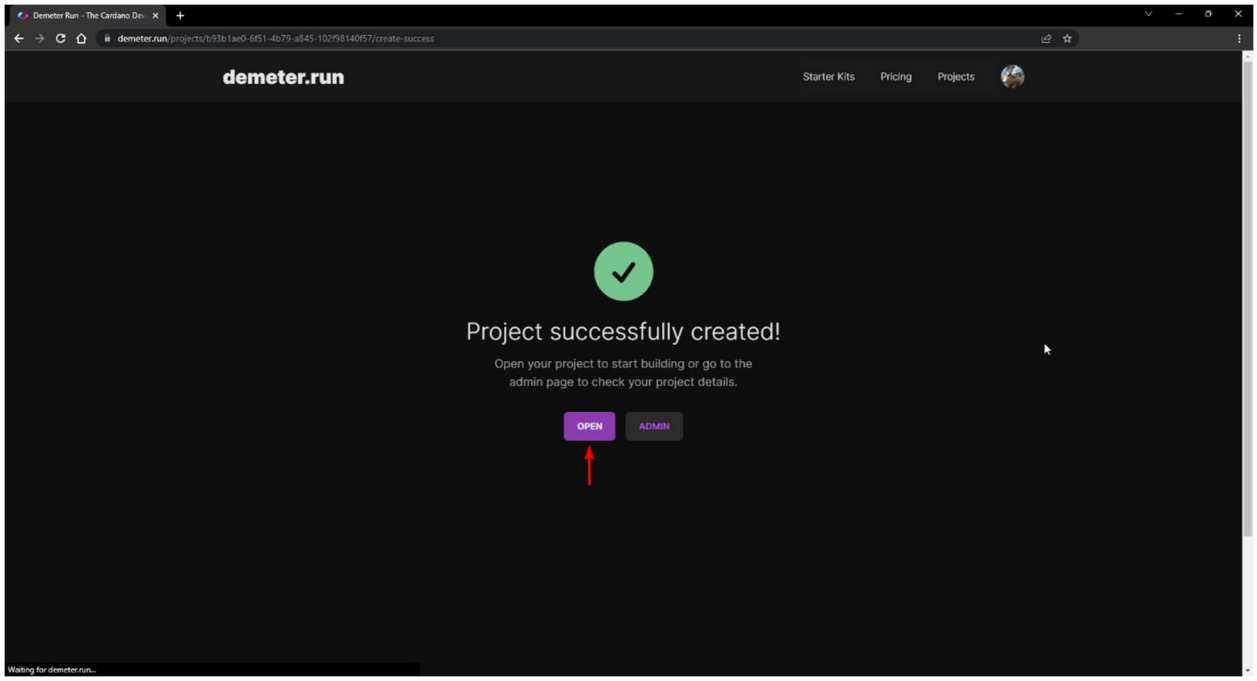
10. Next, you need to choose a network for your project. For testing purposes, we will typically use the **Preview** networks. Select the **Preview** network and click the **NEXT** button to continue, as illustrated below.



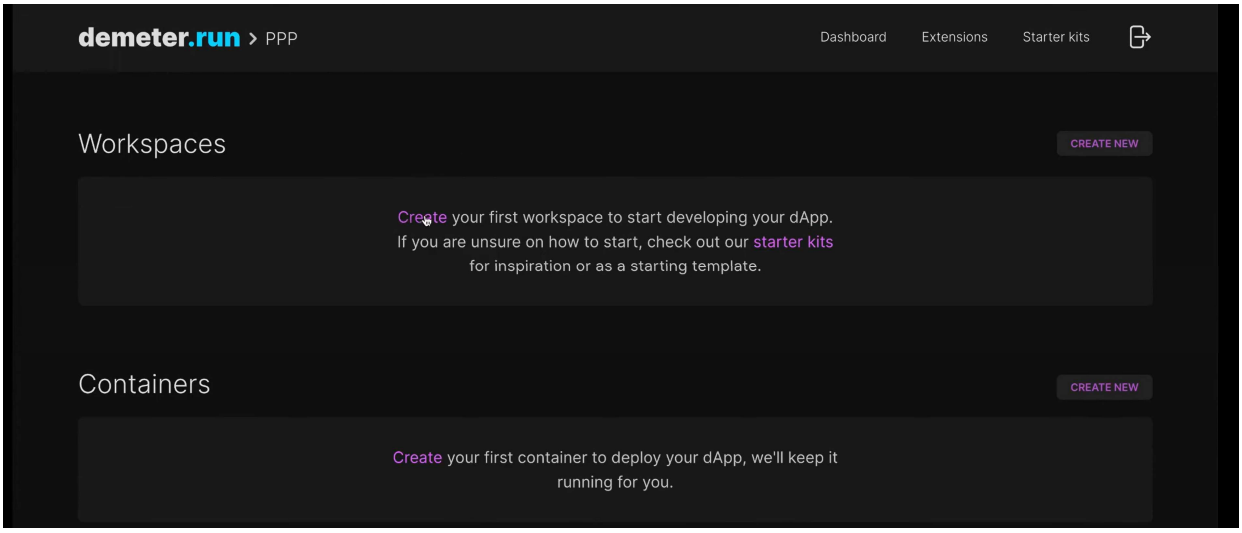
11. The last step is to provide a name for your project. Also, in this step, you can review the project's details; be aware that you can have only one project in the Discover plan. In this demo, the project was named **PPP Demo**. Set a name for your project and click on the **CREATE PROJECT** button to finish, as illustrated below.



12. Once your project is created, it is ready to start building applications. You will see a confirmation message, as illustrated below. Next, you can click the **OPEN** button to navigate to the project's Development Console.

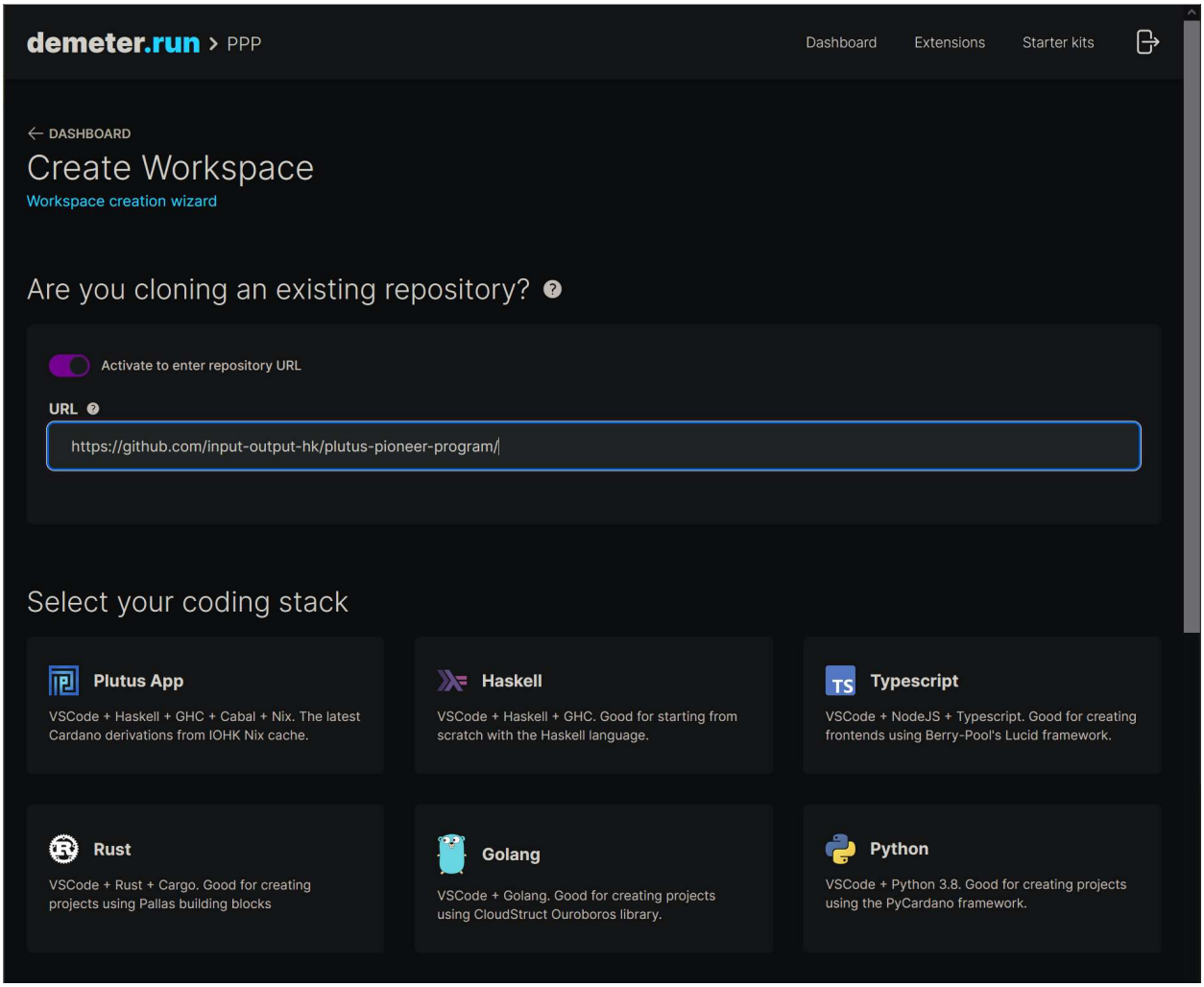


13. After that you see the dashboard. You will need to create a workspace that is tailored to your needs of development tools. Click **Create new** on the upper right corner of the Workspace section.

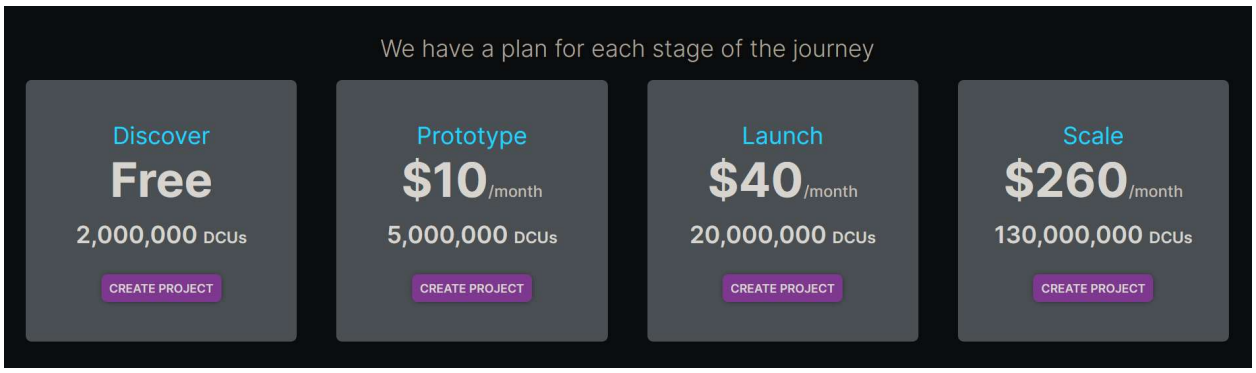


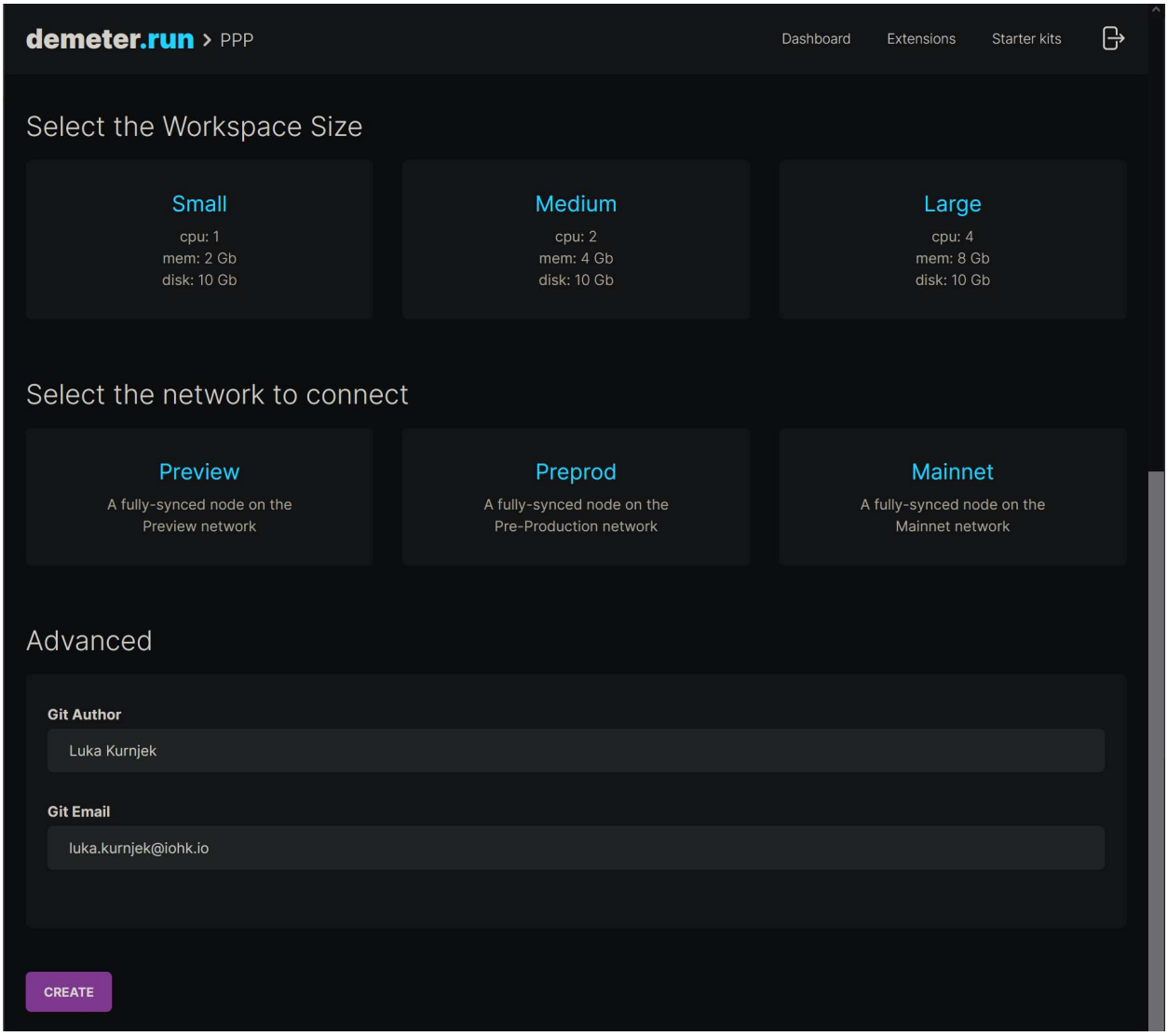
14. In the Create workspace section we click the toggle button for using an existing repository and input the Plutus Pioneer Program GitHub repository. If you have a GitHub

account, you can clone the PPP repo to your account and then input a link to your repository. In the **Select coding stack** section choose the Plutus App option.



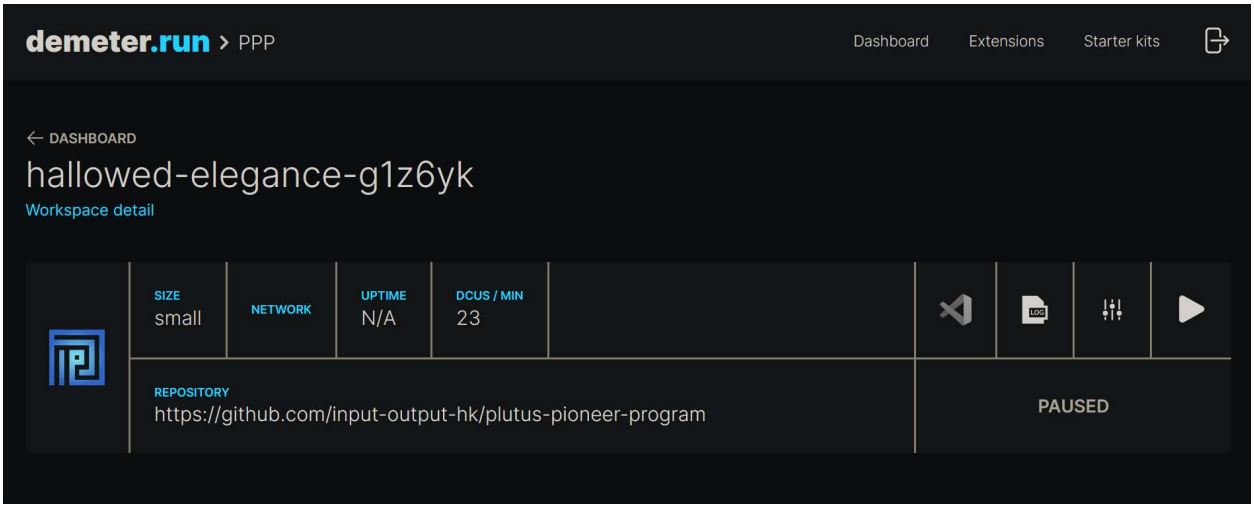
15. At the bottom of this page, you can select your workspace size. The bigger the size is the more credits it consumes. For our project we will choose medium size. You have 2.000.000 DCU (Demeter Computed Units) for free. For the network we select **Preview** and then click the **Create** button at the end of the page.



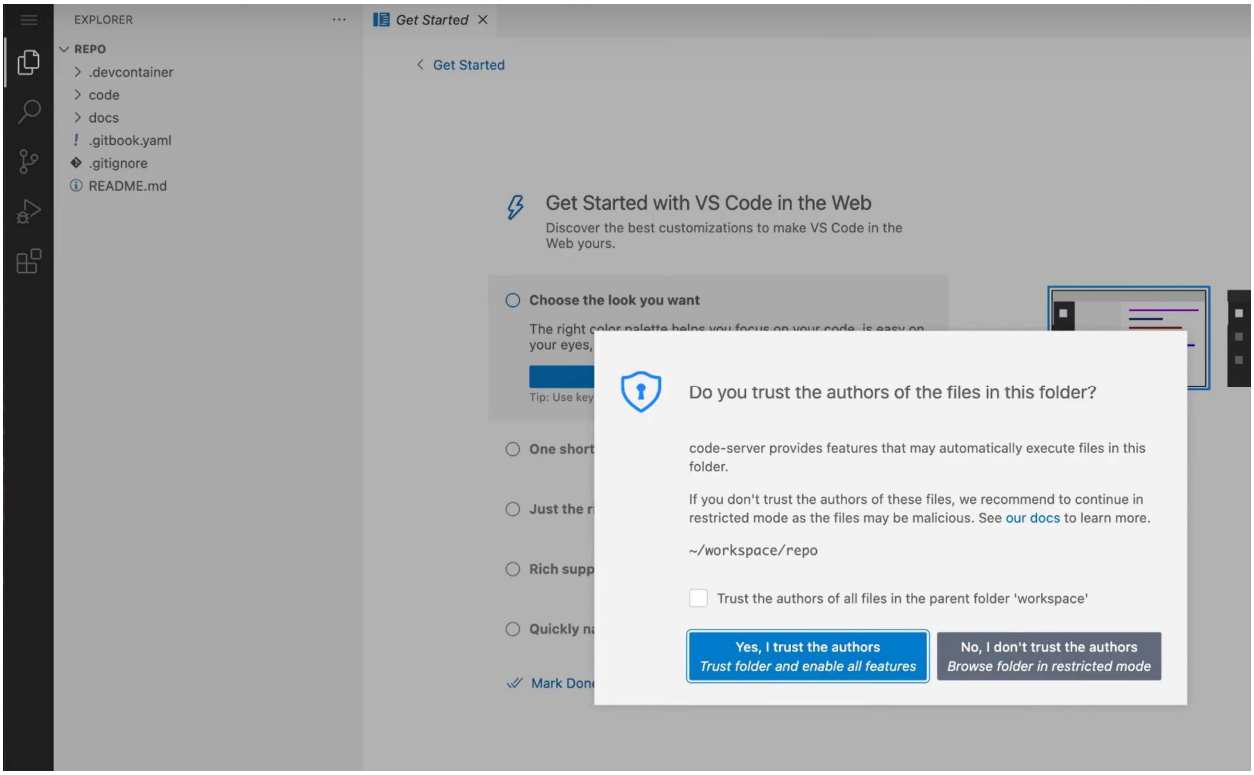


16. After that you will be redirected to the workspace detail page and the workspace information will be displayed. The workspace also needs some time before it builds. After its build, it will automatically go into run mode. You can click the **Pause workspace** button on the right side to pause it and then click again on the same button that will have the tool tip “Run workspace” to run it again.

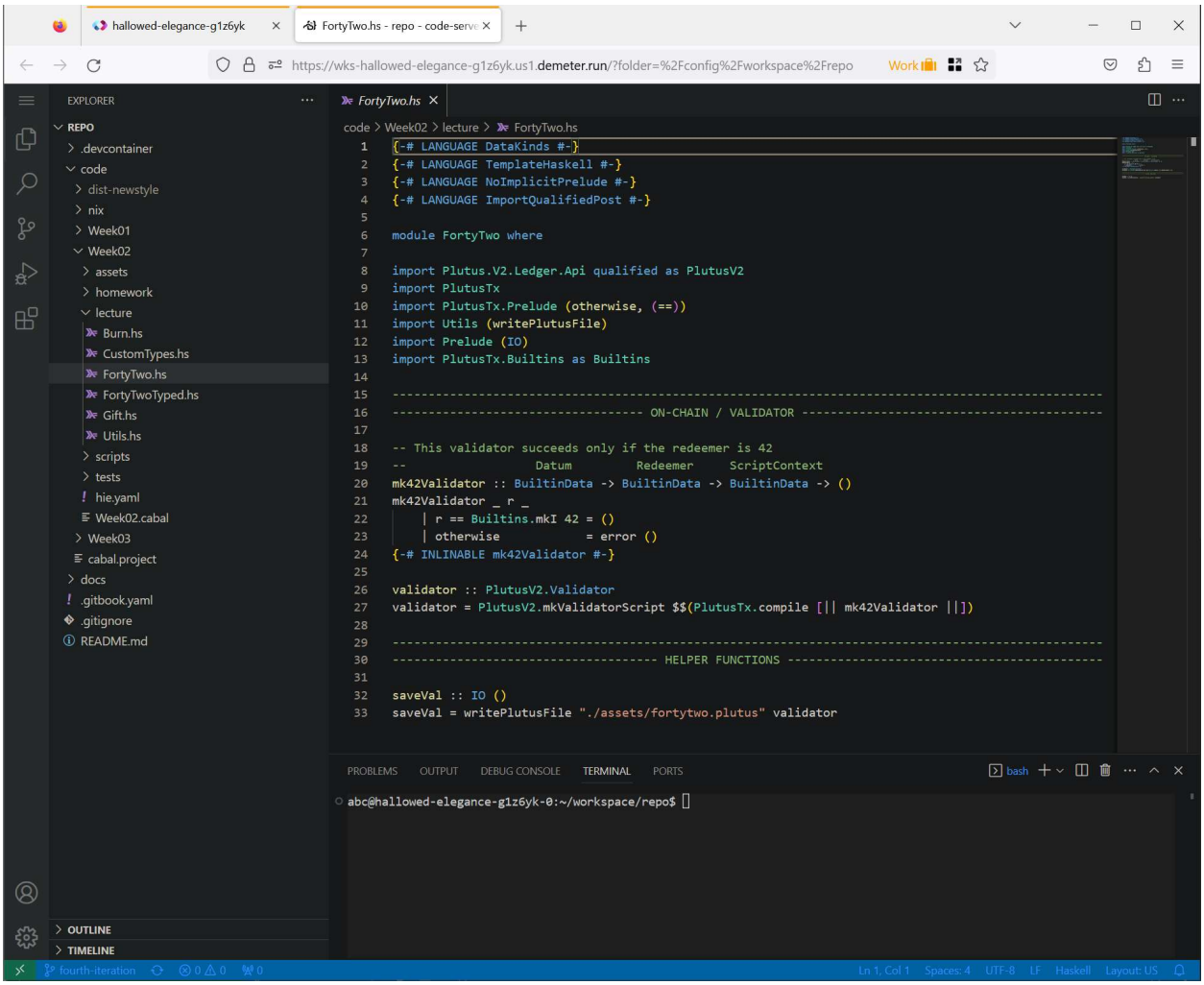
Demeter.run will charge you DCUs for a running workspace. So after you finish with your work, always pause your workspace.



17. Once everything is set up you can click the VSCode logo and a VSCode editor will appear that will contain all the files from the Pioneer Program repository you inputted in the workspace creation step. Check the trust box and click the **Yes** button.



18. Once you are inside the VSCode editor you can click on the menu button in the upper left corner and click **Terminal -> New Terminal**. This will open a terminal window in your workspace and certain command line tools will become available. The code for each lecture is contained in the *code/WeekXX/lecture* folders.



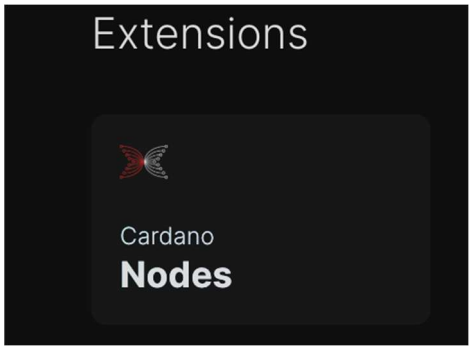
19. With the following commands you can update your project and create a validator script for the *FortyTwo.hs* code example from the Week02 folder.

```
$ cd code
$ cabal update
$ cd Week02
$ cabal repl
Prelude Gift> import FortyTwo
Prelude FortyTwo Gift> FortyTwo.saveVal
```

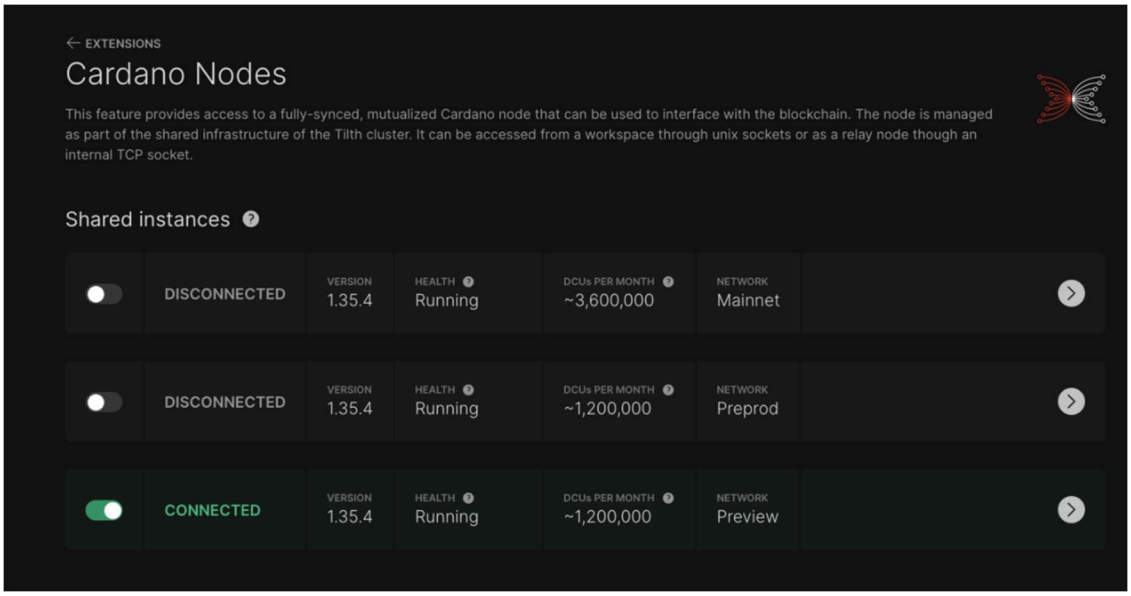
The file *fortytwo.plutus* is saved in the assets folder. You will learn in the next chapter what you can do with the code in the Week02 folder. If you have any questions, you can reach the PPP community on the IOG's technical community on Discord (https://discord.gg/inputoutput) by checking out the *#pioneers-questions* channel.

Later in the book we will use the cardano-cli command line tool which can construct off chain transactions. To use this tool a Cardano node has to be running. To add the node to your demeter.run environment click on the **Extensions** tab from your demeter.run dashboard.

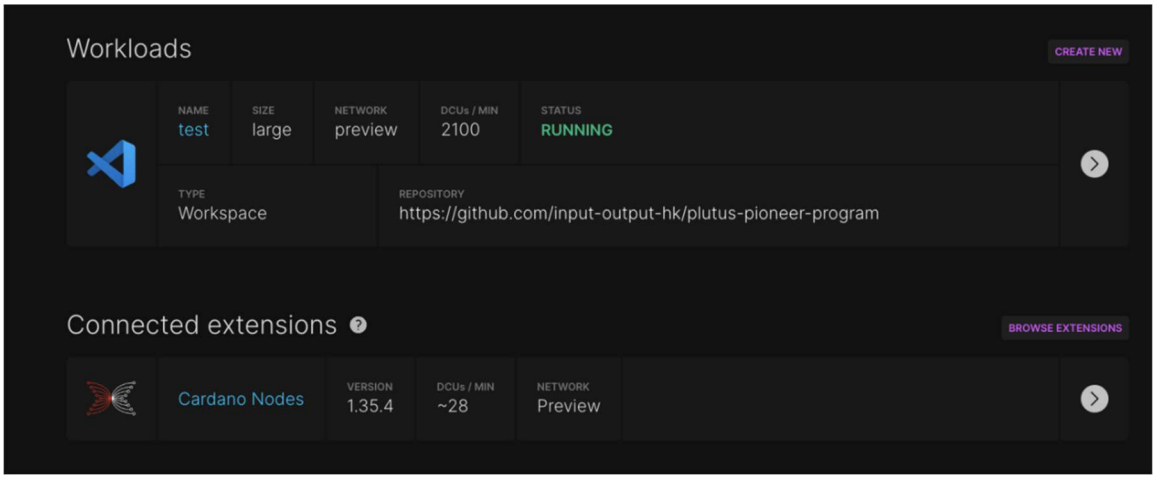
Then select the **Cardano Node** extension. The image is illustrated below.



After the Cardano node settings page opens toggle the setting that says **Preview Network**.



Once you have done that, your dashboard should look like this:



1.1.2 Installing Docker and VSCode

In this guide, you'll learn how to set up a local working environment using Docker and Visual Studio Code. If you have no previous experience using Docker and VSCode, you can follow the steps in this chapter that explain all necessary actions for a successful environment setup.

It is recommended to install and run Docker on your native OS. If you want to run Docker Desktop inside a VM, read through the official Docker documentation for using Windows and a VM.

Docker is a platform designed to help developers build, share, and run applications in an isolated environment on any operating system. To ease setting up a local working environment for this course, the IOG Education team created a Docker container that packages all the dependencies required to follow up the lessons of this program.

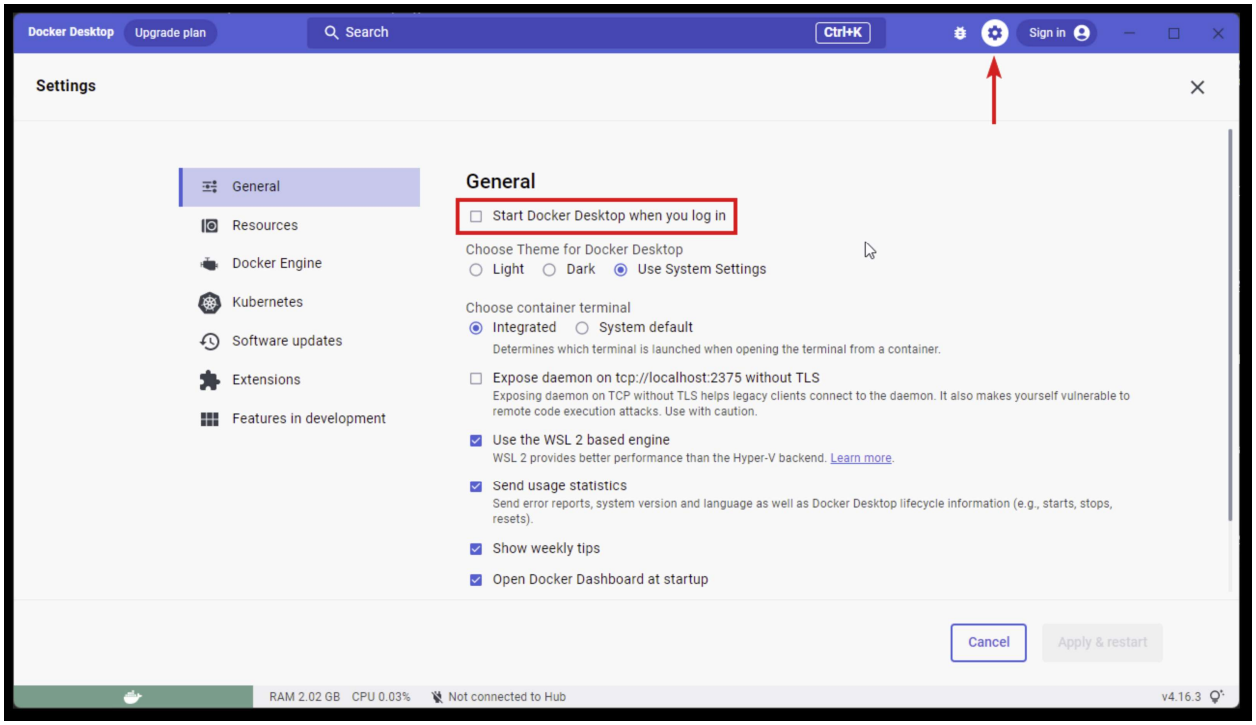
A Docker container is a standard unit of software that packages up code and all its dependencies, so an application runs quickly and reliably from one computing environment to another. From the Docker documentation, you can learn more about Docker containers on the following page: https://www.docker.com/resources/what-container/.

Follow the next steps to install Docker in your computer:

1. Open your browser and navigate to https://www.docker.com. Click the **Download Docker Desktop** button from the Docker's homepage. By default, you'll download a version compatible with your operating system. Docker is available for Linux, Microsoft Windows, and Apple macOS.

Important note for macOS users. Be sure that you download the correct version according to the chip of your computer, for M1 or M2 chips, download the "Apple Chip" version. For Intel chips, download the "Intel Chip" version.

2. After downloading the Docker Desktop installer, execute it and follow the instructions by choosing the default options. Installation options may vary depending on your chip and operating system. If you need detailed instructions, please visit the Get Docker section on the docker docs website: https://docs.docker.com/get-docker/.
3. When Docker Desktop is started, it automatically starts the docker daemon in the background. This will happen every time you login into your computer. You can change this behavior by turning off the **Start Docker Desktop when you log in** option in the Docker Settings configuration. Note that the docker daemon must be running when accessing the docker container in VSCode which we will explain next.



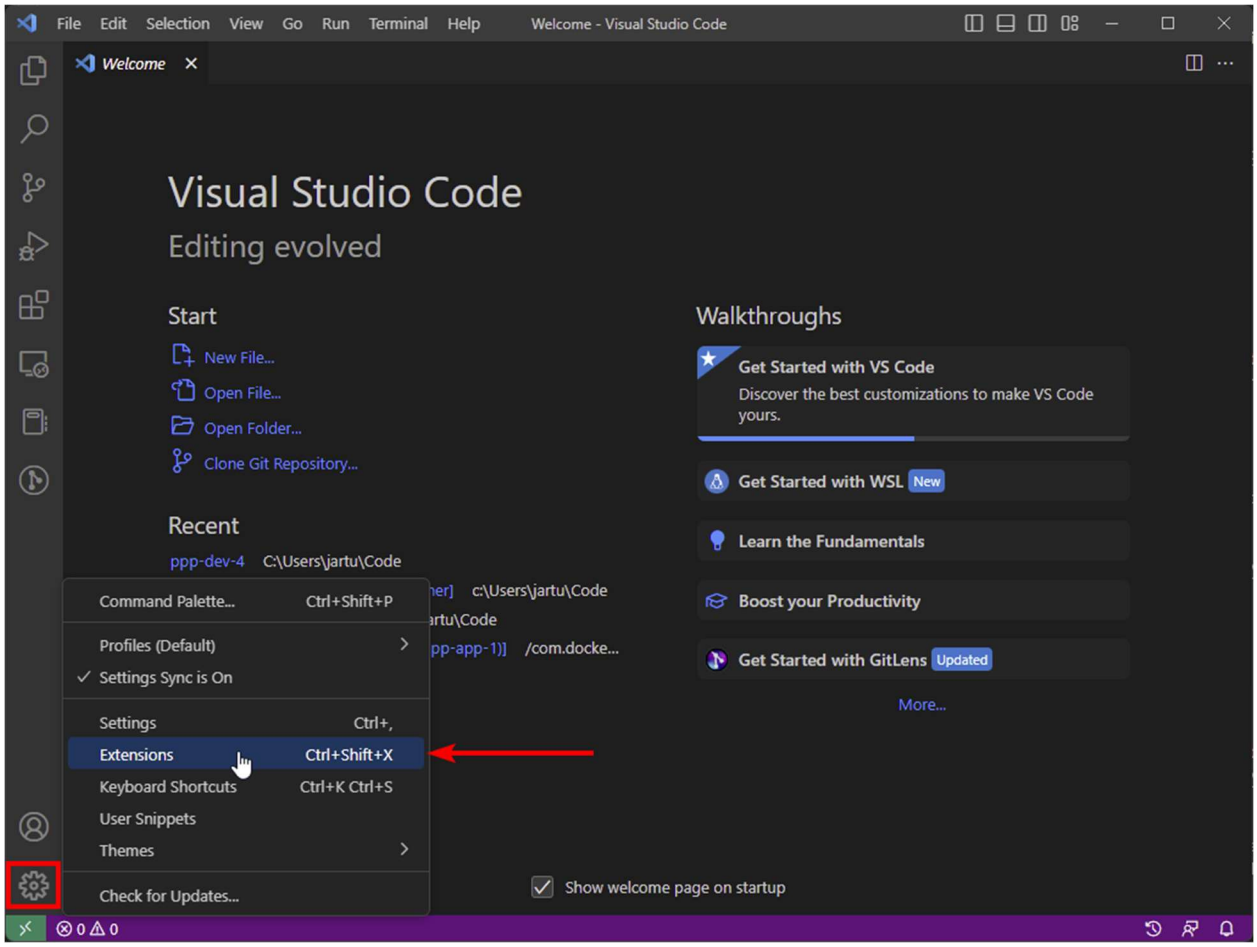
Visual Studio Code (https://code.visualstudio.com/), also known as VS Code, is a source code editor freely distributed by Microsoft that runs on Windows, Linux, and macOS. Additionally, to code editing, VS Code allows you to create and install extensions that eases your daily work.

Do not install any Haskell extension in Visual Studio Code. If you have VS Code installed, you may see several prompts from VS Code asking for permission to install several Haskell extensions when you open the PPP repository. You do not need to install any of those, as the Docker container we deliver will install all the Haskell extensions that VS Code needs. Some Pioneers who installed additional Haskell extensions report issues in completing this install guide; also, the IOG Education team members experienced problems while compiling Plutus scripts.

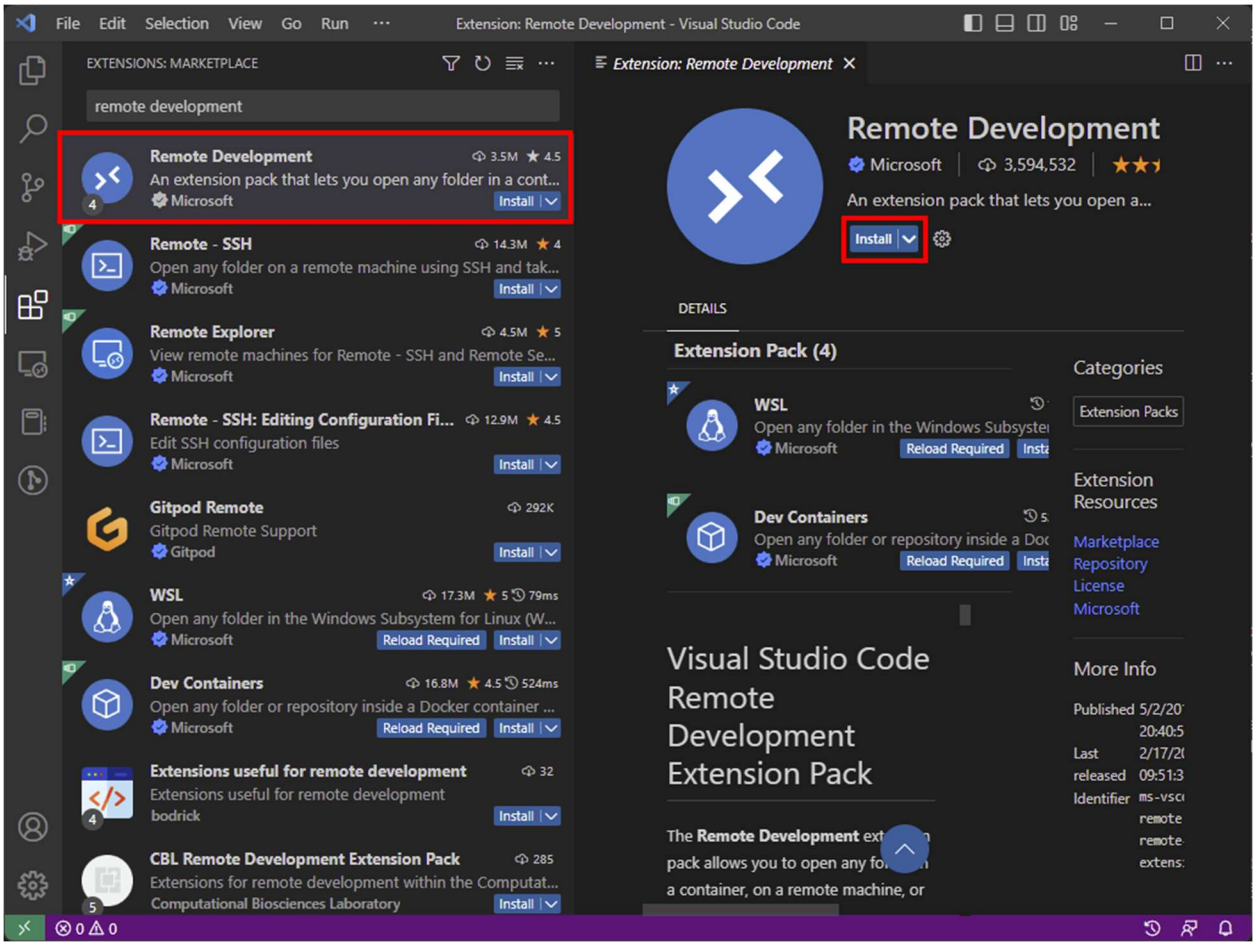
Follow the next steps to install VS Code and a handy extension that you will use in this course:

1. Open your browser and navigate to https://code.visualstudio.com/ to open the Visual Studio Code website. There is a download button that suggests you download the software depending on your operating system. Be sure to download the latest version.
2. After downloading the VS Code installer, execute it and follow the instructions by choosing the default options. Installation options may vary depending on your chip and operating system. If you need detailed instructions, please visit the Setting up Visual Studio Code section in the VS Code docs website: https://code.visualstudio.com/docs/setup/setup-overview

3. Once you have installed VS Code, open it to install an extension. Extensions are additional add-ons that extend the VS Code's functionality; Microsoft provides some extensions, but also, plenty of extensions are created by other companies and developers. To install an extension, click on the **Manage** icon in the bottom left corner and choose the **Extensions** options as illustrated below.



4. Next, a window showing the **Extensions Marketplace** will appear on the left side of the VS Code UI. In the search box, type **remote development** to look for the **Remote Development** extension provided by Microsoft: https://marketplace.visualstudio.com/items?itemName=ms-vscode-remote.vscode-remote-extensionpack
   Once the extension appears, click it. As illustrated below, you should click the **Install** button to install an extension.



After successfully installing the Remote Development extension, you will note that an **Uninstall** button appears in the extension description tab, and the **Open a Remote Window** green icon will appear on the bottom left corner of the VS Code UI.

You have now installed the required software to use the Docker container. Next, we'll guide you through the steps you must follow to load the Docker container and execute it for the first time.

1.1.3 Running the PPP Docker Container

Before moving forward into using the Docker container provided for this cohort of the PPP, you need to close VS Code. Please follow the next section, where we'll guide you through finishing the setup of your local working environment.

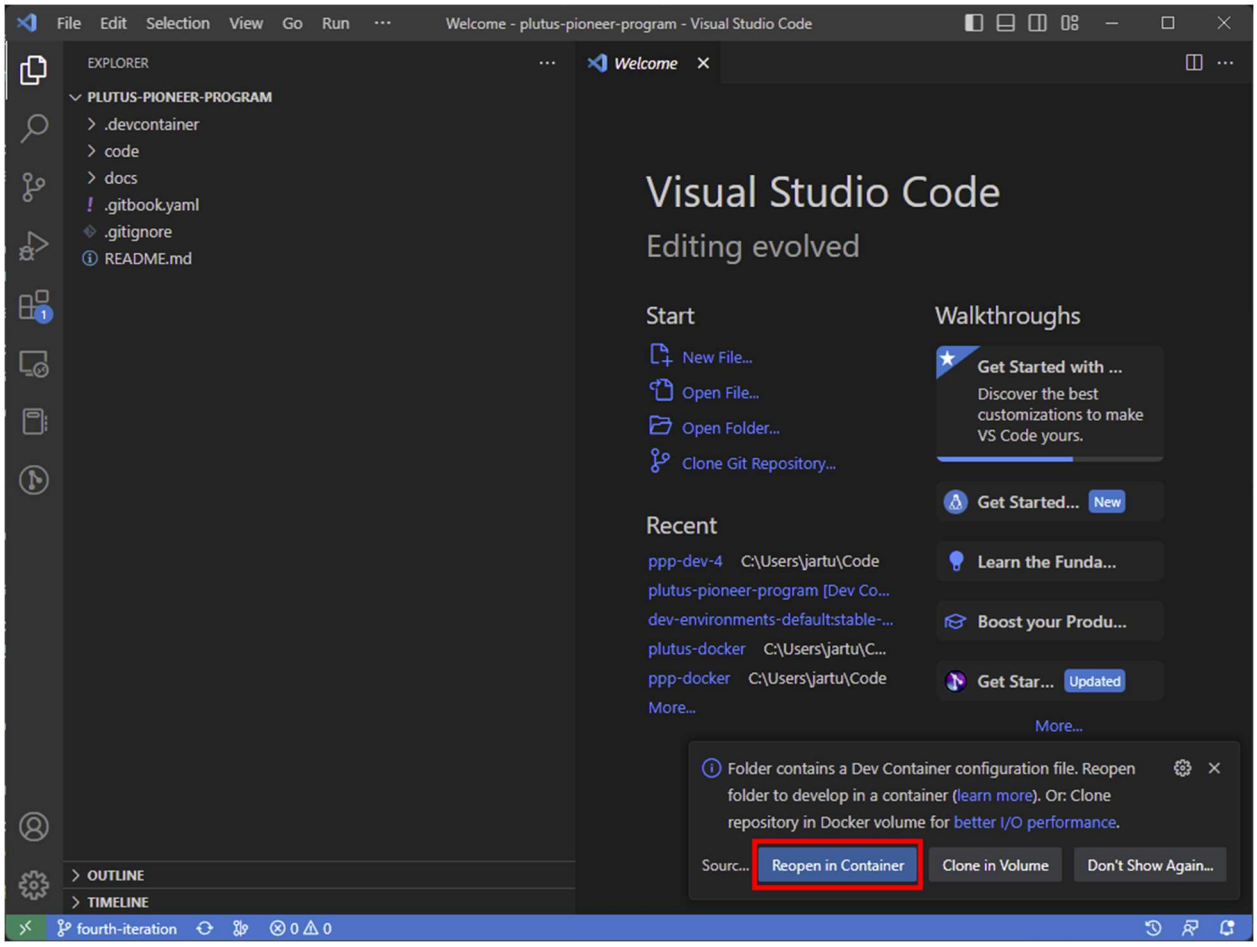
As stated in the demeter.run section you can work with the official GitHub repository or create a fork if you have a GitHub account and want to keep your work online. Instructions on how to create a GitHub fork can be found in the Plutus Pioneer GitBook at the following location:

https://iog-academy.gitbook.io/plutus-pioneers-program-fourth-cohort/preliminary-work/setup/docker#forking-the-ppp-repository

In either case you will need to clone a Plutus Pioneer repository to your computer using Git. If you don't have Git installed, please follow the instructions from the Git documentation https://git-scm.com/book/en/v2/Getting-Started-Installing-Git, where you'll find detailed information about installing Git on Linux, macOS or Windows. Once you have Git installed open up a terminal that has the git command available, cd into you desired location and clone the PPP GitHub repository:

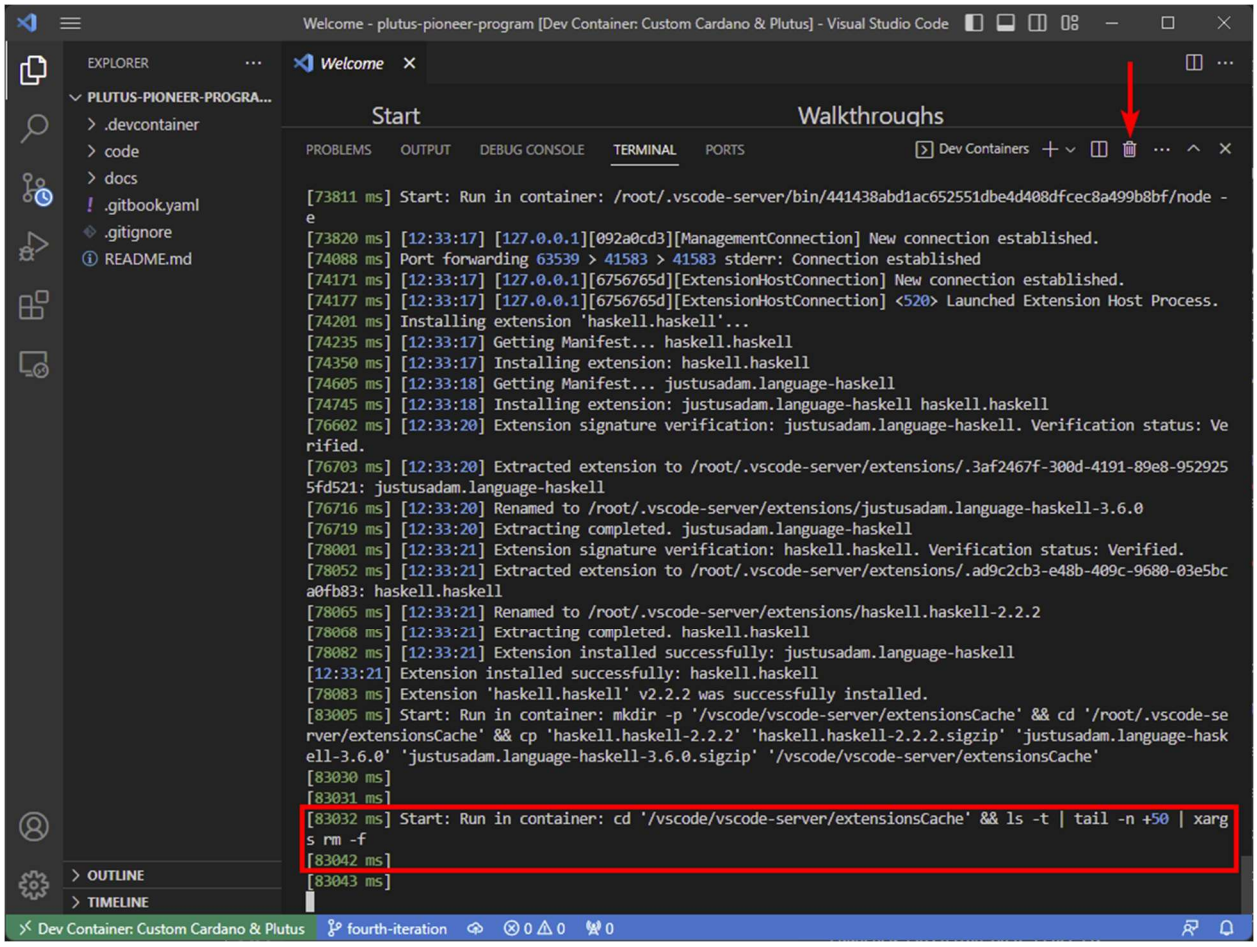
```
$ git clone https://github.com/input-output-hk/plutus-pioneer-program
```

Next open VSCode and in the File menu click **Open Folder** and then select the PPP folder you have cloned with Git. When VS Code opens, the **Remote Development** extension will detect the Docker container. As shown in the image below, you'll see a message asking to reopen your project in the container. Click the **Reopen in Container** button to continue.



After reopening your project in Container, the Docker container will be built. You can click the **Starting Dev Container** message to view the log. Please look at the log to be aware of when the container is ready. The build may take some time. After a few minutes, you will see a message

in the log starting with the text `Start: Run in container` as illustrated below. This message indicates that the dev container is ready.



You can safely close this terminal window by clicking the trash can icon in the upper right corner of the terminal window. Now, open a new terminal window into VS Code by clicking the **Terminal** menu and then **New Terminal**. From the new terminal window cd into the *code* and update the repository with the following command:

```
root@99286bb23f1c:/workspaces/plutus-pioneer-program# cd code
root@99286bb23f1c:/workspaces/plutus-pioneer-program/code# cabal update
```

This step is critical as all the code and updates should run into the code directory. Be patient while running these commands. Depending on your hardware configuration and an internet connection, the time required to execute the command `cabal update` may vary. It takes at least 5 minutes to finish; however, we experienced waiting times of up to 15 minutes in some hardware and internet settings. Now, to finish the dev container setup, type and execute the following command in the VS Code terminal to build all the dependencies required by Plutus.

```
root@99286bb23f1c:/workspaces/plutus-pioneer-program/code# cabal build all
```

After successfully running this command, you will see the system prompt back with no errors. Also, this command can take from 10 to 30 minutes to finish.

In case you are wondering what the docker container includes you can view the *Dockerfile* inside the *.devcontainer* folder. There you can also find the *devcontainer.json* file that specifies how the container is built. If the "image" field is uncommented it pulls the image from DockerHub. If the build field is uncommented the container is built from the local Dockerfile. Only one of them can be uncommented. If you add or update the commands in your Dockerfile you can then build a custom container for your needs. An example would be that you want to update the version of the Cardano node which is hardcoded in the Dockerfile to 1.35.5.

1.1.4 Troubleshooting Guide

The following issues may occur on any operating system:

- If you are getting this message from the Docker terminal in VS Code:

```
The connection to the terminal's pty host process is unresponsive, the terminals may stop working.
```

  The solution to this issue is to uninstall VS Code and install the Insiders version. You can download this version from this page: https://code.visualstudio.com/insiders/.

- **Linux Issues**

  If while opening the Docker container in VS Code, you get the following error message:

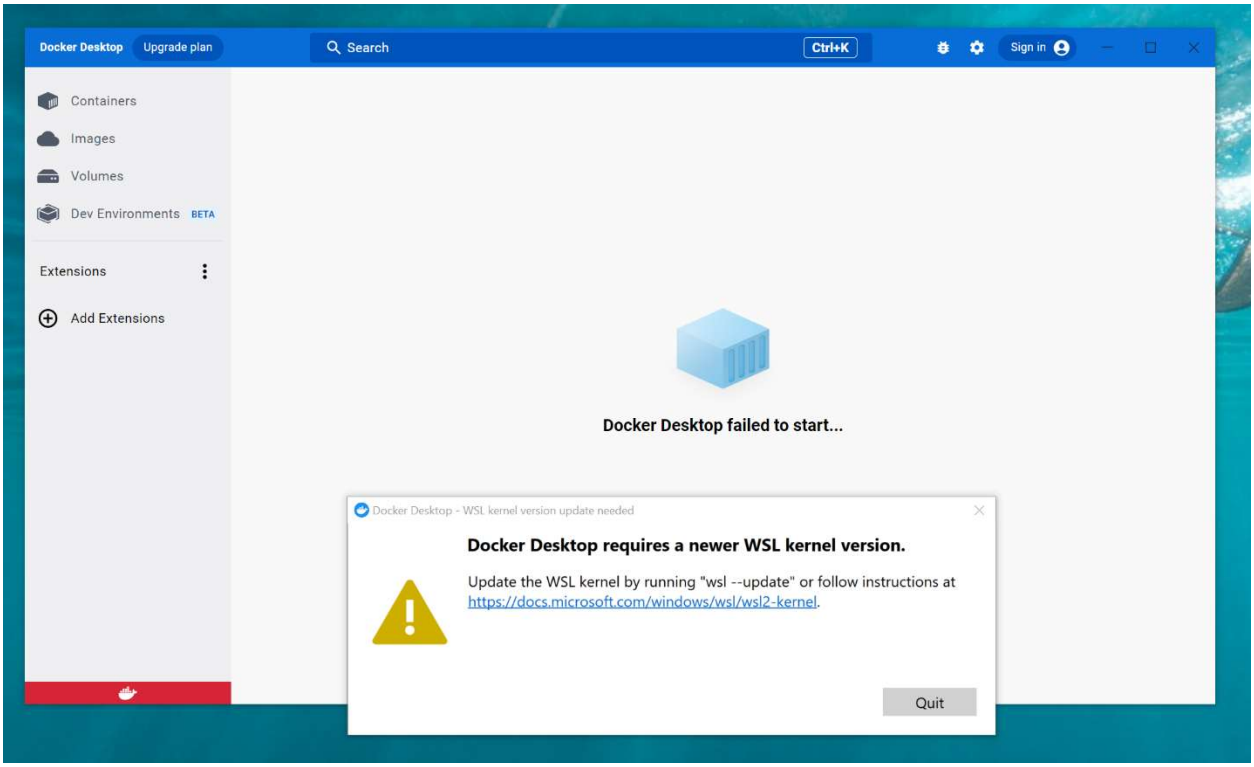
```
View container 'remote' requires 'enabledApiProposals: ["contribViewsRemote"]' to be added to 'Remote'.
```

  you should uninstall VS Code and install the Insiders version. You can download this version from this page: https://code.visualstudio.com/insiders/. To learn more about why this issue can happen, please read the Using Proposed API article https://code.visualstudio.com/api/advanced-topics/using-proposed-api from the Visual Studio Code documentation.

- **Windows Issues**

  If you have installed Docker Desktop before joining the PPP, you might receive an error message asking to install the latest WSL version. An example of the error message is shown in the image below. Please refer to this page from the Windows Subsystem for

Linux Documentation: https://docs.microsoft.com/windows/wsl/wsl2-kernel for further instructions.



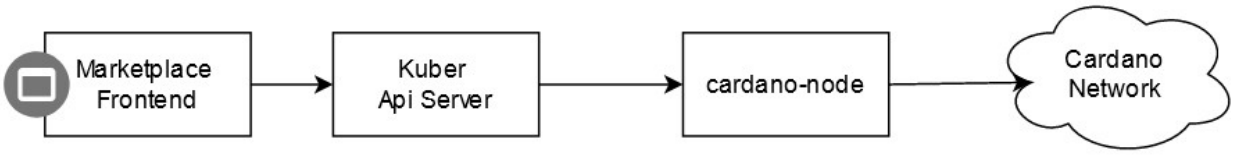
1.2 Kuber marketplace

In this chapter we will demonstrate a DApp that you can build yourself once you are done with this course. The DApp we are going to demonstrate is built by dQuadrant as an example to demonstrate the Kuber library. Kuber is an open-source Haskell library and rest API to easily compose complex EUTxO transactions on the Cardano blockchain. The DApp is called the *cardano-marketplace* and can be found at the dQuadrant GitHub page:

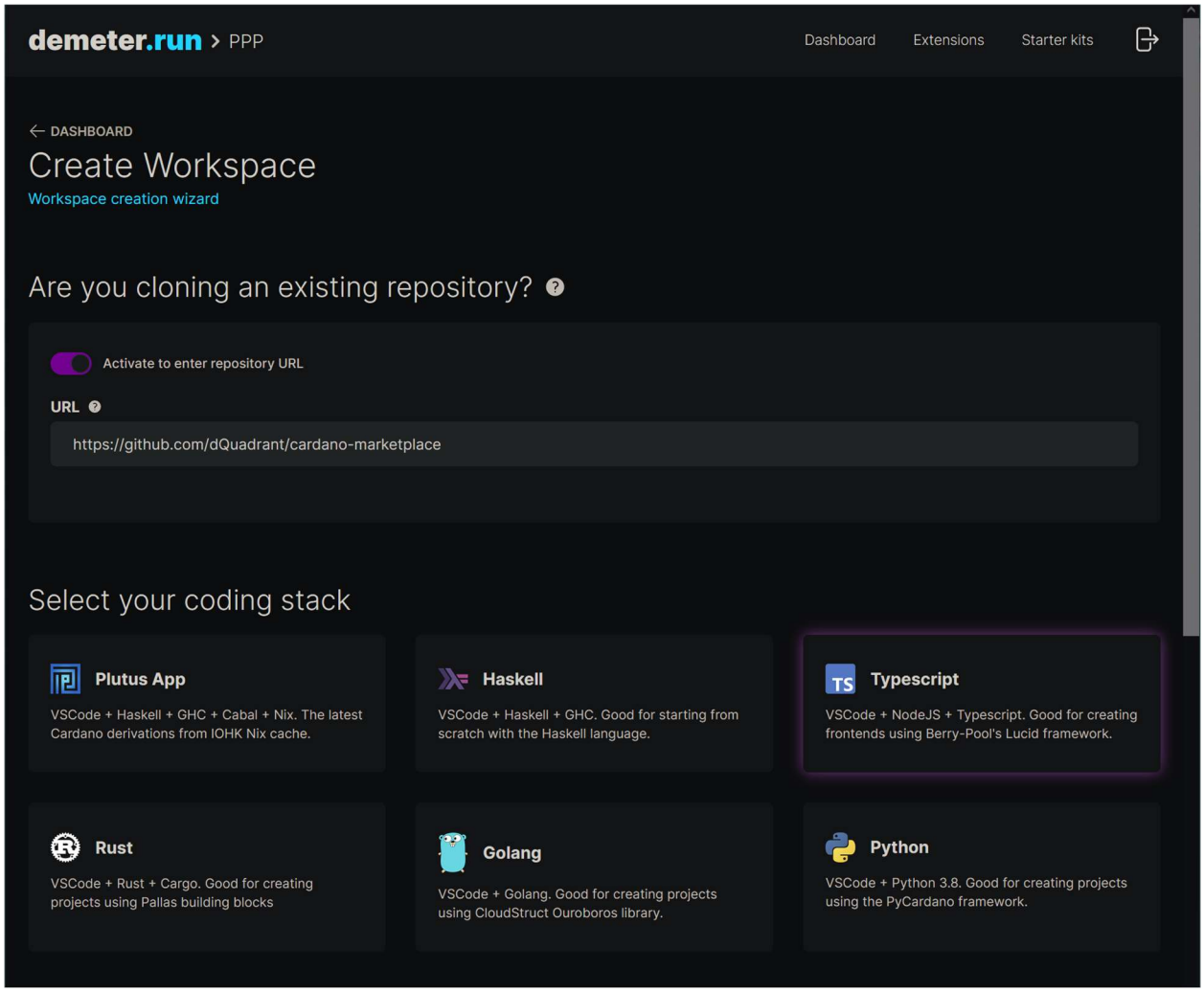
https://github.com/dQuadrant/cardano-marketplace

The marketplace can be deployed in multiple ways, and we're going to take the simple route and deploy a front-end for it, which does the following:

- connects to an existing Kuber server to construct transactions.
- lists tokens for sale using blockfrost API.



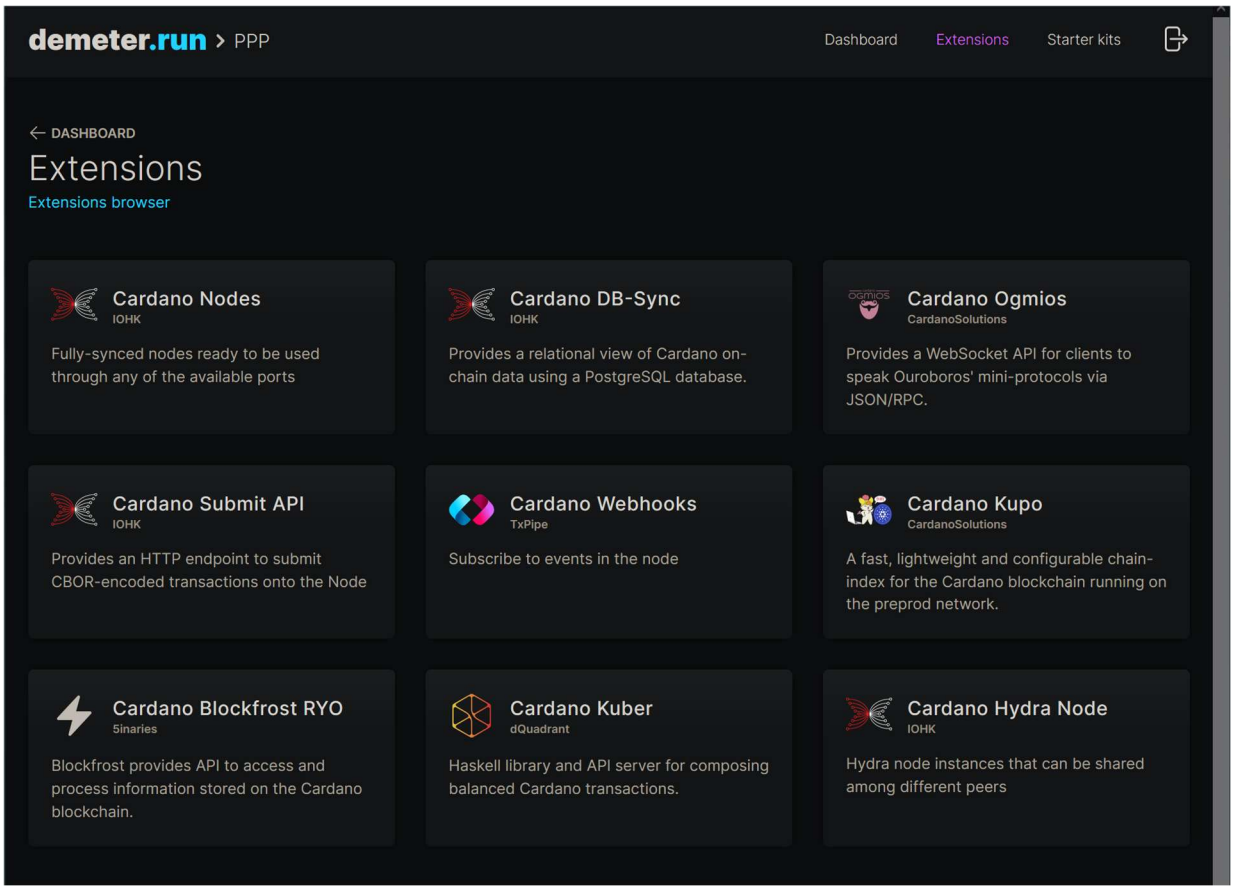
We are going to use Demeter to deploy this architecture. We go to demeter.run and open the project we already created in chapter 1.1.1 Using Demeter.run. If you have not performed these steps go to the chapter and follow the instructions up to creating a workspace. Then click **Create New** under the Workspace section. Input the GitHub link of the cardano-marketplace and select the Typescript coding stack.

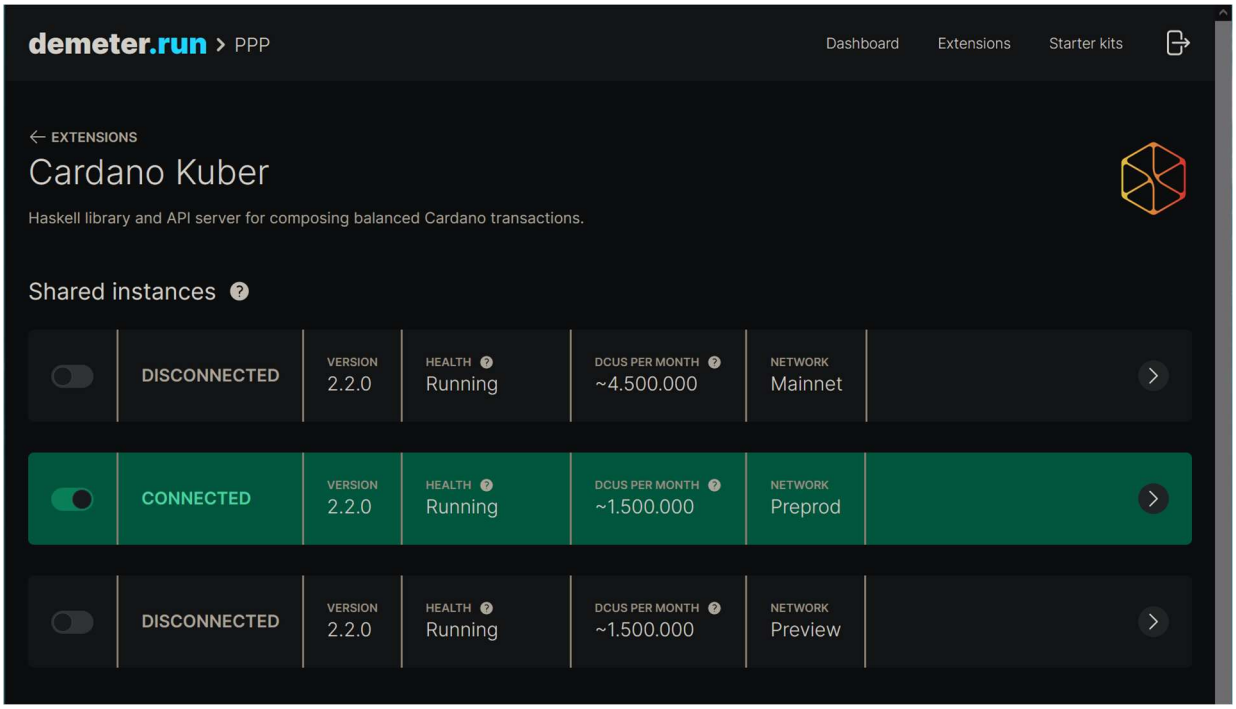


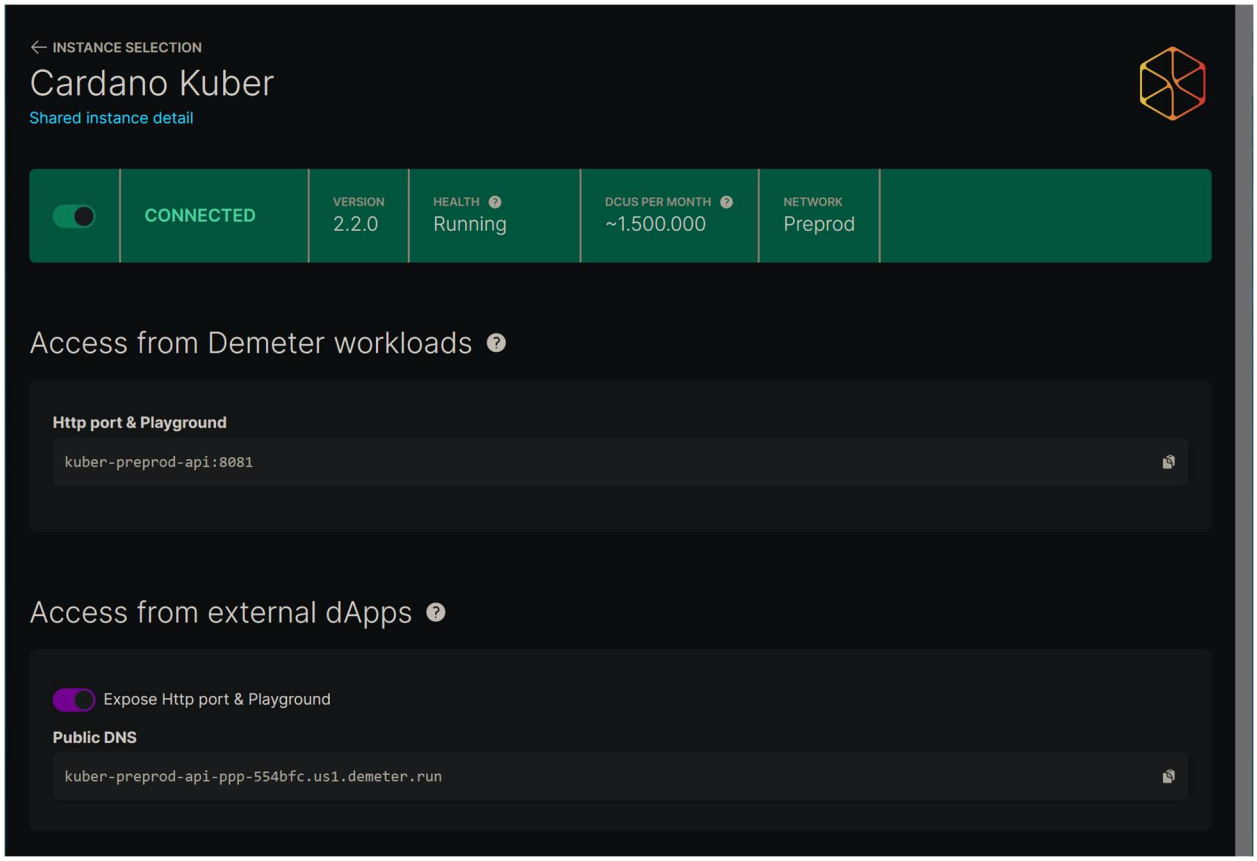
At the bottom of the page select the small workspace size and the Preprod network. Because we need a Kuber API to connect with the Cardano network we will connect the Kuber extension to our workspace. We click the **Browse** extension in the Connected extensions section and choose Cardano Kuber. Next under the Shared instances section we turn on the Preprod net and click the arrow on the right side.

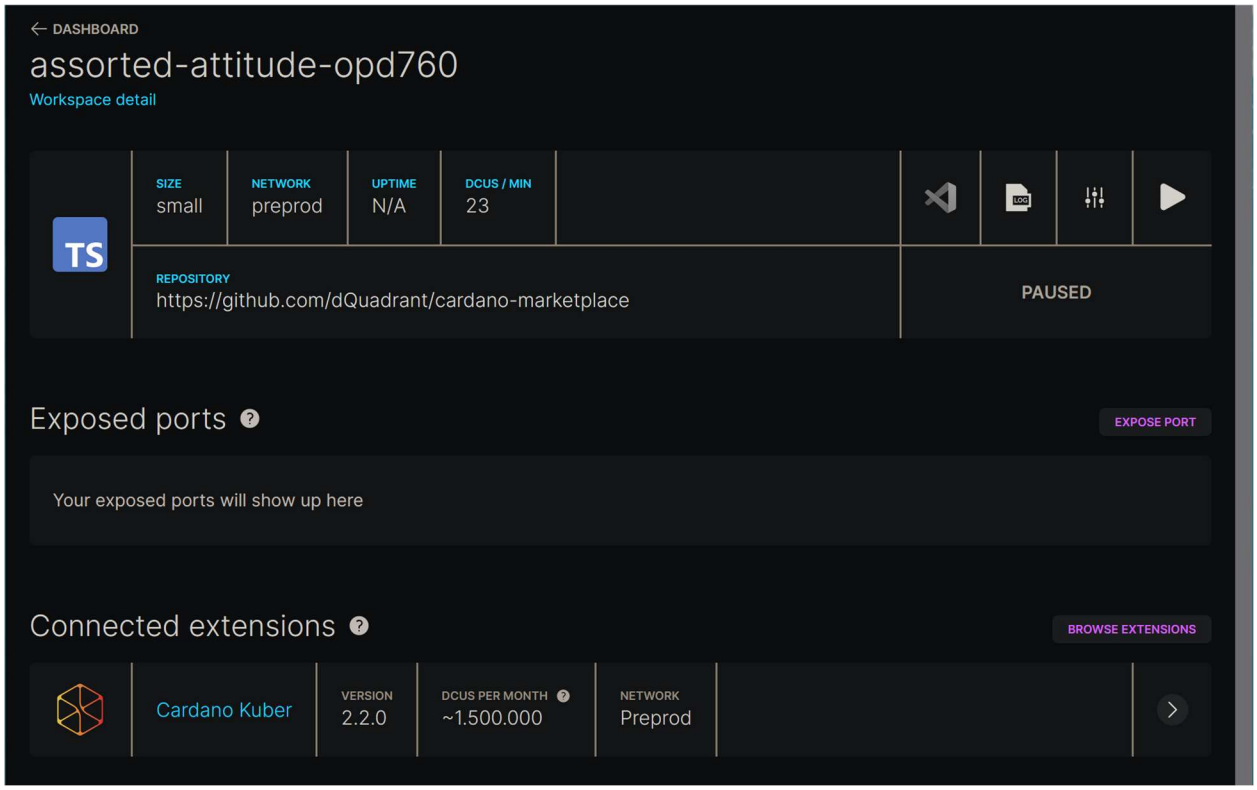
In the access from external dApps section we toggle the button Expose Http port & Playground, which will open an http port of our dApp application such that we can access it from our browser. We copy the URL under the Public DNS section that gets displayed. Then we return to

the dashboard by clicking on the top of the page **Instance selection -> Extensions -> Dashboard**. We can then enter our workspace by clicking the VSCode button.





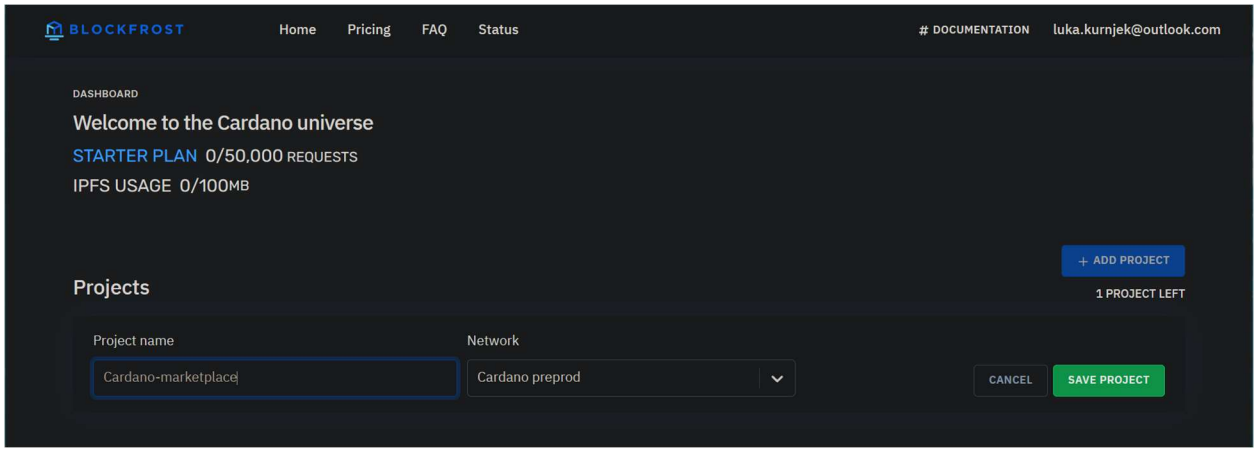

← INSTANCE SELECTION
Cardano Kuber
Shared instance detail
CONNECTED
VERSION
2.2.0
HEALTH
Running
DCUS PER MONTH
~1.500.000
NETWORK
Preprod
Access from Demeter workloads
Http port & Playground
kuber-preprod-api:8081
Access from external dApps
Expose Http port & Playground
Public DNS
kuber-preprod-api-ppp-554bfc.us1.demeter.run


← DASHBOARD
assorted-attitude-opd760
Workspace detail
SIZE
small
NETWORK
preprod
UPTIME
N/A
DCUS / MIN
23
REPOSITORY
https://github.com/dQuadrant/cardano-marketplace
PAUSED
Exposed ports
EXPOSE PORT
Your exposed ports will show up here
Connected extensions
BROWSE EXTENSIONS
Cardano Kuber
VERSION
2.2.0
DCUS PER MONTH
~1.500.000
NETWORK
Preprod

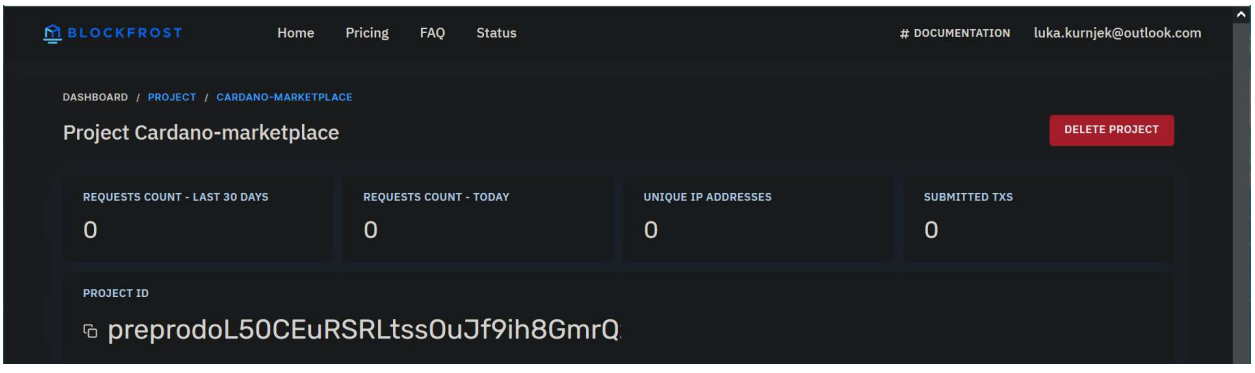
Before we build the frontend, we first need to configure it. Inside the VSCode editor we go to *frontend/src/config* and edit the file as follows:

```
export const kuberApiUrl = "https://kuber-preprod-api-ppp-554bfc.us1.demeter.run"
export const explorerUrl = "https://preprod.cexplorer.io"
export const blockfrost = {
    apiUrl: "https://cardano-preprod.blockfrost.io/api/v0",
    apiKey: "preprodoL50CEuRSRLtssOuJf9ih8GmrQzXgaiy",  // replace the api key
}
```

The link under the *kuberApiUrl* is the one we copied before when we exposed the http port. We also change the URLs to Cardano pre-production network. Under the API key we input the API that you can obtain from blockfrost.io. Go to their webpage and create an account or sign in with GitHub or Google account: https://blockfrost.io/auth/signin. After that under the projects section input your project name, select the Cardano preprod network and click **Save project**.



After that under the section project ID your blockfrost API key will be displayed.

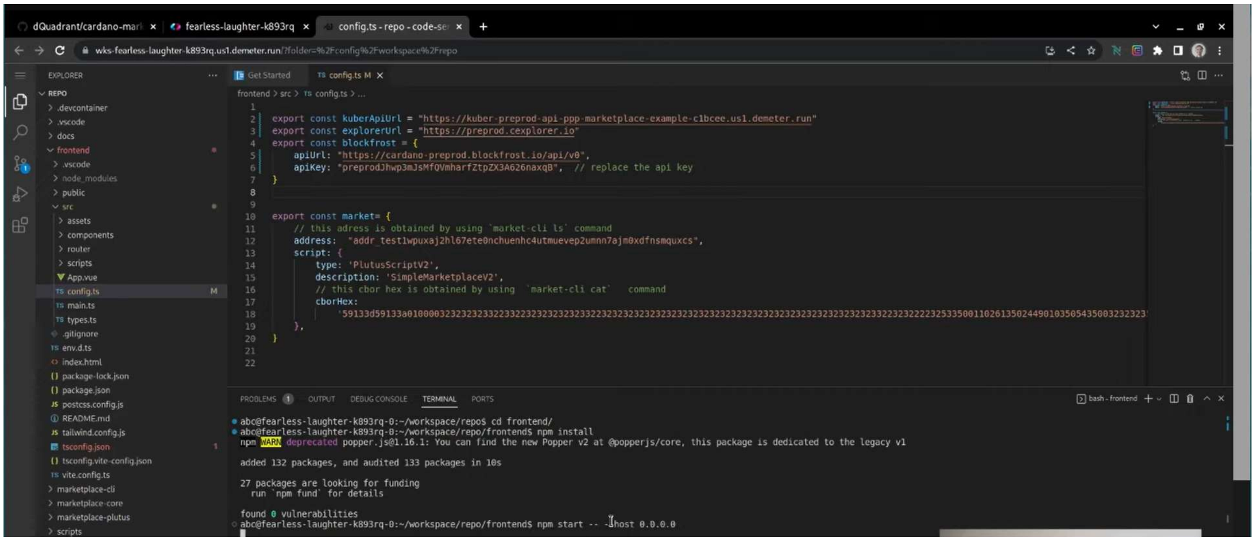


Then we got to the menu **Terminal -> Open Terminal** and in the open terminal we cd into the frontend directory and install the frontend with the npm command:

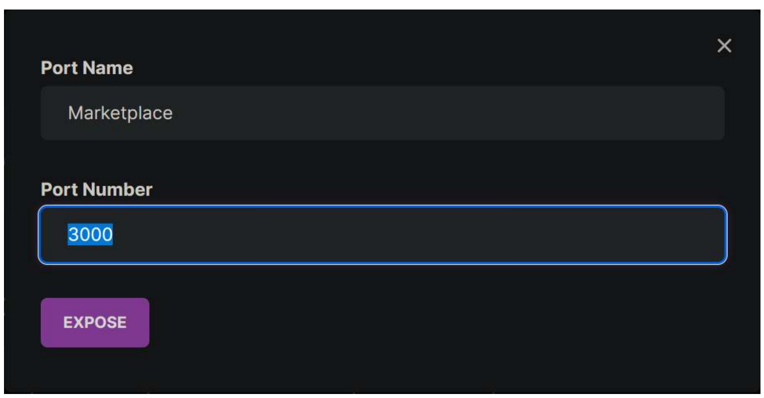
```
~workspace/repo/frontend> npm install
```

Once the installation is done, we are going to run the front-end by starting the web server:

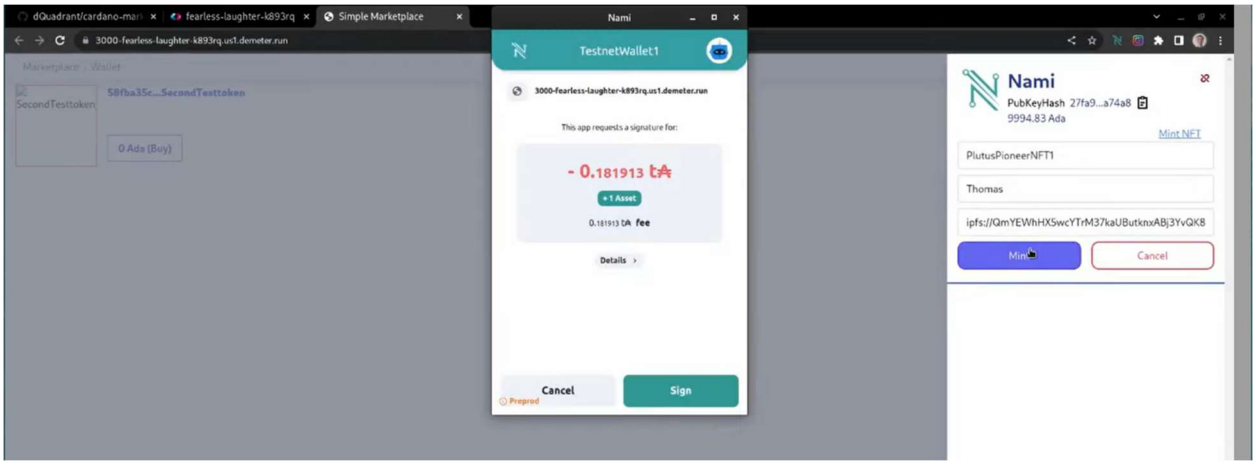
```
~workspace/repo/frontend> npm start -- --host 0.0.0.0
```



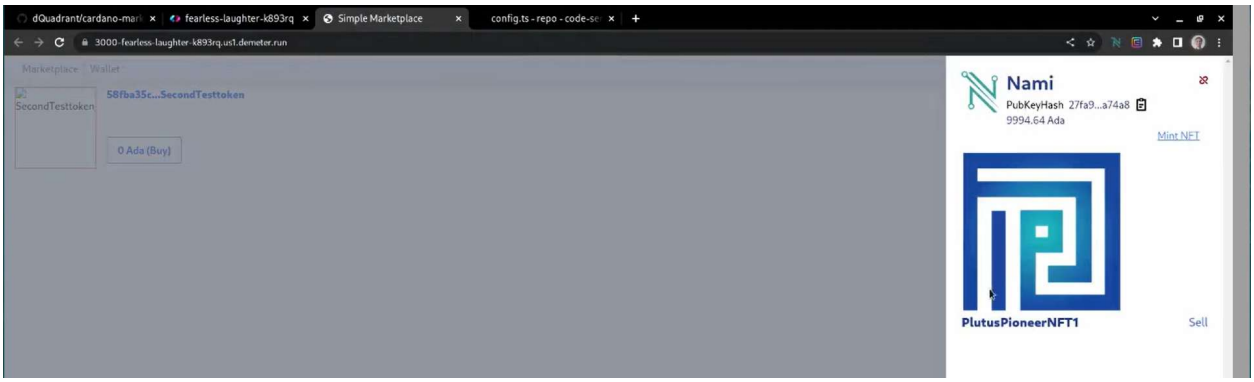
This will open a local host at port 3000 but since you are using demeter this is not reachable from your web browser. You need to go back to your settings on the workspace and in the section **Exposed ports** click on **Expose port**. Under name you can type in **Marketplace** and under port type in 3000 and click **Expose**.



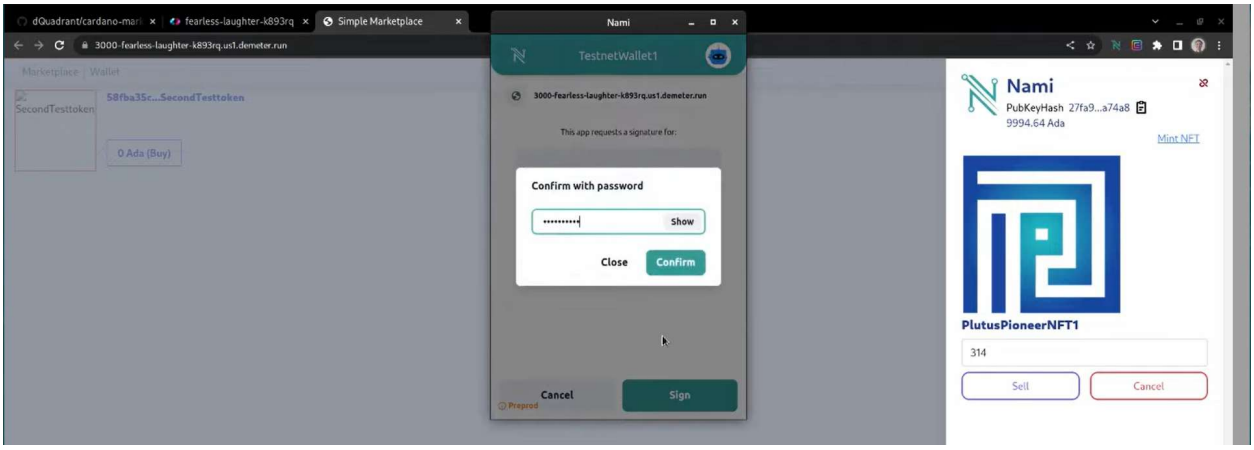
Once this is done you will see a link that you can use to access your marketplace webpage that you have created. You will see on the marketplace one token that is listed for sale. In the upper tab of your marketplace webpage, you can also choose to connect a wallet that you have installed in your web browser. Examples of such wallets are Nami and Eternl that have to be connected to the preproduction network. From the Nami wallet you can click on the MintNFT button and input your token name, artist name and image URL. It will prompt you to sign a transaction. You will of-course need some test tokens in your wallet address to mind the NFT.



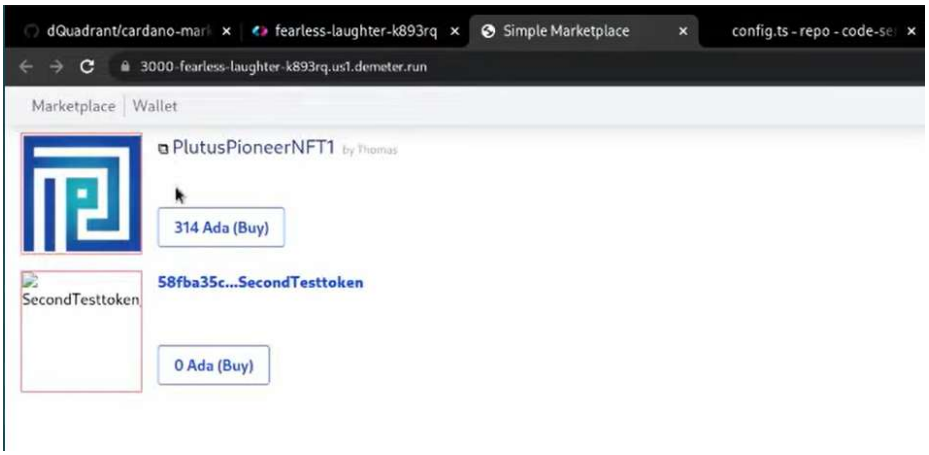
To receive test tokens, you can go to the Cardano testnet faucet and request some tokens for the Preprod testnet: https://docs.cardano.org/cardano-testnet/tools/faucet. After you minted your NFT you want to sell it and connect to the Nami wallet again and click on your NFT that will be shown in your wallet. There will be a Sell highlighted text that you can click.



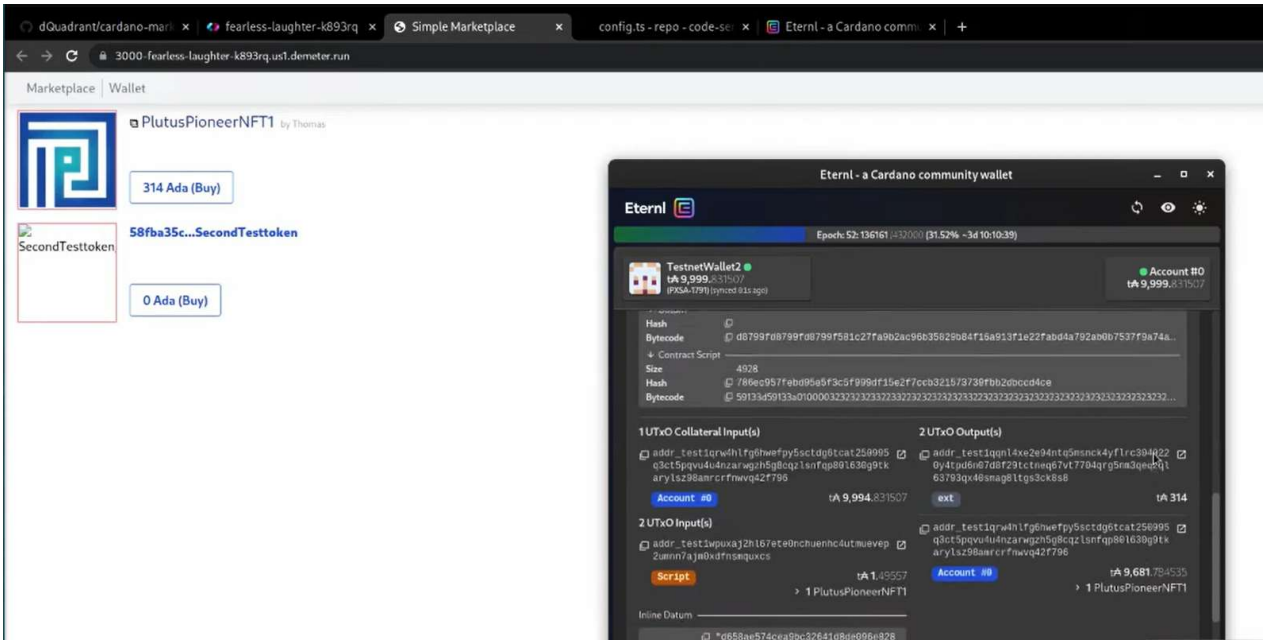
You can then input for how many tokens you want to sell it and click on the **Sell** button and sign again the transaction that will put the NFT on the marketplace.



You can now see the NFT on your marketplace and also the price that you have set is visible.



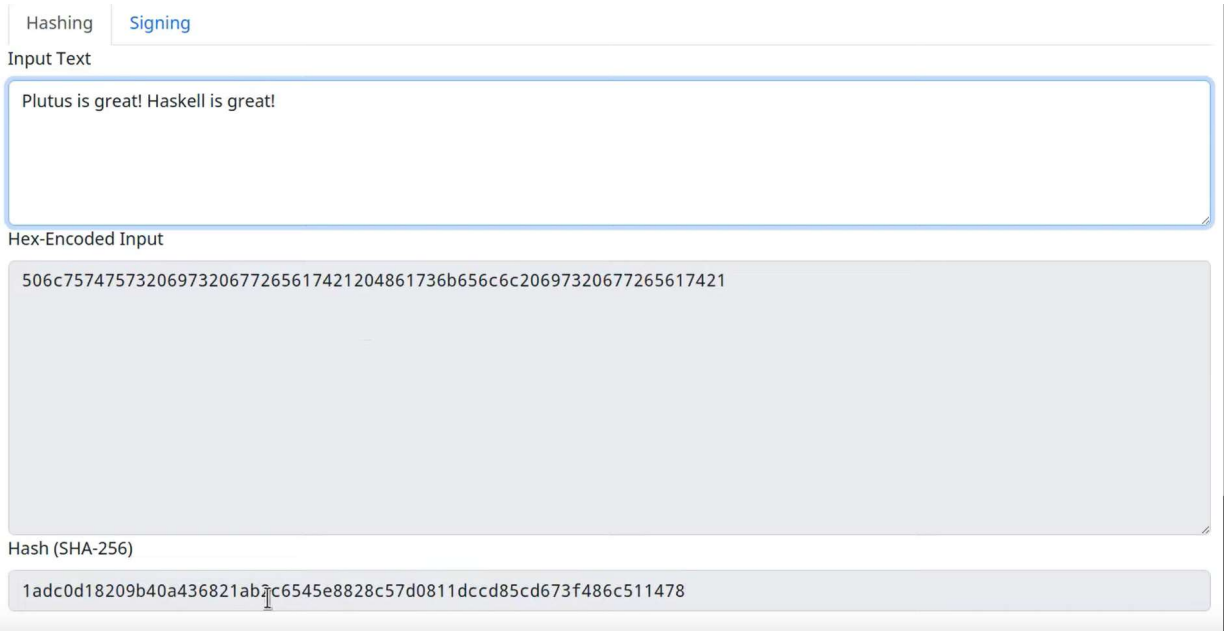
If you want, you can now buy this NFT with another wallet that should be also connected to the pre-production network. You simply have to click the **Buy** button and choose your wallet. If we do this with the Eternl wallet a window will pop-up and display the transaction details.



We sign the transaction, and after it is processed, we can see that we get the NFT in our Eternl wallet and the ada amount for which we were selling the NFT will be sent to our Nami wallet.

1.3 Hashing and digital signatures

These are two cryptographic concepts that play an important role in Cardano and blockchain technology. Hashing takes as input a byte-string. For that we usually use a hexadecimal text that we get when we convert normal text into hexadecimal format. A hashing algorithm takes this hex byte-string and produces another hex byte-string as output as illustrated below.



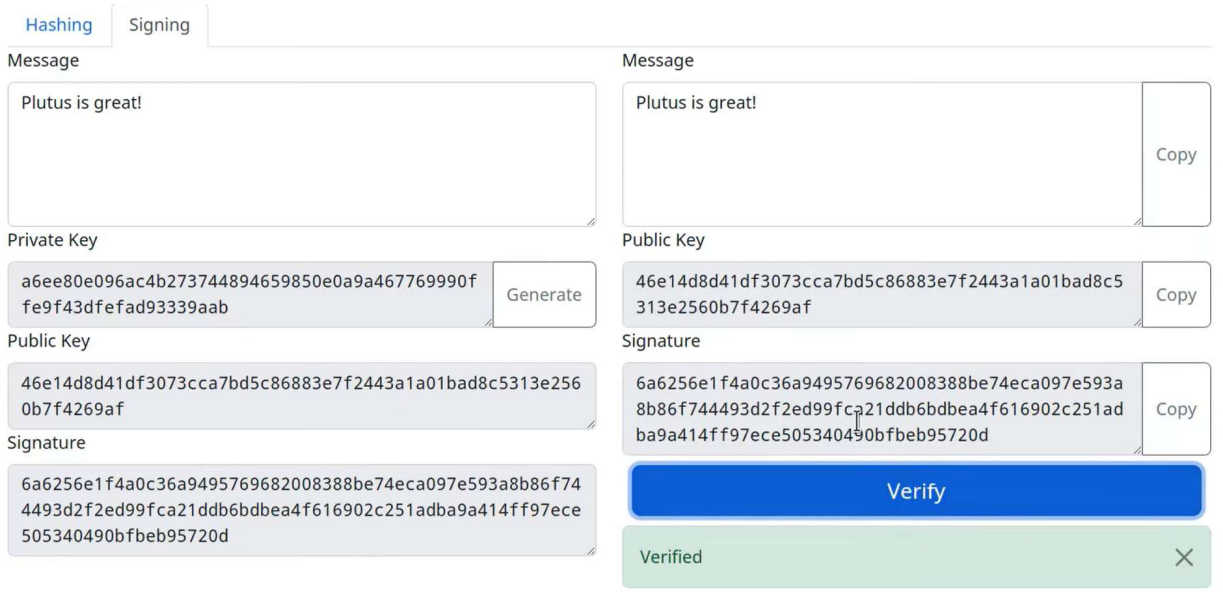
Note in the image above the output byte-string always has the same length no matter the input length. If we use the SHA-256 hashing algorithm the output will always be 256-bits (32-bytes) long. Because one byte can be represented with 2 hexadecimal characters, the SHA-256 hash is 64 characters long. Hexadecimal characters can be numbers from 0 to 9 or letters from A to F.

The second thing to mention is that hashing is very unpredictable. If you make a small change to a long text message the hash of the message will change entirely. Hashing algorithms are designed in a way that makes it practically impossible to reverse engineer the input from a hash value. That is a property on which Cardano and other blockchain technology critically rely on. In proof-of-work blockchains miners are solving little hashing puzzles where they try to produce inputs that have a certain number of zeros in beginning of the resulting hash.

There is an infinite number of input text, but only a finite number of hashes, since their length is fixed which is 2 to the power of 256 for the SHA-256 algorithm. Because of this, it is in theory possible to find two different inputs that would produce the same hash values but such a hash collision has not been found for SHA-256 in practice yet.

Now let us talk about digital signatures. Their point is that you can digitally sign documents. In the background the content is again transformed to a byte-string. Let us take for example a short message instead of an entire document, that is just a long collection of text. To sign a message, we need a key pair consisting of a private key that only the signer knows and a public key that also the verifier knows. The private key can be generated by a program and the public key is then generated also by a program taking the private key into account. But you cannot generate a private key just by knowing the public key.

As there are many hashing algorithms there are many digital signing algorithms. The idea is now that after signing the message with our private key we get an output string that is similar to a hash. We call it the signature of the document we signed. Another person who gets our message, our public key and the signature can check with a program that the signature of the message he received belongs to the public key. Public keys are publicly known. Someone sending a signed email can add at the bottom of the email below his name, also his public key.

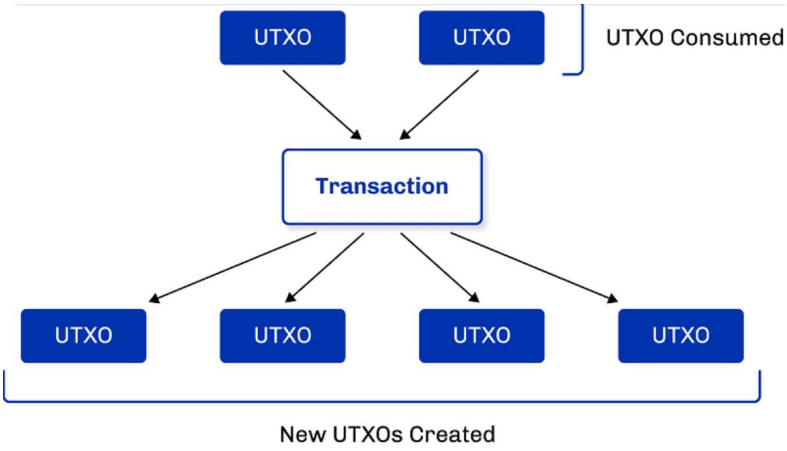


A thing to point out is that a signature is valid only for a specific message and the specific private key. If you change any of them then the verification will fail. This technology is used in Cardano and other blockchains. It is digital signatures that keep your money safe. For a transaction to spend your money you must digitally sign it with your private key. The cryptographic features of digital signatures ensure that it is secure and that only you as the owner of the private key can spend your money.

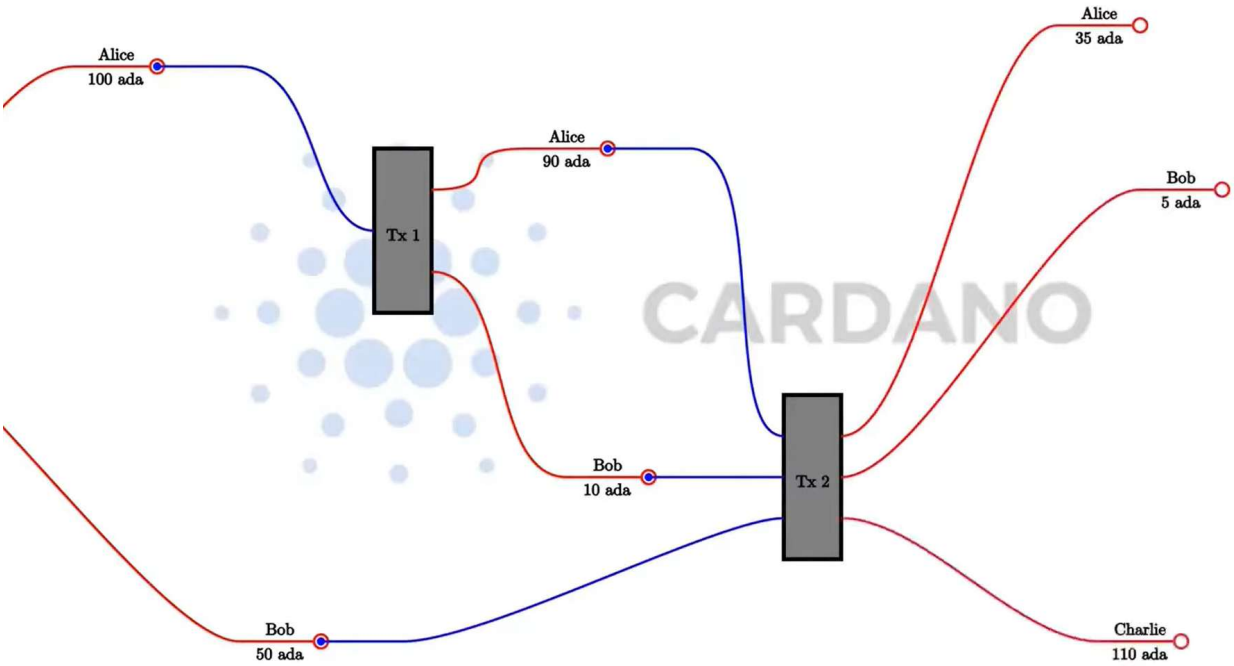
1.4 The EUTxO model

One of the most important things you need to understand in order to write Plutus smart contracts is the accounting model that Cardano uses. It is the so-called EUTXO model, which is an abbreviation for extended UTXO model that is a variant of the Unspent Transaction Output (UTXO) model used by Bitcoin. Transactions consume unspent outputs (UTXOs) that were produced by previous transactions and produce new outputs, which can be used as inputs to later transactions. Unspent outputs are the liquid funds on the blockchain. Users do not have individual accounts, but rather have a software wallet on a smartphone or PC, which manages UTXOs on the blockchain. It can initiate transactions involving UTXOs owned by the user. Every

core node on the blockchain maintains a record of all the currently unspent outputs, the UTXO set. When outputs are spent, they are removed from the UTXO set.



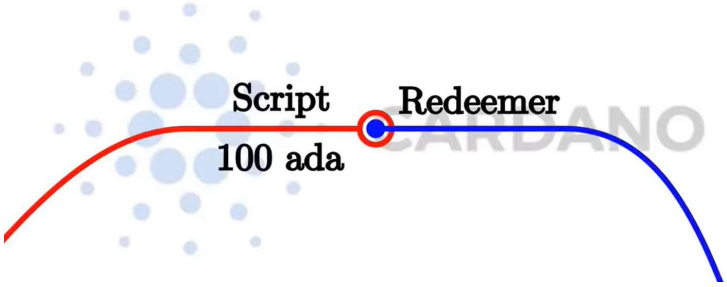
There are models other than UTXO. Ethereum, for example, uses an account-based model, which is what a normal bank uses. There everybody has an account, and each account has a balance. If you transfer money from one account to another, then the balances get updated accordingly. But in the UTXO model, the input is always the entire balance of an UTXO, and the outputs are newly created UTXOs from which one of them could belong to the user that provided his UTXO as input and would represent his change amount. Let us say Alice has 100 ada and wants to send Bob 10 ada, that already has 50 ada. After that she wants to send to Charlie 55 ada and in the same transaction also Bob wants to send Charlie 55 ada.



In the picture above you can see that the inputs are always entire UTXOs, and outputs are newly created UTXOs that can belong to different participants, depending on the transaction. As soon as Alice spends her 100 ada UTXO by using it as input in a transaction, it is no longer a UTXO and can never be used again. When Alice spends her UTXO containing 100 ada she can create outputs for a transaction. Because she wants to pay 10 ada to Bob one output will be 10 ada that will belong to Bob. And then she wants her change back so she creates a second output of 90 ada that belongs to herself. This is how you get your change back when you consume complete UTXOs. In a transaction the sum of the input values must always equal the sum of the output values. This is strictly speaking not true. There are two exceptions. The first exception are transaction fees. On the real blockchain for each transaction you have to pay fees so that means that the sum of input values has to be slightly higher than the sum of output values to accommodate for the fees. The second exception are native tokens that we have on Cardano. So, it is possible for transactions to create new tokens in which case the outputs will be higher than the inputs or to burn tokens in which case the inputs will be higher than the outputs. We will look at minting and burning of native tokens in chapter 5. For our transaction we see that the effect of the transaction is to consume unspent transaction outputs and to produce new ones. In this first example of a transaction, one UTXO has been consumed and two new UTXOs were created. It is important to note that the only thing that happens on a UTXO blockchain when a new transaction is added to the blockchain is that some former UTXOs become spent and new UTXOs appear. In particular nothing is ever changed. No value or any other data associated with the transaction output has ever changed. The only thing that changes with a new transaction is that some of the formerly UTXO disappear and others are created. But the outputs themselves never change. The only thing that changes is whether they are unspent or not. The second transaction on the previous image is when Alice and Bob together want to pay 55 ada each to Charlie. So, they create a transaction together and as input Alice has not much choice since she has only one UTXO so she uses that one. And Bob also does not have a choice because neither of his two UTXOs is large enough to cover 55 ada. So, Bob has to use both his UTXOs as input. This time we need three outputs. One is the 55 plus 55 ada that equals 110 ada that belong to Charlie. And then two change outputs; one for Alice that gets 35 ada change and one for Bob that gets 5 ada change.

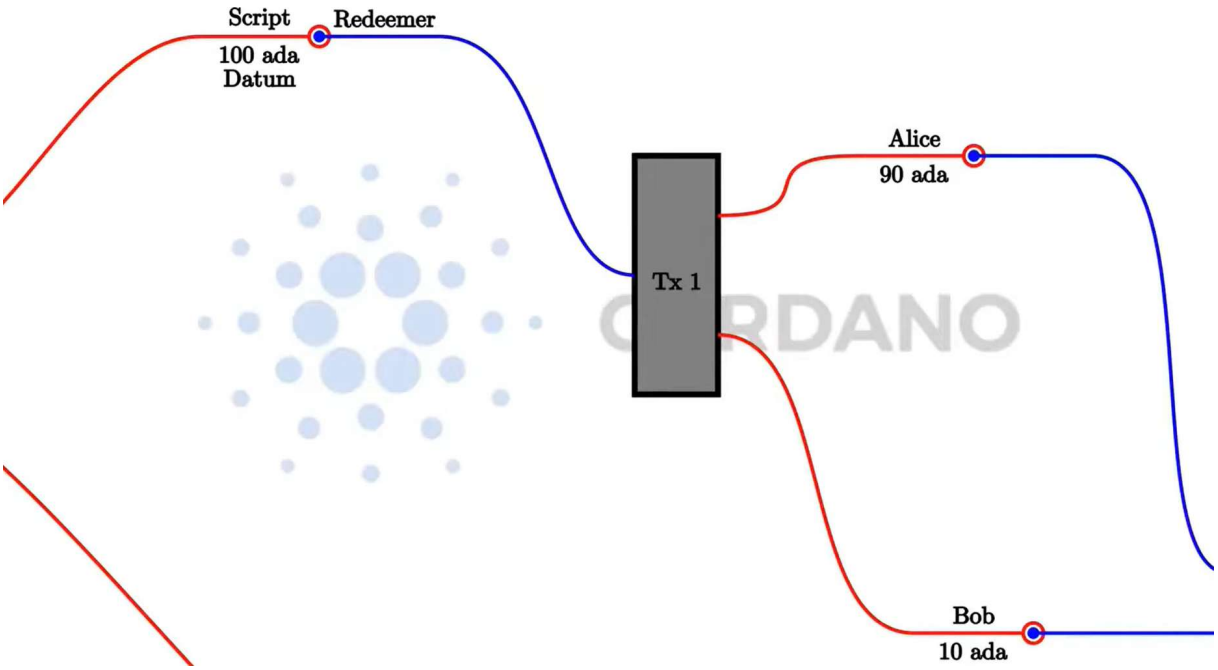
Let us now talk about under which conditions a transaction can spend a UTXO. The way it works is by adding signatures to transactions. So, for the first example where the transaction consumes a UTXO from Alice, Alice's signature has to be added to the transaction. And for the second transaction where the inputs belong to both Alice and Bob, both of them have to sign this transaction. We note that this cannot be done in a wallet as Daedalus. You would need a command line tool as the cardano-cli to create such a complex transaction. Everything we have

explained so far is about the UTXO model, not the extended UTXO model. The extended part comes in when we talk about smart contracts. In order to understand that let us look at one consumption of a UTXO. The validation that decides whether the transaction that an input belongs to is allowed to consume that UTXO in the simple UTXO model relies on digital signatures. The idea of the extended UTXO model is to make this more general. So, instead of having just one condition that the appropriate signature is present in the transaction, we replace this by arbitrary logic and this is where Plutus comes in. The UTXO output is associated with an address which is represented by a public key hash. We call them public key addresses. The ada amount and optionally native tokens of a public key address is the sum of ada and native tokens from all UTXOs belonging to this address. A transaction must be signed by the owner of the private key corresponding to the address that defines the input UTXO. Think of an address as a lock that can only be unlocked by the right key which is the correct signature. The user which controls a private key of an address can create transactions that spend the ada or native tokens sitting at the UTXOs of this address. For every transaction there is also a fee to pay denominated in ada for the Cardano blockchain. In the extended UTXO model we also have more general addresses that are not based on public keys or hashes of public keys but instead contain arbitrary logic that can decide under which condition this specific UTXO can be spent by a transaction. We call them script addresses. So, instead of an address belonging to a public key like Alice's public key in this example there will be an arbitrary script containing arbitrary logic that defines under which conditions the UTXOs sitting at this address can be spent. The address is unlocked by an arbitrary piece of data that we call the *redeemer*, which in the conventional UTXO model would be a private key. The next question is what do we mean by arbitrary logic. It is important to consider what information the script has. There are several options. One option is that the script only sees the redeemer and decides if the transaction can spend a certain UTXO as illustrated in the image below.



That is what Bitcoin does. In Bitcoin there are smart contracts that are very simple and are called Bitcoin script. There is a script on the UTXO site and a redeemer on the input site. The script gets the redeemer and can use it to decide whether it is ok to consume the UTXO or not. The other option is to give more information to the script. Ethereum uses a different concept

where the script can see everything, the whole state of the blockchain. That is the opposite extreme of Bitcoin, where the script has very little context. That enables Ethereum scripts to be more powerful and they can do basically anything but it also comes with problems. Because the scripts there are so powerful it is also very difficult to predict what a given script will do. That opens the door to all sorts of security issues because it is very difficult to predict for the developers of an Ethereum smart contract what can possibly happen because there are so many possibilities. What Cardano does is something in the middle. It does not offer such a restricted view as Bitcoin but also not such a global view as Ethereum and instead chooses a middle way. The script cannot see the state of the whole blockchain but it can see the whole transaction that is being validated. It can see all the inputs of the transaction, the redeemer, all the outputs of the transaction and the transaction itself that carries some data with it. The Plutus script can use that information to decide whether it is ok to consume an output. In our previous example there is only one input but if there would be more, the script could see those as well. There is one last ingredient that Plutus script needs in order to be as powerful and expressive as Ethereum scripts and that is the so-called datum. It is an arbitrary piece of data that can be associated with a UTXO beside the amount of ada and native tokens sitting at the UTXO. So, the datum together with the redeemer, all inputs and outputs and the transaction context are the input information for a script. You can see this in the image below.



We can mathematically prove that every logic you can express in Ethereum you can express in this extended UTXO model that Cardano uses. But it has a lot of important advantages compared to the Ethereum model. In Plutus it is possible to check whether a transaction will

validate in your wallet before you send it to the chain. If it is valid, you can be sure it will be processed on the network, given the condition that all the UTXO inputs are still present at processing time. If they are not the transaction will simply fail and no fee will be charged to the user that sent the transaction. If they are, you can be sure that the transaction will be validated and will have the effect that you predicted when you ran it in your wallet. And this is definitely not the case in Ethereum. There in the time between you constructing a transaction and it being incorporated into the blockchain a lot of stuff can happen concurrently. And that can have unpredictable effects on what can happen when your script executes. So, that means in Ethereum it is always possible that you have to pay gas fees for a transaction although it can eventually fail with an error. And that is guaranteed not to happen in Cardano. In addition to that it is also easier to analyze a Plutus script and to check or even prove that it is secure. You do not have to consider the whole state of the blockchain. You can concentrate on this context that just consists of the spending transaction. You have a much more limited scope and that makes it easier to understand what a script is actually doing and what could possibly go wrong.

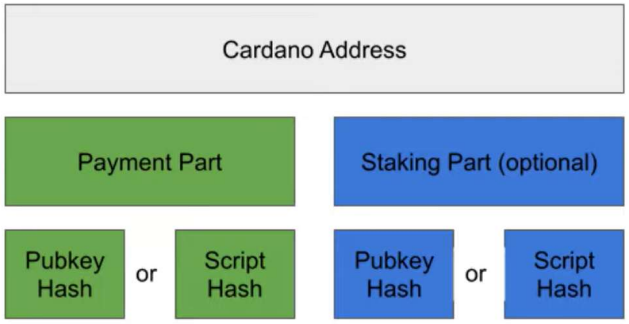
We call the script that validates a transaction the validator. The script address is defined as a hash of the validator code written in Plutus core language. The script addresses are publicly known. We will talk about Plutus core in the next sub-chapter. As said the validator script takes the datum, the redeemer and the transaction context as input information. The input for the datum is collected from each UTXO individually that is sitting at a script address. That means if there are multiple UTXOs specified to be consumed in the transaction, the validation logic is checked for each of them separately. So, in each validation there is only one datum as input coming from one UTXO. A transaction in the EUTXO model can be classified as a producing transaction that produces a script UTXOs or as a spending transaction that spends a script UTXOs. In general, every transaction except the genesis transaction takes at least one UTXO as input and produces at least one UTXO as output. So, we use the terms spending and producing only when we talk about script addresses. If we first send ada to a script address, we call it a producing transaction. After that if we try to collect that ada from the script address, we call this transaction a spending transaction. The producing transaction must include the datum or the hash of the datum that will be attached to the UTXO created at script address. If it includes the hash of the datum, only a person that knows the datum by some other means not by looking at the blockchain is able to ever spend such an UTXO. The spending transaction is responsible for providing the redeemer, the transaction context and optionally also the datum. It also must provide the validator script. In contrast if we construct a transaction that spends funds from a public key address and not a script address, we only need to add a signature with the private key that belongs to the public key.

Let us look now in more detail how the datum or its hash gets provided by the producing transaction. As explained before the datum is part of the output so it is the responsibility of the producing transaction to supply it. However, this is conceptually correct, technically it is a bit more complicated. There are three options to do this. The first option is that the output only contains the hash of the datum. Then the consuming transaction which wants to spend this script output must contain the actual datum. This option is the cheapest for the producing transaction because a hash is small and the transaction costs less fees. But it makes it difficult to consume such a script output because somehow the creator of the consuming transaction must know the actual value of the datum which in this case is not saved on the blockchain. The second option is that beside the datum hash we include the actual datum in the transaction body, which means that people can look up the datum on the blockchain and then include it in their spending transaction. Such a producing transaction costs more than the previous one because it stores the datum in the transaction. However, it's not so convenient because the Cardano node only sees the statement value during validation but then forgets its value. So, for somebody else to later discover the datum on the blockchain another tool is needed like a chain indexer or dbsync. Since the Vasil hard fork there is also the third option where the output itself contains the datum. In this case we call it an inline datum and then it does not have to provide it in the body of the producing transaction. Additionally, the consuming transaction does not have to provide the datum and it becomes smaller and cheaper. From the point of view of the consumer this third option is by far the most convenient. In this case no additional tool is needed. The Cardano node or the Cardano CLI can give the value of such inline datums. In this book we will mostly use the third option. However, for DApps in production there may be reasons to prefer one of the first two options.

Producer Provides		Consumer Provides
In Output	In Tx Body	
Datum Hash	-	Datum
Datum Hash	Datum	Datum
(Inline) Datum	-	-

Before we can talk about how scripts are provided to the blockchain let us briefly look at Cardano addresses. A Cardano address has two parts. The payment part that is always present and optionally the staking part. The payment part is responsible for deciding under which conditions an UTXO sitting at such an address can be spent. It is defined either by the hash of a public key or the hash of a Plutus script. In Cardano we also call the private key the signing key and the public key the verification key. If it contains the public key hash UTXOs sitting at such an address can only be spent if the transaction is signed with the corresponding private key

(signing key). But if it contains the script hash, the corresponding script is executed during validation and its output determines if one or more UTXOs sitting at the script address can be spent. The optional staking part of an address decides who is entitled to staking rewards and is in control of delegation. Also, here you have two options. If the staking part is specified with a public key hash, then the owner of the corresponding private key is entitled to staking rewards. If it is a script hash, then the corresponding Plutus script is executed for transactions that try to withdraw staking rewards for example. In this book we will mostly concentrate on the payment part so most of the scripts we will look at will be used as validators that are relevant for the payment part. But we will also look at how Plutus can be used in staking related questions.



Finally let us look at how the validation script can be referenced by the spending transaction. So, when a transaction wants to produce an output locked by a script it will send that output to an address whose payment part is the hash of that script. However, if somebody wants to consume such an output the Cardano node needs to validate and in order to do that it needs to know the actual script to execute and run it and see whether validation succeeds. There are two ways for this to happen. The traditional way which was the only possibility before the Vasil hard-fork, was for the consuming transaction to contain the script. Some scripts are used over and over again maybe a thousand of times. In the past the same script was included in a lot of transactions. So, that was a lot of redundant data that was repeated over and over again. To address this issue there is now since the Vasil hard-fork another way, so-called reference scripts. We said before that a UTXO can carry value and a datum. Since Vasil it can optionally also carry a script. So, when we create a UTXO we can optionally attach an arbitrary script to it. Usually, you do not want this UTXO to be consumed, so you can send it to a script address where the validation logic fails no matter the inputs. And once that UTXO is on the blockchain other transactions in the future when they want to unlock the UTXO locked by this script can reference this script. So, instead of including the script in the transaction they can just say I want to use the script sitting at that UTXO. That makes such consuming transactions much smaller and avoids the redundancy of repeating the same script over and over again. In both cases whether the script is included in the transaction or referenced the Cardano node will of course make sure that it is the right script. Right in the sense when you hash it you get the right

hash value given by the payment part of the address. Finally, the redeemer is always supplied by the consuming transaction. It has to be because the producing transaction has no idea what redeemer will actually be used. That is solely the responsibility of the consuming transaction. So, whenever a transaction wants to consume a script output it has to include the redeemer in the corresponding input. Also, there are cases where transactions consume and produce new UTXOs for the same script. This is common practice when you want to "update" the datum sitting at an UTXO that you have to spend and then create a new one with the updated datum.

This was the explanation of the EUTXO model and it is not tied to a specific programming language. What we have in Cardano is Plutus which is based on Haskell but in principle you could use the same EUTXO model with a different programming language. There are other blockchains also using EUTXO which are not using plutus, as the Ergo blockchain for example. Cardano uses plutus, or to be more precise Plutus script which you can think of as the assembly language of the Cardano blockchain. The normal way to create a Plutus script is via so-called Plutus TX which is basically Haskell and that is the method we will use in this book. So, we write Haskell programs that get then compiled to Plutus script. There are alternatives however and we will briefly look at some of those in the last lecture of this course.

1.5 Plutus code

The code for Plutus smart contracts can be separated into two parts. First is the "on-chain" code, which consists of the validator function and some additional declarations and variables as the script address. This code gets compiled to Plutus Core language or also called Plutus script. It runs on the Cardano blockchain and once submitted it cannot be changed.

From the official documentation [1] we get the following description for Plutus Core:

Plutus core is the scripting language used by Cardano to implement the EUTXO model. It is a simple, functional language similar to Haskell, and a large subset of Haskell can be used to write Plutus Core scripts. As a smart contract author, you don't write any Plutus Core; rather, all Plutus Core scripts are generated by a Haskell compiler plugin called Plutus Tx.

Community variants exist where you can write the on-chain code in another programming language and then it is compiled to Plutus core. We will talk about them at the end of this book.

The "off-chain" code can also be written in Haskell, just like the on-chain code. Unlike Ethereum where the on-chain code is written in Solidity, but the off-chain code is written in JavaScript. That way, the business logic only needs to be written once. This logic can then be used in the validator script and in the code that builds the transactions that run the validator script.

The off-chain code basically constructs the transaction and submits it to the blockchain. Since both the on-chain and off-chain code can be written in Haskell they can reside in one Haskell file while testing your code, which allows them to share code between them. Another option is also to construct the off-chain transaction with command line tools instead of compiling Haskell code. Cardano provides for this the *cardano-cli* command line tool. There are also community alternatives. Those tools will also be presented in later chapters of this book. In the 4th iteration of the Plutus pioneer program, we will not write any off-chain code in Haskell but use only the cardano-cli and community languages for off-chain code. If you want to see examples of off-chain code written in Haskell, you can look at the videos of the 3rd PPP iteration or also read the accompanying book. However, some of the code examples may not work anymore since the Plutus language got upgraded in the meantime. In the 3rd iteration we used nix to set-up the development environment for using the plutus-apps library. Now it is possible to fetch a pre-prepared nix environment which you can also use to build all the projects from the 4th iteration. You will need to install nix on your system. Instructions and a nix shell is provided by the plinth-template repository. After you have installed nix, you can start the shell by running:

```
nix develop
```

2 Simple validation scripts

In this chapter we will focus on the on-chain part of smart contracts. We outlined earlier that a validation script takes in three parameters: the datum, the redeemer and the transaction context. The transaction context provides information about the submitted transaction, that is the inputs and outputs. Depending on the type of these three parameters we have two possible implementations of Plutus validation scripts. In the low-level implementation of Plutus validation scripts these 3 parameters are represented with the same data type. In the high-level implementation you can use custom Haskell data types for datum and redeemer and a predefined type for the transaction context. You can use both implementations in your smart-contract code. The difference between them is code performance which is better for the low-level implementation data types. We also call the low-level validation scripts untyped validation scripts, and the high-level scripts typed validation scripts.

2.1 Low-level untyped validation scripts

The data type for the low-level implementation is called *BuiltinData*. It contains two conversion functions *builtinDataToData* and *dataToBuiltinData*, that can convert back and forth to the *Data* type. The Data type has its constructor exposed and has the following definition:

Constructors

```
Constr Integer [Data]
Map [(Data, Data)]
List [Data]
I Integer
B ByteString
```

Figure 1 - Data type

If you want to query and look at Plutus data types on your own, you can run the command `serve-docs` from your container terminal in VSCode. It will start up a server that will host the haddock documentation. It can be accessed at http://localhost:8000. In case you have issues in Firefox, use the Chrome or Edge web browsers instead. If you are running this server inside the Demeter platform you also have to open up port 8000 in the Demeter workspace section **Exposed ports**. The procedure for this was already described in chapter 1.2. Once you have the haddock documentation you can use the key combination CTRL+S to open the search bar and type in your desired function or type in the search bar. When you for instance search for the *BuiltinData* type you get several search results. At the top of each section the name of

the module that contains this function is stated. For us the module *Plutus.V2.Ledger.Api* is important because it contains the updates of the Vasil hard fork. Our scripts in this iteration of the Pioneer program will all be Plutus V2 scripts. The HTTP server reads the haddock documentation from our GitHub repository, where the documentation is located.

The haddock documentation is also hosted online where users can select the desired Plutus version. By default, the documentation is displayed for the latest Plutus version.

If you want to build the documentation yourself, you can clone the plutus-apps repository, checkout the latest Tag version and run the nix build command, as follows:

```
$ git clone https://github.com/input-output-hk/plutus-apps.git
$ cd plutus-apps
$ git checkout v1.1.0
$ nix-build default.nix -A docs.site
```

To find the latest tag go to the GitHub page, click on the branch drop-down box and click the Tag tab. You should ignore the tags that start with a date. Look only at tags that contain numbers as v1.1.0. The documentation will be built inside the plutus-apps/result folder. The build may take a while. You can then start a webserver with the command:

```
$ python3 -m http.server -d /path/to/haddock/
```

Usually, we can also explore types in the Haskell REPL. For instance if we type in the REPL `:doc Maybe` then we get the documentation for the Maybe type. But we cannot do this for Plutus types in a normal GHCi REPL. The reason for that is when we write Plutus code and compile it, first the GHC compiler pipeline compiles the code to an intermediate language called Haskell core. And then the Plutus compiler takes this Haskell core language and compiles it to Plutus script language. So GHC is unaware of Plutus types. The Plutus libraries are also not hosted on Hackage, which means the only way to query Plutus types is from a REPL build with a cabal file that imports Plutus libraries or with the haddock documentation.

If we want to play and create some variables of the above-mentioned Data type, we can do this by going to Week02 folder in the VSCode terminal and type in the following commands:

```
/workspaces/plutus-pioneer-program# cd code/Week02/
/workspaces/plutus-pioneer-program/code/Week02# cabal repl
Prelude Gift> :set -XOverloadedStrings
Prelude Gift> import Plutus.V2.Ledger.Api
Prelude Plutus.V2.Ledger.Api Gift> Map [(I 42, B "Haskell"), (List [I 0], I 1000)]
Map [(I 42,B "Haskell"),(List [I 0],I 1000)]
```

The reason why there is no collision from Haskell's Map object is that we do not make use of the standard Prelude module when we load out REPL. Plutus uses a custom prelude where all of the functions are defined strict instead of lazy.

Let us look at the code from the *Gift.hs* file in the week02 folder.

```
{-# LANGUAGE DataKinds           #-}
{-# LANGUAGE ImportQualifiedPost #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE TemplateHaskell     #-}

module Gift where

import qualified Plutus.V2.Ledger.Api as PlutusV2
import           PlutusTx             (BuiltinData, compile)
import           Prelude              (IO)
import           Utilities            (writeValidatorToFile)

------------------------------------------------------------------------------
--------------------------- ON-CHAIN CODE / VALIDATOR ------------------------

-- This validator always succeeds
--                     Datum         Redeemer     ScriptContext
mkGiftValidator :: BuiltinData -> BuiltinData -> BuiltinData -> ()
mkGiftValidator _ _ _ = ()
{-# INLINABLE mkGiftValidator #-}

validator :: PlutusV2.Validator
validator = PlutusV2.mkValidatorScript $$(PlutusTx.compile
                                          [|| mkGiftValidator ||])

------------------------------------------------------------------------------
------------------------------- HELPER FUNCTIONS -----------------------------

saveVal :: IO ()
saveVal = writeValidatorToFile "./assets/gift.plutus" validator
```

There are various language pragmas added in the beginning. One that is worth noticing is the `NoImplicitPrelude` extension which allows you to use a custom prelude. To get a list of all functions of the custom prelude you can search the Plutus haddock documentation for the keyword Prelude. You will see that the module which defined Plutus Prelude functions is called *PlutusTx.Prelude*. Next, we make some import statements of the types and functions we need. We can also import types and functions from Haskell standard Prelude and we do this for the IO type. Then we define our validator function that we call `mkGiftValidator`. The validator takes

in the datum, the redeemer and the script context which are all of type `BuiltinData`. It returns an empty tuple, which we call unit. It is a type that has one value and could be compared to the Void type that is used in other programming languages such as Python, Java or C++. We see in the body of the function we ignore all of the input values and always return unit.

We said earlier the validator validates a given transaction and for that reason you would expect a Bool value as return type. This is the case if we use the high-level implementation. For the low-level we have instead the unit, which is returned if the validation passes. If it does not pass an error is raised instead. We will cover such a case in our next example. Now we can also understand why this module is called Gift, because anyone will be able to claim funds from this address since the validation will always pass.

Next, we define the *`validator`* parameter which is of type `PlutusV2.Validator`. To get it we need to compile the validator function to Plutus Core script. The `PlutusTx.compile` function takes as input a syntax tree of a function which we can get if we put the oxford brackets `[|| mkGiftValidator ||]` around our validator function. The compile function produces another syntax tree that is written in the Plutus core language. Then the `$$` symbol called splice, takes a syntax tree and splices it back to Haskell source code, which is what we need to input to our `mkValidatorScript` function. The splice operator and the oxford brackets can be used because we added the `TemplateHaskell` language pragma.

An important thing to mention is that normally in the oxford brackets you cannot reference anything that is defined outside the brackets. This can become a problem when validator functions are long expressions or you call some library functions in their body. A workaround to that is if we make the function inlinable. We can do this by adding the `INLINABLE` pragma statement after the function definition and then the GHC compiler will replace the function call in the oxford brackets with the actual function body.

In REPL we can import the corresponding module of the Validator type and check its definition.

```
Prelude Gift> import Plutus.V2.Ledger.Api
Prelude Plutus.V2.Ledger.Api Gift> :i Validator
type Validator :: *
newtype Validator = Validator {getValidator :: Script}
        -- Defined in `Plutus.V1.Ledger.Scripts'
```

We see it's just a wrapper around a script type. We can also check that type.

```
Prelude Plutus.V2.Ledger.Api Gift> :i Script
type Script :: *
newtype Script
  = Plutus.V1.Ledger.Scripts.Script {Plutus.V1.Ledger.Scripts.unScript ::
      plutus-core-1.0.0.1:UntypedPlutusCore.Core.Type.Program
      plutus-core-1.0.0.1:PlutusCore.DeBruijn.Internal.DeBruijn
```

```
        plutus-core-1.0.0.1:PlutusCore.Default.Universe.DefaultUni
        plutus-core-1.0.0.1:PlutusCore.Default.Builtins.DefaultFun
        ()}
```

For Script we see it's something of type Program. If use the *unScript* function on the Script variable inside the validator we get the following definition:

```
Prelude Gift> import Plutus.V1.Ledger.Scripts
Prelude Plutus.V1.Ledger.Scripts Gift> unScript $ getValidator validator
Program {_progAnn = (), _progVer = Version () 1 0 0, _progTerm = LamAbs () (DeBruijn
{dbnIndex = 0}) (LamAbs () (DeBruijn {dbnIndex = 0}) (LamAbs () (DeBruijn {dbnIndex =
0}) (Delay () (LamAbs () (DeBruijn {dbnIndex = 0}) (Var () (DeBruijn {dbnIndex =
1})))))))}
```

We also needed to import the `Plutus.V1.Ledger.Scripts` module so we could use the *unScript* record function. The Plutus core expression we get is a version of the lambda calculus. If we want to interact with the smart contract that is defined with our validator function, we need to have the validator variable in serialized form. To be able to do this we use the function `writeValidatorToFile` that is not a standard Plutus function. It was defined for the purpose of this course and can be found in the *Week02/Utilities/src/Utilities/Serialize.hs* file. It uses a bunch of Haskell library functions that do some ByteString manipulation and serialization. This function takes a file path and a Plutus validator, then serializes the validator and writes it to a file. We write the validator to the file *./assests/gift.plutus*. To create this file, we can simply run the *saveVal* function from the REPL and then we can look at the Plutus file.

```
{
    "type": "PlutusScriptV2",
    "description": "",
    "cborHex": "49480100002221200101"
}
```

It is presented in a nice textual format and tells us it's a Plutus version 2 script. Under the *cborHex* field we see the actual serialized contract. In our case it is short because we have a simple contract. If the contract is more complicated also this string gets longer. Let us look now at the second example of a Plutus script that you can find in the file Burn.hs.

```
{-# LANGUAGE DataKinds         #-}
{-# LANGUAGE NoImplicitPrelude #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE TemplateHaskell   #-}

module Burn where

import qualified Plutus.V2.Ledger.Api as PlutusV2
import           PlutusTx             (BuiltinData, compile)
```

```
import           PlutusTx.Prelude     (traceError)
import           Prelude              (IO)
import           Utilities            (writeValidatorToFile)

------------------------------------------------------------------------------
----------------------------- ON-CHAIN / VALIDATOR ---------------------------

-- This validator always fails
--                     Datum         Redeemer     ScriptContext
mkBurnValidator :: BuiltinData -> BuiltinData -> BuiltinData -> ()
mkBurnValidator _ _ _ = traceError "it burns!!!"
{-# INLINABLE mkBurnValidator #-}

validator :: PlutusV2.Validator
validator = PlutusV2.mkValidatorScript $$(PlutusTx.compile
                                          [|| mkBurnValidator ||])

------------------------------------------------------------------------------
------------------------------- HELPER FUNCTIONS -----------------------------

saveVal :: IO ()
saveVal = writeValidatorToFile "./assets/burn.plutus" validator
```

We see that all of the code is the same except the body of the *mkBurnValidator* function. In the body of the validator function, we again ignore the datum, redeemer and script context and raise a trace error with an error message. This means that the validation will fail no matter the inputs. For this reason, we call that module Burn, because all funds sent to this address can never again be retrieved and are therefore burnt. If you would want to raise an error without an error message you could import the *error* function instead of *traceError* and provide it only with an empty tuple. Another thing to mention is that we import these functions from the Plutus Prelude. One of the reasons why we do not use the standard Prelude is that most of the functions defined there cannot be declared inlinable, which we need to do in our Plutus code.

2.2 Using the Cardano CLI to interact with Plutus

Now that we have 2 examples of smart contracts in Plutus let us interact with them. The easiest way to do this is with the Cardano CLI which is a command line tool. This tool comes installed with the installation of the Cardano node that is included on the Demeter platform and in the container, image prepared for the local setup. In order to use the Cardano CLI, the node has to be running. On the Demeter that is taken care of and on your local setup the Cardano node also automatically starts for the preview testnet when the docker container is started.

There is also a pre-production testnet that is used to test polished software. In comparison to the preview testnet where things may possibly break on the pre-production testnet things should not break. But it also takes a longer time to synchronize than the preview testnet, so we use the later one.

Once the node has synchronized, we can open up another terminal in VSCode and we can run the Cardano CLI commands. Some scripts that contain CLI commands can be found in the scripts/ folder at the top of the pioneer repo. These are more general scripts that will be used in a couple of lessons. And other more specific scripts tailored to a lesson can be found in the Week folders also under the scripts/ folder.

The *query-tip.sh* script takes in the *testnet-magic* option and is followed by a number. The number 2 denotes the preview network and number 1 would be for the pre-production network. In case we would want to use this command on the main network the CLI option would be *mainnet* instead of *testnet-magic* without a number following the command option.

```
cardano-cli query tip --testnet-magic 2
```

If you execute this command, you will get an output that will tell you in the field *syncProgress* how much are you already synced to your network.

```
{
    "block": 495537,
    "epoch": 126,
    "era": "Babbage",
    "hash": "2664c8b1928e89c458af8981a973270d665c76a0511a7c5e496e450312f5c1f9",
    "slot": 10921361,
    "syncProgress": "95.34"
}
```

Before the node has fully synchronized, we can use the script *create-key-pair.sh* that expects an input argument as a user name and then generates three files which are the public key, the private key and an address. The Cardano CLI commands that do this are:

```
cardano-cli address key-gen --verification-key-file "$vkey" --signing-key-file "$skey"
cardano-cli address build --payment-verification-key-file "$vkey" \
                          --testnet-magic 2 --out-file "$addr"
```

The *$skey, $vkey* and *$addr* are the names of the files. They have all the same input name that we provided as input to the script and a different extension name. The verification key is the public key and the signing key is the private key. The keys and address will be created in the keys/ folder that will also be located at the top of the pioneer repo. If we look at the address file, we will see that a test address starts with addr_test1 prefix:

```
addr_test1vr8ldcu7ckeulp93q7yhdjv8q6mnwaxjc4fr4ax64a79zzgm5xd7e
```

The next script is the *query-address.sh* script that accepts an address file as input and outputs the address funds:

```
cardano-cli query utxo --address "$1" --testnet-magic 2
```

Of course, if we query our newly created address there won't be any funds. We can use the testnet faucet to send some test ada to our address. It can be found at the following web page:

https://docs.cardano.org/cardano-testnet/tools/faucet

Once we have sent some funds to our address after around 20 seconds the query command will return an output that indicates we have 10.000 test ada sitting at our address:

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
73ad0a24920bca90327af6d479332feca387de6012eec1d410df8f76d2c2e329     0     10000000000
                                                                          lovelace +
                                                                     TxOutDatumNone
```

We see that a UTXO sitting at this address is specified with a transaction hash and a transaction index. The hash is the same for multiple UTXOs if they were produced with the same transaction so you can separate them apart with the transaction index that goes from 0 on.

You can only request test tokens every 24h, so in order to request funds for another address we can send some funds from our first address to the second one. We will show how to do this in a later chapter. Let us now look at the script *make-gift.sh* located in the Week02 folder. The script first builds an address from a *.plutus* script file that we have learned how to produce in the previous chapter. The output is the *gift.addr* file which contains the script address.

```
cardano-cli address build \
    --payment-script-file "$assets/gift.plutus" \
    --testnet-magic 2 \
    --out-file "$assets/gift.addr"
```

In order to send now some funds to this address we must first build the transaction. We do this with the following command:

```
cardano-cli transaction build \
    --babbage-era \
    --testnet-magic 2 \
    --tx-in "$txin" \
    --tx-out "$(cat "$assets/gift.addr") + 3000000 lovelace" \
    --tx-out-inline-datum-file "$assets/unit.json" \
    --change-address "$(cat "$keypath/$name.addr")" \
```

```
    --out-file "$body"
```

For this command you need to specify several options. First, we need to specify the era. The Cardano development timeline is separated in multiple eras that mark significant changes to the protocols and software of the ecosystem. As of time of writing you can choose only one era, previously you could choose multiple eras. Next you need to input the testnet magic number. Again, here we state that running this command for the mainnet would change this option to *--mainnet* without a number. Then you have to specify the UTXO transaction input. You get this with the query command. As said before the UTXO is specified by the TX hash and Tx index and they are separated with a # symbol. So, for us this option would look like:

```
--tx-in 73ad0a24920bca90327af6d479332feca387de6012eec1d410df8f76d2c2e329#0
```

Next, we have the transaction output that takes in a string which needs to contain the address of the script we are sending funds to, the amount we are sending and the units which are lovelace for us. As a side note we state that 1 ada is a 1.000.000 lovelace. Then we have to add the option where we say how do we add the datum. This has to be done because we are sending funds to a script address. As said in the EUTxO chapter there are several options and we chose to inline the datum to the output UTXO. We need to provide the datum file name which is in JSON format. Since we are ignoring the datum, we can input any datum. You can find the *unit.json* file in the Week02/assets folder which corresponds to Haskell's unit type. We will show later how we can generate a datum file. We have now specified one input and one output but you could specify multiple inputs and outputs in the transaction build command.

Then we specify the change address, which should be our address. It tells the transaction that all the leftover funds from the input UTXO will be sent to this change address. In the end we specify the name of the output file that will be produced by this transaction build command. This file represents the body of the transaction but it still needs to be signed by the sender with his private key (signing key). We sign it with the following command:

```
cardano-cli transaction sign \
    --tx-body-file "$body" \
    --signing-key-file "$keypath/$name.skey" \
    --testnet-magic 2 \
    --out-file "$tx"
```

The sign command takes in the body file, our signing key, the testnet magic number and the name of the output file which will be the signed transaction. At the end we use the transaction submit command to submit the transaction to the preview testnet. There we only need to input the testnet magic number and the signed transaction file.

```
cardano-cli transaction submit \
    --testnet-magic 2 \
    --tx-file "$tx"
```

There is also the command *transaction build raw* that gives you more control but you have to take care of more parameters. Now that we have submitted the transaction, we can use the transaction ID command to get the transaction hash.

```
cardano-cli transaction txid --tx-file "$tx"
```

As input we need to provide the signed transaction file. We can use this hash and a blockchain explorer as for example Cardano scan, to look at this transaction. The simplest way to do this is to use the URL https://preview.cardanoscan.io/transaction/<txid> where the *<txid>* is the id we get with the transaction *txid* command. There you can see the transaction inputs, outputs and also look at the attached datum or the datum hash. If we would like now to collect the test ada sitting at the gift address we can use the script *collect-gift.sh*. In the beginning of the script there is the command that produces the protocol parameters file.

```
cardano-cli query protocol-parameters \
    --testnet-magic 2 \
    --out-file "$pp"
```

It takes as an option only the testnet magic number and the output file name. We will need these parameters in the transaction build command. Then we build our transaction, as follows:

```
cardano-cli transaction build \
    --babbage-era \
    --testnet-magic 2 \
    --tx-in "$txin" \
    --tx-in-script-file "$assets/gift.plutus" \
    --tx-in-inline-datum-present \
    --tx-in-redeemer-file "$assets/unit.json" \
    --tx-in-collateral "$collateral" \
    --change-address "$(cat "$keypath/$name.addr")" \
    --protocol-params-file "$pp" \
    --out-file "$body"
```

We see that there is no transaction output in this command. All the funds go to the change address we specify. When we want to consume funds of a script address we also need to provide the *.plutus* script file. When we generated this file we stated that the *cborHex* paramter in the script file gets longer as the validation logic of the scripts gets longer. So this hex makes it possible to read out the validation logic and perform it for a given node. In that sense it is different from a hash where you can not reverse engineer the given input from the hash. Next we can state the option *–tx-in-inline-datum-present* because we inlined the datum with the

producing transaction. Then we need to input the redeemer file which can be the *unit.json* file since the validation passes in any case. After that we need to specify a collateral UTXO that we have. First we state that a transaction is checked locally by the node before it is submitted to the network. The checks that are performed before submission are for e.g. checking if the inputs are still present, or if the datum hashes match if we attach a datum to the transaction. There are other checks done as well. In case any of these checks fails the transaction is not submitted to the network and no fees are charged to the user. But if you really insist you can force a transaction to be submitted even if some of the checks stated before fail before submission. In this case the network would charge a fee from the collateral UTXO. This is done because you could perform attacks on the network by submitting a large amount of failed transactions that would congest the network. But if you need to pay a fee for each of such transactions then such an attack would become very expensive. Usually 5 ada is enough per one transaction. If your transaction fails validation then this fee gets collected from your UTXO. If you would input a UTXO that you do not own the transaction can not be submitted since the signature of such a transaction would not be valid.

In the end we specify the protocol parameter file that we generated earlier. Because we are interacting with a smart contract the validator that we defined and compiled needs to be executed. And in order to calculate the costs and the transaction fees for that we need the protocol parameters that specify how expensive certain operations of a smart contract are. The rest of the script is the same as before. We need to sign the transaction and submit it.

If we now query the funds at the gift script address we will see many UTXOs sitting there, since many students have tried out to send something to this address but have not collected the funds. Because of how we defined our spending transaction we can only collect UTXOs that have an inline datum attached to it. There is also a possibility we will see UTXOs which do not have any datums attached to them. As of now, such UTXOs are impossible to spend, so if you send funds and address and do not attach a datum or datum hash those funds are burnt.

2.3 High-level typed validation scripts

First let us look now at another example of a low-level validation script where we take the redeemer into account in the validation logic. The code is contained in the file *FortyTwo.hs*.

```
{-# LANGUAGE DataKinds           #-}
{-# LANGUAGE ImportQualifiedPost #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE OverloadedStrings   #-}
{-# LANGUAGE TemplateHaskell     #-}
```

```haskell
module FortyTwo where

import qualified Plutus.V2.Ledger.Api as PlutusV2
import           PlutusTx             (BuiltinData, compile)
import           PlutusTx.Builtins    as Builtins (mkI)
import           PlutusTx.Prelude     (otherwise, traceError, (==))
import           Prelude              (IO)
import           Utilities            (writeValidatorToFile)

------------------------------------------------------------------------------
------------------------------ ON-CHAIN / VALIDATOR --------------------------

-- This validator succeeds only if the redeemer is 42
--                  Datum         Redeemer     ScriptContext
mk42Validator :: BuiltinData -> BuiltinData -> BuiltinData -> ()
mk42Validator _ r _
    | r == Builtins.mkI 42 = ()
    | otherwise            = traceError "expected 42"
{-# INLINABLE mk42Validator #-}

validator :: PlutusV2.Validator
validator = PlutusV2.mkValidatorScript $$(PlutusTx.compile [|| mk42Validator ||])

------------------------------------------------------------------------------
--------------------------------- HELPER FUNCTIONS ---------------------------

saveVal :: IO ()
saveVal = writeValidatorToFile "./assets/fortytwo.plutus" validator
```

In the body of the validator function, we say that if the redeemer is represented by the integer number 42 the validation should pass and in any other case it should fail with an error message. The datum and the script context are both ignored. To create a `BuiltinData` type out of the number 42 we use the function `Builtins.mkI`. Now the reason why it would be an advantage to use proper types instead of `BuiltinData` type the compiler would have more information about our types and could therefore more easily catch an error. So, let us look now at the typed version of this validator function. The code can be found in the *FortyTwoTyped.hs* file.

```haskell
{-# LANGUAGE DataKinds           #-}
{-# LANGUAGE ImportQualifiedPost #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE OverloadedStrings   #-}
{-# LANGUAGE TemplateHaskell     #-}

module FortyTwoTyped where
```

```
import qualified Plutus.V2.Ledger.Api as PlutusV2
import           PlutusTx             (compile)
import           PlutusTx.Prelude     (Bool, Eq ((==)), Integer, traceIfFalse,
                                       ($))
import           Prelude              (IO)
import           Utilities            (wrapValidator, writeValidatorToFile)

------------------------------------------------------------------------------
------------------------------ ON-CHAIN / VALIDATOR ----------------------------

-- This validator succeeds only if the redeemer is 42
--               Datum  Redeemer        ScriptContext
mk42Validator :: () -> Integer -> PlutusV2.ScriptContext -> Bool
mk42Validator _ r _ = traceIfFalse "expected 42" $ r == 42
{-# INLINABLE mk42Validator #-}

validator :: PlutusV2.Validator
validator = PlutusV2.mkValidatorScript $$(PlutusTx.compile
                                            [|| wrapValidator mk42Validator ||])

------------------------------------------------------------------------------
------------------------------ HELPER FUNCTIONS --------------------------------

saveVal :: IO ()
saveVal = writeValidatorToFile "./assets/fortytwotyped.plutus" validator
```

We see that the imported language pragmas and the *saveVal* function stay the same, but the code for the import statements, the validator function and the validator change. In the validator function we now use the unit type for the datum, and integer type for the redeemer and the `PlutusV2.ScriptContext` type for the script context, which can be only of this type when writing typed validators. In the body of the validator function, we use the `traceIfFalse` function that takes in an error message and a Boolean expression. It returns unit in case the Boolean expression is True or raises an error with the input message in case it is false. And in the Boolean expression we can simply compare the value of the redeemer to the integer number 42. The problem now we have is that the `PlutusTx.compile` function still expects a function that takes in three parameters of type `BuiltinData` and returns the unit type. We can solve this issue by applying the `wrapValidator` function to our validator function before compiling it. This function is defined in the *Utilities* folder in the module *PlutusTx*.

```
{-# LANGUAGE NoImplicitPrelude #-}
{-# LANGUAGE RankNTypes        #-}
```

```
module Utilities.PlutusTx
  ( wrapValidator
  ) where

import           Plutus.V2.Ledger.Api (ScriptContext, UnsafeFromData,
                                        unsafeFromBuiltinData)
import           PlutusTx.Prelude     (Bool, BuiltinData, check, ($))

{-# INLINABLE wrapValidator #-}
wrapValidator :: forall a b.
        ( UnsafeFromData a
        , UnsafeFromData b
        )
      => (a -> b -> ScriptContext -> Bool)
      -> (BuiltinData -> BuiltinData -> BuiltinData -> ())
wrapValidator f a b ctx =
  check $ f
      (unsafeFromBuiltinData a)
      (unsafeFromBuiltinData b)
      (unsafeFromBuiltinData ctx)
```

We see that the *wrapValidator* function takes in the type signature of a typed validator function and returns a signature of an untyped validator. To convert the custom types to the BuiltinData type we use the conversion function unsafeFromBuiltinData. To be able to apply this function to the types a and b they both need to have an instance of the UnsafeFromData type class. If we look at this type class it has the single method unsafeFromBuiltinData.

```
Prelude Gift> import Plutus.V2.Ledger.Api
Prelude Plutus.V2.Ledger.Api Gift> :i UnsafeFromData
type UnsafeFromData :: * -> Constraint
class UnsafeFromData a where
  unsafeFromBuiltinData :: BuiltinData -> a
```

So for every type we use in our typed validator code this function has to be defined.

From the command above you will also get a long list of data types that have an instance including integer and unit. We can try now to manually convert a BuiltinData type to an integer. But for this we need to also say that we expect an integer number by adding a type signature at the end of the expression.

```
Prelude Plutus.V2.Ledger.Api PlutusTx.Builtins> unsafeFromBuiltinData (mkI 42) ::
Integer
42
```

We can also do the conversion for a *ByteString* if we add the appropriate language pragma.

```
Prelude Plutus.V2.Ledger.Api PlutusTx.Builtins> :set -XOverloadedStrings
Prelude    Plutus.V2.Ledger.Api    PlutusTx.Builtins>    unsafeFromBuiltinData    (mkB
"Haskell") :: BuiltinByteString
"Haskell"
```

As said before there is a performance advantage for the untyped version. If you look at the files *fortytwo.plutus* and *fortytwotyped.plutus* you see that in the *cborHex* field the string gets much longer for the typed version in comparison with the untyped one. Still in this book we will mostly use the typed version which is easier to understand. There exist also tools that can optimize your Plutus core script in order for it to be more performant.

The last thing we will show in this chapter is how to use custom defined types in our on-chain code. If you have a specific business problem your code will have very specific data structures often in the form of record types and ideally you want to use those for datum and redeemer. One way to use these types in validator functions is to write an instance of the `UnsafeFromData` type class for them. But because there is template Haskell support for doing this you can choose another way. Let us look now at the code from the *CustomTypes.hs* file.

```
{-# LANGUAGE DataKinds            #-}
{-# LANGUAGE ImportQualifiedPost #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE OverloadedStrings   #-}
{-# LANGUAGE TemplateHaskell     #-}

module CustomTypes where

import qualified Plutus.V2.Ledger.Api as PlutusV2
import           PlutusTx             (BuiltinData, compile, unstableMakeIsData)
import           PlutusTx.Prelude     (Bool, Eq ((==)), Integer, traceIfFalse,
                                       ($))
import           Prelude              (IO)
import           Utilities            (wrapValidator, writeValidatorToFile)

-------------------------------------------------------------------------------
------------------------------ ON-CHAIN / VALIDATOR ---------------------------

-- We can create custom data types for our datum and redeemer like this:
newtype MySillyRedeemer = MkMySillyRedeemer Integer
PlutusTx.unstableMakeIsData ''MySillyRedeemer -- Use TH to create an instance for
IsData.

-- This validator succeeds only if the redeemer is `MkMySillyRedeemer 42`
--              Datum     Redeemer            ScriptContext
mkCTValidator :: () -> MySillyRedeemer -> PlutusV2.ScriptContext -> Bool
```

```
mkCTValidator _ (MkMySillyRedeemer r) _ = traceIfFalse "expected 42" $ r == 42
{-# INLINABLE mkCTValidator #-}

wrappedMkVal :: BuiltinData -> BuiltinData -> BuiltinData -> ()
wrappedMkVal = wrapValidator mkCTValidator
{-# INLINABLE wrappedMkVal #-}

validator :: PlutusV2.Validator
validator = PlutusV2.mkValidatorScript $$(PlutusTx.compile [|| wrappedMkVal ||])

-------------------------------------------------------------------------------
------------------------------- HELPER FUNCTIONS ------------------------------

saveVal :: IO ()
saveVal = writeValidatorToFile "./assets/customtypes.plutus" validator
```

We look again at part of the code to better understand what is happening. As shown below we define in the code a new type called `MySillyRedeemer` which is just a wrapper for an integer.

```
newtype MySillyRedeemer = MkMySillyRedeemer Integer
PlutusTx.unstableMakeIsData ''MySillyRedeemer
```

We will use this type as redeemer and again validate that the integer it holds is equal to 42. We can use the `PlutusTx.unstableMakeIsData` to define an `UnsafeFromData` instance for our type. There is also a stable version of this function. The difference is you have more control over how exactly values are deserialized when you use the stable version but you also have to provide more details. In template Haskell you need to add two single quotes in front of the type's name in order to use it as an input variable. This line of code also creates *toData* and *fromData* instances. If we import from the REPL *PlutusTx* we can then lookup this type.

```
Prelude CustomTypes> import PlutusTx
Prelude CustomTypes PlutusTx> :i MySillyRedeemer
type MySillyRedeemer :: *
newtype MySillyRedeemer = MkMySillyRedeemer Integer
        -- Defined in `CustomTypes'
instance FromData MySillyRedeemer -- Defined in `CustomTypes'
instance ToData MySillyRedeemer -- Defined in `CustomTypes'
instance UnsafeFromData MySillyRedeemer -- Defined in `CustomTypes'
```

If we check those two classes, we can see that they define the functions `fromBuiltinData` and `toBuiltinData` which can be used to convert data types back and forth between the `Data` and `BuiltinData` types about which we talked in the beginning of chapter 2.

```
Prelude CustomTypes PlutusTx> :i FromData
type FromData :: * -> Constraint
```

```
class FromData a where
  fromBuiltinData :: BuiltinData -> Maybe a

Prelude CustomTypes PlutusTx> :i ToData
type ToData :: * -> Constraint
class ToData a where
  toBuiltinData :: a -> BuiltinData
```

We saw already before that the `UnsafeFromData` type class defines the `unsafeFromBuiltinData` function. The difference to the `fromBuiltinData` function above is that the safe version returns a Nothing value instead of throwing an error if the conversion fails.

```
Prelude PlutusTx> :set -XOverloadedStrings
Prelude PlutusTx> import PlutusTx.Builtins
Prelude PlutusTx PlutusTx.Builtins> fromBuiltinData (mkB "Haskell") :: Maybe Integer
Nothing
```

So now that we have the needed instances for the `MySillyRedeemer` type we can also apply the `wrapValidator` function to our validator function that converts it to an un-typed version. The rest of the Plutus code is the same as in previous examples.

At the end of this chapter let us discuss how to write data which has a custom defined Haskell type to a file. This is needed when we use the cardano-cli and want to attach to a transaction the redeemer and the datum as a file. The function that can write data of a user defined Haskell type to a file is in the `Utilities.Serialise` module and is called *`writeDataToFile`*.

```
writeDataToFile :: ToData a => FilePath -> a -> IO ()
writeDataToFile filePath x = do
  let v = scriptDataToJsonDetailedSchema $ fromPlutusData $ PlutusV2.toData x
  writeFileJSON filePath v >>= \case
   Left err -> print $ displayError err
   Right () -> printf "Wrote data to: %s\n%s\n" filePath $ BS8.unpack $
prettyPrintJSON v
```

We can use this function to write arbitrary data to a JSON file. It takes in a file path which will be the file we write to and a polymorphic type which needs to have an instance of the `ToData` type class, such that the conversion to the `BuiltinData` type can be performed. The result is also printed in the REPL when we apply this function to our data.

```
Prelude CustomTypes Utilities> writeDataToFile "silly.json" $ MkMySillyRedeemer 42
Wrote data to: silly.json
{
    "constructor": 0,
    "fields": [
        {
            "int": 42
        }
```

```
    ]
}
```

We could also use that function for a more structured data type as a tuple that contains a list of integers and a Boolean for example.

```
Prelude CustomTypes Utilities> writeDataToFile "tuple.json" $ ([42 :: Integer, 35],
True)
Wrote data to: tuple.json
{
    "constructor": 0,
    "fields": [
        {
            "list": [
                {
                    "int": 42
                },
                {
                    "int": 35
                }
            ]
        },
        {
            "constructor": 1,
            "fields": []
        }
    ]
}
```

We see that the JSON object becomes quite complex to write by hand for a simple tuple. So, this function and others in the Utilities module are very useful.

2.4 Homework

In the first homework you will need to define a typed validator function that ignores the datum and the script context and for the redeemer takes in a tuple of two Booleans. If the Booleans are both true, then the validator function should return true, otherwise false.

```
{-# LANGUAGE DataKinds           #-}
{-# LANGUAGE ImportQualifiedPost #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE TemplateHaskell     #-}

module Homework1 where

import qualified Plutus.V2.Ledger.Api as PlutusV2
import           PlutusTx             (compile)
import           PlutusTx.Prelude     (Bool (..), BuiltinData)
import           Utilities            (wrap)
```

```
-------------------------------------------------------------------------------
------------------------------ ON-CHAIN / VALIDATOR ----------------------------

{-# INLINABLE mkValidator #-}
-- This should validate if and only if the two Booleans in the redeemer are True!
mkValidator :: () -> (Bool, Bool) -> PlutusV2.ScriptContext -> Bool
mkValidator _ _ _ = False

wrappedVal :: BuiltinData -> BuiltinData -> BuiltinData -> ()
wrappedVal = wrapValidator mkValidator

validator :: PlutusV2.Validator
validator = PlutusV2.mkValidatorScript $$(PlutusTx.compile [|| wrappedVal ||])
```

In the second homework example you have a custom type defined called `MyRedeemer` that wraps two Boolean values. Write a typed validator function that has this type for the redeemer type returns true if the Boolean values are different. Otherwise, it returns false. You will also need to define the *`wrappedVal`* function and the *`validator`* variable.

```
{-# LANGUAGE DataKinds           #-}
{-# LANGUAGE DeriveAnyClass      #-}
{-# LANGUAGE DeriveGeneric       #-}
{-# LANGUAGE ImportQualifiedPost #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE TemplateHaskell     #-}

module Homework2 where

import qualified Plutus.V2.Ledger.Api as PlutusV2
import           PlutusTx             (unstableMakeIsData)
import           PlutusTx.Prelude     (Bool, BuiltinData)
import           Prelude              (undefined)
--import           Utilities            (wrap)

-------------------------------------------------------------------------------
------------------------------ ON-CHAIN / VALIDATOR ----------------------------

data MyRedeemer = MyRedeemer
    { flag1 :: Bool
    , flag2 :: Bool
    }

PlutusTx.unstableMakeIsData ''MyRedeemer
```

```haskell
{-# INLINABLE mkValidator #-}
-- Create a validator that unlocks the funds if MyRedemeer's flags are different
mkValidator :: () -> MyRedeemer -> PlutusV2.ScriptContext -> Bool
mkValidator = undefined

wrappedVal :: BuiltinData -> BuiltinData -> BuiltinData -> ()
wrappedVal = undefined

validator :: PlutusV2.Validator
validator = undefined
```

There are also tests written for both of the homework files. You can cd into the Week02 folder and run the tests with the following command:

```
workspaces/plutus-pioneer-program# cd code/Week02
workspaces/plutus-pioneer-program/code/Week02# cabal test
```

You will get an output that lets you know how many tests have passed the validation. The default output for the unmodified homework should look like this:

```
Running 1 test suites...
Test suite week02-homework: RUNNING...
Homework tests
Testing Homework1
    Case: (True, True):           FAIL (0.47s)
    Case: (True, False):          OK
    Case: (False, True):          OK
    Case: (False, False):         OK
  Testing Homework2
    Case: MyRedeemer True True:   FAIL
    Case: MyRedeemer True False:  FAIL
    Case: MyRedeemer False True:  FAIL
    Case: MyRedeemer False False: FAIL

5 out of 8 tests failed (0.49s)
Test suite week02-homework: FAIL
```

And the output with the correct solutions should look like this:

```
Running 1 test suites...
Test suite week02-homework: RUNNING...
Homework tests
  Testing Homework1
    Case: (True, True):           OK (0.48s)
    Case: (True, False):          OK
    Case: (False, True):          OK
    Case: (False, False):         OK
  Testing Homework2
    Case: MyRedeemer True True:   OK
```

```
    Case: MyRedeemer True False:  OK
    Case: MyRedeemer False True:  OK
    Case: MyRedeemer False False: OK

All 8 tests passed (0.51s)
Test suite week02-homework: PASS
```

3 Script context, time and parameterized contracts

In this chapter we will look at the data type that defines the script context of a transaction, then we will show how to use this data in the validation logic. We will also look at functions that are used to manage time intervals of a transaction. And in the end, we will show how to write parameterized contracts, which are contracts that take in an additional parameter beside the datum, redeemer and the script context.

3.1 Script context

In Bitcoin for instance the Bitcoin script sees the transaction context that is the UTXO to be consumed and the redeemer, which is represented by the private key. This is very little information all together. On the other side we have Ethereum where solidity scripts see and can access information on the complete global state of the blockchain. This is on the other side a lot of information. Cardano chooses a middle way where the script context is the transaction being validated. So in Plutus the script sees the datum, the redeemer and the transaction context that provides information about the inputs and outputs of the transaction being validated. The script context is implemented as a Haskell data type. There are two data type versions of the script context for because there are two versions of Plutus scripts. In our examples we will look at the `ScriptContextV2` type. It is a type synonym for the `ScriptContext` type which is defined in the *Plutus.Script.Utils.V2.Contexts* module.

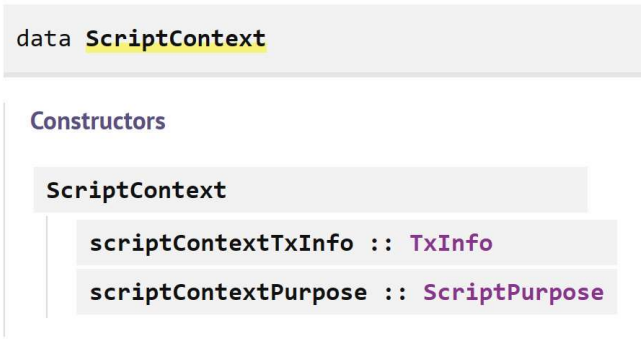


Figure 2 - ScriptContext type

We see it's a record syntax type with two fields that contain transaction information and purpose. The purpose tells us for which purpose the Plutus script in question is being used. For now, we have only talked about the spending purpose which holds the *TxOutRef* type that is a reference to a UTXO that is being validated. The UTXO reference is composed of the transaction ID which is a SHA-256 hash and the transaction index that is an integer number (Figure 4). We have used this data already in chapter 2 when we had to reference an UTXO as input in the transaction build command for the Cardano CLI tool. There our input string was composed from

the transaction hash and index and they were separated by the # symbol. Then minting is used when we try to mint or burn native tokens. We will talk about that in chapter 5. Rewarding is used for withdrawing staking rewards. Certifying is for issuing certificates. So, for all these 4 purposes validation logic in Plutus scripts can be used.

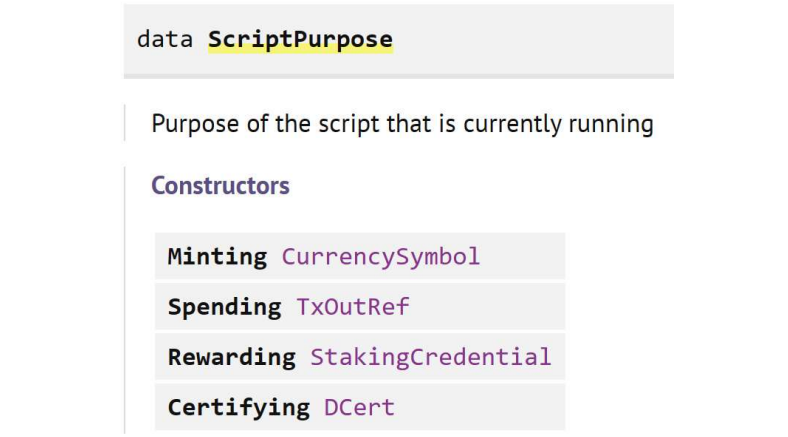


Figure 3 - ScriptPurpose type

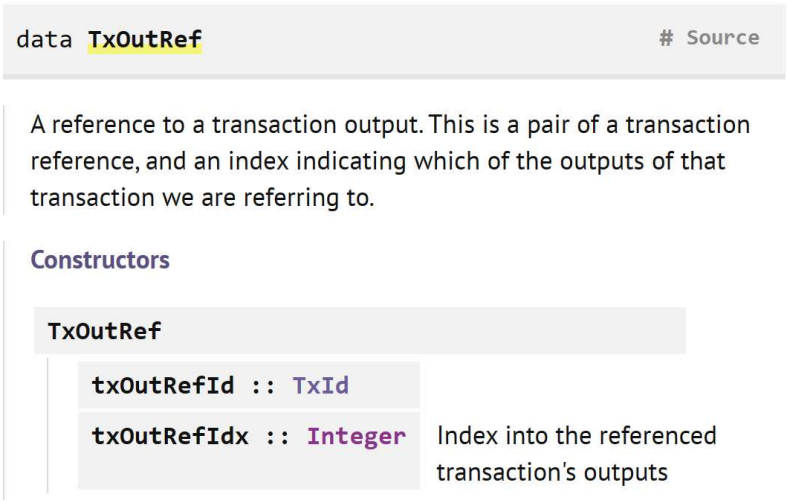


Figure 4 - TxOutRef type

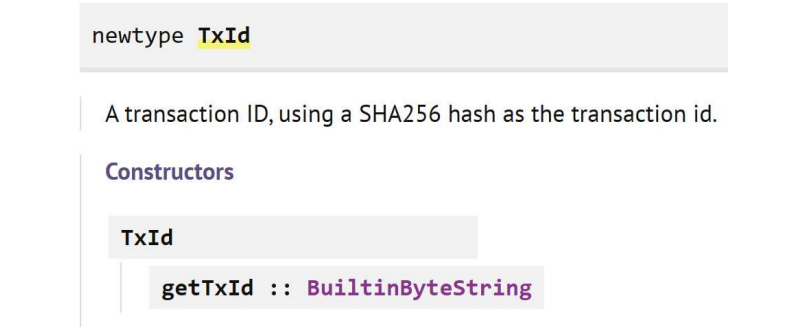


Figure 5 - TxId type

The *TxInfo* type holds information about the transaction. From Figure 6 we can see it holds many data constructors that define various other types, list of types or map of types.

```
data TxInfo                                                    # Source
```

A pending transaction. This is the view as seen by validator scripts, so some details are stripped out.

Constructors

TxInfo

txInfoInputs :: [TxInInfo]	Transaction inputs
txInfoReferenceInputs :: [TxInInfo]	Transaction reference inputs
txInfoOutputs :: [TxOut]	Transaction outputs
txInfoFee :: Value	The fee paid by this transaction.
txInfoMint :: Value	The `Value` minted by this transaction.
txInfoDCert :: [DCert]	Digests of certificates included in this transaction
txInfoWdrl :: Map StakingCredential Integer	Withdrawals
txInfoValidRange :: POSIXTimeRange	The valid range for the transaction.
txInfoSignatories :: [PubKeyHash]	Signatures provided with the transaction, attested that they all signed the tx
txInfoRedeemers :: Map ScriptPurpose Redeemer	
txInfoData :: Map DatumHash Datum	
txInfoId :: TxId	Hash of the pending transaction (excluding witnesses)

Figure 6 - TxInfo type

First it defines the *txInfoInputs* field which is a list of transaction inputs.

```
data TxInInfo
```

An input of a pending transaction.

Constructors

TxInInfo

txInInfoOutRef :: TxOutRef

txInInfoResolved :: TxOut

Figure 7 - TxInInfo type

A transaction input holds a UTXO reference and a description of this UTXO output which is defined with the *TxOut* type (Figure 8).

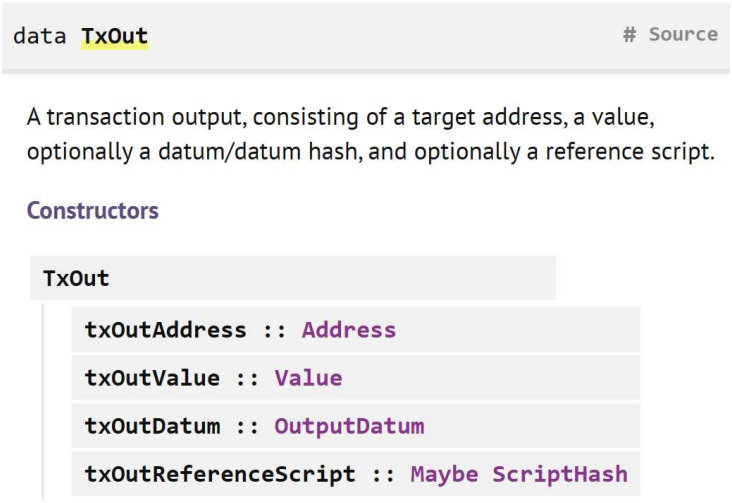


Figure 8 - TxOut type

It is again a record type with 4 fields. It holds the address at which the UTXO sits. Then the value that the UTXO contains, which can be ada, native tokens or a combination of both. Next is the datum which can have one of three possible constructors (Figure 9). The *DatumHash* is a wrapper around a `BuiltinByteString` type. And the *Datum* type is just a wrapper for the `BuiltinData` type we have talked about in chapter 2. In the end we have a possible reference to Plutus validation script that is of type *Maybe ScriptHash*. Under the hood the *ScriptHash* is a `BuiltinByteString` type same as the *DatumHash* type.

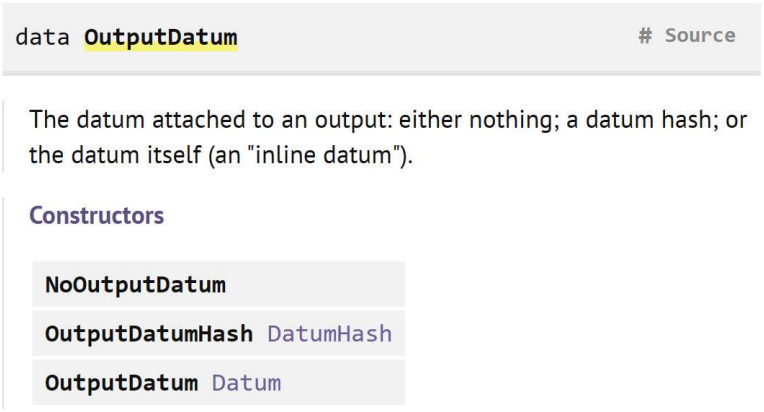


Figure 9 - OutputDatum type

The next data type in the *TxOut* type is *txInfoReferenceInputs* which is again a list of transaction inputs. The difference to the previous list of transaction inputs is that those inputs will not be spent by this transaction and are just referenced for the purpose of adding input information to the validation logic. This could be done because for instance we want to access a datum value sitting at an oracle address or access a reference script sitting and an UTXO.

The nice thing about these input references is that several transactions in the same block can use the same UTXO as reference inputs. Before the Vasil hard fork there could be a UTXO at a script address that was carrying some important information which a lot of transactions were accessing and was representing a bottleneck because only one transaction at a time could access this UTXO. Reference inputs allow you to look at the state of a UTXO without changing and processing it.

After the reference inputs come the outputs of a transaction which is a list of *TxOut* type variables. In this case this type specifies for an UTXO that will be created the address where the UTXO will go to, then the value that will be sit at this UTXO, the output datum that will be added to the UTXO and possibly a reference script that will be attached to the UTXO. Next is the transaction fee (*txInfoFee*) that is of type value. At the moment transaction fees on Cardano are always denominated in ada but it could happen in the future they will also be possible to pay them with native tokens. The *txInfoMint* field denotes minting or burning of native tokens. We will talk about minting and burning in chapter 5 of this book. Then we have *txInfoDCert* field that holds data about digital certificates. A certificate is represented with the *DCert* type.

Figure 10 - DCert type

We see we have a list of 7 possible certificates. The first one is if you register a staking credential. The second one is if you de-register a staking credential.

The third one is if you delegate to a stake pool. The fourth one is for registering a stake pool. The fifth one is for retiring a stake pool. The sixth one is probably used only for the genesis block and the seventh one is used for paying catalyst winners from the treasury. If you want to learn more about project catalyst you can look at this web page: https://projectcatalyst.io/. The Mir credentials stand for "move instantaneous rewards". This field is rarely used.

After the certificates come the withdrawals. Which is used for withdrawing staking rewards. It is a Map object from staking credentials to the amount you withdraw. Then comes the time validity range of the transaction which we will talk about later in chapter 3. Next comes a list of public key hashes that have signed this transaction. After that comes a Map of redeemers that contains Map keys of type *ScriptPurpose* and Map elements of type *Redeemer*. The *Redeemer* type is just a wrapper around the *BuiltinData* type that is also used by the Datum type. Then comes the transaction data that contains a Map of datum hashes that map to variables of Datum type. If any of your input UTXOs contain only datum hashes and not the datums itself you need to provide the datums in the transaction which is done in this *txInfoData* field. At the end comes the transaction ID of type *TxId* that we said already before is a wrapper for a *BuiltingByteString* and gets assigned to every transaction.

3.2 Handling time

As we stated before the advantage of Cardano over Ethereum is that the validation can happen in the wallet. If an input is missing at the time of submitting the transaction it will simply fail without any fees being charged to the user. For this reason, under normal circumstances a validation script should never run and fail on the network. But if you think about this it is not clear how to manage time in that context. We want to be able to express validation logic that says a certain transaction is only valid after a certain time has been reached. And because we said we can validate a transaction in the wallet before submitting it to the network there should be a way to address this time issue. The way Cardano solves this is by adding the POSIX time range field to the transaction info (*TxInfo*) data type. The POSIXTimeRange type gives a valid time interval for the transaction. Before validation on the network the time range of the transaction is checked with the help of this field. So, in that way the validation is completely deterministic again because a node can do the time check before the validation is run.

By default, all transactions use the infinite time range that starts at the genesis block and lasts forever. Such transactions are always valid no matter the time they arrive. Ouroboros the consensus algorithm of Cardano uses slots instead of POSIX time. Each slot, a slot leader is determined that can produce a block. Slots are the native measure of time in Cardano. Plutus uses real time so we need to be able to convert back and forth between real time and slots.

Right now, the slot length is fixed to 1 second. But this can change in the future with a hard fork. For this reason, slot intervals specified in the transaction must not have a definite upper bound that is too far in the future. It can be only that far in the future as it is possible to know what the slot length will be. And that happens to be something like 36 hours, because it is known at least 36 hours in advance if there will be a change to the protocol parameters that could influence a slot length. This applies only to the transaction itself, not the Plutus contracts where we can mention arbitrary dates. The idea is that Plutus contracts are long running and can reference time that is far ahead in the future as e.g., 10 years. Let us look at the POSIX time range type in more detail. You can see it in Figure 11.

```
type POSIXTimeRange = Interval POSIXTime
```

An `Interval` of `POSIXTime`s.

Figure 11 - POSIXTimeRange type

We see it contains the *Interval* type parameterized with the *POSIXTime* type, that is just a wrapper for an integer number that states the number of milliseconds which have passed for a certain date and time since 1st of January 1970 midnight. The interval type holds data for the lower and upper bound of the validity interval for the specified transaction.

```
data Interval a                                    # Source
```

An interval of as.

The interval may be either closed or open at either end, meaning that the endpoints may or may not be included in the interval.

The interval can also be unbounded on either side.

Constructors

```
Interval
    ivFrom :: LowerBound a
    ivTo :: UpperBound a
```

Figure 12 - Interval type

The lower bound type holds the *Extended* and *Closure* types. The closure type is just a wrapper for a Boolean which says whether the bound is included in the interval or not. The extended type has three possible constructor values which represent negative infinity, positive infinity or a finite bound that is parameterized by the type a, which is for us of type *POSIXTime*.

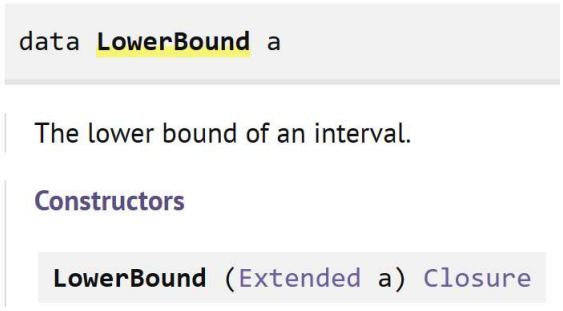


Figure 13 - LowerBound type

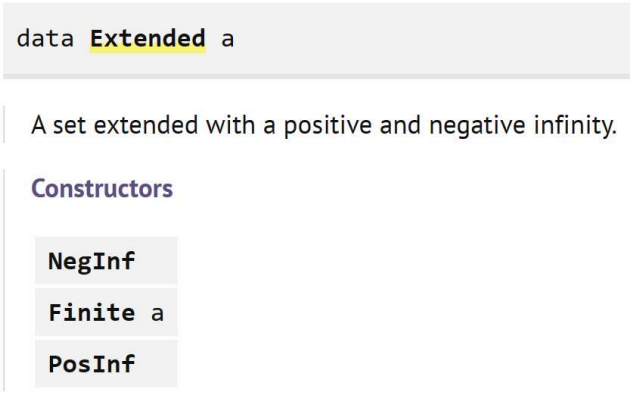


Figure 14 - Extended type

The *UpperBound* type is defined the same way as the *LowerBound* type. If we further explore the module *Plutus.V1.Ledger.Interval* we can see there are a lot of functions that work with time intervals which do the following things:

- *member*: Checks whether a value is in an interval.
- *interval*: takes in two parameters and constructs an *Interval a* with boundaries included.
- *from*: gives an *Interval a* that includes all values that are greater than or equal to *a*.
- *to*: gives an *Interval a* that includes all values that are smaller than or equal to *a*.
- *always*: An interval that covers every slot.
- *never*: An interval that is empty.
- *singleton*: An *Interval a* that only contains the *a*.
- *hull*: *hull Interval a Interval b* is the smallest interval containing both intervals.
- *intersection*: *intersection Interval a Interval b* is the largest interval that is contained in both of the intervals, if it exists.
- *overlap*: checks whether two intervals have a value in common and returns a Boolean.
- *contains*: checks whether the second interval is contained in the first one.
- *isEmpty*: checks whether an interval is empty.
- *before*: checks whether a given time is before the given interval.
- *after*: checks whether a given time is after the given interval.

The last four functions *lowerBound, upperBound, strictLowerBound, strictUpperBound* are used to construct a lower or upper bound that can be inclusive or exclusive. Let us look at some code examples that use these functions:

```
Prelude> import Plutus.V1.Ledger.Interval

Prelude Plutus.V1.Ledger.Interval> interval (10 :: Integer) 20
Interval {ivFrom = LowerBound (Finite 10) True, ivTo = UpperBound (Finite 20) True}

Prelude Plutus.V1.Ledger.Interval> member 9 $ interval (10 :: Integer) 20
False

Prelude Plutus.V1.Ledger.Interval> member 10 $ interval (10 :: Integer) 20
True

Prelude Plutus.V1.Ledger.Interval> member 29 $ from (30 :: Integer)
False

Prelude Plutus.V1.Ledger.Interval> member 30 $ from (30 :: Integer)
True

Prelude Plutus.V1.Ledger.Interval> member 31 $ to (30 :: Integer)
False

Prelude Plutus.V1.Ledger.Interval> member 30 $ to (30 :: Integer)
True

Prelude Plutus.V1.Ledger.Interval> intersection (interval (10 :: Integer) 20)
$ interval 18 30
Interval {ivFrom = LowerBound (Finite 18) True, ivTo = UpperBound (Finite 20) True}

Prelude Plutus.V1.Ledger.Interval> contains (to (100 :: Integer)) $ interval 30 80
True

Prelude Plutus.V1.Ledger.Interval> contains (to (100 :: Integer)) $ interval 30 101
False

Prelude Plutus.V1.Ledger.Interval> overlaps (to (100 :: Integer)) $ interval 30 101
True
```

3.3 A vesting example

Imagine you want to make a gift of ada to a child but such that he can access the ada when he or she becomes 18. Using Plutus such a vesting scheme is easy to implement. In the Week03 folder we find the file *Vesting.hs* which is a simple implementation of a vesting scheme.

```
{-# LANGUAGE DataKinds         #-}
{-# LANGUAGE NoImplicitPrelude #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE TemplateHaskell   #-}
```

```haskell
module Vesting where

import           Data.Maybe                (fromJust)
import           Plutus.V1.Ledger.Interval (contains)
import           Plutus.V2.Ledger.Api      (BuiltinData, POSIXTime, PubKeyHash,
                                            ScriptContext (scriptContextTxInfo),
                                            TxInfo (txInfoValidRange),
                                            Validator, from, mkValidatorScript)
import           Plutus.V2.Ledger.Contexts (txSignedBy)
import           PlutusTx                  (compile, unstableMakeIsData)
import           PlutusTx.Prelude          (Bool, traceIfFalse, ($), (&&))
import           Prelude                   (IO, String)
import           Utilities                 (Network, posixTimeFromIso8601,
                                            printDataToJSON,
                                            validatorAddressBech32,
                                            wrapValidator,
                                            writeValidatorToFile)

-------------------------------------------------------------------------------
------------------------------ ON-CHAIN / VALIDATOR ---------------------------

data VestingDatum = VestingDatum
    { beneficiary :: PubKeyHash
    , deadline    :: POSIXTime
    }

unstableMakeIsData ''VestingDatum

{-# INLINABLE mkVestingValidator #-}
mkVestingValidator :: VestingDatum -> () -> ScriptContext -> Bool
mkVestingValidator dat () ctx = traceIfFalse "beneficiary's signature missing"
signedByBeneficiary &&
                                traceIfFalse "deadline not reached"
deadlineReached
  where
    info :: TxInfo
    info = scriptContextTxInfo ctx

    signedByBeneficiary :: Bool
    signedByBeneficiary = txSignedBy info $ beneficiary dat

    deadlineReached :: Bool
    deadlineReached = contains (from $ deadline dat) $ txInfoValidRange info

{-# INLINABLE  mkWrappedVestingValidator #-}
```

```haskell
mkWrappedVestingValidator :: BuiltinData -> BuiltinData -> BuiltinData -> ()
mkWrappedVestingValidator = wrapValidator mkVestingValidator

validator :: Validator
validator = mkValidatorScript $$(compile [|| mkWrappedVestingValidator ||])

-------------------------------------------------------------------------------
--------------------------------- HELPER FUNCTIONS ----------------------------

saveVal :: IO ()
saveVal = writeValidatorToFile "./assets/vesting.plutus" validator

vestingAddressBech32 :: Network -> String
vestingAddressBech32 network = validatorAddressBech32 network validator

printVestingDatumJSON :: PubKeyHash -> String -> IO ()
printVestingDatumJSON pkh time = printDataToJSON $ VestingDatum
    { beneficiary = pkh
    , deadline    = fromJust $ posixTimeFromIso8601 time
    }
```

We see that in this smart contract we use typed high-level code with a custom data type for the datum that we call `VestingDatum`. It contains the beneficiary which is a `PubKeyHash` type and the deadline which is of type `POSIXTime`. So, the validation logic says that the funds can be unlocked only when the deadline has been reached and the transaction is signed by the beneficiary. In the validation code we have the helper variables *`signedByBeneficiary`* and *`deadlineReached`* which are of type Bool. In the body of the first variable, we use the helper function `txSignedBy` that takes in a transaction info and a public key hash and checks whether this transaction has been signed with this public key hash. In the body of the second variable, we access the validity range of the transaction and check that it is contained inside the interval starting with the deadline and going to infinity. Then we wrap the validator function and compile it to a *Validator* type. In the helper functions section we have again the *`saveVal`* function that writes the validator to a *.plutus* file. And then we also have the *`printVestingDatumJSON`* function that takes in a public key hash and a string which contains the time in ISO 8601 format and prints the datum in JSON format to the terminal. We can try and run this function for the current date and time.

```
Prelude> import Vesting
Prelude Vesting> :set -XOverloadedStrings
Prelude Vesting> import Plutus.V2.Ledger.Api
Prelude Vesting> pkh1 = "cff6e39ec5b3cf84b1078976c98706b73774d2c5523af4daaf7c5109"
Prelude Vesting Plutus.V1.Ledger.Api>
printVestingDatumJSON pkh1 "2023-03-11T13:12:11.123Z"
```

```
{
    "constructor": 0,
    "fields": [
        {
            "bytes": "cff6e39ec5b3cf84b1078976c98706b73774d2c5523af4daaf7c5109"
        },
        {
            "int": 1678540331123
        }
    ]
}
```

Now you can save this JSON output to the file *vesting-datum.JSON* and use it with the cardano-cli. The time 13:12:11.123Z denotes the GMT time 13:12 with 11 seconds and 123 milliseconds. You can check the conversion to POSIX time at https://www.epochconverter.com/. For the pkh1 variable you might notice it is just a *ByteString* but the *PubKeyHash* type is defined as:

```
Prelude Plutus.V2.Ledger.Api> :i PubKeyHash
type PubKeyHash :: *
newtype PubKeyHash
  = PubKeyHash {getPubKeyHash :: BuiltinByteString}
```

If you try to define the pkh1 variable as:

```
Prelude> pkh1 = PubKeyHash "cff6e39ec5b3cf84b1078976c98706b73774d2c5523af4daaf7c5109"
```

and then call the *printVestingDatumJSON* function on the pkh1 variable the encoding of the bytestring will not be correct and the cardano-cli transaction build command will tell you that that the signature of the beneficiary is missing when you will try to build the spending transaction. A workaround to that is to add some imports and code to the *Vesting.hs* file that will handle the conversion from *hex -> text -> bytestring* correctly.

```
{-# LANGUAGE ImportQualifiedPost   #-}

import PlutusTx.Prelude (Bool, traceIfFalse, ($), (&&), toBuiltin)
import Prelude          (IO, String, Either(..), (.), Show(..), error)

import Data.ByteString qualified as BS (ByteString)
import PlutusTx.Builtins qualified as BI
import Plutus.V1.Ledger.Bytes qualified as P (bytes, fromHex)

-- At the end of the Vesting.hs file add this function
bytesFromHex :: BS.ByteString -> BS.ByteString
bytesFromHex = P.bytes . fromEither . P.fromHex
  where
    fromEither (Left e)  = error $ show e
    fromEither (Right b) = b
```

After we have added this to the *Vesting.hs* code, we can construct the vesting datum with the `PubKeyHash` type by using the commands below:

```
Prelude> pkh2 = PubKeyHash $ toBuiltin $ bytesFromHex
"cff6e39ec5b3cf84b1078976c98706b73774d2c5523af4daaf7c5109"
Prelude> printVestingDatumJSON pkh2 "2023-03-11T13:12:11.123Z"
{
    "constructor": 0,
    "fields": [
        {
            "bytes": "cff6e39ec5b3cf84b1078976c98706b73774d2c5523af4daaf7c5109"
        },
        {
            "int": 1678540331123
        }
    ]
}
```

3.4 Parameterized contracts

In the examples we have seen so far, we had one specific validator that was a Haskell value of type `Validator`. If there was any variability in the contract, we model that by using the datum as in the vesting example where the datum contained the beneficiary and the deadline. There is an alternative to this that are so-called parameterized contracts. The idea is to build the variability into the contract itself by adding a parameter variable to the validator function. We showcase this in the *ParameterizedVestin.hs* file which contains a modified version of the code from the *Vesting.hs* example where we create a parameterized contract.

```
{-# LANGUAGE DataKinds              #-}
{-# LANGUAGE MultiParamTypeClasses #-}
{-# LANGUAGE NoImplicitPrelude     #-}
{-# LANGUAGE OverloadedStrings     #-}
{-# LANGUAGE ScopedTypeVariables   #-}
{-# LANGUAGE TemplateHaskell       #-}

module ParameterizedVesting where

import           Plutus.V1.Ledger.Interval (contains)
import           Plutus.V2.Ledger.Api      (BuiltinData, POSIXTime, PubKeyHash,
                                            ScriptContext (scriptContextTxInfo),
                                            TxInfo (txInfoValidRange),
                                            Validator, from, mkValidatorScript)
import           Plutus.V2.Ledger.Contexts (txSignedBy)
import           PlutusTx                  (applyCode, compile, liftCode,
                                            makeLift)
import           PlutusTx.Prelude          (Bool, traceIfFalse, ($), (&&), (.))
```

```
import           Prelude                 (IO)
import           Utilities               (wrapValidator120,
writeValidatorToFile)

-------------------------------------------------------------------------------
------------------------------ ON-CHAIN / VALIDATOR ---------------------------

data VestingParams = VestingParams
    { beneficiary :: PubKeyHash
    , deadline    :: POSIXTime
    }
makeLift ''VestingParams

{-# INLINABLE mkParameterizedVestingValidator #-}
mkParameterizedVestingValidator :: VestingParams -> () -> () -> ScriptContext ->
Bool
mkParameterizedVestingValidator params () () ctx =
    traceIfFalse "beneficiary's signature missing" signedByBeneficiary &&
    traceIfFalse "deadline not reached" deadlineReached
  where
    info :: TxInfo
    info = scriptContextTxInfo ctx

    signedByBeneficiary :: Bool
    signedByBeneficiary = txSignedBy info $ beneficiary params

    deadlineReached :: Bool
    deadlineReached = contains (from $ deadline params) $ txInfoValidRange info

{-# INLINABLE  mkWrappedParameterizedVestingValidator #-}
mkWrappedParameterizedVestingValidator :: VestingParams -> BuiltinData ->
BuiltinData -> BuiltinData -> ()
mkWrappedParameterizedVestingValidator = wrapValidator .
mkParameterizedVestingValidator

validator :: VestingParams -> Validator
validator params = mkValidatorScript ($$(compile [||
                                      mkWrappedParameterizedVestingValidator ||])
                                      `applyCode` liftCode params)

-------------------------------------------------------------------------------
------------------------------- HELPER FUNCTIONS ------------------------------

saveVal :: VestingParams -> IO ()
saveVal = writeValidatorToFile "./assets/parameterized-vesting.plutus" .
```

```
          validator
```

First, we create the type `VestingParams` which was previously called `VestingDatum`. It holds the beneficiary's public key hash and the deadline after which the beneficiary can claim the funds. We do not need the `unstableMakeIsData` instance. The type signature of the validator function changes such that it takes in the additional vesting parameter. And the datums type changes to unit. We note that a validator function can take in any number of parameters. We are using in our example only one parameter. Then in the body of the validator function we change the name of the `dat` parameter to the name `params`. We need to update the type signature of the *`mkWrappedParameterizedVestingValidator`* function. And we write the body of the function with point free style using only the dot . operator that we also have to import. For the *`validator`* we now expect a function instead of a single value, so we update its type signature accordingly which now represents a family of validators. Now in the body of the validator we cannot simply pass the input parameter to the `mkWrappedParameterizedVestingValidator` function. Because we are using template Haskell all the data that template Haskell splices need must be known at compile time because this step runs before out program is even compiled. But the parameter we are passing to the validator function is only known at runtime. So, template Haskell does not know what code to generate and splice into the source code. But what we can do is to apply the compiled validator function to the compiled input parameter. Because the compiled validator function in Plutus script is still a function. And we do this with the help of the `applyCode` function which allows us to pass in a parameter to a Plutus script function. But we still need to know the value of the input parameter at compile time, which is not possible. However, the input parameter is not some arbitrary Haskell function, it is static data. And for such data there is a way to pass it at runtime to a script function if we create for them an instance of the type class Lift. Then we only need to apply the function `liftCode` to the data in order to compile the input parameter at runtime. Let us have a look at the Lift class.

```
class Lift uni a where                                    # Source
```

Class for types which can be lifted into Plutus IR. Instances should be derived, do not write your own instance!

Methods

```
lift :: a -> RTCompile uni fun (Term TyName Name uni fun ())
                                                          # Source
```

Get a Plutus IR term corresponding to the given value.

Figure 15 - Lift class

It contains only the *lift* method but we will not use that directly. There are a lot of instances for this type class for all possible data types such as *BuiltinData*, *Data*, *Integer*, *Bool* and so on. But not for functions so we cannot use it to compile Haskell validators to Plutus core validators. The *PlutusTx* module defines this type class and it also defines the *liftCode* function.

```
liftCode :: (Lift uni a, Throwable uni fun, Typecheckable uni fun) =>
a -> CompiledCodeIn uni fun a                             # Source
```

Get a Plutus Core program corresponding to the given value as a `CompiledCodeIn`, throwing any errors that occur as exceptions and ignoring fresh names.

Figure 16 - liftCode function

We see that the input variable of type a has to have an instance of the *Lift* type class. Because we are using a custom defined type in our on-chain code we can use a template Haskell mechanism to define an instance for our type. And we do this with in the line where we say:

```
makeLift ''VestingParams
```

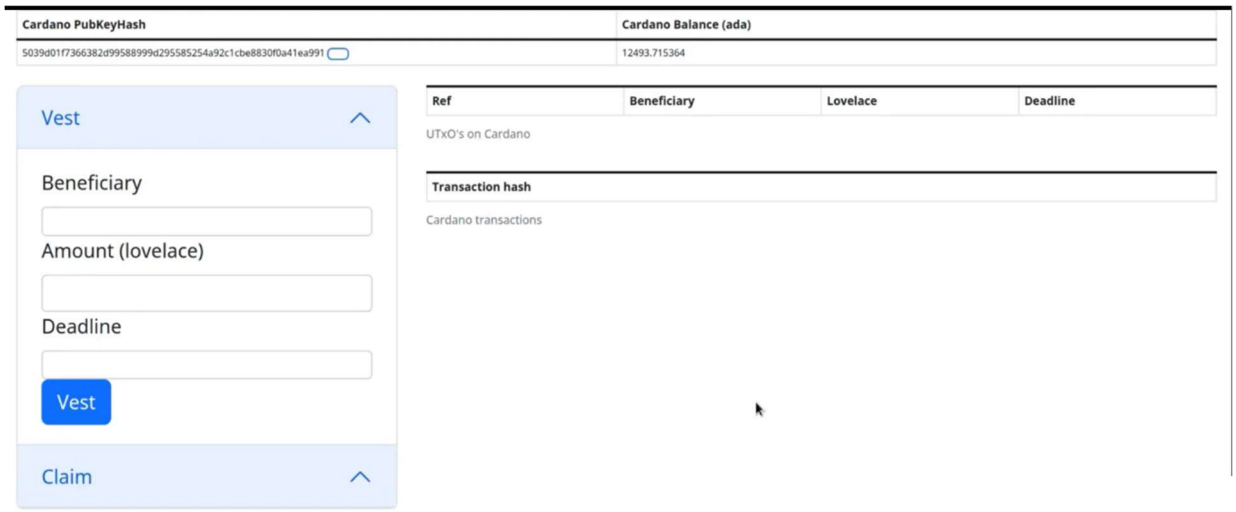
This line only compiles if the `MultiParamTypeClasses` and `ScopedTypeVariables` language extensions are enabled. So, this is all together the procedure on how we can write parameterized contracts. If there would be more than one parameter, we would need to use the `applyCode` function multiple times for each of the lifted parameters one after the other.

3.5 Offchain code with Lucid

Off-chain code is interaction with the blockchain. You want to be able to query the blockchain to see which UTXOs are there and also to construct and submit transactions. You can do this with the Cardano CLI but it requires a lot of manual work. So, now we will demonstrate another tool to manage off-chain code called lucid. If you google Cardano *lucid* you can click on the *npmjs.com* link which is the repository of node JavaScript packages, so you can install it as a npm package. You can use lucid writing JavaScript or TypeScript code. At the previously mentioned web page you can look up some basic examples and there is a link to the API documentation. We will write a DApp that can interact with the vesting contract we wrote in chapter 3.3. Below are the commands to start the DApp.

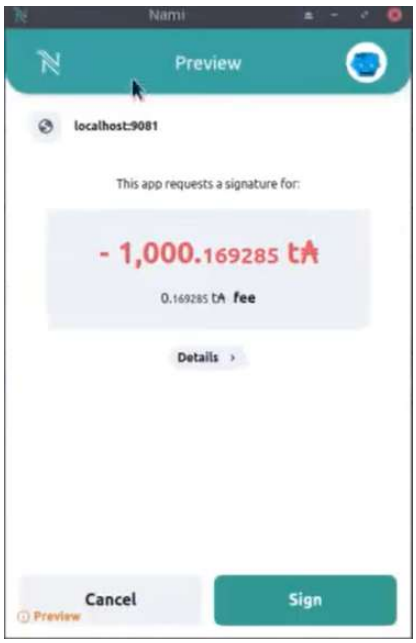
```
/workspace# cd code/Week03/lucid
/workspace/code/Week03/lucid# npm install
/workspace/code/Week03/lucid# npm start
```

That will open the DApp in your browser as illustrated below.



The DApp is written such that it works specifically with the Nami browser wallet. When you will run it for the first time the Nami wallet plugin will pop up and ask you weather you want to connect to the DApp. At the top you can see your public key hash and the balance of your test ada. In our case we are connected to the preview testnet. For our example we have two Nami accounts that are called preview and preview2. We use the preview wallet to create the vesting and the preview2 wallet will be the beneficiary. So, we copy the public key hash of wallet 2 and we switch to wallet 1, where we input the data under the Vest tab on the left side. We input the public key hash, the amount in lovelace we want to vest and we set the deadline. For the deadline a calendar appears once we click on the input box and we can choose a date and time.

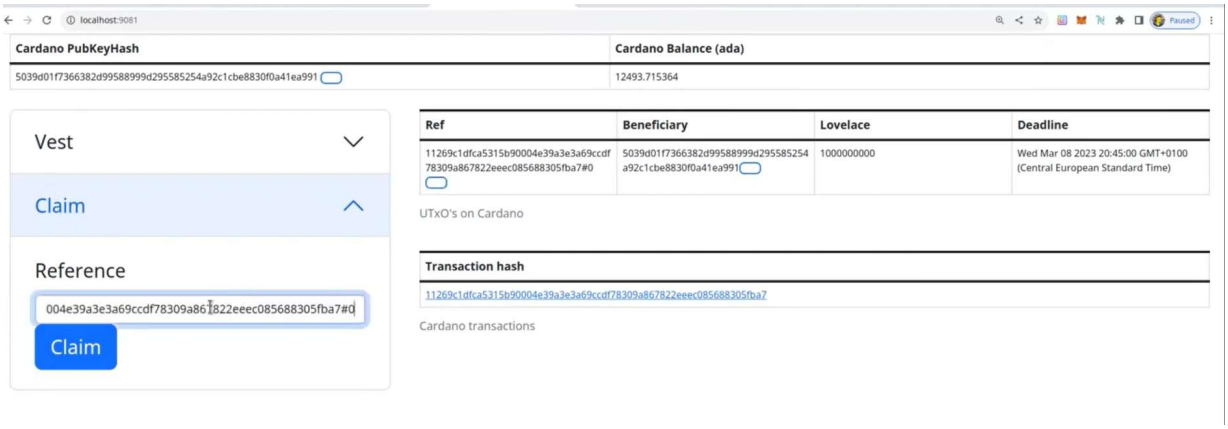
After that we click on **Vest** button and a window from our Nami wallet appears that displays the amount we are sending and gives us the option to sign the transaction.



If you click on **Details**, you can also see some details of the transaction. Once we have signed the transaction, we will see in our DApp under the **Transaction hash** section a hash of the transaction we have just created. If we click on it, it redirects us to the cardano-scan web page. The table above the transaction hash shows all the vesting's that have been found on the blockchain. It displays the following information:

- the transaction output reference
- the beneficiaries public key hash
- the amount vested
- the deadline

We now switch to wallet 2 and click on the **Claim** tab on the left side. There we can input the transaction reference that we copy from the table. After that we can click on the **Claim** button.



We will again see a pop-up window that prompts us to sign the transaction. After we do that if the deadline has passed, we will see that we have successfully claimed the ada. We can now look at the index.js file that contains the lucid code for this DApp.

```
import 'bootstrap';
import 'bootstrap/dist/css/bootstrap.min.css';
import './bootstrap-datetimepicker.min.css';
import * as L from 'lucid-cardano';

const vestingScript = {
    type: "PlutusV2",
    script: "590b30590b2d01000032323233223232332232323232323232332…",
};

const VestingDatum = L.Data.Object({
    beneficiary: L.Data.String,
    deadline: L.Data.BigInt,
});

function removeChildren(elt) {
    while (elt.firstChild) {
        elt.removeChild(elt.lastChild);
    }
}
```

In the beginning we import lucid. Then we define our vesting script where we copy the CBOR hex from our vesting.plutus file. Then we define the datum type which needs to be converted to a JSON type. For this we define a schema for the datum type where the beneficiary is of type *String* and the deadline is of type *BigInt*. We will use it later so that we can convert our datum type to JSON. The *removeChildren* function is just part of the UI (user interface) code.

```
async function loadCardano() {
    const nami = window.cardano.nami;
    if (!nami) {
        setTimeout(loadCardano);
    } else {
        const api = await nami.enable();
        console.log('nami enabled');
        const lucid = await L.Lucid.new(
            new L.Blockfrost("https://cardano-preview.blockfrost.io/api/v0",
"preview1JXEDVldkIyBkxEUrEx3n9ll4afFK1Xj"),
            "Preview",
        );
        console.log('lucid active');
```

```
        lucid.selectWallet(api);
        return lucid;
    }
}
```

With the *loadCardano* function we initialize lucid and connect it to our Nami wallet. The Nami wallet plugin is available as a global object *window.cardano.nami*. First, we enable it, then we initialize lucid and we use blockfrost for that. If you want to use this code you will of course need to input your own blockfrost API key that we explained how to get in chapter 1.2. In the end we link the Nami wallet with lucid, so all activities that require a wallet will use Nami.

```
async function submitCardanoTx(signedTx) {
    const tid = await signedTx.submit();
    console.log("Cardano tx submitted: " + tid);
    addLinkToTable("cardanoTxTable",
"https://preview.cardanoscan.io/transaction/" + tid, tid);
}

async function signAndSubmitCardanoTx(tx) {
    try {
        const signedTx = await tx.sign().complete();
        await submitCardanoTx(signedTx);
    } catch (err) {
        alert(`Cardano transaction:\ninfo: ${err.info}\nmessage:
${err.message}`);
        throw (err);
    }
}
```

With the *submitCardanoTx* function we submit a signed transaction. The return value is the transaction ID. We log that in the console and update the table of the DApp that displays this information. Another helper function is *signAndSubmitCardanoTx* that takes an unsigned lucid transaction and signs it before submitting it. The line where we call the *sign* function on the transaction will cause our Nami wallet to pop up and prompt us to sign the transaction.

```
async function getCardanoPKH() {
    const addr = await lucid.wallet.address();
    const details = await L.getAddressDetails(addr);
    return details.paymentCredential.hash;
}

async function getStatus() {
    const pkh = await getCardanoPKH();
```

```
    const utxos = await lucid.wallet.getUtxos();
    const lovelace = utxos.reduce((acc, utxo) => acc + utxo.assets.lovelace, 0n);

    const vestings = await vestingUTxOs();

    return {
        cardanoPKH: pkh,
        cardanoBalance: lovelace,
        vestingUTxOs: vestings,
    };
}
```

The *getCardanoPKH* function is used to get the public key hash belonging to the connected wallet. The wallet has an address function which gives me the address of the Nami wallet. Then we extract details for this address which also contain the public key hash. The *getStatus* function is used to return the status which is composed of the pkh, the wallet balance and the vesting UTXO. We use the previously defined function to display the public key hash and then we also display the balance of the wallet. It does this by getting all UTXOs from the connected wallet and then we sum the lovelace value sitting at these UTXOs. In the end we also retrieve the vesting UTXO with the *vestingUTxOs* helper function.

```
function addCell(tr, content) {
    const td = document.createElement('td');
    tr.appendChild(td);
    td.appendChild(document.createTextNode(content));
}

function addLinkToTable(tableId, href, text) {
    const txTable = document.getElementById('cardanoTxTable');
    const tr = document.createElement('tr');
    txTable.appendChild(tr);
    const td = document.createElement('td');
    tr.appendChild(td);
    const a = document.createElement('a');
    td.appendChild(a);
    a.setAttribute('href', href);
    a.setAttribute('target', '_blank');
    a.appendChild(document.createTextNode(text));
}

function addCopyCell(row, text) {
    const td = document.createElement("td");
    row.appendChild(td);
```

```
    const span = document.createElement("span");
    td.appendChild(span);
    const uid = String(Math.random()).slice(2);
    span.setAttribute("id", uid);
    span.appendChild(document.createTextNode(text));
    const button = document.createElement("button");
    td.appendChild(button);
    button.setAttribute("type", "button");
    button.classList.add("btn");
    button.classList.add("btn-outline-primary");
    button.classList.add("btn-sm");
    button.addEventListener("click", () => onCopy(uid));
}

async function setStatus() {
    const status = await getStatus();

    const cardanoPKH = document.getElementById('cardanoPKH');
    removeChildren(cardanoPKH);
    cardanoPKH.appendChild(document.createTextNode(status.cardanoPKH));

    const cardanoBalance = document.getElementById('cardanoBalance');
    const ada = Number(status.cardanoBalance) / 1000000;
    removeChildren(cardanoBalance);
    cardanoBalance.appendChild(document.createTextNode(ada));

    const vestingUTxOsTable = document.getElementById('vestingUTxOsTable');
    removeChildren(vestingUTxOsTable);
    for (const x of status.vestingUTxOs) {
        const tr = document.createElement('tr');
        vestingUTxOsTable.appendChild(tr);

        addCopyCell(tr, x.utxo.txHash + '#' + x.utxo.outputIndex);
        addCopyCell(tr, x.datum.beneficiary);
        addCell(tr, x.utxo.assets.lovelace);
        addCell(tr, new Date(Number(x.datum.deadline)));
    }
}
```

The helper functions `addCell`, `addLinkToTable` and `addCopyCell` are used for handling the user interface. The function `setStatus` uses the function `getStatus` to populate the DApp with the status information.

```
async function vestingUTxOs() {
```

```
    const utxos = await lucid.utxosAt(vestingAddress);
    const res = [];
    for (const utxo of utxos) {
        const datum = utxo.datum;
        if (datum) {
            try {
                const d = L.Data.from(datum, VestingDatum);
                res.push({
                    utxo: utxo,
                    datum: d
                });
            } catch (err) {
            }
        }
    }
    return res;
}

async function findUTxO(ref) {
    const chunks = ref.split('#');
    const tid = chunks[0];
    const ix = parseInt(chunks[1]);
    const utxos = await vestingUTxOs();
    for (const utxo of utxos) {
        if (utxo.utxo.txHash == tid && utxo.utxo.outputIndex == ix) {
            return utxo;
        }
    }
    return null;
}
```

With the `vestingUTxOs` function we fetch the vesting UTXOs. So, we have the vesting address and from it we can retrieve a list of all UTXOs belonging to this address. And then we filter out those vesting UTXOs that have a the `VestingDatum` structure that we defined in the beginning. The `findUTxO` function is used for the claim functionality where I want to redeem a specific vesting UTXO. The *ref* input parameter is the transaction output reference of the UTXO we want to claim. From it we extract the transaction hash and index. Then we get all vesting UTXOs that have an appropriate datum structure and we filter out the one that has the matching transaction hash and ID.

```
async function onVest() {
    const beneficiaryText = document.getElementById('vestBeneficiaryText');
    const beneficiary = beneficiaryText.value;
```

```
    const amountText = document.getElementById('vestAmountText');
    const amount = BigInt(parseInt(vestAmountText.value));
    const deadlineText = document.getElementById('vestDeadlineText');
    const deadline = BigInt(Date.parse(deadlineText.value));

    const d = {
        beneficiary: beneficiary,
        deadline: deadline,
    };
    const datum = L.Data.to(d, VestingDatum);
    const tx = await lucid
        .newTx()
        .payToContract(vestingAddress, { inline: datum }, { lovelace: amount })
        .complete();
    signAndSubmitCardanoTx(tx);

    beneficiaryText.value = "";
    amountText.value = "";
    deadlineText.value = "";
}
```

The `onVest` function handles the button click of the **Vest** button. The first six lines just extract the beneficiary and the amount from the input fields in the HTML. Then we compose the datum that we want to use where we set the beneficiary and the deadline. After that we convert it to JSON format with the `L.Data.to` function. Then we construct our transaction where we say how much we want to pay to which address and what datum we attach to it. And in the end, we sign and submit the transaction we have created. What lucid takes care of automatically is to look at the UTXOs of our connected wallet and do so-called coin selection which means to find appropriate inputs and also add change outputs when necessary.

```
async function onClaim() {
    const pkh = await getCardanoPKH();

    const referenceText = document.getElementById('claimReferenceText');
    const reference = referenceText.value;

    const utxo = await findUTxO(reference);
    if (utxo) {
        const tx = await lucid
            .newTx()
            .collectFrom([utxo.utxo], L.Data.to(new L.Constr(0, [])))
            .attachSpendingValidator(vestingScript)
            .addSignerKey(pkh)
```

```
            .validFrom(Number(utxo.datum.deadline))
            .complete();
        signAndSubmitCardanoTx(tx);
    } else {
        console.log("UTxO not found");
    }

    referenceText.value = "";
}
```

The *onClaim* function is executed when we click on the **Claim** button. We first look up our public key hash. Then we look up the reference from the input field that we want to claim. Once we have the reference, we use the function *findUTxO*. And then we construct our transaction where we use the *collectFrom* function where we add an empty redeemer, we attach the vesting script, add our signature and specify the validity interval. Since we have in lined the datum, we do not need to attach it now.

```
function onCopy(elt) {
    navigator.clipboard.writeText(document.getElementById(elt).firstChild.textCon
tent);
}

window.L = L;
window.lucid = await loadCardano();
const vestingAddress = lucid.utils.validatorToAddress(vestingScript);

$(function () {
    $(".dtp").datetimepicker({
        minuteStep: 1,
        autoclose: true,
        format: 'yyyy-mm-dd hh:ii'
    });
});

setStatus();
setInterval(setStatus, 5000);

document.getElementById("vestButton").addEventListener("click", onVest);
document.getElementById("claimButton").addEventListener("click", onClaim);
document.getElementById('cardanoPKHButton').addEventListener("click", () =>
onCopy("cardanoPKH"));
```

At the end we have the *onCopy* function that gets triggered when you click on a copy to clipboard button, that can be found beside the public key hash field. After that we call the *LoadCardano* function, we define the vesting address and we pull the status periodically. The rest of the code is dealing with UI, where we hook up the event handlers for the button actions. This DApp is serverless which means that it completely runs in the browser and does not rely on any database or server.

3.6 Reference scripts

We will look now at a variant of the lucid script from the previous chapter which we can find in the *code/Week03/lucid-ref-script* folder. In this variant we will use reference scripts. You can again build and start this project with the *npm install* and *npm start* command. Before the Vasil hard fork an output consisted of an address, a value and an optional datum. After the hard fork it is possible to optionally include a script to the output. Once such a UTXO exists then if a transaction wants to spend a UTXO sitting at the address given by this script it does not have to include the script but instead point to the UTXO that has the script attached. It is relatively costly to create these reference script outputs, but once they are created, to use them is cheaper than including the script in the transaction, because a pointer is generally much smaller than the actual script. The question that arises is where do you send such a reference script. It can be an arbitrary UTXO but what address should you choose. You can of course send it to your own wallet and later spend it again, such that you collect the deposit you needed to pay to put that reference script on the blockchain. For production applications the other option is to put it somewhere where it cannot be retrieved which can be for example the address of the burn script, we presented in chapter 2. Then you can rely on that script that it will be there forever and hardcode the *TxOutRef* that points to that script in your code. So let us look at the code changes in the *index.js* file compared to the previous chapter.

```
const burnScript = {
    type: "PlutusV2",
    script: "581f581d01000022232632498cd5ce24810b6974206275726e7321212100120011"
};

async function getReferenceUTxO() {
    const utxos = await lucid.utxosByOutRef([{
        txHash:
"902aba5b2b72df2423ee33d4af294e21e0903c31e34917a4de8570b3e8d08c7b",
        outputIndex: 0
    }]);
    return utxos[0];
}
```

First, we add the burn script such that we can compute its address in the code. Then most of the code stays the same as in the previous chapter. What we add is the *getReferenceUTxO* function. This function is returning the reference UTXO, that contains the reference vesting script. We use the function *utxosByOutRef* that takes in a transaction hash and an output index and gets the UTXO that is defined by them. It returns an array of 1 element. We have previously created a UTXO that has the vesting script attached to it at the burn address and now we can hardcode the transaction hash and index in our code.

```
async function onClaim() {
    const pkh = await getCardanoPKH();

    const referenceText = document.getElementById('claimReferenceText');
    const reference = referenceText.value;

    const utxo = await findUTxO(reference);
    if (utxo) {
        const tx = await lucid
            .newTx()
            .collectFrom([utxo.utxo], L.Data.void())
            .readFrom([referenceUTxO])
            .addSignerKey(pkh)
            .validFrom(Number(utxo.datum.deadline))
            .complete();
        signAndSubmitCardanoTx(tx);
    } else {
        console.log("UTxO not found");
    }

    referenceText.value = "";
}

async function onDeploy() {
    const tx = await lucid
        .newTx()
        .payToContract(burnAddress, { inline: L.Data.void(), scriptRef:
vestingScript }, {})
        .complete();
    signAndSubmitCardanoTx(tx);
}

const referenceUTxO = await getReferenceUTxO();
```

The function *onClaim* also changes. Instead of attaching the vesting script to the spending transaction we can simply use the *readFrom* function that reads the script from the UTXO we input to that function. And we input it as the global variable *referenceUTxO* that we define at the end of our code. Such a spending transaction does only look at reference inputs and does not consume them. And reference scripts are a special case and are implemented using reference inputs. In our DApp we also have a new button that connects to the *onDeploy* function which allows the user to deploy the vesting script to the burn address. In the function *payToContract* we attach the vesting script under the field `vestingScript` and then we sign and submit the transaction. Lucid will take care automatically of the minimal deposit that needs to be sent to the burn address in order that we can create our UTXO with the reference script. We notice that the **Deploy** button in the DApp needs to be used only once and never again. If we test this out, we see that the cost for creating this transaction is around 13.6 ada, which is not that much but still 10 times more compared to the fee of a simple transaction. The actual fee of this transaction is around 0.3 ada and the rest is the minimal deposit we had to make because the vesting script is that big. The deposit depends on the size of the data you attach to a UTXO. If we would not send it to a burn address, we could also get this deposit back when we spend that UTXO again. When we complete the transaction, we will see a transaction hash displayed in the DApp and we can use the Cardano scan explorer to check which index is the correct one that contains our UTXO with the reference script.

3.7 Homework

In the first part of the homework, you will have to implement a variation of the vesting validator. The datum contains two beneficiaries and a deadline. The validator script should validate if either beneficiary 1 has signed the transaction and the current slot is before or at the deadline or if beneficiary 2 has signed the transaction and the deadline has passed.

```
{-# LANGUAGE DataKinds         #-}
{-# LANGUAGE NoImplicitPrelude #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE TemplateHaskell   #-}
{-# LANGUAGE TypeApplications  #-}
{-# LANGUAGE TypeFamilies      #-}

module Homework1 where

import           Plutus.V2.Ledger.Api (BuiltinData, POSIXTime, PubKeyHash,
                                        ScriptContext, Validator,
                                        mkValidatorScript)
import           PlutusTx             (compile, unstableMakeIsData)
```

```
import           PlutusTx.Prelude     (Bool (..))
import           Utilities            (wrapValidator)

-------------------------------------------------------------------------------
------------------------------ ON-CHAIN / VALIDATOR ------------------------------

data VestingDatum = VestingDatum
    { beneficiary1 :: PubKeyHash
    , beneficiary2 :: PubKeyHash
    , deadline     :: POSIXTime
    }

unstableMakeIsData ''VestingDatum

{-# INLINABLE mkVestingValidator #-}
-- This should validate if either beneficiary1 has signed the transaction and the
-- current slot is before or at the deadline or if beneficiary2 has signed the
-- transaction and the deadline has passed.
mkVestingValidator :: VestingDatum -> () -> ScriptContext -> Bool
mkVestingValidator _dat () _ctx = False -- FIX ME!

{-# INLINABLE  mkWrappedVestingValidator #-}
mkWrappedVestingValidator :: BuiltinData -> BuiltinData -> BuiltinData -> ()
mkWrappedVestingValidator = wrapValidator mkVestingValidator

validator :: Validator
validator = mkValidatorScript $$(compile [|| mkWrappedVestingValidator ||])
```

In the second homework you will have to write a parameterized vesting contract where the parameter is only the beneficiary and the deadline is contained in the datum.

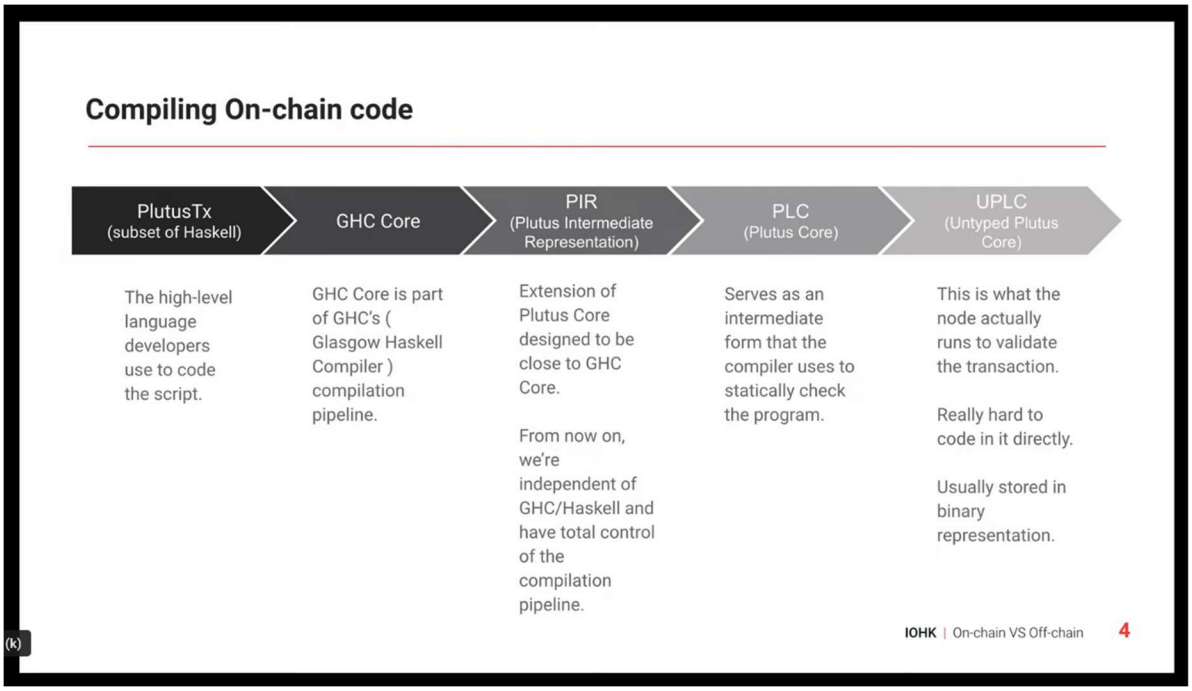
```
{-# LANGUAGE DataKinds             #-}
{-# LANGUAGE MultiParamTypeClasses #-}
{-# LANGUAGE NoImplicitPrelude     #-}
{-# LANGUAGE OverloadedStrings     #-}
{-# LANGUAGE ScopedTypeVariables   #-}
{-# LANGUAGE TemplateHaskell       #-}

module Homework2 where

import           Plutus.V2.Ledger.Api (BuiltinData, POSIXTime, PubKeyHash,
                                       ScriptContext, Validator,
                                       mkValidatorScript)
import           PlutusTx             (applyCode, compile, liftCode)
import           PlutusTx.Prelude     (Bool (False), (.))
```

```
import           Utilities            (wrapValidator)

-------------------------------------------------------------------------------
------------------------------ ON-CHAIN / VALIDATOR ---------------------------

{-# INLINABLE mkParameterizedVestingValidator #-}
-- This should validate if the transaction has a signature from the parameterized
-- beneficiary and the deadline has passed.
mkParameterizedVestingValidator :: PubKeyHash -> POSIXTime -> () ->
                                   ScriptContext -> Bool
mkParameterizedVestingValidator _beneficiary _deadline () _ctx = False -- FIX ME!

{-# INLINABLE  mkWrappedParameterizedVestingValidator #-}
mkWrappedParameterizedVestingValidator :: PubKeyHash -> BuiltinData ->
                                          BuiltinData -> BuiltinData -> ()
mkWrappedParameterizedVestingValidator = wrapValidator .
                                         mkParameterizedVestingValidator

validator :: PubKeyHash -> Validator
validator beneficiary = mkValidatorScript ($$(compile [||
mkWrappedParameterizedVestingValidator ||]) `applyCode` liftCode beneficiary)
```

4 Off-chain code possibilities

This chapter mainly focuses on writing off-chain code. Not so long ago we had only one way of coding smart contracts, which in the beginning of 2022 changed, when additional tools were created. We will try to explain the concept behind those tools now. To briefly recap, on-chain code is the code that runs in the node during the inclusion of new transactions that want to spend a UTXO at a script address. When we talk about off-chain code we mean the code that runs in the users or a service provider's device which queries the blockchain and builds and submits transactions. You could also have a running process to check the state of a transaction and show it in a UI. The blockchain is a passive entity, it does not do anything by itself. The only way something happens on a blockchain is when a transaction is submitted by a user. Off-chain code does not need to have the same performance and security standards as on-chain code.

Why is it necessary to decompose our code? Because both on-chain and off-chain code have inherent advantages and limitations. On-chain code ensures integrity but it is expensive, because all the data in the blockchain has to be synchronized and validated in all the nodes. Which will change after Mithril goes live. And also, all code that runs on the blockchain has to be paid for. Because the script's integrity is crucial, we store it in our cryptographically immutable blockchain to prevent bad actors from modifying it. Off-chain code does not ensure integrity but has access to the user's wallet and resources. For this reason, the user's signature is required when we are constructing a transaction that spends the user's funds. Let us look now at the compiling pipeline our code has to go through to become something a node can run.



We start with PlutusTx which is the high-level language developers use to code the smart contract. When we compile this code, it goes through several steps. First it gets transformed to GHC core which is part of the GHC compilation pipeline. Then this gets compiled to the Plutus intermediate representation or PIR. From now on we are independent of GHC and Haskell and have total control of the compilation pipeline. Then we transform it to Plutus core. This serves as an intermediate form that the compiler uses to statically check the program. And finally, we get to untyped Plutus core which is what the node actually runs to validate the transaction. This language is hard to code in directly and it is usually stored in binary representation. It is wise to break down bit compilation pipelines by introducing intermediate languages to ensure no step is too large and to test each step independently. You could theoretically use a different high-level language if you have a compiler that compiles to any of the Plutus languages. That is what members of the community are doing, by replacing parts or all of the compilation pipeline. You can code validators in pre-existing languages like typescript, embedded domain specific languages like plutarch or completely new languages like Aiken.

Cardano validators have access only to the information related to the transactions we are making, which is the transaction context, all the inputs and outputs of the transaction and the redeemer. If you want to consume a specific UTXO, access a specific datum or use a script there are many ways to find them. One option is to run your own node or you can have your own local database that is synchronized with a local or remote node usually through a chain indexer which follows the blockchain and stores its information in databases. Examples of chain indexers tools are db-sync, Kupo, Scrolls and Carp. These tools can also be used to find and extract datum information that was attached to a transaction. You can also use third-party tools that run their own nodes and databases as e.g., Blockfrost. What you will use will probably depend on your project needs for speed and security. Once you have all the information you need you can build, sign and submit a transaction. When building the transaction most of the tools do balancing of transactions automatically in the background. That means that the input funds equal the output funds. With the *cardano-cli transaction build-raw* command balancing can be also done manually. Every transaction needs to be signed by at least one key, because it spends at least one UTXO. For UTXOs sitting at payment addresses it is checked whether the user is allowed to spend this UTXO, and for UTXOs sitting at script addresses this check can also possibly be performed, so in both cases the signature has to be present. When submitting the transaction multiple checks are performed in two phases. In phase one it is checked whether the transaction is built correctly and can be added to the ledger. For example, it is checked whether the UTXO inputs actually exist on the ledger, if the correct amounts of fees are paid and also the correct amount of collateral is provided. Then also if all the hashes are correct and all the data required for executing your Plutus scripts is included.

If any of the checks from phase one fails the transaction is rejected without being included in a block and no fees are charged. This is work that the node does not get compensated for. If this phase succeeds the check in phase two is performed which is the execution of the validation script. In one transaction also multiple validation scripts can be runed. If all of the scripts validate the transaction actions then the node will add the transaction to the ledger. If just one of the scripts fails the entire transaction fails. In this case the collateral is collected by the node. Determinism ensures whenever a transaction is accepted it will have predictable effects on the state of the ledger. Applying a transaction in the EUTXO model is deterministic. The script will always return the same result given the same inputs. So, users acting in good faith should not encounter any surprises and will never lose their collateral.

4.1 Cardano CLI GUI

The Cardano CLI GUI is a program written in Python and the popular PyQt library that offers the user to construct simple transactions and interact with smart contracts. The source code and the installer files can be found on the following web page:
https://github.com/input-output-hk/cardano-cli-gui

In order to be able to use the gui one has to download and install the Cardano node. The executables from the node can be found at the following web page:
https://github.com/input-output-hk/cardano-node/releases

After that also the configuration files need to be downloaded. We will be using the preview testnet and its configuration files can be found at the following link:
https://book.world.dev.cardano.org/environments.html#preview-testnet

Once you have everything you can create a bash script with the following command:

```
cardano-node run \
 --topology topology.json \
 --database-path db \
 --socket-path node.socket \
 --host-addr 0.0.0.0 \
 --port 3001 \
 --config config.json
```

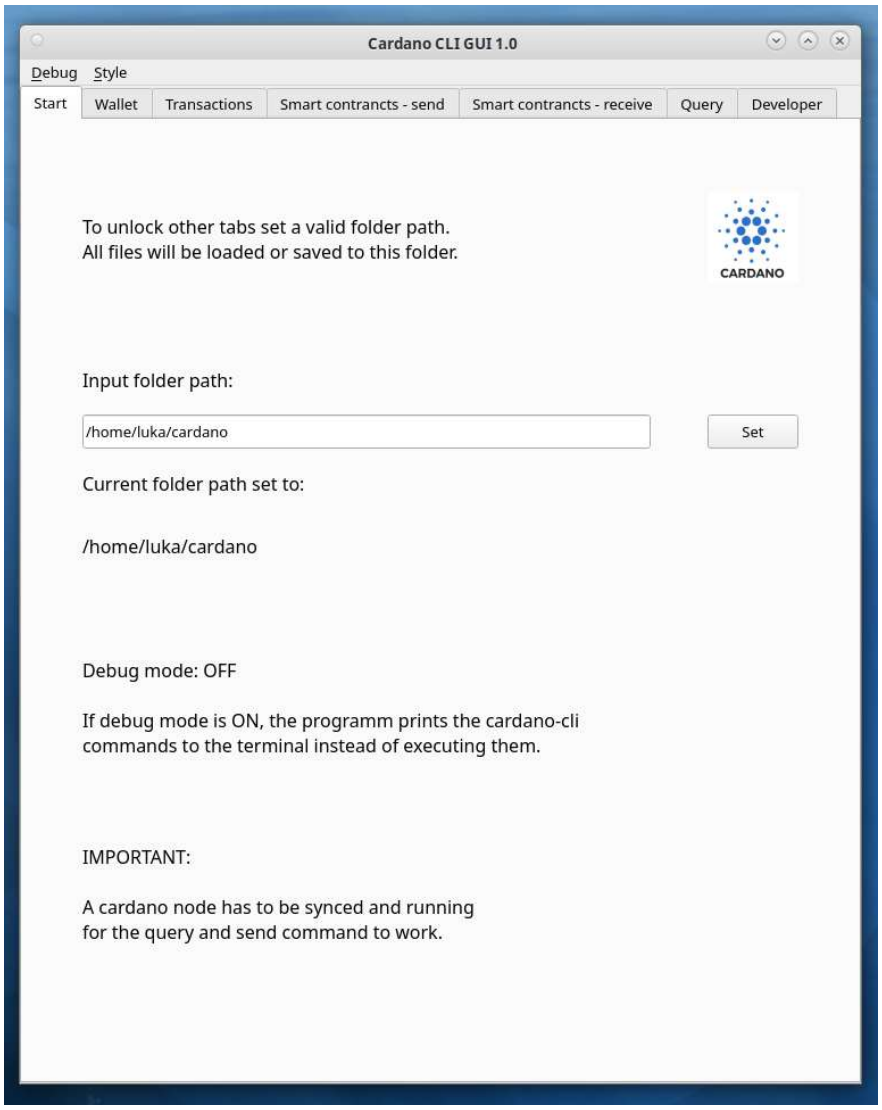
Place this bash script into the folder that contains the configuration scripts and run it. After you have started the node the node.socket file will be created. You can stop the node then and add the following line to your .bashrc file if you work on a Linux type OS:

```
export CARDANO_NODE_SOCKET_PATH="$HOME/path/to/node.socket"
```

Source the .bashrc file and start the node again. If you are using another OS you will have to add this variable to your path in another way. Now that the node is running you can clone the Cardano CLI GUI repository to your computer. Once this is done you can go to the executable folder inside of it and run the executable file for your OS. In case you are using Windows some fonts of the gui may get distorted so it is advised to use the gui on a Unix based OS. You could also run the gui from the command like with the following command:

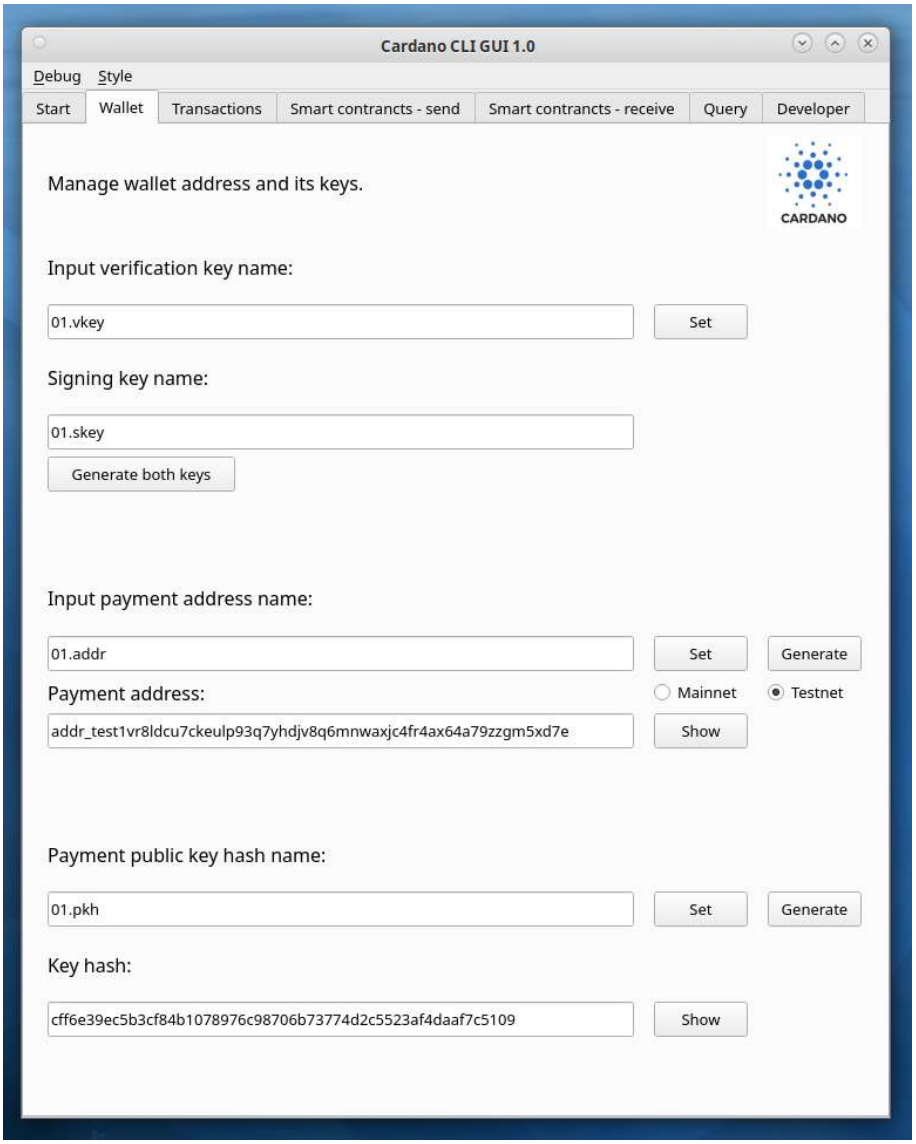
```
$ cd /path/to/cardano-cli-gui/
$ python cardano-cli-gui.py
```

In that case you need to have python 3 and PyQt 5 installed for the GUI to work. Once the gui has started you will need to input a folder path which will be used to store all the files generated and used by the gui. Once you have done this click on the **Set** button and other tabs in the gui will get unlocked.

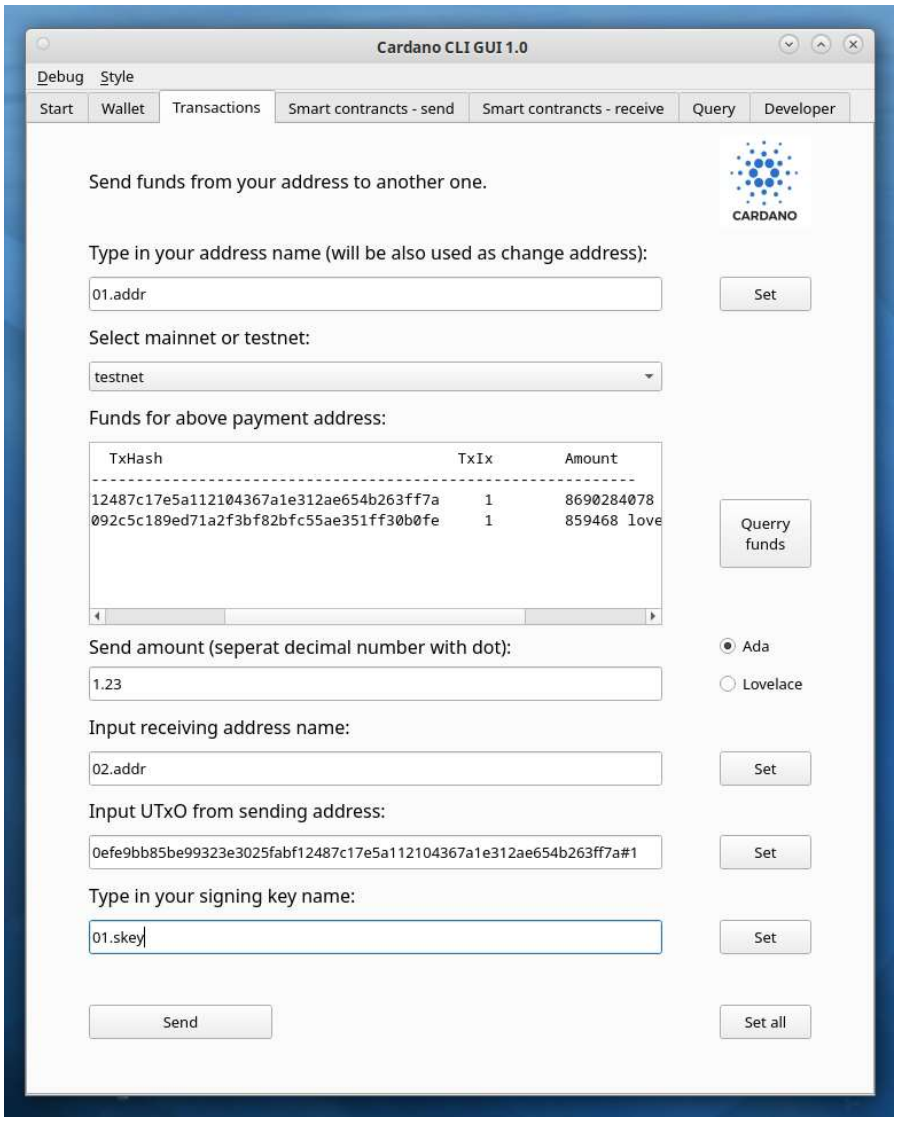


If you go now to the Query tab you can select the testnet option at the top of the tab and then click on the **Query info** button that will display some information regarding the testnet the Cardano node you are running is connected to. There you will also see the field syncProgress that tells you how much the node is synchronized to the testnet. Once this number is 100.00 you can start using the gui for constructing and submitting transactions. In the menu of the gui you have the option to switch the debug mode to ON or OFF. When the debug mode is set to ON the gui prints the cardano-cli commands to the terminal window instead of executing them.

If we go now to the Wallet tab, we see we can generate a signing and verification key pair. If we already have an existing key pair in the folder that we have set in the start tab, we can input the verification key name and click on the **Set** button. The signing key has then to have the same name as verification key in order that both are set.

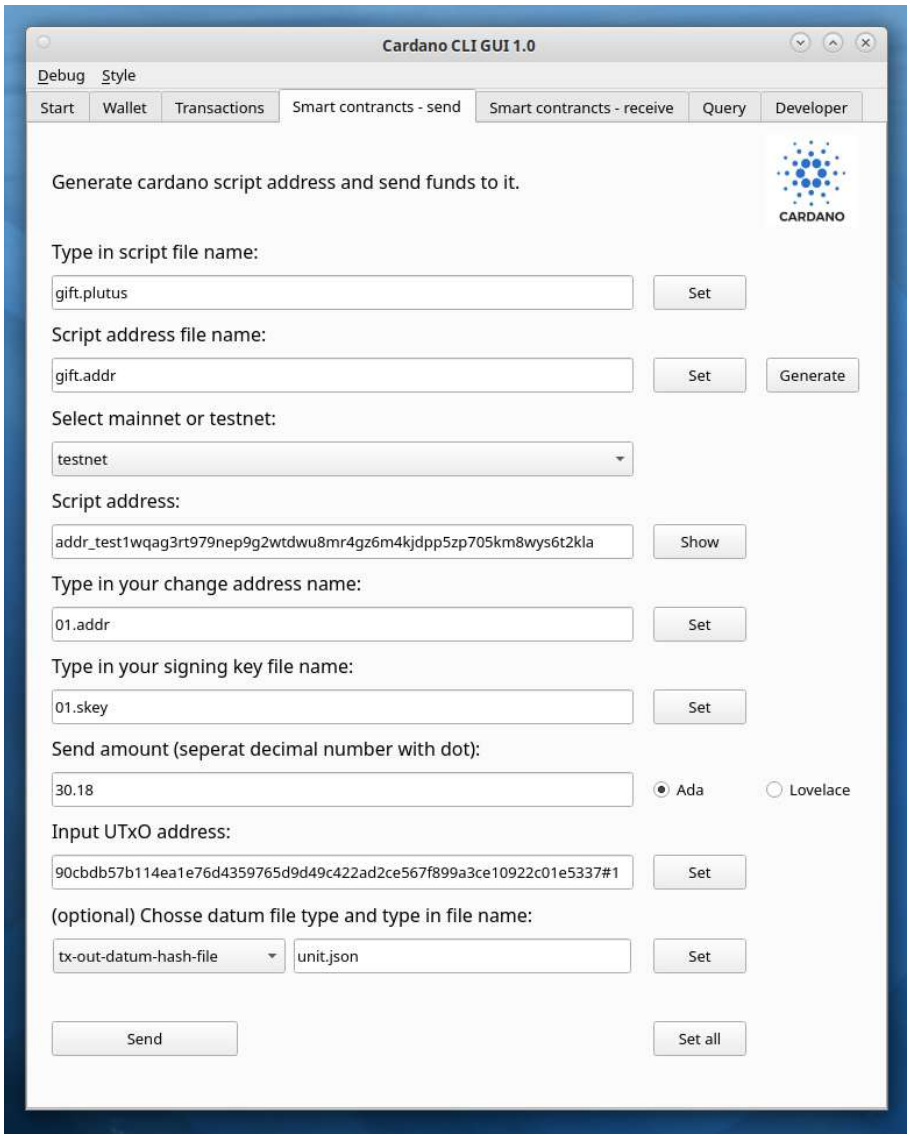


Now that the keys are set, we can click on the **Generate** button in the payment address section that will generate a new payment address belonging to those keys. We have to also select the option mainnet or testnet. Once the address is generated or manually set, we can display it by clicking on the **Show** button. Similarly, as for the payment address, we can generate or set a payment public key hash and then display it. In the transaction tab we can send some funds from one payment address to another. First, we need to set the payment address from which we are sending the funds and then select the testnet option. If we click on the **Query funds** button we can see if any funds are sitting at our address. We can do the same in the Query tab.



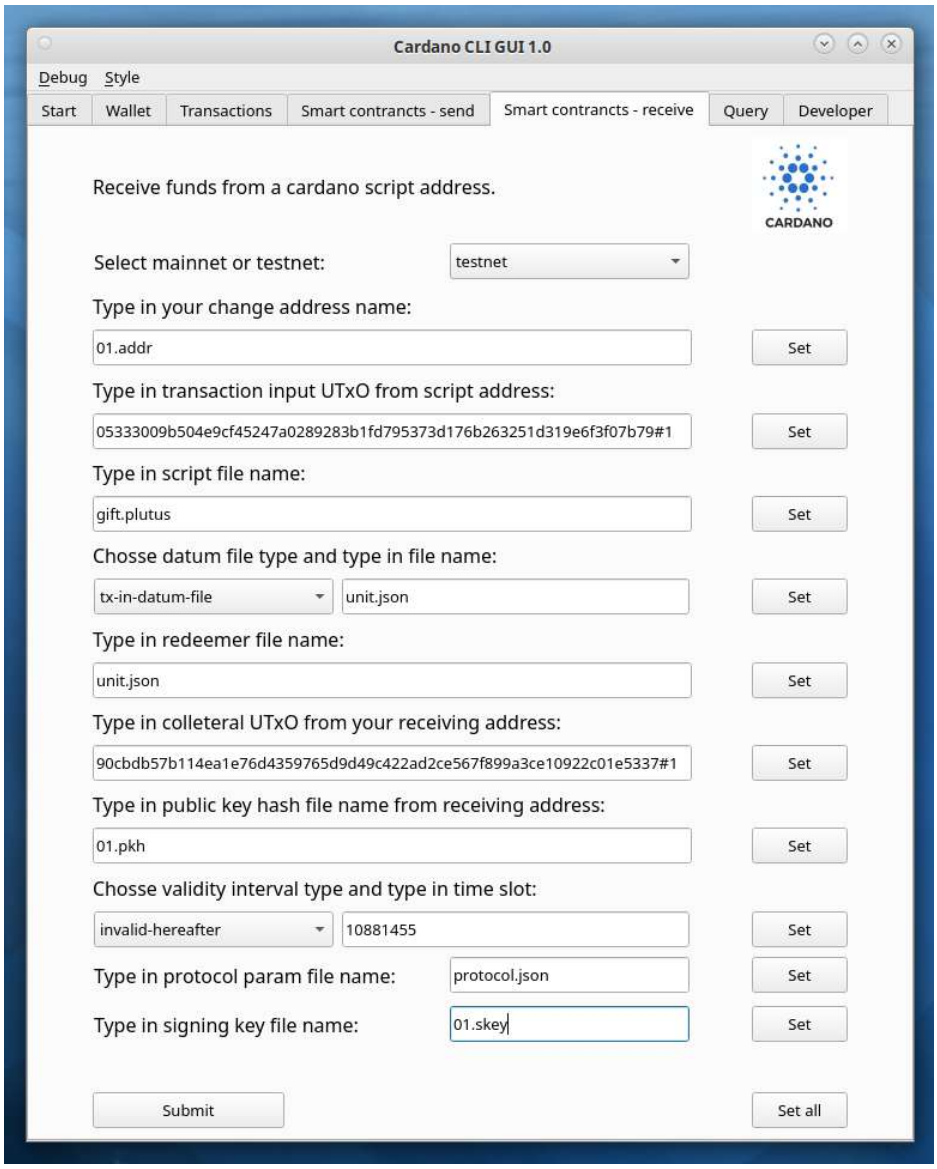
After that we input the amount, we want to send in lovelace or ada, we input the receiving address name, a transaction hash and index from the UTXO that we want to spend and our signing key belonging to our address. The GUI makes some basic checks as if the address and signing files exist, if the number input in the amount field is in correct format and if the UTXO

transaction hash is 64 characters long and if followed by a hash symbol. When we have set everything, we can click on the Send button and check the terminal for the output message that says if the transaction was successfully submitted. If it was, we can query the address where we have sent some funds and check if they have arrived. Next, we go to the Smart contract – send address. There we can send some funds to a script address. At the top if we input and set a *.plutus* file we can generate the payment address for this script file. The funds we will send will go to this address. Before we can do that, we have to select the testnet option. After that we can click on the Show button that will display the script address.



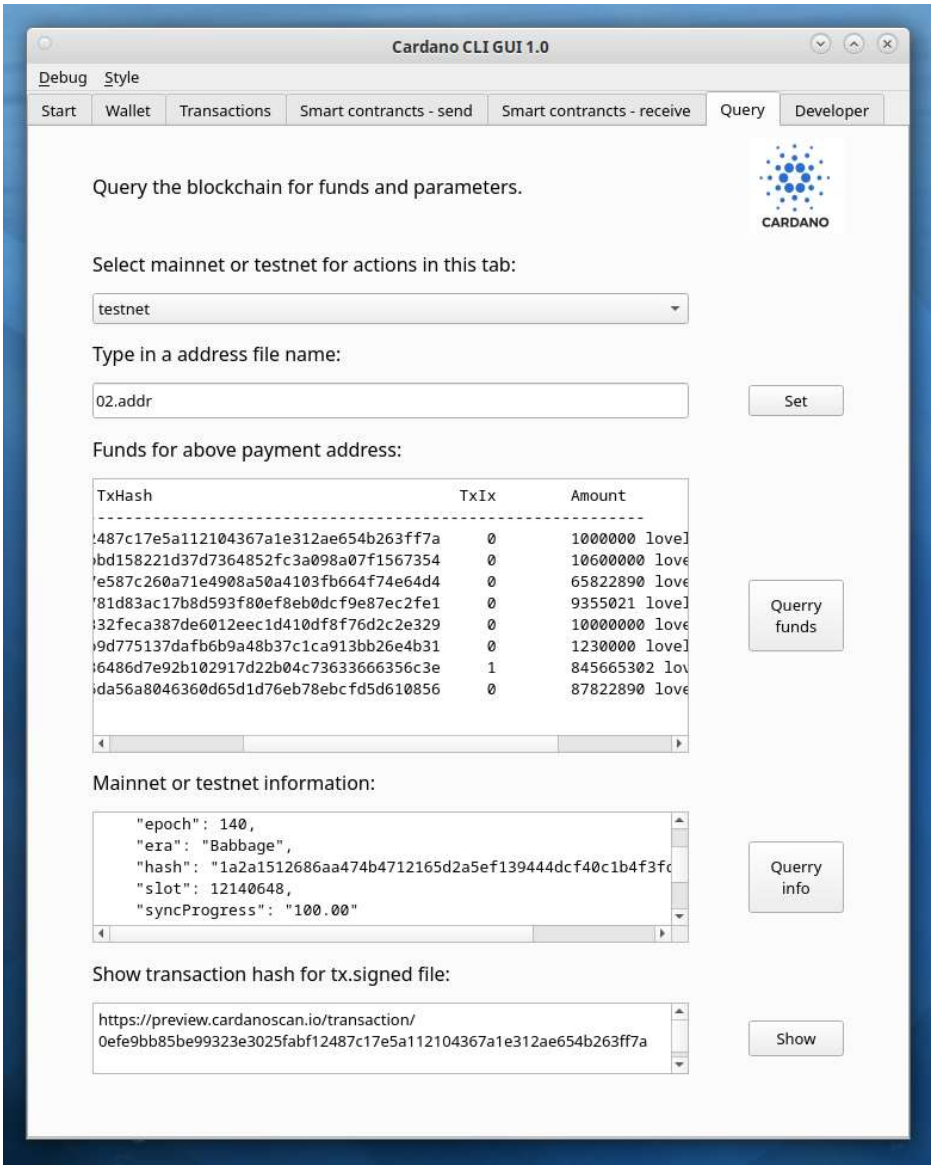
Next, we have to input our change address name, our signing key belonging to our address, the amount we want to send in ada or lovelace, the transaction hash and index of the UTXO that we want to spend and optionally we can include the datum as well, where we can pick one of several options. As of now UTXOs that have no datum or datum hash attach are not spendable

anymore. After we have set everything, we can click on **Send** and check after some time if the funds have arrived at the smart contract address. At the Smart contracts – receive tab we can submit a transaction to claim some funds from a script address. First, we select the testnet option, then we pick our change address. We type in the transaction input UTXO from the script address we want to spend. We have to add the script file and choose if we want to add the datum to the transaction. Next, we set our redeemer JSON file. We also have to specify a collateral UTXO from our address and our public key hash. In the end we set the validity interval, the protocol file and our signing key file name.



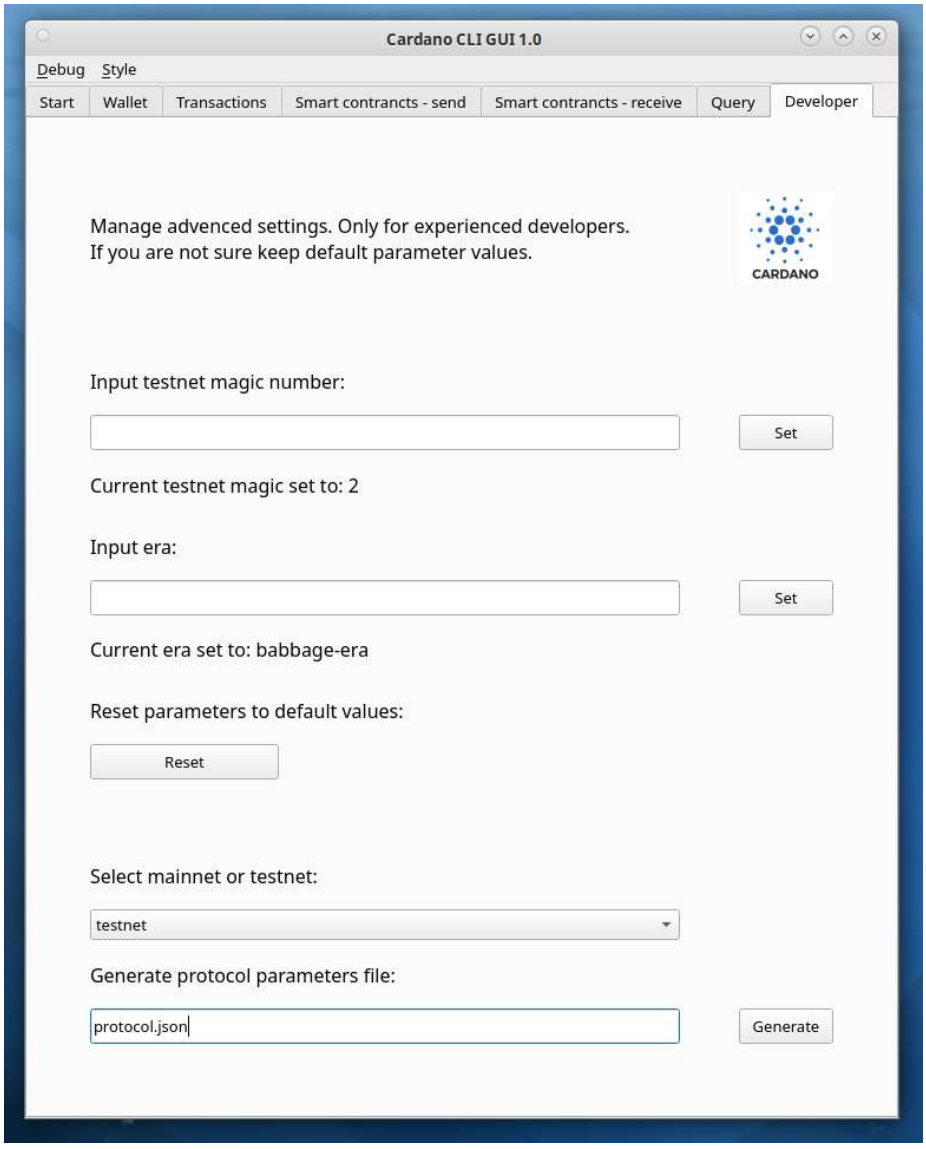
The validity interval is specified in slots and the current slot we can obtain in the Query tab if we click on the **Query info** button. The protocol parameter file can be generated at the bottom of the Developer tab, where we first select the testnet option and click on the **Generate** button.

In the Query tab beside querying funds of an address and querying information for the testnet we can also display the transaction hash for the last submitted transaction and check it on the Cardano scan link that gets displayed. The transaction hash is embedded inside the link.



In the Developer tab we can set the testnet magic number which is by default set to 2, that corresponds to the preview testnet. For the preprod (pre-production) testnet this number would have to be set to 1. The preview testnet is used for software that is still in its development phase and can possibly break or have issues. Once the release candidate is polished enough it goes to the preprod testnet where it should successfully run for some time without any issues. You can also set the current era, which is by default set to Babbage-era. As of the version 1.35.5 of the Cardano node, the only allowed era is the current era we are in, which is at the time of writing the babbage-era. This option is available if the era would change

and the GUI would not be updated in time, so then you can change the era manually. The suffix "-era" has to be appended to the name. You can also retest these to parameters to default values if you click on the **Reset** button.



We state that the limitation of the gui is that it can only consume one UTXOs and send funds to one address. However, if you run the gui in the debug mode you can copy the generated command and modify it such that you can spend multiple UTXOs or send funds to multiple addresses. Below is an example of a transaction that spends funds from multiple UTXOs belonging to a user's address and sends them all to one address.

```
cardano-cli transaction build  \
    --babbage-era  \
    --testnet-magic 2  \
    --tx-in 438356bd480c7be067589af95fa5ee08dcb469f865a4ddf791b0e89c609781b6#0  \
```

```
    --tx-in 82c8ba1008b51b4b864ed7ef7441151ac3231ed560ca7d5f3bdf20364f54ab7a#0  \
    --tx-in d51bf7746c6045064cb4b7d7c9ed581aba5b28136d843ba467b3a7273432ab4d#0  \
    --tx-in d51bf7746c6045064cb4b7d7c9ed581aba5b28136d843ba467b3a7273432ab4d#1  \
    --tx-out "addr_test1vr8ldcu7ckeulp93q7yhdjv8q6mnwaxjc4fr4ax64a79zzgm5xd7e 10000000
             lovelace" \
    --change-address addr_test1vr8ldcu7ckeulp93q7yhdjv8q6mnwaxjc4fr4ax64a79zzgm5xd7e \
    --out-file tx.body

cardano-cli transaction sign  \
    --tx-body-file tx.body  \
    --signing-key-file 02.skey  \
    --testnet-magic 2  \
    --out-file tx.signed

cardano-cli transaction submit  \
    --testnet-magic 2  \
    --tx-file tx.signed
```

We can get the other UTXO transaction hashes and indexes from the Query tab and add them to the generated command. What we also have to modify is to put the data from the --tx-out option inside quotes. We can now copy the transaction build, sign and submit commands and execute them one by one from a terminal window at the location where our files are sitting. This comes handy when we are constructing transactions that deal with ada. But if our transaction includes native tokens you have to do the balancing by hand and cannot use the gui to help yourself. There are also alternatives to the Cardano CLI which will be presented next.

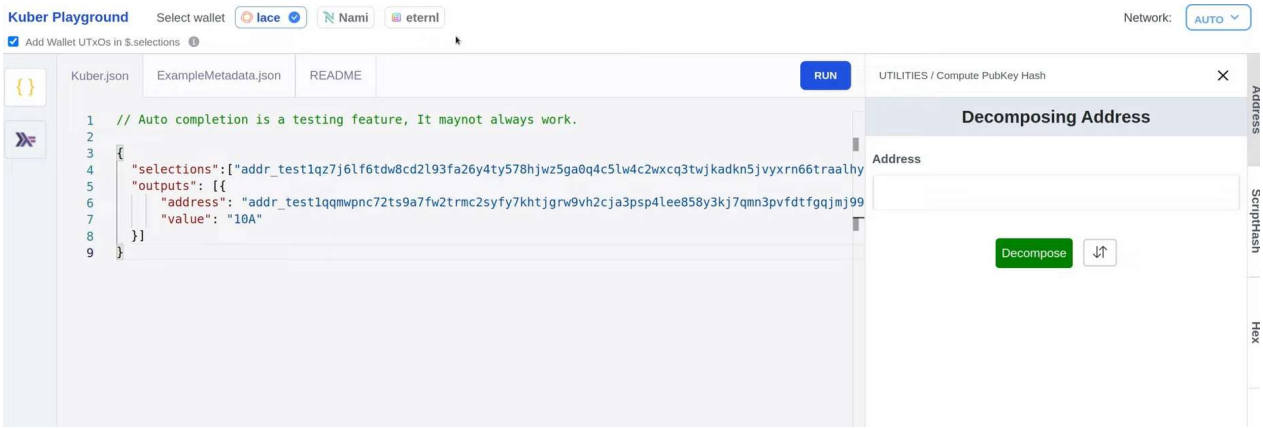
4.2 Off-chain code with Kuber

In this lecture we will talk about off-chain code using kuber. Kuber which is developed by dQuadrant can be found at the following GitHub repository:
https://github.com/dQuadrant/kuber

Kuber tries to achieve a high-level interface for constructing Cardano transactions. In contrast to the Cardano CLI where we specify precisely how a transaction is constructed in kuber we can achieve the same in a declarative way and the kuber backend will do the balancing of the transaction for us. You can build and run kuber yourself by following the instructions in the README file on the GitHub page or you can use an open API endpoint for example that Demeter exposes. In this lecture we will use the kuber playground where you do not need to install anything. Below is the link to the kuber playground:
https://dquadrant.github.io/kuber/

The kuber API itself expects JSON formatted that are constructed using a set of keys described in their JSON API reference that you can find in the GitHub page under the Docs section. In there you will find all keys and value pairs that you can use to declare your transactions. To showcase this, we will not introduce any new code but use the kuber playground together with

the vesting example of last week. The playground is a frontend for interacting with the kuber API. In it we can construct and edit JSON queries in the Kuber.json tab that first opens on the playground. We can run the queries if we click on the **Run** button. The run button will send the JSON queries to kuber which will auto-balance it and then send it back for signing. After that the frontend will prompt our wallet so that we can sign it. If we have multiple web browser wallets they are displayed at the top of the playground and we can select one of them.

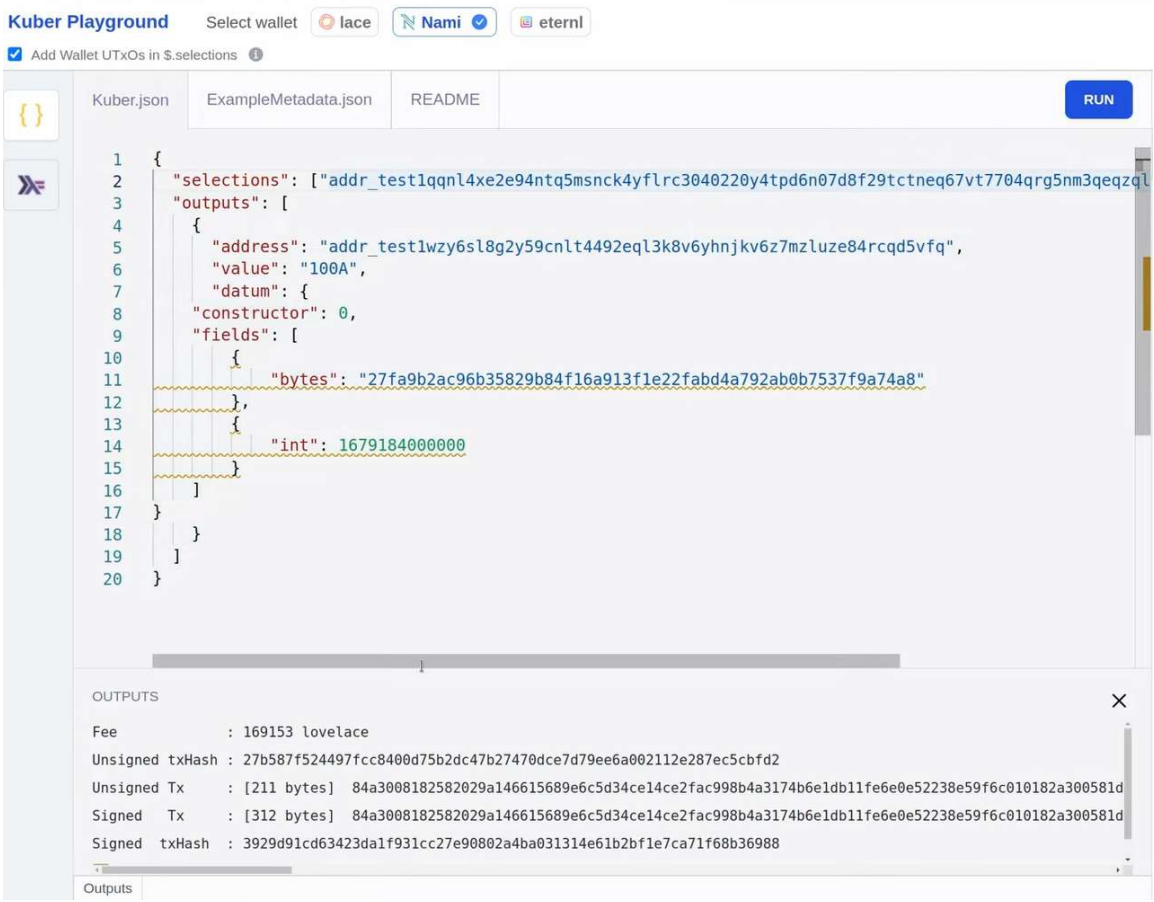


For the vesting contract we need to create an appropriate datum with the help of our VSCode editor. We open up the terminal inside our container and execute following commands:

```
/workspace/code# cabal repl Week03
Prelude> :set -XoverloadedStrings
Prelude> import Utilities
Prelude> import Plutus.V2.Ledger.Api
Prelude> import Vesting
Prelude> pkh = PubKeyHash $ toBuiltin $ bytesFromHex
               "cff6e39ec5b3cf84b1078976c98706b73774d2c5523af4daaf7c5109"
Prelude> printVestingDatumJSON pkh "2023-03-11T13:12:11.123Z"
{
    "constructor": 0,
    "fields": [
        {
            "bytes": "cff6e39ec5b3cf84b1078976c98706b73774d2c5523af4daaf7c5109"
        },
        {
            "int": 1678540331123
        }
    ]
}
```

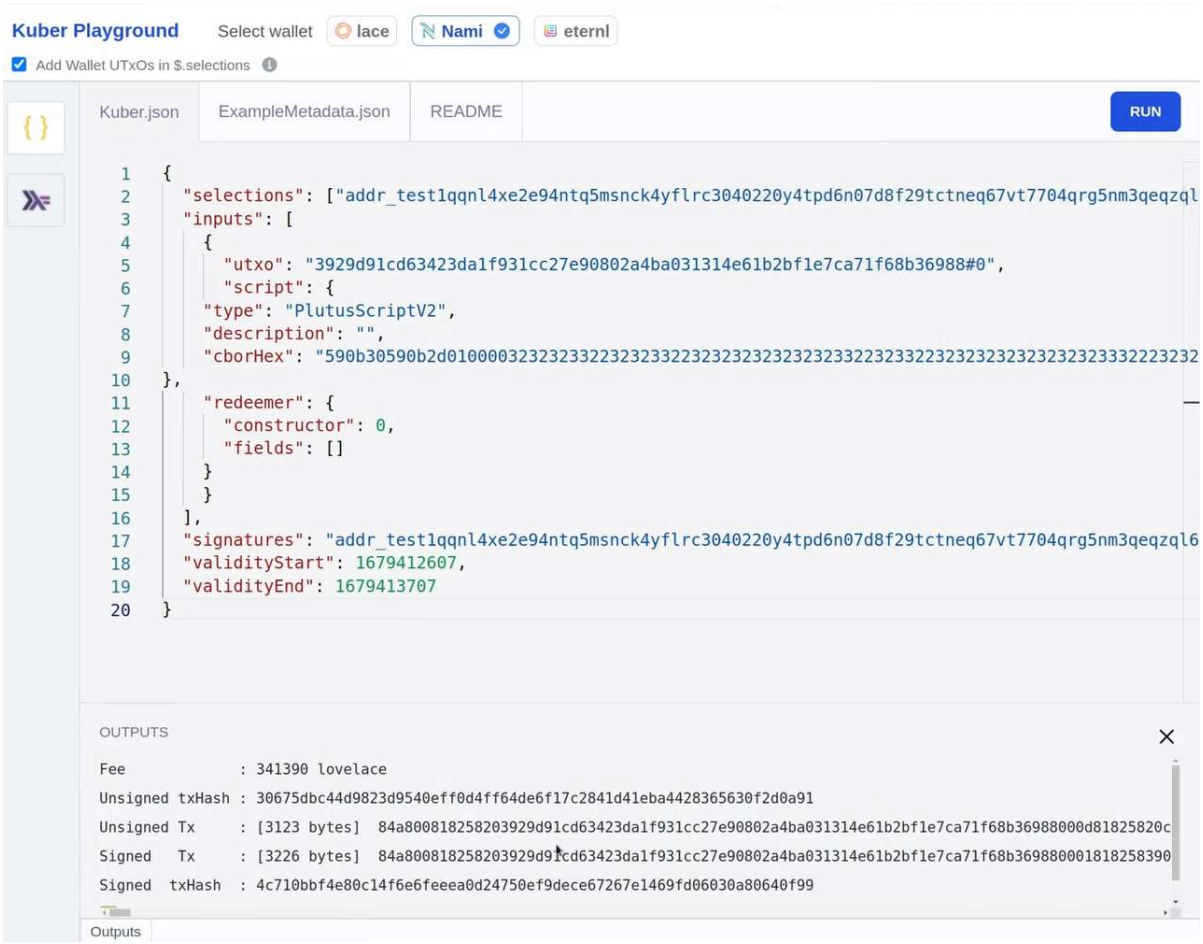
The public key hash we got with help of kuber where we can click on the Address tab on the right side and then paste in the address of e.g., our Nami wallet. When we click on the **Decompose** button, we get our public key hash. The format is hexadecimal and we need to convert it to a byte string. We combine the *toBuiltin* and *bytesFromHex* functions to accomplish this. Then we use the *printVestingDatumJSON* function to print the datum out and we can store

it in a JSON file. Now we start to construct the transaction in kuber. We first use the selection option where we add the address of our Nami wallet. We also have to select the Nami wallet at the top row of the kuber playground. Next, we specify the output option where we can decide how many output UTXOs we create at which addresses. We input the address of our vesting script here. To that address we add a value to it where say how much ada we want to send to that address and the datum we are attaching to it. The datum we can copy from the output we got in our VSCode editor that we previously constructed. If we then click on the **Run** button a pop-up window opens that prompts you to sign the transaction with your Nami wallet. At the bottom we see the output of the transaction that contains also the transaction hash of it.



We can use this transaction hash and look up the transaction and the attached datum on the https://preview.cardanoscan.io/transaction/ webpage. Next, we want to claim the vested funds. We will now create a new transaction that spends this newly created UTXO at the vesting address. We again need the selection field. After that we use the inputs field where we specify the UTXO we want to spend, the script where this UTXO is sitting at and the redeemer. For the UTXO we have to write out the transaction hash and the index. For the script which we can copy the content from the *vesting.plutus* file found in the *code/Week03/assets* folder of the

pioneer program GitHub repository. For the redeemer we can use the content of the unit.json file also found in the *code/Week03/assets* folder. Then we also add the signatures option where we input our own address that should sign the transaction. At the end we specify the validity interval and we add the *validityStart* and *validityEnd* options and add the POSIX time in seconds. We can get the current time at the https://www.epochconverter.com/ webpage.



After we click on the Run button, we again have to sign the transaction with our Nami wallet and then the transaction hash gets displayed at the bottom in the output console. We can now check the transaction on cardanoscan.io webpage if the funds were successfully claimed.

4.3 Off-chain code with Lucid

Lucid is a library that also allows you to create Cardano transactions in JavaScript and Node.js. The lucid GitHub repository can be found at the following location:
https://github.com/spacebudz/lucid

This library can be run via two runtimes that are Node.js or Deno. Because lucid code is written in JavaScript it easily allows you to also write a front-end for your DApp. Install instructions for

lucid are provided at the GitHub page. In this repository you can also find examples how to use Lucid under the Examples section. Another great learning resource is if you go to the Deno webpage where you will have access to all lucid name spaces, classes, variables, functions, interfaces and type aliases and for each of them there is a short code example shown: https://deno.land/x/lucid@0.9.1/mod.ts

You can also write lucid code by using TypeScript. This is what we will do in this example. We will be using Deno and lucid version 0.9.1. The code is located in the *lucid-vesting.ts* file inside the *Week04/lecture* folder.

```
import {
    Data,
    Lucid,
    Blockfrost,
    getAddressDetails,
    SpendingValidator,
    TxHash,
    Datum,
    UTxO,
    Address,
    AddressDetails,
} from "https://deno.land/x/lucid@0.9.1/mod.ts"
// create a seed.ts file with your seed
import { secretSeed } from "./seed.ts"

// set blockfrost endpoint
const lucid = await Lucid.new(
  new Blockfrost(
    "https://cardano-preprod.blockfrost.io/api/v0",
    "insert you own api key here"
  ),
  "Preprod"
);

// load local stored seed as a wallet into lucid
lucid.selectWalletFromSeed(secretSeed);
const addr: Address = await lucid.wallet.address();
console.log(addr);
```

First, we import the things we need. The *seed.ts* file should contain the following content:

```
export const secretSeed = "15, 18, 21 or 25 seed phrase here with spaces between the
                           words";
```

We initiate lucid as a constant where we use the method *new*, which requires a provider and a network. Provider means a way to fetch submit data to the blockchain and the network indicates which blockchain (mainnet, pre-production or preview). We can use blockfrost for the provider that runs a full node for us. We can get an API key from the blockfrost webpage so lucid can then request data from the blockchain in various formats. You can also connect lucid to a provider that you host yourself. Next, we tell lucid how to connect to a wallet which can be done in multiple ways if you look at the documentation. We select the wallet via a seed phrase that we have imported in the beginning. At the end we log the address of our wallet. To get the address we can run the following command from our VSCode terminal:

```
/workspace/code/Week04/lecture# deno run -A lucid-vesting.ts
addr_test1vr8ldcu7ckeulp93q7yhdjv8q6mnwaxjc4fr4ax64a79zzgm5xd7e
```

Next let us look at some variables we define in our TypeScript code.

```
// Define the vesting Plutus script
const vestingScript: SpendingValidator = {
    type: "PlutusV2",
    script: "590b30590b2d01000032323233223232332…",
};
const vestingAddress: Address = lucid.utils.validatorToAddress(vestingScript);

// Create the vesting datum type
const VestingDatum = Data.Object({
    beneficiary: Data.String,
    deadline: Data.BigInt,
});
type VestingDatum = Data.Static<typeof VestingDatum>;

// Set the vesting deadline
const deadlineDate: Date = new Date("2023-03-19T00:00:00Z")
const deadlinePosIx = BigInt(deadlineDate.getTime());

// Set the vesting beneficiary to our own key.
const details: AddressDetails = getAddressDetails(addr);
const beneficiaryPKH: string = details.paymentCredential.hash

// Creating a datum with a beneficiary and deadline
const datum: VestingDatum = {
    beneficiary: beneficiaryPKH,
    deadline: deadlinePosIx,
};
```

First we define the testing script that will be of type *SpendingValidator* and we can copy it from our *vesting.plutus* file. Then we compute the vesting address from the script. Next, we define the vesting datum type. To play around with the types we start a deno repl:

```
/workspace/code/Week04/lecture# deno repl
> import * as L from "https://deno.land/x/lucid@0.9.1/mod.ts"
> L.data
```

The last command gives us a list of possible types that we can use to construct the datum. After that we can create a specific datum and we first define the datum type. Then we compute the deadline variable where we use a date that has already passed. We convert the ISO UTC time to POSIX time. We compute the address details and define the beneficiary's payment public key hash. At the end we define the specific datum that we will use in the contract. Next, we try to construct the producing transaction and sign it.

```
// An asynchronous function that sends an amount of Lovelace to the script with
the above datum.
async function vestFunds(amount: bigint): Promise<TxHash> {
    const dtm: Datum = Data.to<VestingDatum>(datum,VestingDatum);
    const tx = await lucid
      .newTx()
      .payToContract(vestingAddress, { inline: dtm }, { lovelace: amount })
      .complete();
    const signedTx = await tx.sign().complete();
    const txHash = await signedTx.submit();
    return txHash
}

console.log(await vestFunds(100000000n));
```

The classes used to manage a transaction are called *Tx, TxComplete* and *TxSigned*. To construct a transaction, we use the *Lucid.newTx()* function, which returns a variable of *Tx* type. Then we use the *payToContract* and *complete* functions of the *Tx* class. In the first function we input the address to which we are sending the funds, the amount we are sending and the datum that we inline. The second function returns a variable of type *TxComplete*. After that we can use the functions *sign* and *complete* from the *TxComplete* class. The last one returns a variable of type *TxSigned*. Now we can use the function *submit* from the *TxSigned* class to submit the transaction which returns a transaction hash that we return at the end of the *vestFunds* function. Now we can look at the function that claims the vesting funds.

```
async function claimVestedFunds(): Promise<TxHash> {
    const dtm: Datum = Data.to<VestingDatum>(datum,VestingDatum);
```

```
    const utxoAtScript: UTxO[] = await lucid.utxosAt(vestingAddress);
    const ourUTxO: UTxO[] = utxoAtScript.filter((utxo) => utxo.datum == dtm);

    if (ourUTxO && ourUTxO.length > 0) {
        const tx = await lucid
            .newTx()
            .collectFrom(ourUTxO, Data.void())
            .addSignerKey(beneficiaryPKH)
            .attachSpendingValidator(vestingScript)
            .validFrom(Date.now()-100000)
            .complete();

        const signedTx = await tx.sign().complete();
        const txHash = await signedTx.submit();
        return txHash
    }
    else return "No UTxO's found that can be claimed"
}

console.log(await claimVestedFunds());
```

First, we define the datum. Then we lookup which UTXOs reside at our script address. Lucid uses Blockfrost for this because we have initialized lucid with it. After that we filter out the UTXOs that contain our datum. Then we check that our list of filtered UTXOs exists and that it is not empty. If yes, we construct the transaction. We specify which UTXOs we want to spend, then add the public key hash of the beneficiary, attach the validator script, define the validity interval and in the end sign and submit the transaction. The function returns the transaction hash in case we have found a UTXO with our datum or a message that says no UTXO with the specified datum could be claimed at the vesting address. We can run the entire *lucid-vesting.ts* file from the container terminal in VSCode.

```
/workspace/code/Week04/lecture# deno run -A lucid-vesting.ts
```

After the transactions are completed, we can use the transaction hashes that we output to the console and check on the cardanoscan.io webpage details of the transactions.

4.4 Homework

In the homework we provide two validator functions for which you have to first figure out what they do and then write the off-chain code with the tool of your choice that covers all possible interactions with the smart contracts. The second contract is a parameterized script.

```haskell
{-# LANGUAGE DataKinds         #-}
{-# LANGUAGE NoImplicitPrelude #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE TemplateHaskell   #-}
{-# LANGUAGE TypeApplications  #-}
{-# LANGUAGE TypeFamilies      #-}

module Homework1 where

import           Data.Maybe                (fromJust)
import           Plutus.V1.Ledger.Interval (contains, to)
import           Plutus.V2.Ledger.Api      (BuiltinData, POSIXTime,
                                            POSIXTimeRange, PubKeyHash,
                                            ScriptContext (scriptContextTxInfo),
                                            TxInfo (txInfoValidRange),
                                            Validator, from, mkValidatorScript)
import           Plutus.V2.Ledger.Contexts (txSignedBy)
import           PlutusTx                  (compile, unstableMakeIsData)
import           PlutusTx.Prelude          (Bool, traceIfFalse, ($), (&&), (+),
                                            (||))
import           Prelude                   (IO, String)
import           Utilities                 (Network, posixTimeFromIso8601,
                                            printDataToJSON,
                                            validatorAddressBech32,
                                            wrapValidator, writeValidatorToFile)

-------------------------------------------------------------------------------
------------------------------------ PROMPT -----------------------------------

{-
1- Figure out what this (already finished) validator does using all the tools at
your disposal.
2- Write the off-chain code necessary to cover all possible interactions with the
validator using the off-chain tool of your choosing.
HINT: If you get stuck, take a look at Week03's lecture
-}

-------------------------------------------------------------------------------
------------------------------- ON-CHAIN / VALIDATOR --------------------------

data MisteryDatum = MisteryDatum
    { beneficiary1 :: PubKeyHash
    , beneficiary2 :: PubKeyHash
    , deadline     :: POSIXTime
    }
```

```haskell
unstableMakeIsData ''MisteryDatum

{-# INLINABLE mkMisteryValidator #-}
mkMisteryValidator :: MisteryDatum -> () -> ScriptContext -> Bool
mkMisteryValidator dat () ctx =
    traceIfFalse "Benificiary1 did not sign or to late" checkCondition1 ||
    traceIfFalse "Benificiary2 did not sign or is to early" checkCondition2
    where
        txInfo :: TxInfo
        txInfo = scriptContextTxInfo ctx

        txValidRange :: POSIXTimeRange
        txValidRange  = txInfoValidRange txInfo

        checkCondition1 :: Bool
        checkCondition1 = txSignedBy txInfo (beneficiary1 dat) &&
                          contains (to (deadline dat)) txValidRange

        checkCondition2 :: Bool
        checkCondition2 = txSignedBy txInfo (beneficiary2 dat) &&
                          contains (from (1 + deadline dat)) txValidRange

{-# INLINABLE  mkWrappedMisteryValidator #-}
mkWrappedMisteryValidator :: BuiltinData -> BuiltinData -> BuiltinData -> ()
mkWrappedMisteryValidator = wrapValidator mkMisteryValidator

validator :: Validator
validator = mkValidatorScript $$(compile [|| mkWrappedMisteryValidator ||])

-------------------------------------------------------------------------------
--------------------------------- HELPER FUNCTIONS ----------------------------

saveVal :: IO ()
saveVal = writeValidatorToFile "./assets/mistery1.plutus" validator

misteryAddressBech32 :: Network -> String
misteryAddressBech32 network = validatorAddressBech32 network validator

printMisteryDatumJSON :: PubKeyHash -> PubKeyHash -> String -> IO ()
printMisteryDatumJSON pkh1 pkh2 time = printDataToJSON $ MisteryDatum
    { beneficiary1 = pkh1
    , beneficiary2 = pkh2
    , deadline     = fromJust $ posixTimeFromIso8601 time
    }
```

```haskell
{-# LANGUAGE DataKinds             #-}
{-# LANGUAGE MultiParamTypeClasses #-}
{-# LANGUAGE NoImplicitPrelude     #-}
{-# LANGUAGE OverloadedStrings     #-}
{-# LANGUAGE ScopedTypeVariables   #-}
{-# LANGUAGE TemplateHaskell       #-}

module Homework2 where

import           Plutus.V1.Ledger.Interval (contains)
import           Plutus.V2.Ledger.Api
import           Plutus.V2.Ledger.Contexts (txSignedBy)
import           PlutusTx                  (applyCode, compile, liftCode)
import           PlutusTx.Prelude          (Bool (..), traceIfFalse, (&&), (.))
import           Prelude                   (IO)
import           Utilities                 (wrapValidator, writeValidatorToFile)

-------------------------------------------------------------------------------
------------------------------------ PROMPT -----------------------------------

{-
1- Figure out what this (already finished) validator does using all the tools at
your disposal.
2- Write the off-chain code necessary to cover all possible interactions with the
validator using the off-chain tool of your choosing.
HINT: If you get stuck, take a look at Week03's lecture.
-}

-------------------------------------------------------------------------------
------------------------------ ON-CHAIN / VALIDATOR ---------------------------

{-# INLINABLE mkParameterizedMisteryValidator #-}
mkParameterizedMisteryValidator :: PubKeyHash -> POSIXTime -> ()
                                -> ScriptContext -> Bool
mkParameterizedMisteryValidator beneficiary deadline () ctx =
    traceIfFalse "not signed by beneficiary" checkSig &&
    traceIfFalse "deadline has not passed yet" checkDeadline
    where
        txInfo :: TxInfo
        txInfo = scriptContextTxInfo ctx

        txValidRange :: POSIXTimeRange
        txValidRange  = txInfoValidRange txInfo

        checkSig :: Bool
```

```
        checkSig = txSignedBy txInfo beneficiary

        checkDeadline :: Bool
        checkDeadline = contains (from deadline) txValidRange

{-# INLINABLE  mkWrappedParameterizedMisteryValidator #-}
mkWrappedParameterizedMisteryValidator :: PubKeyHash -> BuiltinData ->
BuiltinData -> BuiltinData -> ()
mkWrappedParameterizedMisteryValidator = wrapValidator .
                                         mkParameterizedMisteryValidator

validator :: PubKeyHash -> Validator
validator beneficiary = mkValidatorScript ($$(compile
                        [|| mkWrappedParameterizedMisteryValidator ||])
                        `applyCode` liftCode beneficiary)


-------------------------------------------------------------------------------
--------------------------------- HELPER FUNCTIONS ----------------------------

saveVal :: PubKeyHash -> IO ()
saveVal = writeValidatorToFile "./assets/parameterized-Mistery.plutus" .
        validator
```

5 Native tokens

In this chapter we will talk about native tokens and how they are supported by Plutus. The idea is that you can define a minting policy that specifies under which conditions native tokens can be minted. In the extended UTXO chapter we explained every UTXO has an address and a value and possibly a datum. In the examples we have seen so far, the value was always ada. If you want native tokens to be put at a UTXO you have to explicitly create them. In the first chapter we will talk about the *Value* type in Plutus and how to work with it.

5.1 Values

The *Value* type is defined in the module *Plutus.V1.Ledger.Value*. Even if we are using Plutus V2 scripts this module has not changed and we can still use the V1 version for it.

Figure 17 - Value type

It is a Map object from a so-called currency symbol to another Map that has token names for its keys. Token name and currency symbol are just wrappers for the type *BuiltinByteString* that represents a byte string. These two byte strings define a native token or coin. The integer represents the amount of the token. There is also another type called *AssetClass.*

Figure 18 - AssetClass type

It combines the currency symbol and token name that together define an asset class which is a native token or also ada. This module also provides the variables *adaSymbol* and *adaToken* that are of type currency symbol and token name. And they are both wrappers around an empty byte string. Because the value type is a Map it can contain different tokens and amounts including ada. We can look now at how to use some of the functions from this module to create different variables of type *Value* in the repl. To construct a value, we can use the function *assetClass* that takes in a currency symbol and token name and returns a variable of type asset class. Then we can use the function *assetClassValue* that takes in an asset class and integer and returns a variable of type value. Let us define a value type variable of 100 ada.

```
Prelude> import Plutus.V1.Ledger.Value
Prelude Plutus.V1.Ledger.Value> :t assetClassValue
assetClassValue :: AssetClass -> Integer -> Value
Prelude Plutus.V1.Ledger.Value> :t assetClass
assetClass :: CurrencySymbol -> TokenName -> AssetClass
Prelude Plutus.V1.Ledger.Value> ada = assetClass adaSymbol adaToken
Prelude Plutus.V1.Ledger.Value> ada
(,"")
Prelude Plutus.V1.Ledger.Value> adaValue = assetClassValue ada 100000000
Value (Map [(,Map [("",100000000)])])
Prelude Plutus.V1.Ledger.Value> :t assetClassValueOf
assetClassValueOf :: Value -> AssetClass -> Integer
Prelude Plutus.V1.Ledger.Value> assetClassValueOf adaValue ada
100000000
```

With the *assetClassValueOf* function we can check for an asset class how many tokens are contained in a value type variable. Let us try now to work with a different token than ada.

```
Prelude Plutus.V1.Ledger.Value> :set -XOverloadedStrings
Prelude Plutus.V1.Ledger.Value> assetClassValue (assetClass "a507ff33" "MyToken") 77
Value (Map [(a507ff33,Map [("MyToken",77)])])
Prelude Plutus.V1.Ledger.Value Homework1> :i Value
type Value :: *
newtype Value
  = Value {getValue :: PlutusTx.AssocMap.Map
                         CurrencySymbol (PlutusTx.AssocMap.Map TokenName Integer)}
        -- Defined in `Plutus.V1.Ledger.Value'
instance Eq Value -- Defined in `Plutus.V1.Ledger.Value'
instance Monoid Value -- Defined in `Plutus.V1.Ledger.Value'
instance Semigroup Value -- Defined in `Plutus.V1.Ledger.Value'
instance Show Value -- Defined in `Plutus.V1.Ledger.Value'
Prelude Plutus.V1.Ledger.Value> mempty :: Value
Value (Map [])
```

Because token names and currency symbols have instances of the *IsString* class we can use literal strings to define values of them if we import the overloaded strings pragma. We also see that the value type has an instance of the monoid type class. We can use the semigroup operator <> to combine variables of type value. For the value type the *mempty* parameter is just an empty Map. Let us try now to add our ada value and the value of the custom token.

```
Prelude Plutus.V1.Ledger.Value> myAssetClass = assetClass "a507ff33" "MyToken"
Prelude Plutus.V1.Ledger.Value> myTokenValue = assetClassValue myAssetClass 77
Prelude Plutus.V1.Ledger.Value> combined = myTokenValue <> adaValue
Value (Map [(,Map [("",100000000)]),(a507ff33,Map [("MyToken",77)])])
Prelude Plutus.V1.Ledger.Value> assetClassValueOf combined myAssetClass
77
```

We see we can also extract the amount of our custom defined token from the combined value parameter. If we look at the value type again it has many instances of which one is the *Group* type class. It allows us to also subtract values. It is defined in the *PlutusTx.Monoid* module.

For subtracting value, we use the *gsub* function from this module and we can subtract for instance 10 ada from our combined value variable.

```
Prelude Plutus.V1.Ledger.Value> import PlutusTx.Monoid
Prelude Plutus.V1.Ledger.Value PlutusTx.Monoid> :i gsub
gsub :: Group a => a -> a -> a  -- Defined in `PlutusTx.Monoid'
Prelude Plutus.V1.Ledger.Value PlutusTx.Monoid> gsub combined $ assetClassValue ada
10000000
Value (Map [(,Map [("",90000000)]),(a507ff33,Map [("MyToken",77)])])
```

If we look at the *Plutus.V1.Ledger.Value* module under the Partial order operations section we see there are also other helper functions that let us compare two values. This works only for values that are comparable, such that contain the same asset class. Then in the Etc section we also have some other helper functions. With the *isZero* function we can check whether a value is zero. With the *split* function we can split a value in two parts. With the flattenValue function we can convert a value type variable into a list of triples that contain the currency symbol, the token name and the amount of the tokens.

```
Prelude Plutus.V1.Ledger.Value> flattenValue combined
[(,"",100000000),(a507ff33,"MyToken",77)]
```

Let us now explain why we need both the currency symbol and the token name to identify a native token. We said before the native tokens can be only created with minting scripts. And the currency symbol is actually the hash of the minting script used to mint some native tokens. The purpose of minting scripts is that they say whether a transaction has the right to mint or burn tokens. Because the currency symbol of ada is an empty byte string it is not the hash of any script. So, ada cannot be minted or burned. All the ada that exists comes from the genesis block or from monetary expansion in form of rewards that are paid after each epoch.

5.2 A simple minting policy

Let us recall how validation works. When we have a script address the transaction that wants to spend a UTXO at this address has to provide the script context. So far when we looked at the S*criptPurpose* type variable of the *ScriptContext* data type we were only concerned with the *Spending* constructor that takes as input a transaction reference. In the transaction information *TxInfo* data type we can specify a *Value* data type for the *txInfoMint* variable. Minting policies are triggered if this field contains a non-zero value. For each currency symbol defined in this value the corresponding minting policy is triggered. Because the currency symbol is a hash of the minting policy it is possible to link the both together. A minting policy has only two inputs, the redeemer and the context. This is because datums sit at existing UTXOs and minting scripts

are not consuming existing UTXOs, they are only producing new UTXOs. In a minting transaction the script purpose is then set to *Minting*. There might be several different minting policies triggered by a transaction if you mint multiple native tokens that have different currency symbols. And each of them gets a redeemer and transaction context provided as input. All of the minting policies contained in a transaction have to pass in order that the transaction passes, otherwise it fails. Let us look now at an example of a minting policy.

```
{-# LANGUAGE DataKinds         #-}
{-# LANGUAGE NoImplicitPrelude #-}
{-# LANGUAGE TemplateHaskell   #-}

module Free where

import           Plutus.V2.Ledger.Api (BuiltinData, CurrencySymbol,
                                       MintingPolicy, ScriptContext,
                                       mkMintingPolicyScript)
import qualified PlutusTx
import           PlutusTx.Prelude     (Bool (True))
import           Prelude              (IO)
import           Utilities            (currencySymbol, wrapPolicy,
                                       writePolicyToFile)

{-# INLINABLE mkFreePolicy #-}
mkFreePolicy :: () -> ScriptContext -> Bool
mkFreePolicy () _ = True

{-# INLINABLE mkWrappedFreePolicy #-}
mkWrappedFreePolicy :: BuiltinData -> BuiltinData -> ()
mkWrappedFreePolicy = wrapPolicy mkFreePolicy

freePolicy :: MintingPolicy
freePolicy = mkMintingPolicyScript $$(PlutusTx.compile
                                        [|| mkWrappedFreePolicy ||])

-------------------------------------------------------------------------------
--------------------------------- HELPER FUNCTIONS ----------------------------

saveFreePolicy :: IO ()
saveFreePolicy = writePolicyToFile "assets/free.plutus" freePolicy

freeCurrencySymbol :: CurrencySymbol
freeCurrencySymbol = currencySymbol freePolicy
```

As for normal validators we first define a Haskell function *mkFreePolicy* that represents the policy. It takes in a unit redeemer and script context and always evaluates to true. It represents a minting policy that allows all sorts of minting and burning for the currency symbol given by this policy. We have to convert this typed minting policy to an untyped version and we do this with the helper function `wrapPolicy` that is defined in the utilities module. This function is very similar to the `wrapValidator` function just that it does not take in a datum. Then we need to compile this policy and we use the `mkMintingPolicyScript` function to accomplish this, from the `Plutus.V2.Ledger.Api` module. We also need to add the inlinable pragmas to the policy and wrapped policy function. In the end there are two helper functions. One serializes the resulting policy and saves it to the hard drive. The other computes the currency symbol of the script. The helper function `writePolicyToFile` is defined in the *Serialize* module.

```
Prelude Free> freeCurrencySymbol
79dc2cb93b706af32fe1ef3b3fb014b98ef83be6b5c1a0c6e9aa8f83
```

The token name can be arbitrarily picked. If someone would offer to sell you tokens with this currency symbol you should not pay any money for them since you can mint them yourself for free. We can now look at the off-chain code that is stored in the file *lucid-free.ts*.

```
import {
    Lucid,
    Blockfrost,
    Address,
    MintingPolicy,
    PolicyId,
    Unit,
    fromText,
    Data
} from "https://deno.land/x/lucid@0.9.1/mod.ts"
import { blockfrostKey, secretSeed } from "./secret.ts"

function readAmount(): bigint {
    const input = prompt("amount: ");
    return input ? BigInt(Number.parseInt(input)) : 1000000n;
}

const freePolicy: MintingPolicy = {
    type: "PlutusV2",
    script:
"5830582e0100003232223200533333573466e1cd55ce9baa0024800080148c98c8014cd5ce2490350
54310000500349848005"
};
```

```
// set blockfrost endpoint
const lucid = await Lucid.new(
    new Blockfrost(
        "https://cardano-preprod.blockfrost.io/api/v0",
        blockfrostKey
    ),
    "Preprod"
);
```

From the *secret.ts* file we import our blockfrost key and the seed passphrase. The *readAmount* function asks the user for a number of tokens that you want to mint. Then we define the policy script variable. Next, we initialize lucid with our blockfrost API key. We connect to the pre-production network in this example.

```
// load local stored seed as a wallet into lucid
lucid.selectWalletFromSeed(secretSeed);
const addr: Address = await lucid.wallet.address();
console.log("own address: " + addr);

const policyId: PolicyId = lucid.utils.mintingPolicyToId(freePolicy);
console.log("minting policy: " + policyId);

const unit: Unit = policyId + fromText("PPP Free");

const amount: bigint = readAmount();

const tx = await lucid
    .newTx()
    .mintAssets({[unit]: amount}, Data.void())
    .attachMintingPolicy(freePolicy)
    .complete();

const signedTx = await tx.sign().complete();
const txHash = await signedTx.submit();
console.log("tid: " + txHash);
```

In the second part of the code, we connect our wallet with the seed phrase and we log our own address. Then we compute the policy ID with the lucid function *mintingPolicyToId*. In the *unit* variable we see lucid's way to represent asset classes. It is a concatenation of the currency symbol and the hexadecimal representation of the token name. The length of the token name is restricted to 32 bytes. If you use token names in the Cardano CLI they are also supposed to be hex encoded. And the *fromText* helper function does that encoding. Then we read the amount from the user and construct the transaction. For constructing the transaction, we use the

mintAssets function that takes in our unit variable and the amount of the tokens we want to mint and then also the redeemer that can be for us of type void. Then we also attach the minting policy. After the transaction is constructed, we sign and submit it and also log the transaction hash to the console. We can execute this code with the following command:

```
/workspace/code/Week05# deno run -A lucid-free.ts
```

If we input a negative amount of the tokens we burn the tokens, which have to exist.

5.3 A More realistic minting policy

We will now look at a more realistic example of a minting policy. Our condition will be that only the owner of a specific public key that signs the transaction is allowed to mint or burn tokens. You can imagine the key is associated with some project or company that takes the role of a central bank in traditional finance which is allowed to mint and burn fiat currencies. We will call that module Signed and our minting policy will take in an additional parameter which will be a public key hash, so it will be a parameterized policy. Let us look at the code.

```
{-# LANGUAGE DataKinds         #-}
{-# LANGUAGE NoImplicitPrelude #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE TemplateHaskell   #-}

module Signed where

import           Plutus.V2.Ledger.Api      (BuiltinData, CurrencySymbol,
                                            MintingPolicy, PubKeyHash,
                                            ScriptContext (scriptContextTxInfo),
                                            mkMintingPolicyScript)
import           Plutus.V2.Ledger.Contexts (txSignedBy)
import qualified PlutusTx
import           PlutusTx.Prelude          (Bool, traceIfFalse, ($), (.))
import           Prelude                   (IO, Show (show))
import           Text.Printf               (printf)
import           Utilities                 (currencySymbol, wrapPolicy,
                                            writeCodeToFile, writePolicyToFile)

{-# INLINABLE mkSignedPolicy #-}
mkSignedPolicy :: PubKeyHash -> () -> ScriptContext -> Bool
mkSignedPolicy pkh () ctx = traceIfFalse "missing signature" $
                            txSignedBy (scriptContextTxInfo ctx) pkh

{-# INLINABLE mkWrappedSignedPolicy #-}
```

```
mkWrappedSignedPolicy :: BuiltinData -> BuiltinData -> BuiltinData -> ()
mkWrappedSignedPolicy pkh = wrapPolicy (mkSignedPolicy $
                                        PlutusTx.unsafeFromBuiltinData pkh)

signedCode :: PlutusTx.CompiledCode (BuiltinData -> BuiltinData ->
                                     BuiltinData -> ())
signedCode = $$(PlutusTx.compile [|| mkWrappedSignedPolicy ||])

signedPolicy :: PubKeyHash -> MintingPolicy
signedPolicy pkh = mkMintingPolicyScript $ signedCode
                   `PlutusTx.applyCode` PlutusTx.liftCode
                   (PlutusTx.toBuiltinData pkh)

-------------------------------------------------------------------------------
------------------------------- HELPER FUNCTIONS ------------------------------

saveSignedCode :: IO ()
saveSignedCode = writeCodeToFile "assets/signed.plutus" signedCode

saveSignedPolicy :: PubKeyHash -> IO ()
saveSignedPolicy pkh = writePolicyToFile (printf "assets/signed-%s.plutus" $
                                          show pkh) $ signedPolicy pkh

signedCurrencySymbol :: PubKeyHash -> CurrencySymbol
signedCurrencySymbol = currencySymbol . signedPolicy
```

We define our minting policy with the *mkSignedPolicy* function that takes in a public key hash as an additional parameter to the redeemer and script context. Note that in chapter three in the parameterized example our parameter kept its type when we compiled the validator. In this example we chose to change the additional parameter's type to `BuiltinData`. To accomplish this, we first use the `unsafeFromBuiltinData` function that converts the `BuiltinData` to the actual type. Then the *signedPolicy* function does take in a parameter of type `PubKeyHash` and we convert it to `BuiltinData` with the help of the `toBuiltinData` function. The helper function *saveSignedPolicy* allows us to write the minting policy to a Plutus script file for a given public key hash. And with the *signedCurrencySymbol* function we can also compute the currency symbol of the minting script given a concrete public key hash. The `writeCodeToFile` helper function can take arbitrary compiled code and serialize it to disk. The *saveSignedCode* function that uses this helper function produces a script serialized on disk where the parameter, the public key hash, has not been applied yet. It saves it to the *signed.plutus* file. Now we can demonstrate how to use lucid off-chain code to apply this parameter. The off-chain lucid code can be found in the *lucid-signed.ts* TypeScript file.

```
import {
    Lucid,
    Blockfrost,
    Address,
    MintingPolicy,
    PolicyId,
    Unit,
    fromText,
    Data,
    getAddressDetails,
    applyParamsToScript
} from "https://deno.land/x/lucid@0.9.1/mod.ts"
import { blockfrostKey, secretSeed } from "./secret.ts"

function readAmount(): bigint {
    const input = prompt("amount: ");
    return input ? BigInt(Number.parseInt(input)) : 1000000n;
}

// set blockfrost endpoint
const lucid = await Lucid.new(
    new Blockfrost(
        "https://cardano-preprod.blockfrost.io/api/v0",
        blockfrostKey
    ),
    "Preprod"
);

// load local stored seed as a wallet into lucid
lucid.selectWalletFromSeed(secretSeed);
const addr: Address = await lucid.wallet.address();
console.log("own address: " + addr);

const pkh: string = getAddressDetails(addr).paymentCredential?.hash || "";
console.log("own pubkey hash: " + pkh);
```

First, we import all necessary lucid classes, functions and types, our blockfrost API key and the secret seed of our wallet. Then we define the function that reads an integer amount from the user and in case of no input sets the amount to 1000000. After that we initialize lucid with blockfrost using the pre-production network. Next, we select a wallet, retrieve the address from it and log it. And in the end, we compute the public key hash and also log it. So, this script will enable the owner of the secret seed to mint and burn tokens. Then comes the code for constructing the minting transaction with the parameterized script.

```
const Params = Data.Tuple([Data.String]);
type Params = Data.Static<typeof Params>;

const signedPolicy: MintingPolicy = {
    type: "PlutusV2",
    script: applyParamsToScript<Params>(
        "59081a590817010000323232332232323232323232323323232332323232…",
        [pkh],
        Params)
};

const policyId: PolicyId = lucid.utils.mintingPolicyToId(signedPolicy);
console.log("minting policy: " + policyId);

const unit: Unit = policyId + fromText("PPP Signed");
const amount: bigint = readAmount();

const tx = await lucid
    .newTx()
    .mintAssets({[unit]: amount}, Data.void())
    .attachMintingPolicy(signedPolicy)
    .addSignerKey(pkh)
    .complete();

const signedTx = await tx.sign().complete();
const txHash = await signedTx.submit();
console.log("tid: " + txHash);
```

We first define our `Params` type which is a tuple with one list parameter. You can have more than multiple parameters in this list, which we will see in the case of NFTs. For this example, our list will contain only one parameter that is of type string which represents the public key hash. To create our minting policy we take the cbor hex from the *signed.plutus* file and we use the function *applyParamsToScript* to apply our public key hash to this script. Next, we compute the policy ID and log it. We compute our asset class that we call *Unit* and we read from the user the number of tokens we want to mint. After that we construct the transaction where we use the *mintAssets* function that takes in an asset class and the number of tokens we want to mint. It also takes in a redeemer which is in our case of type void. Then we attach the minting policy and we also add the signature of our address that corresponds to our public key hash. In the end we sign and submit the transaction and we log the transaction hash. We can mint now our tokens by running this code with the following command:

```
/workspace/code/Week05# deno run -A lucid-signed.ts
```

We can then input the number of tokens we want to mint. After some time, we minted the tokens, we will see in our wallet that the tokens with the name “PPP Signed” will arrive. If the number is negative, we burn tokens. In that case we have to have already existing tokens in our wallet. We note even though every transaction we submit in lucid will be signed with the signing key that belongs to our wallet we have to explicitly add the signature to the transaction with the `addSignerKey` function for the minting policy to accept our transaction. This was an example of a more realistic minting policy. If some other wallet would try to mint such tokens, they would have the same token name but a different currency symbol.

5.4 NFTs

The name NFT stands for non-fungible token. That means this is a token that can only exist once. If we would specify in the transaction context in the *txInfoMint* field that the forged value has an amount of one, we could still produce more of such tokens if we constructed multiple of such transactions. A second option would be to set a deadline in our policy and allow minting of tokens only before this deadline has been reached. If you then want an NFT you mint one coin before the deadline has passed. But in order to check that you minted only one token before the deadline you need something like a blockchain explorer. So, in this sense this Merry era deadline mechanism does not give us true NFTs in the sense that the currency symbol by itself guarantees that they are NFTs. Using Plutus, it is possible to mint true NFTs where if you know the policy script that corresponds to the currency symbol you can be sure that only one such coin can be in existence. So, the trick to writing such a policy script is that you need something unique on Cardano blockchain that you can refer to. And for that we use UTXOs because UTXOs can exist only once and once it is consumed as input to a transaction there will never exist the same UTXO again, because the transaction hash that defines a UTXO is unique. They are unique because every transaction needs to pay some fees, which come from a previous UTXO so by a simple induction argument we see there can never be two identical transactions since the hash of a transaction is computer from the previous transaction hashes that produced the UTXOs which will be used up in the new transaction. So, the idea is to name a specific parameter in our minting policy which is the UTXO transaction hash and id and then in the policy check that the transaction that does the minting consumes this specific UTXO. Let us look at such a minting script that we find in the NFT module.

```
{-# LANGUAGE DataKinds         #-}
{-# LANGUAGE NoImplicitPrelude #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE TemplateHaskell   #-}
```

```
module NFT where

import qualified Data.ByteString.Char8      as BS8
import           Plutus.V1.Ledger.Value     (flattenValue)
import           Plutus.V2.Ledger.Api       (BuiltinData, CurrencySymbol,
                                             MintingPolicy,
                                             ScriptContext (scriptContextTxInfo),
                                             TokenName (unTokenName),
                                             TxId (TxId, getTxId),
                                             TxInInfo (txInInfoOutRef),
                                             TxInfo (txInfoInputs, txInfoMint),
                                             TxOutRef (TxOutRef, txOutRefId,
                                                       txOutRefIdx),
                                             mkMintingPolicyScript)
import qualified PlutusTx
import           PlutusTx.Builtins.Internal (BuiltinByteString
                                             (BuiltinByteString))
import           PlutusTx.Prelude           (Bool (False), Eq ((==)), any,
                                             traceIfFalse, ($), (&&))
import           Prelude                    (IO, Show (show), String)
import           Text.Printf                (printf)
import           Utilities                  (bytesToHex, currencySymbol,
                                             wrapPolicy, writeCodeToFile,
                                             writePolicyToFile)

{-# INLINABLE mkNFTPolicy #-}
mkNFTPolicy :: TxOutRef -> TokenName -> () -> ScriptContext -> Bool
mkNFTPolicy oref tn () ctx = traceIfFalse "UTxO not consumed"   hasUTxO &&
                             traceIfFalse "wrong amount minted" checkMintedAmount
  where
    info :: TxInfo
    info = scriptContextTxInfo ctx

    hasUTxO :: Bool
    hasUTxO = any (\i -> txInInfoOutRef i == oref) $ txInfoInputs info

    checkMintedAmount :: Bool
    checkMintedAmount = case flattenValue (txInfoMint info) of
        [(_, tn'', amt)] -> tn'' == tn && amt == 1
        _                -> False
```

Our script will be parameterized by two parameters. The transaction output reference of a UTXO that we are spending in this transaction and by the token name.

The condition that we check is that the minting transaction consumes the `TxOutRef` that we provided as a parameter to the script. We achieve this with the `any` function that takes in a function that returns a bool and applies it to a list. It returns true if the input function returns true for at least one of the elements in a list. And we also check we mint only one coin with the specified token name. For this we can use the `flattenValue` function that takes in a *Value* type variable and flattens it to a list of triples. These both checks guarantee there will only ever be one coin with the currency symbol given by a concrete parameterized script. Next, we look at the functions that convert the validator function to an untyped version and compile it.

```
{-# INLINABLE mkWrappedNFTPolicy #-}
mkWrappedNFTPolicy :: BuiltinData -> BuiltinData -> BuiltinData ->
                      BuiltinData -> BuiltinData -> ()
mkWrappedNFTPolicy tid ix tn' = wrapPolicy $ mkNFTPolicy oref tn
  where
    oref :: TxOutRef
    oref = TxOutRef
        (TxId $ PlutusTx.unsafeFromBuiltinData tid)
        (PlutusTx.unsafeFromBuiltinData ix)

    tn :: TokenName
    tn = PlutusTx.unsafeFromBuiltinData tn'

nftCode :: PlutusTx.CompiledCode (BuiltinData -> BuiltinData -> BuiltinData ->
                                  BuiltinData -> BuiltinData -> ())
nftCode = $$(PlutusTx.compile [|| mkWrappedNFTPolicy ||])

nftPolicy :: TxOutRef -> TokenName -> MintingPolicy
nftPolicy oref tn = mkMintingPolicyScript $ nftCode
        `PlutusTx.applyCode` PlutusTx.liftCode (PlutusTx.toBuiltinData $
                                                getTxId $ txOutRefId oref)
        `PlutusTx.applyCode` PlutusTx.liftCode (PlutusTx.toBuiltinData $
                                                txOutRefIdx oref)
        `PlutusTx.applyCode` PlutusTx.liftCode (PlutusTx.toBuiltinData tn)
```

We now have three input parameters to our script because we have to convert the transaction hash also called transaction ID and the transaction index from `BuiltinData` to their custom data types. We then also compose the transaction ID and index to the transaction output reference which we use as an input parameter to the *`mkNFTPolicy`* validator function. Then we compile the wrapped validator function and after that define our *`nftPolicy`* function that given two concrete input parameters creates a minting policy. Now let us look at some helper function that can serialize the compiled minting policy to disk.

```
-------------------------------------------------------------------------------
------------------------------ HELPER FUNCTIONS --------------------------------

saveNFTCode :: IO ()
saveNFTCode = writeCodeToFile "assets/nft.plutus" nftCode

saveNFTPolicy :: TxOutRef -> TokenName -> IO ()
saveNFTPolicy oref tn = writePolicyToFile
    (printf "assets/nft-%s#%d-%s.plutus"
        (show $ txOutRefId oref)
        (txOutRefIdx oref) $
        tn') $
    nftPolicy oref tn
  where
    tn' :: String
    tn' = case unTokenName tn of
        (BuiltinByteString bs) -> BS8.unpack $ bytesToHex bs

nftCurrencySymbol :: TxOutRef -> TokenName -> CurrencySymbol
nftCurrencySymbol oref tn = currencySymbol $ nftPolicy oref tn
```

The *saveNFTCode* function writes the unparameterized Plutus script to disk. This is useful because we get an unparameterized script that can be then for instance used in a DApp by multiple users. The *saveNFTPolicy* function takes in a transaction output reference and a token name and writes the parameterized minting policy to a Plutus script and saves it to disk. This is useful if we only want to create one NFT and already know the UTXO that we will spend. The *nftCurrencySymbol* function computes the currency symbol of the script that is parameterized with two concrete parameters. Let us look now at the off-chain code.

```
import {
    Lucid,
    Blockfrost,
    Address,
    MintingPolicy,
    PolicyId,
    Unit,
    fromText,
    Data,
    applyParamsToScript
} from "https://deno.land/x/lucid@0.9.1/mod.ts"
import { blockfrostKey, secretSeed } from "./secret.ts"

// set blockfrost endpoint
const lucid = await Lucid.new(
```

```
    new Blockfrost(
        "https://cardano-preprod.blockfrost.io/api/v0",
        blockfrostKey
    ),
    "Preprod"
);

// load local stored seed as a wallet into lucid
lucid.selectWalletFromSeed(secretSeed);
const addr: Address = await lucid.wallet.address();
console.log("own address: " + addr);

const utxos = await lucid.utxosAt(addr);
const utxo = utxos[0];
console.log("utxo: " + utxo.txHash + "#" + utxo.outputIndex);
```

Again, we import all the necessary lucid functions, classes and types, our API key and the secret seed. Then we initialize lucid with our API key and connect to the pre-production network. Next, we compute and log our address and then we log information about the first UTXO that we get when we query them at our address. Next, comes the code that handles the minting.

```
const tn = fromText("PPP NFT");
const Params = Data.Tuple([Data.String, Data.BigInt, Data.String]);
type Params = Data.Static<typeof Params>;
const nftPolicy: MintingPolicy = {
    type: "PlutusV2",
    script: applyParamsToScript<Params>(
        "590906590903010000323322332232323232323232323232323232323232323232…",
        [utxo.txHash, BigInt(utxo.outputIndex), tn],
        Params)
};

const policyId: PolicyId = lucid.utils.mintingPolicyToId(nftPolicy);
console.log("minting policy: " + policyId);

const unit: Unit = policyId + tn;

const tx = await lucid
    .newTx()
    .mintAssets({[unit]: 1n}, Data.void())
    .attachMintingPolicy(nftPolicy)
    .collectFrom([utxo])
    .complete();
```

```
const signedTx = await tx.sign().complete();
const txHash = await signedTx.submit();
console.log("tid: " + txHash);
```

First, we define our token name, then we define our *Params* variable, that holds the data types of our parameters. Next, we can define the actual minting policy. We use the cbor hex from the *nft.plutus* script and apply our three parameters to it. Then we define our policy ID and log it. And we also define the asset class that we call *Unit*. After that we construct the transaction where we again specify what assed we want to mint. Here we say we want to mint only 1 coin. Then we attach the minting policy and after that we specify with the help of the *collectFrom* function which UTXO we want to spend when paying for the fees. At the end we sign and submit the transaction and log the transaction hash. We can now run this code with the following command:

```
/workspace/code/Week05# deno run -A lucid-nft.ts
```

If we would run this code multiple times, we would get tokens with the same name but different currency symbols. And that means that they truly are NFTs since the asset class of a token is also defined by the currency symbol that is different for each of them.

5.5 Homework

In the first homework you have to extend the signed example where the additional parameter in the minting script was the public key hash and the owner of this key had to sign the transaction. Now you have to add a second parameter of type *POSIXTime* and the added condition should be that the minting or burning happens before this deadline. This is a realistic minting policy that some projects may use, since you can fix the number of tokens you mint, because after the deadline those tokens cannot be minted again. You can also modify the off-chain code *lucid-signed.ts* such that it works with this minting policy.

```
{-# LANGUAGE DataKinds         #-}
{-# LANGUAGE NoImplicitPrelude #-}
{-# LANGUAGE TemplateHaskell   #-}

module Homework1 where

import           Plutus.V2.Ledger.Api (BuiltinData, MintingPolicy, POSIXTime,
                                        PubKeyHash, ScriptContext,
                                        mkMintingPolicyScript)
import qualified PlutusTx
import           PlutusTx.Prelude     (Bool (False), ($))
```

```
import           Utilities            (wrapPolicy)

{-# INLINABLE mkDeadlinePolicy #-}
-- This policy should only allow minting (or burning) of tokens if the owner of
-- the specified PubKeyHash has signed the transaction and if the specified
-- deadline has not passed.
mkDeadlinePolicy :: PubKeyHash -> POSIXTime -> () -> ScriptContext -> Bool
mkDeadlinePolicy _pkh _deadline () _ctx = False -- FIX ME!

{-# INLINABLE mkWrappedDeadlinePolicy #-}
mkWrappedDeadlinePolicy :: PubKeyHash -> POSIXTime -> BuiltinData ->
                           BuiltinData -> ()
mkWrappedDeadlinePolicy pkh deadline = wrapPolicy $ mkDeadlinePolicy pkh deadline

deadlinePolicy :: PubKeyHash -> POSIXTime -> MintingPolicy
deadlinePolicy pkh deadline = mkMintingPolicyScript $
    $$(PlutusTx.compile [|| mkWrappedDeadlinePolicy ||])
        `PlutusTx.applyCode` PlutusTx.liftCode pkh
        `PlutusTx.applyCode` PlutusTx.liftCode deadline
```

For the second homework change the NFT policy such that the token name can no longer be freely chosen but has to be the empty byte sting.

```
{-# LANGUAGE DataKinds         #-}
{-# LANGUAGE NoImplicitPrelude #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE TemplateHaskell   #-}

module Homework2 where

import           Plutus.V2.Ledger.Api (BuiltinData, MintingPolicy,
                                       ScriptContext, TokenName, TxOutRef,
                                       mkMintingPolicyScript)
import qualified PlutusTx
import           PlutusTx.Prelude     (Bool (False), ($), (.))
import           Utilities            (wrapPolicy)

{-# INLINABLE mkEmptyNFTPolicy #-}
-- Minting policy for an NFT, where the minting transaction must consume the
-- given UTxO as input and where the TokenName will be the empty ByteString.
mkEmptyNFTPolicy :: TxOutRef -> () -> ScriptContext -> Bool
mkEmptyNFTPolicy _oref () _ctx = False -- FIX ME!

{-# INLINABLE mkWrappedEmptyNFTPolicy #-}
mkWrappedEmptyNFTPolicy :: TxOutRef -> BuiltinData -> BuiltinData -> ()
```

```
mkWrappedEmptyNFTPolicy = wrapPolicy . mkEmptyNFTPolicy

nftPolicy :: TxOutRef -> TokenName -> MintingPolicy
nftPolicy oref tn = mkMintingPolicyScript $ $$(PlutusTx.compile
                                               [|| mkWrappedEmptyNFTPolicy ||])
                                               `PlutusTx.applyCode`
                                               PlutusTx.liftCode oref
```

In both homework examples the template code uses the typed version when compiling the code which is the standard way but is less well suited for off-chain code with Lucid.

6 Testing smart contracts

In this chapter we will go through different methods of testing smart contracts. Because smart contracts can hold millions worth of assets, we want to make sure that they work as we expect them to do. Unit and property are a big part of testing smart contracts and this chapter is all about that. We will start with an introduction to the state monad because the Plutus simple model library that we will use to test our validators uses it. Then we will learn how to use this library to unit test our validators and how to combine it with the QuickCheck library to property test our validator. After that we are going to show how to write the same tests by using lucid.

6.1 The state monad in practice

We will be using fairly simple Haskell to demonstrate how a state monad can be used to keep track of a blockchains state. The Plutus simple model library also uses a state monad to keep track of the state of the blockchain. The code presented in this chapter is used to demonstrate in a simplified manner how the actual testing library models the blockchain. The first thing we do in the code is create the types that allow us to model a blockchain.

```
module State where

import           Control.Monad.State (State, get, put, runState)

------------------------------------------------------------------------------
----------------------------- HELPER FUNCTIONS/TYPES --------------------------

-- Mock UTxO type
data UTxO = UTxO { owner :: String , value :: Integer }
    deriving (Show, Eq)

-- Mock blockchain type
newtype Mock = Mock { utxos :: [UTxO] }
    deriving (Show, Eq)

-- Initial blockchain state
initialMockS :: Mock
initialMockS = Mock [ UTxO "Alice" 1000 ]
```

The `UTxO` data type contains the owner of the UTXO and the value sitting at the UTXO which in our case is just an integer number. Then we create the `Mock` blockchain type that is just a wrapper around a list of UTXOs. So, our mock chain is just a list of UTXOs. At the end we create an initial state of our blockchain where we say that Alice has a 1000 ada.

Next, we create a function that allows us to transfer some value from one user to another and then use it in a couple of transactions. First, we show the solution without using a state monad.

```
-------------------------------------------------------------------------------
------------------------------ WITHOUT STATE MONAD ----------------------------

sendValue :: String -> Integer -> String -> Mock -> (Bool, Mock)
sendValue from amount to mockS =
    let senderUtxos = filter ((== from) . owner) (utxos mockS)
        blockchainWithoutSenderUtxos = filter ((/= from) . owner) (utxos mockS)
        totalSenderFunds = sum (map value senderUtxos)
        receiverUtxo = UTxO to amount
        senderChange = UTxO from (totalSenderFunds - amount)
    in if totalSenderFunds >= amount
        then (True, Mock $ [receiverUtxo] ++ [senderChange] ++
                           blockchainWithoutSenderUtxos)
        else (False, mockS)


multipleTx :: (Bool, Mock)
multipleTx =
    let (isOk,  mockS1) = sendValue "Alice" 100 "Bob"   initialMockS
        (isOk2, mockS2) = sendValue "Alice" 300 "Bob"   mockS1
        (isOk3, mockS3) = sendValue "Bob"   200 "Rick"  mockS2
    in (isOk && isOk2 && isOk3, mockS3)
```

The *sendValue* function takes in following parameters in the specified order:

- The name of the user that sends the funds.
- The amount of value we want to send.
- The name of the user that receives the funds.
- The current state of the blockchain before our transaction.

This function returns a tuple with a Boolean that says if the transaction was successful and the new state of the mock blockchain. This state will be the same as before if the transaction fails. In the body of the function, we first compute all the senders UTXOs from the blockchain and then all the UTXOs on the blockchain without the sender ones. Then we compute the total value that the sender owns, the UTXO that the receiver will get and the change amount that will go back to the sender. Next, we check if the sender has enough funds to create the transaction and in case he does, we return the updated mock blockchain with two newly created UTXOs. If he does not, we return the original state of the blockchain. In the *multipleTx* function we create three transactions and return the final result of the *sendValue* function. We can now look in the repl at the results that the *multipleTx* function returns.

```
Prelude State> fst multipleTx
True
Prelude State> snd multipleTx
Mock {utxos = [UTxO {owner = "Rick", value = 200},UTxO {owner = "Bob", value =
200},UTxO {owner = "Alice", value = 600}]}
```

Let us now rewrite those the *multipleTx* and *sendValue* functions such that we use a state monad to implement them.

```
-------------------------------------------------------------------------------
------------------------------- WITH STATE MONAD ------------------------------

-- newtype State s a = State { runState :: s -> (a, s) }

sendValue' :: String -> Integer -> String -> State Mock Bool
sendValue' from amount to = do
    mockS <- get
    let senderUtxos = filter ((== from) . owner) (utxos mockS)
        blockchainWithoutSenderUtxos = filter ((/= from) . owner) (utxos mockS)
        totalSenderFunds = sum (map value senderUtxos)
        receiverUtxo = UTxO to amount
        senderChange = UTxO from (totalSenderFunds - amount)
    if totalSenderFunds >= amount
        then do
            put $ Mock $ [receiverUtxo] ++ [senderChange] ++
                         blockchainWithoutSenderUtxos
            return True
        else return False

multipleTx' :: (Bool, Mock)
multipleTx' = runState (do
    isOk  <- sendValue' "Alice" 100 "Bob"
    isOk2 <- sendValue' "Alice" 300 "Bob"
    isOk3 <- sendValue' "Bob"   200 "Rick"
    return (isOk && isOk2 && isOk3))
    initialMockS

type Run a = State Mock a
```

The *sendValue'* function the first three parameters we take in are the same. Then we return a state monad that is parameterized by the mock blockchain and a bool. Now we can use the functions `get` and `put` to get the current state of the blockchain and to set a new state. The rest of the code stays the same. In the *multipleTx'* function we use the `runState` helper function to evaluate the result of a state monad in which we defined the same transactions as in the first

example. We also provide to this function the initial state of the mock blockchain. In the end we define a type synonym for the state monad where the first parameter is fixed to the mock blockchain. Such a parameter is also used in the Plutus simple model library. If we look at the source code of this library, we can see how the actual Mock and Run types are defined:

```
data Mock = Mock
  { mockUsers :: !(Map PubKeyHash User)
  , mockAddresses :: !(Map Address (Set TxOutRef))
  , mockUtxos :: !(Map TxOutRef TxOut)
  -- ^ Since 0.4, reference script UTxOs are also included.
  , mockDatums :: !(Map DatumHash Datum)
  , mockStake :: !Stake
  , mockTxs :: !(Log TxStat)
  , mockConfig :: !MockConfig
  , mockCurrentSlot :: !Slot
  , mockUserStep :: !Integer
  , mockFails :: !(Log FailReason)
  , mockInfo :: !(Log String)
  , mustFailLog :: !(Log MustFailLog)
  , mockNames :: !MockNames
  -- ^ human readable names. Idea is to substitute for them
  -- in pretty printers for error logs, user names, script names.
  }

-- | State monad wrapper to run blockchain.
newtype Run a = Run (State Mock a)
  deriving newtype (Functor, Applicative, Monad, MonadState Mock)
```

Source code where the Mock data type is defined: https://github.com/mlabs-haskell/plutus-simple-model/blob/main/psm/src/Plutus/Model/Mock.hs

6.2 Introduction to the Plutus simple model library

From the PPP container in VSCode we can open the documentation for the Plutus simple model library by running the following command:

```
/workspace/code/Week06# serve-testing-docs
```

Let us now look at a simple code example that uses this library. The code is contained in the file *PSM.hs* and we can find it in the *code/Week06/lecture_tests* folder.

```
{-# LANGUAGE NumericUnderscores #-}

module Main where

import           Prelude
import           Test.Tasty           (defaultMain, testGroup)
```

```
import           Control.Monad        (replicateM)
import           Plutus.Model         (Ada (Lovelace), Run, ada, adaValue,
                                       defaultBabbage, mustFail, newUser,
                                       noErrors, sendValue, testNoErrors,
                                       valueAt)
import           Plutus.V1.Ledger.Api (PubKeyHash)

--------------------------------------------------------------------------------
--------------------------------- TESTING MAIN ---------------------------------

main :: IO ()
main = defaultMain $ do
    testGroup
      "Test simple user transactions"
      [ good "Simple spend" simpleSpend
      , bad  "Not enough funds" notEnoughFunds
      ]
      where
        bad msg = good msg . mustFail
        good = testNoErrors (adaValue 10_000_000) defaultBabbage
```

The *main* function is the entry point of our code. In the beginning there is some standard code that repeats itself in every test. We have the `defaultMain` function that is part of the tasty module that transforms a tree of tests into the IO action the main needs to run. The `testGroup` function also provided by the tasty module allows us to give a name to a list of tests. This is handy when you have multiple groups of tests and you want to classify them in some way. Finally, we have the list of actual tests each with its own description and the actual test. In the "Simple spend" test we make user one sends a transaction to user two and everything should be OK. In the "Not enough funds" test user one also sends some funds to user two but user one has not enough funds to cover the transaction, so this test should fail. We can run the tests with the following command that first compiles the code and then runs the tests:

```
/workspace/code/Week06# cabal test week06-PSM
...
Running 1 test suites...
Test suite week06-PSM: RUNNING...
Test simple user transactions
  Simple spend:     OK (0.40s)
  Not enough funds: OK

All 2 tests passed (0.41s)
Test suite week06-PSM: PASS
```

You can see that in the output of the test results the descriptions of the tests are used and the OK sign indicates that the tests have passed. In the code we see that we run the tests with the `good` and `bad` functions. The `good` function is a partially applied `testNoErrors` function.

```
testNoErrors :: Value -> MockConfig -> String -> Run a -> TestTree
```

The first parameter is the value we provide to the admin user, same as in the previous chapter we will have a single UTXO in the beginning. We provide 10.000.000 coins to the `adaValue` function that takes in an integer and returns a value.

```
adaValue :: Integer -> Value
```

And we also provide the default Babbage mock configuration that configures our mock blockchain. The description of the test and the run action that the `testNoErrors` function also takes in are then provided in the list of tests. At the end it produces a variable of type TestTree. In the case of the `bad` function, we use the `mustFail` function that logs an error if everything goes well and if there is an error it will succeed. In the first case we anticipate that the test will succeed and in the second case we anticipate it will fail. Now let us look at the actual tests.

```
-------------------------------------------------------------------------------
------------------------------ HELPER FUNCTIONS -------------------------------

-- Set many users at once
setupUsers :: Run [PubKeyHash]
setupUsers = replicateM 3 $ newUser $ ada (Lovelace 1000)

-------------------------------------------------------------------------------
----------------------------- TESTING TRANSACTIONS ----------------------------

-- Function to test that a simple transaction works
simpleSpend :: Run Bool
simpleSpend = do
    -- Create 3 users and assign each 1000 lovelaces
    users <- setupUsers
    -- Give names to individual users
    let [u1, u2, u3] = users
    -- Send 100 lovelaces from user 1 to user 2
    sendValue u1 (adaValue 100) u2
    -- Send 100 lovelaces from user 2 to user 3
    sendValue u2 (adaValue 100) u3
    -- Check that all TXs were accepted without errors
    isOk <- noErrors
    -- Read user values
```

```
    vals <- mapM valueAt users
    -- Check isOk and that all users have correct values
    return $ isOk &&
           (vals == fmap adaValue [900, 1000, 1100])

-- Function to test that a transaction fails if there are not enough funds
notEnoughFunds :: Run Bool
notEnoughFunds = do
  -- Create 3 users and assign each 1000 lovelaces
  users <- setupUsers
  -- Give names to individual users
  let [u1, u2, _u3] = users
  -- Send 10.000 lovelaces from user 1 to user 2
  sendValue u1 (adaValue 10000) u2
  -- Check that all TXs were accepted without errors (should fail)
  noErrors
```

First, we have to create the helper function *setupUsers* that creates three new users. The `newUser` function creates a new user and transfers the input number of coins from the admin user to the new user. We transfer a 1000 lovelace and replicate this function three times. The function returns the run monad parameterized by a list of three public key hashes. The *simpleSpend* function is also a run monad parameterized by a Boolean that says whether the given transaction has passed or failed. All the actions in this monad influence the state of the blockchain in case they pass. In the body of this state monad, we first create our list of public key hashes that represents the three users and assign them names. The actual transaction we create with the `sendValue` function that takes in the public key hash of the persons which send the funds and the one which receives it and the lovelace value that we want to transfer. The function returns the run monad parameterized by the unit type. In the case of the `sendValue` function the success of a transaction is locked as a part of the state of the blockchain. We then use the `noErrors` action with which we check that all the transactions went well. We then also check if the final balances are the ones that we expect. With the `valueAt` function we get the value of a certain user and we use it to map over the list of public key hashes that belong to our three users. In the end we compare the list of the actual balances with the expected ones and also if all the transactions succeeded. In the *notEnoughFunds* function we again create three users and then send 10000 lovelace from user u1 to user u2. Since this is more than u1 has, this transaction should fail. And we return the result of the `noErrors` action.

6.3 Unit testing a smart contract

We will test the smart contract contained in the *NegativeRTimed.hs* file. This smart contract checks that the redeemer which is an integer number is smaller or equal to zero. And it also checks that the deadline contained in the datum, which is a custom data type, has passed.

```
{-# LANGUAGE DataKinds           #-}
{-# LANGUAGE ImportQualifiedPost #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE OverloadedStrings   #-}
{-# LANGUAGE TemplateHaskell     #-}

module NegativeRTimed where

import           Plutus.V1.Ledger.Interval (contains)
import           Plutus.V2.Ledger.Api      (POSIXTime,
                                            ScriptContext (scriptContextTxInfo),
                                            TxInfo (txInfoValidRange), from)
import qualified Plutus.V2.Ledger.Api      as PlutusV2
import           PlutusTx                  (compile, unstableMakeIsData)
import           PlutusTx.Builtins         (BuiltinData, Integer)
import           PlutusTx.Prelude          (Bool, Ord ((<=)), traceIfFalse, ($),
                                            (&&))
import           Utilities                 (wrapValidator)

-------------------------------------------------------------------------------
---------------------------------- ON-CHAIN / VALIDATOR -------------------------
------------------

newtype CustomDatum = MkCustomDatum { deadline :: POSIXTime }
unstableMakeIsData ''CustomDatum

{-# INLINABLE mkValidator #-}
mkValidator :: CustomDatum -> Integer -> ScriptContext -> Bool
mkValidator (MkCustomDatum d) r ctx = traceIfFalse "expected a negative redeemer"
                                      $ r <= 0 &&
                                      traceIfFalse "deadline not reached"
                                      deadlineReached
    where
        info :: TxInfo
        info = scriptContextTxInfo ctx

        deadlineReached :: Bool
        deadlineReached = contains (from d) $ txInfoValidRange info
```

```
{-# INLINABLE  mkWrappedValidator #-}
mkWrappedValidator :: BuiltinData -> BuiltinData -> BuiltinData -> ()
mkWrappedValidator = wrapValidator mkValidator

validator :: PlutusV2.Validator
validator = PlutusV2.mkValidatorScript $$(PlutusTx.compile
                                          [|| mkWrappedValidator ||])
```

We see we check that the validity range lies in the interval that starts with the deadline. And we also check that the redeemer is lower or equal to zero. Then we wrap the validator and compile it. Let us look now at our test that is contained in the *UTNegativeRTimed.hs* file.

```
{-# LANGUAGE DataKinds          #-}
{-# LANGUAGE NoImplicitPrelude  #-}
{-# LANGUAGE NumericUnderscores #-}

module Main where

import           Control.Monad        (mapM, replicateM, unless)
import qualified NegativeRTimed       as OnChain
import           Plutus.Model         (Ada (Lovelace), DatumMode (HashDatum),
                                       Run, Tx, TypedValidator (TypedValidator),
                                       UserSpend, ada, adaValue, currentTimeRad,
                                       defaultBabbage, logError, mustFail,
                                       newUser, payToKey, payToScript, spend,
                                       spendScript, submitTx, testNoErrors,
                                       toV2, userSpend, utxoAt, validateIn,
                                       valueAt, waitUntil)
import           Plutus.V2.Ledger.Api (POSIXTime, PubKeyHash,
                                       TxOut (txOutValue), TxOutRef, Value)
import           PlutusTx.Builtins    (Integer, mkI)
import           PlutusTx.Prelude     (Eq ((==)), ($), (&&), (.))
import           Prelude              (IO, mconcat)
import           Test.Tasty           (defaultMain, testGroup)

------------------------------------------------------------------------------
--------------------------------- TESTING MAIN -------------------------------

main :: IO ()
main = defaultMain $ do
    testGroup
      "Testing validator with some sensible values"
      [ good "User 1 locks and user 2 takes with R = -42 after deadline succeeds"
             $ testScript 50 (-42)
      , good "User 1 locks and user 2 takes with R = 0   after deadline succeeds"
```

```
            $ testScript 50 0
      , bad  "User 1 locks and user 2 takes with R = 42  after deadline fails"
            $ testScript 50 42
      , bad  "User 1 locks and user 2 takes with R = -42 before deadline fails"
            $ testScript 5000 (-42)
      , bad  "User 1 locks and user 2 takes with R = 0   before deadline fails"
            $ testScript 5000 0
      , bad  "User 1 locks and user 2 takes with R = 42  before deadline fails"
            $ testScript 5000 42
      ]
    where
      bad msg = good msg . mustFail
      good = testNoErrors (adaValue 10_000_000) defaultBabbage
```

Same as in the previous chapter we have our *main* function that contains the list of tests with their descriptions. The first two tests should pass and the rest of the tests should fail. We use the same test in all six cases, just the parameters we pass to the test are different for all of the cases. The first parameter we pass is the POSIX time for the deadline and the second one is the redeemer. The deadline of 50 should be OK, because we are hardcoding the time that we wait before we submit the consuming transaction to 1000. Let us look now at the helper functions.

```
--------------------------------------------------------------------------------
------------------------------ HELPER FUNCTIONS --------------------------------

waitBeforeConsumingTx :: POSIXTime
waitBeforeConsumingTx = 1000

-- Set many users at once
setupUsers :: Run [PubKeyHash]
setupUsers = replicateM 2 $ newUser $ ada (Lovelace 1000)

-- Validator's script
valScript :: TypedValidator datum redeemer
valScript = TypedValidator $ toV2 OnChain.validator

-- Create transaction that spends "usp" to lock "val" in "valScript"
lockingTx :: POSIXTime -> UserSpend -> Value -> Tx
lockingTx dl usp val =
  mconcat
    [ userSpend usp
    , payToScript valScript (HashDatum (OnChain.MkCustomDatum dl)) val
    ]
```

```
-- Create transaction that spends "ref" to unlock "val" from the "valScript"
-- validator
consumingTx :: POSIXTime -> Integer -> PubKeyHash -> TxOutRef -> Value -> Tx
consumingTx dl redeemer usr ref val =
  mconcat
    [ spendScript valScript ref (mkI redeemer) (OnChain.MkCustomDatum dl)
    , payToKey usr val
    ]
```

First, we define the time that we wait in between the locking transaction and the consuming transaction. Then we define the function to set up 2 users and give them 1000 lovelace. Next, we import the actual validator from the `NegativeRTimed` module that we named `OnChain`. At the end we define two functions that produce helper transactions. One to create the UTXO at the script address and one to consume this UTXO. Transactions have an instance of the Monoid type class so you can combine them with the `mconcat` operator. For the locking transaction we first define the part which will contain the information which user UTXOs we want to spend and how much value we want to send to the contract. And then in the second part we define to which script the funds will go and what datum hash, we attach to it. The `spendScript` function takes in a script, a datum mode and a value and produces a transaction. The *DatumMode* type can be either an inline datum or a datum hash.

```
Prelude> import Plutus.Model
Prelude Plutus.Model> :i payToScript
payToScript ::
  (HasDatum script, HasAddress script) =>
  script
  -> DatumMode (DatumType script)
  -> Plutus.V1.Ledger.Value.Value
  -> Tx

Prelude Plutus.Model> :i DatumMode
data DatumMode a = InlineDatum a | HashDatum a
```

We follow a similar logic for building the consuming transaction. Because we want to consume a UTXO sitting at a script address we need to provide the datum, the redeemer and also a reference to the UTXO. These are then together with the script the input parameters to the `spendScript` function that defines under which conditions a script UTXO can be spent.

```
Prelude Plutus.Model> :i spendScript
spendScript ::
  IsValidator script =>
  script
  -> Plutus.V1.Ledger.Tx.TxOutRef
  -> RedeemerType script
  -> DatumType script
```

```
    -> Tx
```

Then in the second part of the transaction we say to which user how much value should be paid. Let us now look at the *testScript* function that defines our actual test.

```
--------------------------------------------------------------------------------
------------------------------ TESTING REDEEMERS -------------------------------

-- Function to test if both creating and consuming script UTxOs works properly
testScript :: POSIXTime -> Integer -> Run ()
testScript d r = do
  -- SETUP USERS
  [u1, u2] <- setupUsers
  -- USER 1 LOCKS 100 lovelace ("val") IN VALIDATOR
  -- Define value to be transfered
  let val = adaValue 100
  -- Get user's UTXO that we should spend
  sp <- spend u1 val
  -- User 1 submits "lockingTx" transaction
  submitTx u1 $ lockingTx d sp val
  -- WAIT FOR A BIT
  waitUntil waitBeforeConsumingTx
  -- USER 2 TAKES "val" FROM VALIDATOR
  -- Query blockchain to get all UTxOs at script
  utxos <- utxoAt valScript
  -- We know there is only one UTXO (the one we created before)
  let [(ref, out)] = utxos
  -- Create time interval with equal radius around current time
  ct <- currentTimeRad 100
  -- Build Tx
  tx <- validateIn ct $ consumingTx d r u2 ref (txOutValue out)
  -- User 2 submits "consumingTx" transaction
  submitTx u2 tx
  -- CHECK THAT FINAL BALANCES MATCH EXPECTED BALANCES
  -- Get final balances of both users
  [v1, v2] <- mapM valueAt [u1, u2]
  -- Check if final balances match expected balances
  unless (v1 == adaValue 900 && v2 == adaValue 1100) $
    logError "Final balances are incorrect"
```

It takes in the datum in POSIX time and the redeemer and returns the updated blockchain in the run monad parameterized by the unit type. The first thing we do is set up two users. Then we create a value of 100 lovelace. Then we use the `spend` function that takes in a public key hash and a value and returns all the UTXOs owned by that user needed to pay for that value.

```
Prelude Plutus.Model> :i spend
spend :: PubKeyHash -> Value -> Run UserSpend
```

Now we can submit the locking transaction by passing to the `lockingTx` function the spend value that contains the UTXOs together with the value we want to send and the datum that we attach. Then we submit this transaction with the help of the `submitTx` function that also takes in the public key hash of the user that is submitting the transaction.

```
Prelude Plutus.Model> :i submitTx
submitTx :: PubKeyHash -> Tx -> Run ()
```

Now our validator has a UTXO containing the value that we have sent and the datum we have attached. Next, we wait for 1000 milliseconds and then start the process of consuming the script UTXO. First, we query all the UTXOs from the script address. Because we know there is only one UTXO, we pattern match to extract the reference and the actual output. Then we create a time interval with the `currentTimeRad` function that takes in a value and creates the upper and lower bound by subtracting and adding this value from the current time. So, our time interval will be [900, 1100]. Now we create the actual transaction by providing all the input parameters to the `consumingTx` function and then applying the `validateIn` function to it, which takes in a time interval and a transaction and creates a new transaction.

```
Prelude Plutus.Model> :i validateIn
validateIn :: POSIXTimeRange -> Tx -> Run Tx
```

At the end we say that user two submits the transaction. At the end of the test, we check that the balances of user one and two are as expected and in case they are not we log and error.

```
Prelude Plutus.Model> :i logError
logError :: String -> Run ()
```

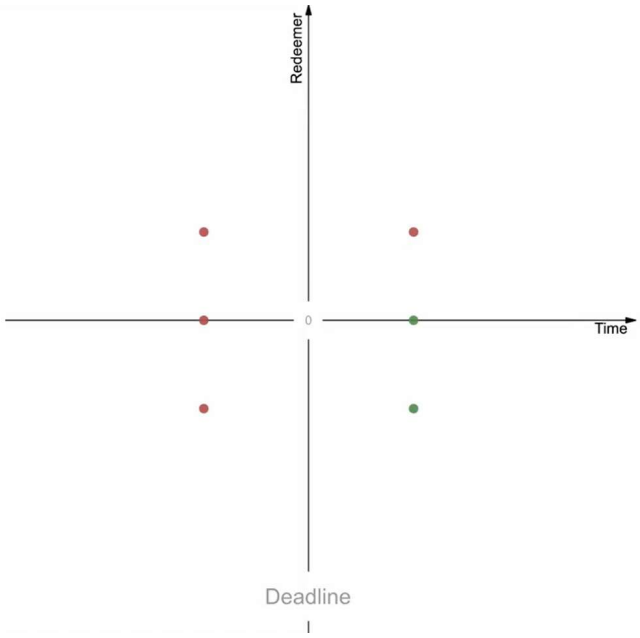
The `logError` function takes a string and returns a run monad parameterized by unit. We can run now the tests with the following command:

```
/workspace/code/Week06# cabal test week06-UTNegativeRTimed
...
Running 1 test suites...
Test suite week06-UTNegativeRTimed: RUNNING...
Testing validator with some sensible values
  User 1 locks and user 2 takes with R = -42 after dealine succeeds: OK (0.54s)
  User 1 locks and user 2 takes with R = 0   after dealine succeeds: OK
  User 1 locks and user 2 takes with R = 42  after dealine fails   : OK
  User 1 locks and user 2 takes with R = -42 before dealine fails  : OK
  User 1 locks and user 2 takes with R = 0   before dealine fails  : OK
  User 1 locks and user 2 takes with R = 42  before dealine fails  : OK
```

```
All 6 tests passed (0.57s)
```

6.4 Property testing a smart contract

With unit testing if we choose our tests carefully, we can pinpoint logic errors and issues in our smart contracts. But with unit testing we cannot say much about the properties of our smart contracts other than that they pass the tests in these specific scenarios. We can graphically display the unit tests from the previous chapter as illustrated below.



The green dots represent the tests that should pass and the red dots represent the tests that should fail. The surface of the diagram represents all possible combinations for the two values that we input to our test function. What we would like to achieve is that we cover as much as possible of this surface which is hard to do with unit testing. With property testing we can specify a property and then let the QuickCheck testing library generate a large number of random tests so our confidence in the behavior of the smart contract increases. This library is a Haskell library and you can use it to test any code, not just smart contracts. Let us look now at our property tests that are contained in the *PTNegativeRTimed.hs* file.

```
{-# LANGUAGE DataKinds          #-}
{-# LANGUAGE FlexibleInstances  #-}
{-# LANGUAGE NoImplicitPrelude  #-}
{-# LANGUAGE NumericUnderscores #-}
{-# OPTIONS_GHC -Wno-orphans #-}

module Main where
```

```
import qualified NegativeRTimed       as OnChain
import           Control.Monad           (mapM, replicateM)
import           Plutus.Model            (Ada (Lovelace), DatumMode (HashDatum),
                                          Run, Tx,
                                          TypedValidator (TypedValidator),
                                          UserSpend, ada, adaValue,
                                          defaultBabbage, initMock, mustFail,
                                          newUser, payToKey, payToScript,
                                          runMock, spend, spendScript, submitTx,
                                          toV2, userSpend, utxoAt, valueAt,
                                          waitUntil, currentTimeRad, validateIn)
import           Plutus.V2.Ledger.Api    (PubKeyHash, TxOut (txOutValue),
                                          TxOutRef, Value, POSIXTime (POSIXTime),
                                          getPOSIXTime))
import           PlutusTx.Builtins       (Integer, mkI)
import           PlutusTx.Prelude        (Bool (..), Eq ((==)),
                                          return, ($), (&&), (.))
import           Prelude                 (IO, Ord ((<), (>)),
                                          mconcat)
import           Test.QuickCheck         (Property, Testable (property),
                                          collect, (==>), Arbitrary (arbitrary),
choose)
import           Test.QuickCheck.Monadic (assert, monadic, run)
import           Test.Tasty              (defaultMain, testGroup)
import           Test.Tasty.QuickCheck   as QC (testProperty)

-------------------------------------------------------------------------------
--------------------------------- TESTING MAIN --------------------------------

-- | Test the validator script
main :: IO ()
main = defaultMain $ do
    testGroup
      "Testing script properties"
      [ testProperty "Anything before the deadline always fails"
                     prop_Before_Fails
      , testProperty "Positive redeemer after deadline always fails"
                     prop_PositiveAfter_Fails
      , testProperty "Negative redeemer after deadline always succeeds"
                     prop_NegativeAfter_Succeeds
      ]
```

We again use the functions `defaultMain` and `testGroup` from the tasty module. To generate a test tree, we now use the `testProperty` function that is also contained in the tasty package.

```
Prelude> import Test.Tasty.QuickCheck
Prelude Test.Tasty.QuickCheck> :i testProperty
testProperty :: Testable a => TestName -> a -> TestTree
```

The final testing functions we provide to the `testProperty` function also take in parameters but we do not provide any since this does the QuickCheck testing library for us. The first property we test tells us that any transaction submitted before the deadline always fails no matter which redeemer you choose. Then we test for the property that a transaction with a positive redeemer after the deadline also always fails. And the final property is that a transaction with a negative redeemer submitted after the deadline should always succeed. Now let us look over the helper functions that we will use in the testing function.

```
--------------------------------------------------------------------------------
-------------------------- HELPER FUNCTIONS/INSTANCES --------------------------

-- | Make Run an instance of Testable so we can use it with QuickCheck
instance Testable a => Testable (Run a) where
  property rp = let (a,_) = runMock rp $ initMock defaultBabbage
                                         (adaValue 10_000_000) in property a

-- Make POSIXTime an instance of Arbitrary so QuickCheck can generate random
-- values to test
instance Arbitrary POSIXTime where
  arbitrary = do
    n <- choose (0, 2000)
    return (POSIXTime n)

-- Time to wait before consumming UTxO from script
waitBeforeConsumingTx :: POSIXTime
waitBeforeConsumingTx = 1000

-- Set many users at once
setupUsers :: Run [PubKeyHash]
setupUsers = replicateM 2 $ newUser $ ada (Lovelace 1000)

-- Validator's script
valScript :: TypedValidator datum redeemer
valScript = TypedValidator $ toV2 OnChain.validator

-- Create transaction that spends "usp" to lock "val" in "giftScript"
lockingTx :: POSIXTime -> UserSpend -> Value -> Tx
lockingTx dl usp val =
  mconcat
    [ userSpend usp
```

```
    , payToScript valScript (HashDatum (OnChain.MkCustomDatum dl)) val
    ]

-- Create transaction that spends "giftRef" to unlock "giftVal" from the
-- "valScript" validator
consumingTx :: POSIXTime -> Integer -> PubKeyHash -> TxOutRef -> Value -> Tx
consumingTx dl redeemer usr ref val =
  mconcat
    [ spendScript valScript ref (mkI redeemer) (OnChain.MkCustomDatum dl)
    , payToKey usr val
    ]
```

We use the same helper functions that we have used in the previous chapter when demonstrating how to unit test a smart contract. In addition to those helper functions, we also add two instances. We need a testable instance for the run type because the Plutus simple model library uses this run type to wrap the state monad. Once this instance is created it can be used also for other tests so this code can be reused. But keep in mind here we are initializing the blockchain with the Babbage default configuration and give 10.000.000 coins to the admin user. Then we create an arbitrary instance. QuickCheck has many instances of this arbitrary type class for common types but for the `POSIXTime` type we have to create one. Because this type is just a wrapper around an integer, we can randomly pick a number between 0 and 2000 with the `choose` function and then return it as `POSIXTime`. Let us now look at how we define our property functions that we use for testing.

```
--------------------------------------------------------------------------------
------------------------------ TESTING PROPERTIES ------------------------------

-- All redeemers fail before deadline
prop_Before_Fails :: POSIXTime -> Integer -> Property
prop_Before_Fails d r = (d > 1001) ==> runChecks False d r

-- Positive redeemer always fail after deadline
prop_PositiveAfter_Fails :: POSIXTime -> Integer -> Property
prop_PositiveAfter_Fails d r = (r > 0 && d < 999) ==> runChecks False d r

-- Negative redeemers always succeed after deadline
prop_NegativeAfter_Succeeds :: POSIXTime -> Integer -> Property
prop_NegativeAfter_Succeeds d r = (r < 0 && d < 999) ==> runChecks True d r
```

All of the property functions take in a POSIX time and an integer variable and return a `Property` type variable. Also, all of these functions use the `runChecks` function together with the `==>`

implication for properties operator. The `runChecks` function takes in a Boolean which says if we expect this test to fail and succeed. And it also takes in a POSIX time and an integer which are randomly generated and it returns a `Property` type variable. The operator `==>` takes in a Boolean that says which values are allowed to be passed to the `runChecks` function. And it takes in a type variable that has an instance of the testable type class, which the `Property` type has, that the runChecks function returns. Below are the type signatures of these functions.

```
runChecks :: Bool -> POSIXTime -> Integer -> Property

(==>) :: Testable prop => Bool -> prop -> Property
```

For the first property test we allow only deadlines larger than 1001 milliseconds to be passed to the `runChecks` function so this property test should always fail since the time that we submit the transaction is 1000 milliseconds. For the second property test the deadline should be smaller than 999 milliseconds which is OK, but the redeemer should be positive. So, these tests should also fail. In the third property test the values for the redeemer and the deadline should be in a valid range so these tests should pass. Let us now look at the testing functions.

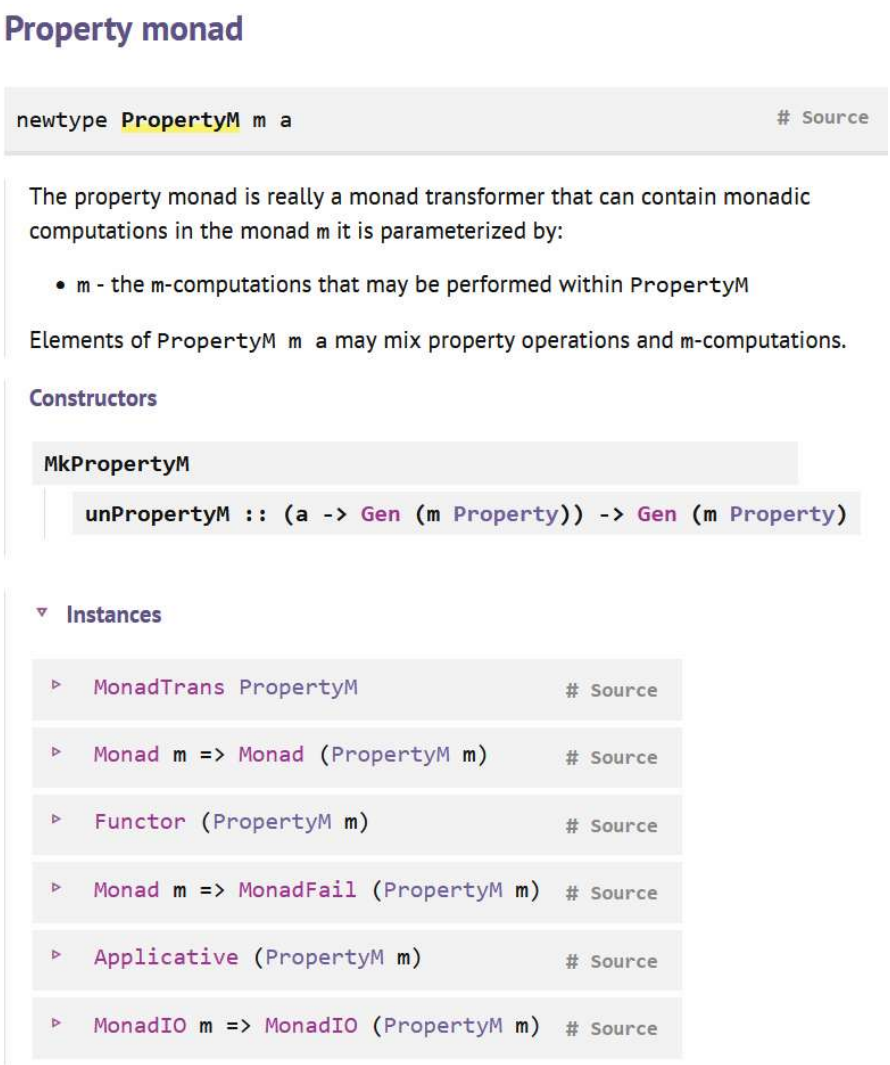
```
--------------------------------------------------------------------------------
------------------------------ RUNNING THE TESTS -------------------------------

-- | Check that the expected and real balances match after using the validator
-- with different redeemers
runChecks :: Bool -> POSIXTime -> Integer -> Property
runChecks shouldConsume deadline redeemer =
  collect (redeemer, getPOSIXTime deadline) $ monadic property check
    where check = do
            balancesMatch <- run $ testValues shouldConsume deadline redeemer
            assert balancesMatch

-- Function to test if both creating and consuming script UTxOs works properly
testValues :: Bool -> POSIXTime -> Integer -> Run Bool
testValues shouldConsume datum redeemer = do
  -- SETUP USERS
  [u1, u2] <- setupUsers
  -- USER 1 LOCKS 100 ADA ("val") IN VALIDATOR
  -- Define value to be transferred
  let val = adaValue 100
  -- Get user's UTXOs that we should spend
  sp <- spend u1 val
  -- User 1 submits "lockingTx" transaction
  submitTx u1 $ lockingTx datum sp val
  -- WAIT FOR A BIT
```

```
waitUntil waitBeforeConsumingTx
-- USER 2 TAKES "val" FROM VALIDATOR
-- Query blockchain to get all UTxOs at script
utxos <- utxoAt valScript
-- We know there is only one UTXO (the one we created before)
let [(oRef, oOut)] = utxos
    -- Define transaction to be submitted
    tx = consumingTx datum redeemer u2 oRef (txOutValue oOut)
    -- Define expected balance for user 2
    v2Expected = if shouldConsume then adaValue 1100 else adaValue 1000
-- Create time interval with equal radius around current time
ct  <- currentTimeRad 100
-- Build final consuming Tx
tx' <- validateIn ct tx
-- User 2 submits "consumingTx" transaction
if shouldConsume then submitTx u2 tx' else mustFail . submitTx u2 $ tx'
-- CHECK THAT FINAL BALANCES MATCH EXPECTED BALANCES
-- Get final balances
[v1, v2] <- mapM valueAt [u1, u2]
-- Check if final balances match expected balances
return $ v1 == adaValue 900 && v2 == v2Expected
```

The `collect` function is used such that QuickCheck actually displays the values that are being generated and tested against our properties. The `monadic` and the `property` functions are used to generate a `Property` type variable from our test that is contained in the `check` variable. You can see the type signature of all those three functions below.

```
Prelude> import Test.QuickCheck
Prelude Test.QuickCheck> :i collect
collect :: (Show a, Testable prop) => a -> prop -> Property

Prelude> import Test.QuickCheck.Monadic
Prelude Test.QuickCheck.Monadic> :i monadic
monadic :: (Testable a, Monad m) => (m Property -> Property) ->
                                    PropertyM m a -> Property

Prelude Test.QuickCheck> :i property
class Testable prop where
  property :: prop -> Property
```

The `check` variable is of type `PropertyM m a` which is a property monad. Given the m has an instance of the monad type class also PropertyM has an instance of the monad type class. In Figure 19 we can also see that the property monad has also an instance of the IO monad (*MonadIO*) and the Fail monad (*MonadFail*) given that also m has an instance of both monads.



Property monad

```
newtype PropertyM m a                                    # Source
```

The property monad is really a monad transformer that can contain monadic computations in the monad `m` it is parameterized by:

- `m` - the `m`-computations that may be performed within `PropertyM`

Elements of `PropertyM m a` may mix property operations and `m`-computations.

Constructors

```
MkPropertyM
  unPropertyM :: (a -> Gen (m Property)) -> Gen (m Property)
```

Instances

```
MonadTrans PropertyM                                     # Source
Monad m => Monad (PropertyM m)                           # Source
Functor (PropertyM m)                                    # Source
Monad m => MonadFail (PropertyM m)                       # Source
Applicative (PropertyM m)                                # Source
MonadIO m => MonadIO (PropertyM m)                       # Source
```

Figure 19 - PropertyM type

In the `check` variable we first call our *`testValues`* function and use the `run` function to raise our run monad to a property monad and then extract the return value which is of type bool. This Boolean says whether the test has passed or failed. After that we use the `assert` function that takes in our Boolean and returns a property monad parameterized by unit. In case the bool is false it raises an error. You can see the type signatures of the `run` and `assert` functions below.

```
Prelude Test.QuickCheck.Monadic> :i run
run :: Monad m => m a -> PropertyM m a

Prelude Test.QuickCheck.Monadic> :i assert
assert :: Monad m => Bool -> PropertyM m ()
assert True  = return ()
assert False = fail "Assertion failed"
```

The *testValues* function is structured very similarly as in the previous chapter. The difference is that we compose the expected final balance of user two (`v2Expected`) depending on the case whether the test should fail or pass. Also, when we submit the claiming transaction, we combine it with the `mustFail` function in the case we expect the test to fail. In the end we compare the expected balances of user one and two with their actual balances. We can now run the property tests with the following command:

```
/workspace/code/Week06# cabal test week06-PTNegativeRTimed
...
Running 1 test suites...
Test suite week06-PTNegativeRTimed: RUNNING...
Testing script properties
  Anything before the deadline always fails       : OK (0.94s)
    +++ OK, passed 100 tests; 127 discarded:
     1% (-1,1162)
     1% (-10,1450)
     ... 96 other tests
     1% (83,1783)
     1% (9,1568)
  Positive redeemer after deadline always fails   : OK (0.33s)
    +++ OK, passed 100 tests; 341 discarded:
     1% (1,208)
     1% (1,248)
     ... 96 other tests
     1% (9,588)
     1% (9,699)
  Negative redeemer after deadline always succeeds: OK (0.30s)
    +++ OK, passed 100 tests; 285 discarded:
     1% (-1,176)
     1% (-1,185)
     ... 96 other tests
     1% (-8,820)
     1% (-80,446)

All 3 tests passed (1.58s)
Test suite week06-PTNegativeRTimed: PASS
```

6.5 Testing smart contracts with Lucid

The smart contract that we are going to test is the one contained in the *NegativeRTimed.hs* file. First let us talk about the tools that we will use for testing the smart contract. They can be divided into three categories. The first one is the software that we are testing which will be the lucid emulator that you can find at the following location:
https://deno.land/x/lucid@0.9.8/mod.ts?s=Emulator

Second is the native implementation for unit testing in Deno that can be found at:
https://deno.land/manual@v1.32.3/basics/testing

And the third is the fast-check property-based testing library written in JavaScript which is similar to QuickCheck, the Haskell library for property-based testing. It can be found at: https://github.com/dubzzz/fast-check

If we look at the lucid emulator, we see that it implements a provider similar to blockfrost that we talked about in previous chapters. The difference is that this emulator can be set up to run a local mocked blockchain used for running tests. Similar to blockfrost this emulator has methods for querying the mocked blockchain, submitting transactions and also methods for controlling the progress of the mock blockchain. This can be done via the *awaitBlock* and the *awaitSlot* methods. These will allow us to emulate our contract and off-chain code. Before we can write our off-chain code for testing we need to write our validator to the file system. We do this with the following commands from our container inside the VSCode editor:

```
/workspace# cd code//Week06/
/workspace/code/Week06# cabal repl
Prelude> import Utilities
Prelude Utilities> import NegativeRTimed
Prelude Utilities NegativeRTimed>
writeValidatorToFile "./assets/NegativeRTimed.plutus" validator
```

Let us look now at the lucid code for testing that is located in the *NegativeRTimed.ts* file.

```
import * as L from "https://deno.land/x/lucid@0.9.8/mod.ts";
import { assert } from "https://deno.land/std@0.90.0/testing/asserts.ts";
import * as fc from 'https://cdn.skypack.dev/fast-check';

// the script that we are testing.
const negativeRTimedValidator: L.SpendingValidator = {
  type: "PlutusV2",
  script: "590a8f590a8c01000323322323232323322323232323232323232332232323…",
};
const negativeRTimedAddr: L.Address = await (await L.Lucid.new(undefined,
"Custom")).utils.validatorToAddress(negativeRTimedValidator);

// the typed datum and redeemer that we are going to use.
const NegativeRTimedDatum = L.Data.Object({
  deadline: L.Data.Integer()
});
type NegativeRTimedDatum = L.Data.Static<typeof NegativeRTimedDatum>;

const NegativeRTimedRedeemer = L.Data.Integer();
type NegativeRTimedRedeemer = L.Data.Static<typeof NegativeRTimedRedeemer>;
```

We see at the beginning we import three things. First is lucid which we import as the letter L. Second, we import something from the standard testing library of Deno. And third we import the fast check library as fc. Then we define what our script is, where we copy the cbor hex from the *NegativeRTimed.plutus* script file. After that we convert that script to an address. We do not initiate lucid yet as a constant but just use it inline. Next, we define our datum which is a wrapper around the deadline that is an integer. We also define the type of the redeemer that is also an integer. Now let us look at the functions that create the locking and consuming transactions.

```
// the function that given the context of lucid, the wallet, the datum and sends
// 10 ada to the script address.
async function sendToScript(
    lucid: L.Lucid,
    userPrivKey: L.PrivateKey,
    dtm: NegativeRTimedDatum
  ): Promise<L.TxHash> {
  lucid.selectWalletFromPrivateKey(userPrivKey);
  const tx = await lucid
    .newTx()
    .payToContract(negativeRTimedAddr, { inline:
                   L.Data.to<NegativeRTimedDatum>(dtm,NegativeRTimedDatum) },
                   { lovelace: 10000000n })
    .complete();
  const signedTx = await tx.sign().complete();
  const txHash = await signedTx.submit();
  return txHash
}

// the function that given the context of lucid and a negative redeemer, grabs
// funds from the script address.
async function grabFunds(
    lucid: L.Lucid,
    emulator: L.Emulator,
    userPrivKey: L.PrivateKey,
    dtm: NegativeRTimedDatum,
    r: NegativeRTimedRedeemer
  ): Promise<L.TxHash> {
  lucid.selectWalletFromPrivateKey(userPrivKey);
  const rdm: L.Redeemer =
            L.Data.to<NegativeRTimedRedeemer>(r,NegativeRTimedRedeemer);
  const utxoAtScript: L.UTxO[] = await lucid.utxosAt(negativeRTimedAddr);
  const ourUTxO: L.UTxO[] = utxoAtScript.filter((utxo) => utxo.datum ==
                L.Data.to<NegativeRTimedDatum>(dtm,NegativeRTimedDatum));
```

```
  if (ourUTxO && ourUTxO.length > 0) {
      const tx = await lucid
          .newTx()
          .collectFrom(ourUTxO, rdm)
          .attachSpendingValidator(negativeRTimedValidator)
          .validFrom(emulator.now())
          .complete();

      const signedTx = await tx.sign().complete();
      const txHash = await signedTx.submit();
      return txHash
  }
  else throw new Error("UTxO's Expected!")
}
```

The *sendToScript* function sends funds to the script and attaches a datum. If we look at the inputs of the function beside the datum and the private key that will sign the transaction we also take in an instance of lucid. We set the wallet with use of the private key and the lucid instance the function received. Then we create a new transaction and pay to the contract 10 ada and inline the datum. At the end we sing and submit the transaction. The *grabFunds* function takes in beside the parameters that the *sendToScript* function took also an emulator and the redeemer. First it selects our wallet, then converts the redeemer to cbor encoded string, it looks at the mocked blockchain at the address that we have created for the script and fetches all the UTXOs at that script and then it filters these UTXOs for those that contain the datum that we have. If there are any such UTXOs we create the consuming transaction where we specify which UTXO we want to spend, attach the script and specify the validity interval. We fetch the current posix time of the emulator when we specify the validity interval. At the end we sign and submit the transaction or log an error message if no UTXO with the datum that we have was found. We need this error because we will test the smart contract and for certain test cases, we expect that an error will be thrown. Let us now look at the function that runs a test.

```
async function runTest(dtm: NegativeRTimedDatum,
                       r: NegativeRTimedRedeemer,
                       n: number) {
  // setup a new privateKey that we can use for testing.
  const user1: L.PrivateKey = L.generatePrivateKey();
  const address1: string = await (await L.Lucid.new(undefined, "Custom"))
                                      .selectWalletFromPrivateKey(user1)
                                      .wallet.address();

  const user2: L.PrivateKey = L.generatePrivateKey();
  const address2: string = await (await L.Lucid.new(undefined, "Custom"))
```

```
                                        .selectWalletFromPrivateKey(user2)
                                        .wallet.address();

// Setup the emulator and give our testing wallet 10000 ada. These funds get
// added to the genesis block.
  const emulator = new L.Emulator([{ address: address1,
                                     assets: { lovelace: 10000000000n } },
                                   { address: address2,
                                     assets: { lovelace: 10000000000n}}]);
  const lucid = await L.Lucid.new(emulator);

  // gets added to the first block in the emulator
  await sendToScript(lucid,user1,dtm);

  // wait n slots
  emulator.awaitSlot(n);

  // gets added to the (n+1)th block
  await grabFunds(lucid,emulator,user2,dtm,r);

  emulator.awaitBlock(10);

  console.log(await emulator.getUtxos(address2));
}
await runTest({deadline:BigInt(Date.now()+20000*5+1000)},-42n,5*20+1);
```

The *runTest* function has three inputs: the datum, the redeemer and a number that represents the slot number at which the grab function will be initiated. So, if that is before the deadline the transaction should fail and if it is after the deadline given a correct redeemer it should process successfully. The reason why we did not initiate lucid in the beginning is because we will set up an emulator and to do that, we first define two users and generate a private key for each of these. Then we also compute their addresses. Given these two addresses we can set up an emulator that takes in a list of outputs that we want to create at the genesis block when our emulator runs. For each address we define what assets we assign to it. We assign 10.000 ada to each of our addresses. Now we can initiate lucid with the provider emulator that we have just defined. So, now we have set the stage for testing. We can now pass this emulator through the lucid variable to the *sendToScript* function. Then we wait for n slots and then call the *grabFunds* function with the updated emulator. At the end we wait for 10 blocks to pass and log the funds from the address of user two. Now we can run our test where we set the deadline. On average every block is processed in 20 seconds which is 20000 milliseconds. So, we set the deadline at 5 blocks plus one slot from the initiation of the creation of our mocked

blockchain. We also set a negative redeemer and the moment that we are going to call our grab function is five blocks after the deadline plus one so when the deadline passes, we immediately send a transaction. We can now run this code with the following command:

```
/workspace# cd code/Week06/
/workspace/code/Week06# deno run ./lecture_tests/NegativeRTimed.ts
✅ Granted network access to "deno.land".
[
  {
    txHash: "3774ad7a837bfbedec3a2f28783cc8de6a776c923e1fffa20eab02cf9b1220bc",
    outputIndex: 0,
    assets: { lovelace: 10009661568n },
    address: "addr_test1vr36a67zgn56hlawvxdcr9jp72wkgn2pmaevdm7qega099s4s4al7",
    datumHash: undefined,
    datum: undefined,
    scriptRef: undefined
  }
]
```

In the output we see that the address two contains a bit less than 1010 ada since some fees were subtracted from the 10 ada we have claimed from the script address. If in the last line we change the last parameter to 5*20 the test will fail because we are trying to grab the funds before the deadline.

```
/workspace/code/Week06# deno run ./lecture_tests/NegativeRTimed.ts
✅ Granted network access to "deno.land".
error: Uncaught (in promise) "Redeemer (Spend, 1): The provided Plutus code called
'error'."
```

Now we can write some unit tests. Let us look at the code for unit tests.

```
// UNIT tests

function testSucceed(
  str: string, // the string to display of the test
  r: bigint,   // the redeemer number
  d: bigint,   // the deadline in seconds from now
  n:number     // the number of slots user 2 waits
) {
  Deno.test(str, async () =>
                 {await runTest({deadline:BigInt(Date.now())+d},r,n)})
}

async function testFails(
  str: string, // the string to display of the test
  r: bigint,   // the redeemer number
  d: bigint,   // the deadline in seconds from now
```

```
  n:number     // the number of slots user 2 waits
) {
  Deno.test(str,async () => {
    let errorThrown = false;
    try {
      await runTest({deadline:BigInt(Date.now())+d},r,n);
    } catch (error) {
      errorThrown = true;
    }
    assert(
      errorThrown,
      "Expected to throw an error, but it completed successfully"
    );
  });
};

// deadline is slot 100 and user 2 claims at slot 120
testSucceed("UT: User 1 locks and user 2 takes with R = -42 after dealine;
             succeeds",-42n,BigInt(1000*100),120);
// deadline is slot 100 and user 2 claims at slot 120
testSucceed("UT: User 1 locks and user 2 takes with R = 0 after dealine;
             succeeds",0n,BigInt(1000*100),120);
// deadline is slot 100 and user 2 claims at slot 120
testFails("UT: User 1 locks and user 2 takes with R = 42 after dealine;
           fails",42n,BigInt(1000*100),120);
// deadline is slot 100 and user 2 claims at slot 80
testFails("UT: User 1 locks and user 2 takes with R = -42 before dealine;
           fails",-42n,BigInt(1000*100),80);
// deadline is slot 100 and user 2 claims at slot 80
testFails("UT: User 1 locks and user 2 takes with R = 0 before dealine; fails",
          -0n,BigInt(1000*100),80);
// deadline is slot 100 and user 2 claims at slot 80
testFails("UT: User 1 locks and user 2 takes with R = 42 before dealine;
           fails",42n,BigInt(1000*100),80);
```

We have two types of tests: the ones that should succeed and the ones that should fail. In the deno test environment we can wrap functions and capture their errors when they appear. In the *testSucceed* function the `Deno.test` function takes in a description of the test and the actual test with concrete parameters. When a test fails that we expect to succeed the `Deno.test` function will grab that error and log that the test that we denoted by the `str` parameter failed. If the test succeeds then it will also log a message that the test passed. In the *testFails* function we first define the *errorThrown* variable that we set to false. Then we call

our test function from within a try-catch statement and in the case, there is an error we set the *errorThrown* variable to true. At the end we raise an error with the *assert* function in the case that the *errorThrown* variable is set to false which means the test did not fail. After that comes the unit test where two of them should succeed and four of them should fail. We can run these tests with the following command:

```
/workspace/code/Week06# deno test -A lecture_tests/NegativeRTimed.ts
Check file:///workspace/code/Week06/lecture_tests/NegativeRTimed.ts
running 9 tests from ./lecture_tests/NegativeRTimed.ts
UT: User 1 locks and user 2 takes with R = -42 after dealine; succeeds ... ok (108ms)
UT: User 1 locks and user 2 takes with R = 0 after dealine; succeeds ... ok (47ms)
UT: User 1 locks and user 2 takes with R = 42 after dealine; fails ... ok (45ms)
UT: User 1 locks and user 2 takes with R = -42 before dealine; fails ... ok (45ms)
UT: User 1 locks and user 2 takes with R = 0 before dealine; fails ... ok (44ms)
UT: User 1 locks and user 2 takes with R = 42 before dealine; fails ... ok (47ms)

ok | 6 passed | 0 failed (8s)
```

We see that all of the six tests have passed. If we change the last input parameter of the first test from 120 to 80 and run the tests again, we will see that the first test fails.

```
/workspace/code/Week06# deno test -A lecture_tests/NegativeRTimed.ts
Check file:///workspace/code/Week06/lecture_tests/NegativeRTimed.ts
running 9 tests from ./lecture_tests/NegativeRTimed.ts
UT: User 1 locks and user 2 takes with R = -42 after dealine; succeeds ...
                                                        FAILED (workspace 80ms)
UT: User 1 locks and user 2 takes with R = 0 after dealine; succeeds ... ok (47ms)
UT: User 1 locks and user 2 takes with R = 42 after dealine; fails ... ok (45ms)
UT: User 1 locks and user 2 takes with R = -42 before dealine; fails ... ok (45ms)
UT: User 1 locks and user 2 takes with R = 0 before dealine; fails ... ok (44ms)
UT: User 1 locks and user 2 takes with R = 42 before dealine; fails ... ok (47ms)

FAILED | 5 passed | 1 failed (8s)
```

Let us look now at the property tests that are written with the help of the fast check library.

```
// Property test
// set up a fixed deadline at slot 100
const dl: number = 100*1000;
// create only random 256 bit negative big integers for r.
const negativeBigIntArbitrary = fc.bigIntN(256).filter((n:bigint) => n <= 0n);
// create only random 256 bit positive big integers for r.
const positiveBigIntArbitrary = fc.bigIntN(256).filter((n:bigint) => n > 0n);
// create only random integers that represent claiming after the deadline
const afterDeadlineWaits = fc.integer().filter((n: number) => n >= dl);
// create only random integers that represent claiming before the deadline
const beforeDeadlineWaits = fc.integer().filter((n: number) => n < dl);

Deno.test("PT: Negative redeemer after deadline always succeeds", () => {
```

```
  fc.assert(fc.asyncProperty(
    negativeBigIntArbitrary, afterDeadlineWaits, async (r: bigint,n: number) => {
      try {
        await runTest({deadline:BigInt(Date.now()+dl)},r,n);
      } catch (error) {
        console.error('Test failed for r= '+ r +' with error: ' + error.message);
        throw error
      };
    }
  ),{numRuns: 100});
});

Deno.test("PT: Positive redeemer after deadline always fails", () => {
  fc.assert(fc.asyncProperty(
    positiveBigIntArbitrary, afterDeadlineWaits,async (r:bigint, n: number) => {
      let errorThrown = false;
      try {
        await runTest({deadline:BigInt(Date.now()+dl)},r,n);
      } catch (error) {
        errorThrown = true;
      }
      assert(errorThrown,'Test failed for r= ' + r + ' and n= '+ n);
    }
  ),{numRuns:100});
})

Deno.test("PT: Anything before the deadline always fails", () => {
  fc.assert(fc.asyncProperty(
    fc.bigIntN(256), beforeDeadlineWaits,async (r:bigint, n: number) => {
      let errorThrown = false;
      try {
        await runTest({deadline:BigInt(Date.now()+dl)},r,n);
      } catch (error) {
        errorThrown = true;
      }
      assert(errorThrown,'Test failed for r= ' + r + ' and n= ' + n);
    }
  ),{numRuns:100});
})
```

Initially for property tests to run you have to define how tests are generated and we do that via the four constants we define in the beginning. Two conditions for the redeemer where once it should be a negative number and once a positive. And then two conditions for the waiting time where once it should be longer than the deadline and once it should be shorter. We use the fast

check library to generate these numbers via the functions *fc.bigIntN(256).filter* and *fc.integer().filter*. The first function generates a random integer of 256 bits and applies the given filter to it. The first function generates a random integer number of arbitrary size and then applies the filter function to it. In the first line of the property test code, we set the deadline to 100 slots and we state this in milliseconds. We remember one slot is 1000 milliseconds. For the property tests we also wrap them inside the Deno.test function. The input to this function is first a string that represents the name of the test and second, we input the fast check *assert* function. This function expects two inputs. First a property which is asynchronous in our case and the number or runs that we set to 100. If we look at the property, we see it contains a function that takes as input the two random generated numbers with the applied conditions that we have set in the beginning. Those numbers are our redeemer and the slot at which we grab the funds. And then this function tries to await the *runTest* function and possibly catches an error. This is repeated 100 times and each time a new set of input parameters are generated taking the applied conditions into account and they are passed to the *runTest* function. In the case that the test fails we log an error message that also prints out the input parameters that caused the test to fail. For the second and third scenarios that we expect to fail we again define the *errorThrown* variable and set its default value to false. Then if the test fails, we catch the error in a try-catch statement and set the variable to true. In the end we check this variable with the *assert* function and throw an error in case the variable is set to false which means that the expected error did not happen. In the last scenario we apply only the condition to the deadline and generate a redeemer as an arbitrary 256-bit number. This is because we test that the claiming transaction should fail for any possible redeemer if we submit the transaction before the deadline. We can run the test with the following command:

```
/workspace/code/Week06# deno test -A lecture_tests/NegativeRTimed.ts
Check file:///workspace/code/Week06/lecture_tests/NegativeRTimed.ts
running 9 tests from ./lecture_tests/NegativeRTimed.ts
PT: Negative redeemer after deadline always succeeds ... ok (3s)
PT: Positive redeemer after deadline always fails ... ok (3s)
PT: Anything before the deadline always fails ... ok (560ms)

ok | 3 passed | 0 failed (8s)
```

If we would now for instance change the condition *negativeBigIntArbitrary* such that the randomly picked redeemer integer is smaller or equal to 10 instead of 0 we would maybe see that some of the tests fail because the outcome depends on the random generated numbers.

```
root@90772a93ea13:/workspace/code/Week06# deno test -A lecture_tests/NegativeRTimed.ts
Check file:///workspace/code/Week06/lecture_tests/NegativeRTimed.ts
running 9 tests from ./lecture_tests/NegativeRTimed.ts
UT: User 1 locks and user 2 takes with R = -42 after dealine; succeeds ...
                                                                  FAILED (108ms)
```

```
UT: User 1 locks and user 2 takes with R = 0 after dealine; succeeds ... ok (51ms)
UT: User 1 locks and user 2 takes with R = 42 after dealine; fails ... ok (44ms)
UT: User 1 locks and user 2 takes with R = -42 before dealine; fails ... ok (47ms)
UT: User 1 locks and user 2 takes with R = 0 before dealine; fails ... ok (43ms)
UT: User 1 locks and user 2 takes with R = 42 before dealine; fails ... ok (44ms)
PT: Negative redeemer after deadline always succeeds ...
------- output -------
Test failed for r= 2 with error: undefined
Test failed for r= 1 with error: undefined
...
FAILED | 5 passed | 5 failed (5s)
```

In the output we also see that the we see also for which redeemer numbers the test failed.

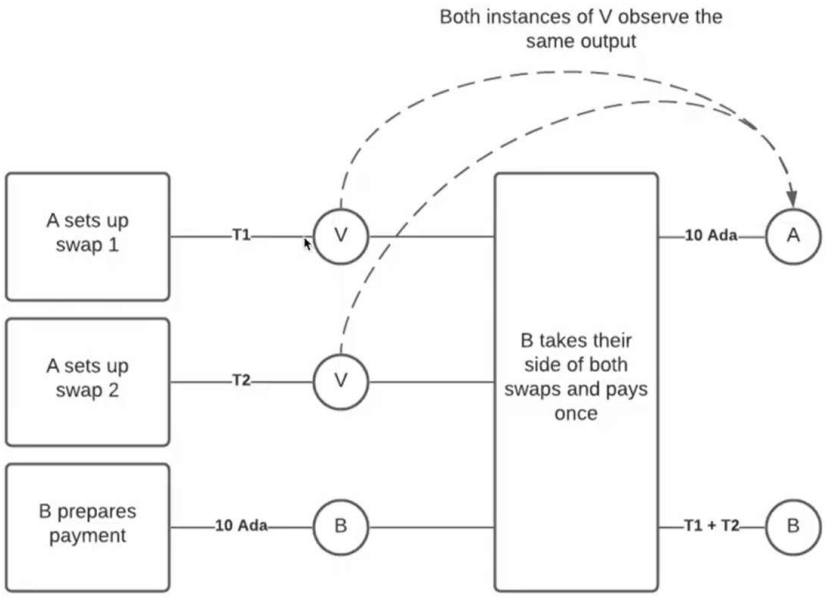
6.6 Double spending and homework

Imagine you want to sell some tokens or trade an NFT. One way to do it is through two transactions. You send the tokens or NFT and then the buyer sends you whatever you agreed to as payment. If the buyer does not send the payment back you lost your funds. To make a more secure trade we will look at a swap contract but it can be exploited.

```
{-# LANGUAGE DataKinds            #-}
{-# LANGUAGE ImportQualifiedPost #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE OverloadedStrings   #-}
{-# LANGUAGE TemplateHaskell     #-}

module ExploitableSwap where

import           Plutus.V2.Ledger.Api      (ScriptContext (scriptContextTxInfo),
                                            PubKeyHash, Validator,
                                            mkValidatorScript, adaToken,
                                            adaSymbol, singleton)
import           Plutus.V2.Ledger.Contexts (valuePaidTo, TxInfo)
import           PlutusTx                  (compile, unstableMakeIsData)
import           PlutusTx.Builtins         (BuiltinData, Integer)
import           PlutusTx.Prelude          (Bool (..), (==), traceIfFalse)
import           Utilities                 (wrapValidator)

-------------------------------------------------------------------------------
------------------------------ ON-CHAIN / VALIDATOR ---------------------------

data DatumSwap = DatumSwap
  { beneficiary :: PubKeyHash
  , price       :: Integer
  }
```

```
PlutusTx.unstableMakeIsData ''DatumSwap

{-# INLINABLE mkValidator #-}
mkValidator :: DatumSwap -> () -> ScriptContext -> Bool
mkValidator ds _ ctx = traceIfFalse "Hey! You have to pay the owner!"
                                    outputToBeneficiary
    where
        txInfo :: TxInfo
        txInfo = scriptContextTxInfo ctx

        outputToBeneficiary :: Bool
        outputToBeneficiary =
          valuePaidTo txInfo (beneficiary ds) == singleton adaSymbol
                                                 adaToken (price ds)

{-# INLINABLE  mkWrappedValidator #-}
mkWrappedValidator :: BuiltinData -> BuiltinData -> BuiltinData -> ()
mkWrappedValidator = wrapValidator mkValidator


validator :: Validator
validator = mkValidatorScript $$(compile [|| mkWrappedValidator ||])
```

The `DatumSwap` data type holds the price of our tokens or NFT and our public key hash. And then if someone wants to actually get those tokens and the NFT it has to pay you the agreed price in the same transaction. The only check we make in the validator function is that when someone consumes this output that there has to be another output to the beneficiary's address with the price in the datum. We do this by using the `valuePaidTo` function that takes in a transaction info type and a public key hash and returns the total value paid to this public key address by the validating transaction. We compare the output of this function with the value constructed by the `singleton` function that takes in the price in lovelace. This smart contract has a vulnerability. If we create two or more outputs with the same price a malicious person could consume all of them UTXOs and only pay for one as illustrated on the figure below. This is a so-called “double-spending” issue. It has several solutions. One of them is adding an identifier to the datum so even if the same person asks for the same price, it has different ID values or reference values. And another solution is that we make sure a single output is being consumed by the transaction. So, this week’s homework is to create the tests needed to catch this vulnerability and once those tests work as expected you have to fix the validator and check with the tests that it is resistant to double spending. You can find a template of the test in the file *TExploitableSwap.hs* inside the *homework* folder.



Once you have implemented your test cases you can run the test with the following command:

```
/workspace/code/Week06# cabal test week06-TExploitableSwap
```

Below you can see the template code for testing the *ExploitableSwap* smart contract.

```
{-# LANGUAGE DataKinds           #-}
{-# LANGUAGE FlexibleInstances   #-}
{-# LANGUAGE NoImplicitPrelude   #-}
{-# LANGUAGE NumericUnderscores #-}
{-# LANGUAGE OverloadedStrings   #-}

module Main where

import           Plutus.Model         (Run,
                                       TypedValidator (TypedValidator),
                                       adaValue, defaultBabbage, mustFail,
testNoErrors,
                                       toV2, FakeCoin (FakeCoin), fakeValue)
import           PlutusTx.Prelude     (($))
import           Prelude              (IO, (.), (<>), undefined)
import qualified ExploitableSwap      as OnChain
import           Test.Tasty           (defaultMain, testGroup)

-------------------------------------------------------------------------------
---------------------------------- TESTING ------------------------------------

main :: IO ()
main = do
```

```
  defaultMain $ do
    testGroup
      "Catch double spend with testing"
      [ good "Normal spending" normalSpending
      , bad  "Double spending" doubleSpending
      ]
 where
    bad msg = good msg . mustFail
    good = testNoErrors (adaValue 10_000_000 <> fakeValue scToken 100)
defaultBabbage

--------------------------------------------------------------------------------
----------------------- HELPER FUNCTIONS/INSTANCES/TYPES -----------------------

scToken :: FakeCoin
scToken = FakeCoin "Super-Cool-Token"

type HomeworkScript = TypedValidator OnChain.DatumSwap ()

swapScript :: HomeworkScript
swapScript = TypedValidator $ toV2 OnChain.validator

lockingTx = undefined

consumingTx = undefined

doubleConsumingTx = undefined

--------------------------------------------------------------------------------
------------------------------- TESTING SPENDING -------------------------------

normalSpending :: Run ()
normalSpending = undefined

doubleSpending :: Run ()
doubleSpending = undefined
```

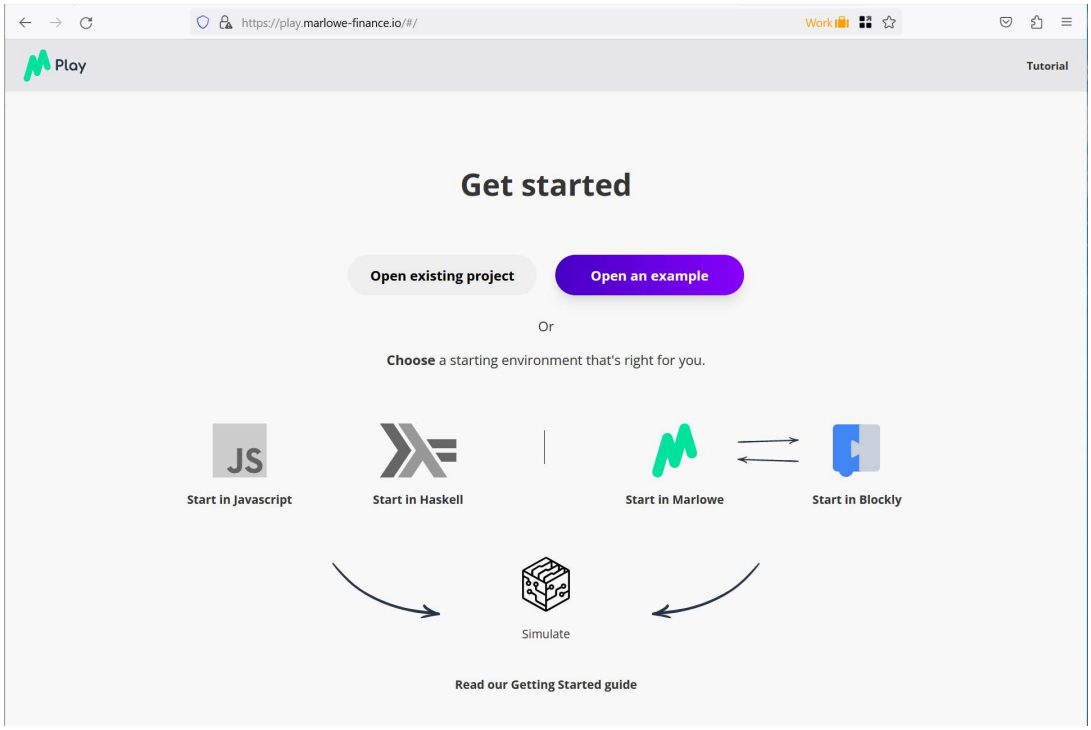
7 Marlowe

Marlowe is a domain-specific language specialized for financial contracts that is built on top of Plutus. Plutus is a very powerful language and allows you to implement other languages on top of it. You can write an interpreter in Plutus for other languages. When you design such languages, you have some trade-offs to make. One of these trade-offs is how powerful the language should be. Plutus is very powerful. It is Turing-complete which means you can express arbitrary logic in it. In computer science the Halting problem states that it is impossible to write a program that takes as input another program written in a Turing-complete languages and decide whether that program will stop or not. So, Turing-complete language are so powerful that it is difficult or impossible to automatically analyze them. You always have to check by hand certain properties of the program. In practice for a large class of programs it may be possible to have automated tools but there can be no general tools that decide whether the program stops or not. On the other hand, if you are willing to not have Turing-completeness you gain the ability to potentially have very powerful program analysis tools. And Marlowe is one such language that is not Turing-complete so you cannot express arbitrary logic in it. For that you gain for example the option before you run the program to statically analyze how long it will run. All Marlowe programs are guaranteed to have a finite lifetime. There are also other things you can do as for instance check that no money will forever be locked in a Marlowe contract. Whereas Plutus is a relatively complicated language, Marlowe is by design extremely simple. It has been designed with non-programmers in mind as for example financial experts that want to express financial contracts. There are only a hand-full of constructors in Marlowe contracts and they are quite intuitive. The design of Marlowe has been inspired by a famous paper that is now more than 20 years old written by Simon Payton Jones, one of the original creators of the Haskell programming language. The name of the paper is “Composing contracts: an adventure in financial engineering”. In this paper the authors look at traditional finance and try to formalize traditional financial contracts in a way that uses very few constructs. And Marlowe follows the same idea. With very few constructs an amazing amount of read world contracts can actually be modeled. In the real world you can enforce contracts by using the legal system. On the blockchain that is not possible. You cannot force anyone to make a payment and Marlowe takes that into account. All Marlowe can do is wait for someone to make a payment. And there always must be contingencies so if you are waiting for a certain action and that action is not happening in Marlowe there is always a timeout that says what action will happen when the timeout is reached. Also, to make Marlowe more intuitively to use it has an account-based approach even though it is implemented on Cardano which uses the UTXO model. In a Marlowe contract there is a concept of internal accounts. There are parties that participate in a Marlowe contract and each of those parties has an associated internal account.

And the internal accounts are under the control of the contract. What happens normally in a Marlowe contract is that all the participants are supposed to make a deposit which means that they send funds from their own wallet to their own internal account inside the Marlowe contract. And then depending on what the contract is about money can be transferred between these internal accounts or also back to one of the external accounts. As mentioned before no money is forever locked in a Marlowe contract so, in the case of these internal accounts once a Marlowe contract ends if there is still money left in one or more internal accounts it automatically goes back to the owner of those internal accounts. Further documentation about Marlowe with interesting examples can be found at: https://marlowe.iohk.io/. In the next chapter we will look at the Marlowe playground which is a graphical web-based environment where you can simulate the execution of Marlowe contracts and also create them. You can do that in several ways: you can use Haskell, JavaScript or also Blockly which a graphical language which allows you to click together a Marlowe contract.

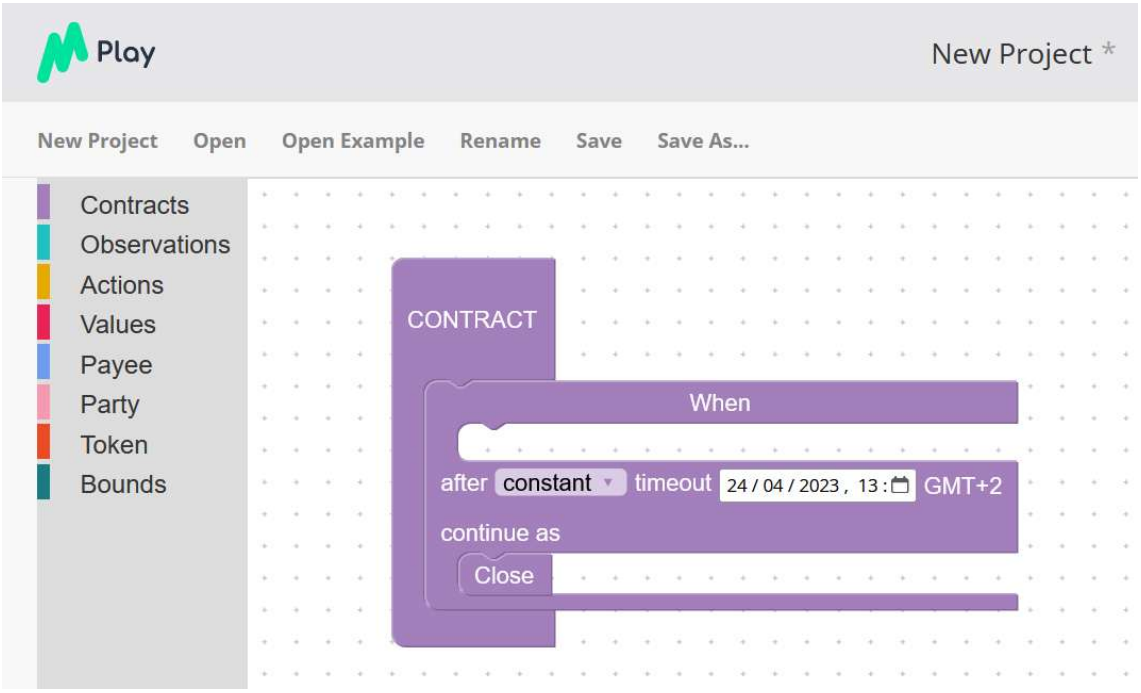
7.1 Marlowe playground demo

You can find the Marlowe playground at https://play.marlowe-finance.io/. You will see several options that you can choose from to create a Marlowe contract. You can use JavaScript, Haskell, the Marlowe language or Blockly the graphical tool.



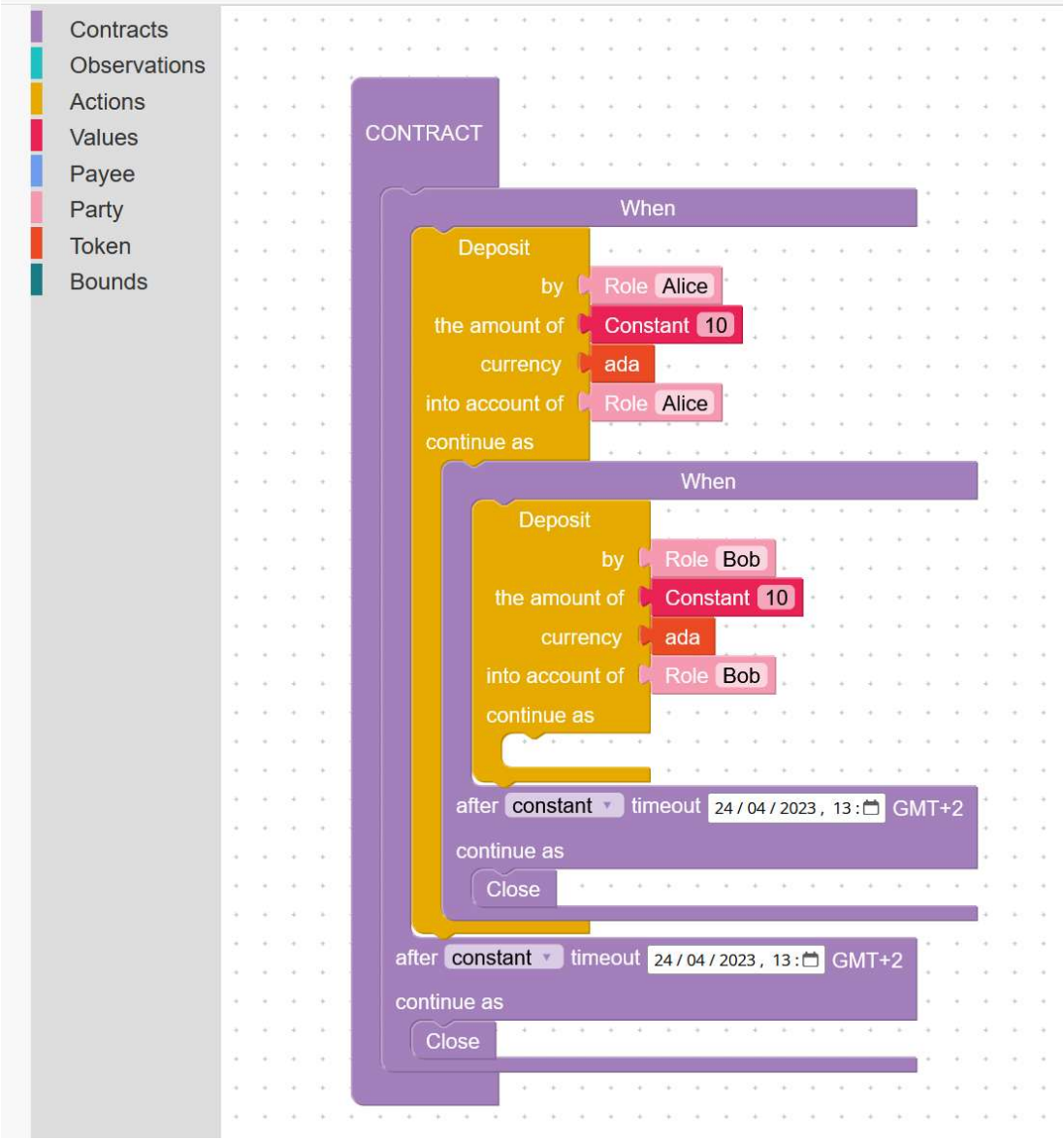
As a side note we state the time in Marlowe is based on POSIX time as Plutus. We will first look at the graphical editor Blockly. As an example, we want to write a contract where there are

three parties Alice, Bob and Charlie and the idea is that Alice and Bob deposit an amount of 10 ada into the contract. Then Charlie decides whether Alice or Bob gets the total amount. There is always the possibility that one of the three does not play along, in which case everybody should be reimbursed for what they have paid up to that point. When we start with Blockly there is already a contract that does not do anything except it closes immediately. If there would be money in the internal accounts it would pay back the money to the owners of the accounts. We can remove the close block by right clicking on it and choosing the remove option. If we click on the Contracts section on the left side, we can choose the **When** block that waits for an external action and slide it to the empty space in the contract also called slot where previously the close block was. Our external action will be a deposit and we say that the deposit should happen until a certain time. And in case it does not happen we can close the contract.



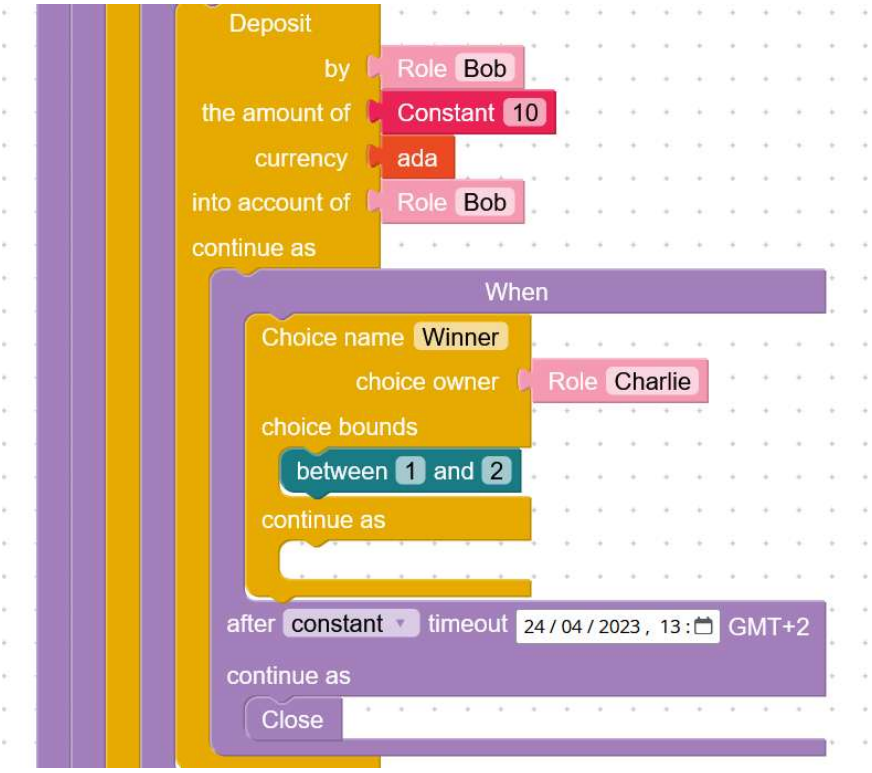
Then in the when slot we can only, we will say we wait for one action, namely that Alice makes her deposit. We click on the actions section, pick the deposit action and drag it to the when slot. There we see we can fill a couple of slots. The first one is who has to make the deposit. We click on the party section and we have to choose to select a public key block or a role. We pick **Role** and assign it the name Alice. Normally this would be the name of the role token so whoever owns that token can incorporate that role. Next, we have to specify the amount. We can click on the values section and we pick a constant amount of 10 lovelace. In the currency slot we can specify that it is ada. The ada block is available if we click on the token section. The actual number that we input is counted as lovelace even though the currency symbol is ada. There is also the option to use other tokens then ada. Then we have to specify to which internal

account Alice will make a deposit and we say she pays it to her own internal account. And then we can say what happens next if Alice makes this deposit. And we can copy the entire **When** block and change the role name to Bob and set another deadline for Bob that should be longer than the one for Alice so Bob has time to make his deposit.

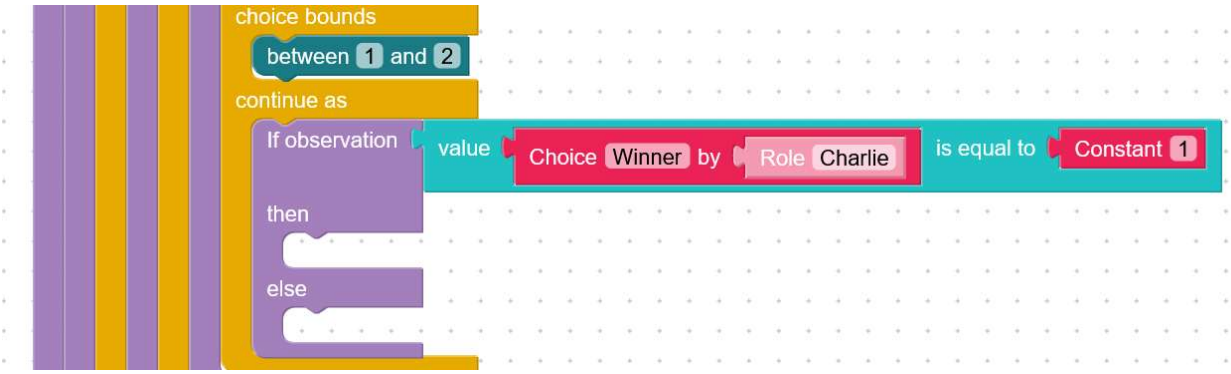


Now that we are sure the both deposits have been made, we want Charlie to make a choice. This is again an external action and we need the when block for this. We set a new timeout for Charlie's action to be made. We can now pick from the actions section the choice block and slide it into the when slot. We can give it a name as for example "Winner". Then we have to decide who makes the choice. We slide in the role block and set it to Charlie.

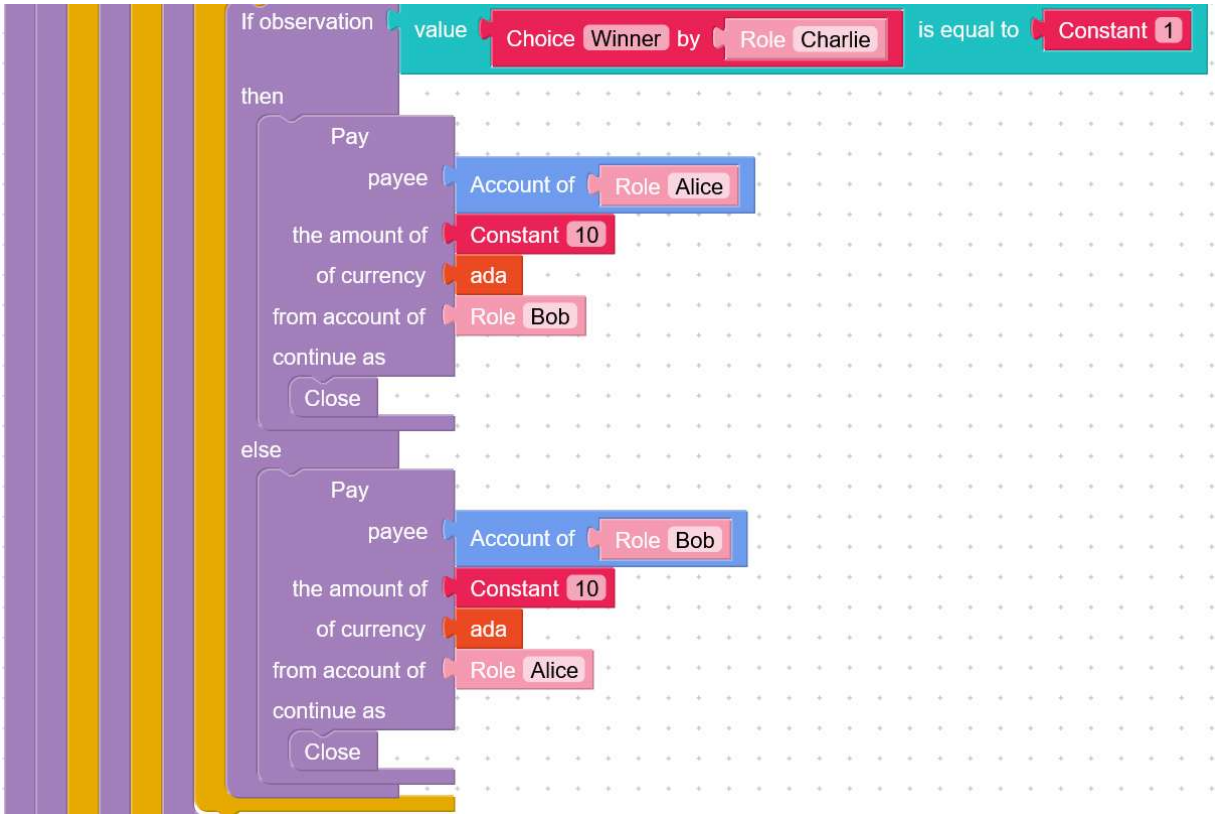
Next, we can specify what values the choice can have. And that is numeric so we can assign the number one to Alice and the number two to Bob. For this we can pick from the bounds section the between block and set the values to one and two.



The next action depends on that choice the Charlie made. In the contracts section we can use the if block and slide it into the empty slot of the choice block. First, we need to set an observation and we pick the value-equality block. We want to compare the choice that Charlie made with the value one. We can click on the value section and select the choice block. There we set the name to the choice name that for us is “Winner” and we set the role to Charlie which made the choice. There can be multiple choices in a smart contract so for this reason we have to specify a name of the choice. Then we set the value to which we compare the choice to, to one. Now we have defined the conditions for the actions that will be executed in this block.



In the then branch we can take a pay block from the contract section. The payee is the person who gets the money. And we have two choices which can be an internal account or it can be an external party. In our case it does not matter because in the end when we close the contract all the parties get the money from their internal accounts. So, from the payee section we pick the **Account** block and in it we chose Alice. Then we specify how much Alice will get and in what currency. We chose 10 lovelace. Next, we have to set who pays this money. This must be an internal account because payments are triggered from internal accounts that are under the control of the smart contract. And we select Bob. So, this means Bob will pay from his internal account 10 lovelace to Alice's internal account and she will have all together now 20 lovelace. Afterwards we can close the contract. When we do this all the money from the internal accounts will be paid to the external accounts so, Alice will get the 20 lovelace to her external account. For the else branch of the if-block we copy the pay block and just reverse the roles.



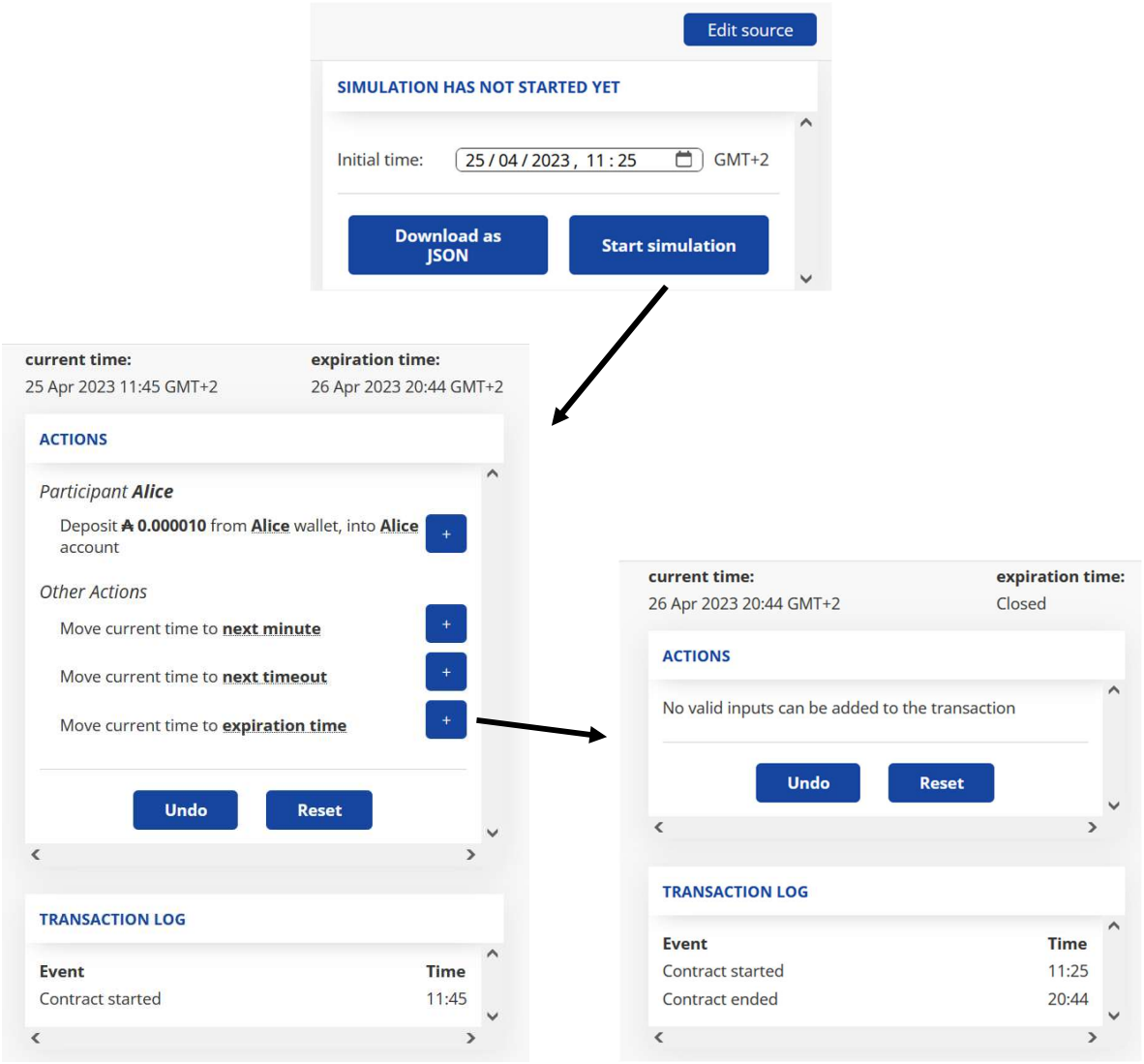
We can now click on the **View as Marlowe** button on the right upper corner of the playground and we will see the pure Marlowe code from the graphical contract that we have made.

```
When
    [Case
        (Deposit
            (Role "Alice")
            (Role "Alice")
            (Token "" "")
```

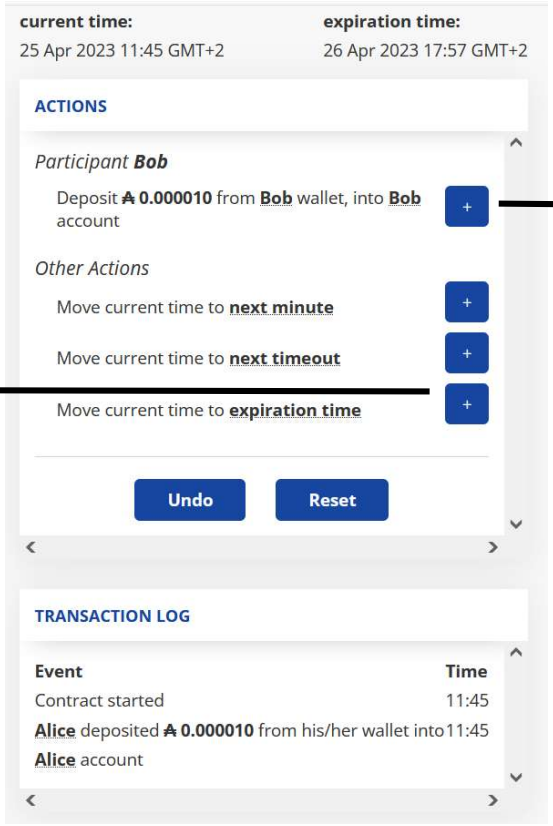
```
            (Constant 10)
        )
        (When
            [Case
                (Deposit
                    (Role "Bob")
                    (Role "Bob")
                    (Token "" "")
                    (Constant 10)
                )
                (When
                    [Case
                        (Choice
                            (ChoiceId
                                "Winner"
                                (Role "Charlie")
                            )
                            [Bound 1 2]
                        )
                        (If
                            (ValueEQ
                                (ChoiceValue
                                    (ChoiceId
                                        "Winner"
                                        (Role "Charlie")
                                    ))
                                (Constant 1)
                            )
                            (Pay
                                (Role "Bob")
                                (Account (Role "Alice"))
                                (Token "" "")
                                (Constant 10)
                                Close
                            )
                            (Pay
                                (Role "Alice")
                                (Account (Role "Bob"))
                                (Token "" "")
                                (Constant 10)
                                Close
                            )
                        )]
                    1682514671117 Close
                )]
            1682524671117 Close
        )]
    1682534671117 Close
```

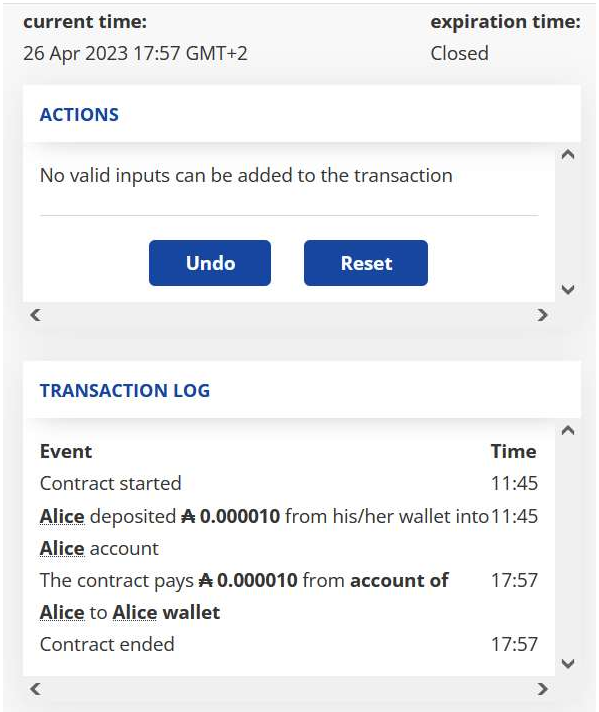
We can send this now to the simulator by clicking on the **Send To Simulator** button also in the upper right corner. Before we start the simulation, we can pick the initial time. We also have the option to download the smart contract as a JSON file. If we click on the **Start simulation** button, we get the first action that we can choose. The possibilities are that Alice makes her deposit, that she waits for a minute, that she waits until the next timeout or that she waits until

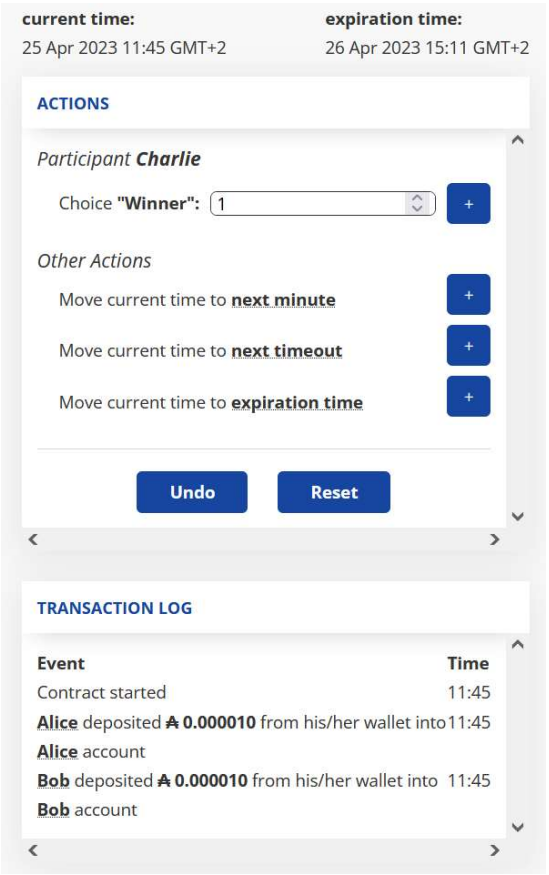
the contract timeouts. If we click on the **+** button for waiting until the contract terminates no more actions will be available and everyone gets their money back.



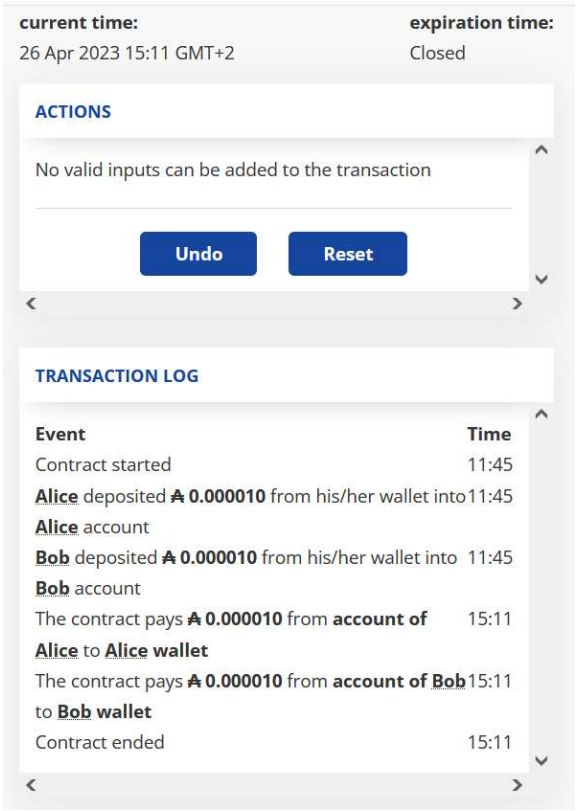
If we however make the deposit then again, the action window for a deposit gets displayed and the name changes from Alice to Bob. On the left side where we see the Marlowe code in the editor the part of the contract that was not executed is highlighted and the part where Alice had to make the deposit is not highlighted anymore. If we choose now for Bob's action to wait until the expiration time of the contract, we will get a transaction log that says that Alice has deposited 10 lovelace from her wallet into her internal account. And then the contract paid back 10 lovelace to Alice's wallet. If however, we choose that Bob makes his deposit another action window opens for Charlie. And he can choose to make a numeric choice or also wait until the contract terminates. We also see that again a smaller part of the Marlowe code gets highlighted to indicate that the smart contract has progressed.

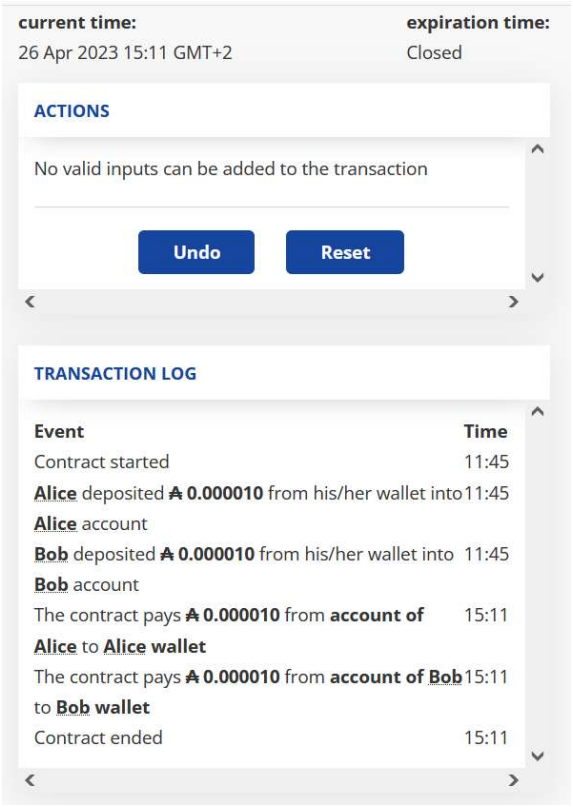

current time:
25 Apr 2023 11:45 GMT+2
expiration time:
26 Apr 2023 17:57 GMT+2
ACTIONS
Participant Bob
Deposit ₳ 0.000010 from Bob wallet, into Bob account
Other Actions
Move current time to next minute
Move current time to next timeout
Move current time to expiration time
Undo
Reset
TRANSACTION LOG
Event
Time
Contract started
11:45
Alice deposited ₳ 0.000010 from his/her wallet into Alice account
11:45


current time:
26 Apr 2023 17:57 GMT+2
expiration time:
Closed
ACTIONS
No valid inputs can be added to the transaction
Undo
Reset
TRANSACTION LOG
Event
Time
Contract started
11:45
Alice deposited ₳ 0.000010 from his/her wallet into Alice account
11:45
The contract pays ₳ 0.000010 from account of Alice to Alice wallet
17:57
Contract ended
17:57


current time:
25 Apr 2023 11:45 GMT+2
expiration time:
26 Apr 2023 15:11 GMT+2
ACTIONS
Participant Charlie
Choice "Winner":
1
Other Actions
Move current time to next minute
Move current time to next timeout
Move current time to expiration time
Undo
Reset
TRANSACTION LOG
Event
Time
Contract started
11:45
Alice deposited ₳ 0.000010 from his/her wallet into Alice account
11:45
Bob deposited ₳ 0.000010 from his/her wallet into Bob account
11:45

If we now choose that Charlie waits for the contract to expire both Alice and Bob get their funds back which we can see in the transaction log on the left picture below. If, however Charlie decides to make a choice and for example chooses the number 1 then the smart contract will pay all the funds to Alice's wallet. We can see this case in the transaction log on the right figure below. In case Charlie would pick the number 2 Bob would get all the funds.





Once the contract terminates in the Marlowe code the close keyword is highlighted at which the contract terminated. Since the contract can terminate in multiple ways there are multiple close keywords in our contract that can get highlighted. We can now copy the Marlowe code and go to the Haskell editor and paste the code in there. We see there some existing code that already has an import statement and a language pragma added for a simple close contract. We can copy our code into the contract variable. We can now leverage the fact that we write Haskell code and generalize our contract by creating some helper functions.

```
{-# LANGUAGE OverloadedStrings #-}

import Language.Marlowe.Extended.V1 hiding (contract)

main :: IO ()
main = printJSON $ contract "Charles" "Simon" "Alex" $ Constant 100

choiceId :: Party -> ChoiceId
choiceId p = ChoiceId "Winner" p
```

```
contract :: Party -> Party -> Party -> Value -> Contract
contract alice bob charlie deposit =
    When
        [ f alice bob
        , f bob alice
        ]
        1682514671117 Close
  where
    f :: Party -> Party -> Case
    f x y =
        Case
            (Deposit
                x
                x
                ada
                deposit
            )
            (When
                [Case
                    (Deposit
                        y
                        y
                        ada
                        deposit
                    )
                    (When
                        [Case
                            (Choice
                                (choiceId charlie)
                                [Bound 1 2]
                            )
                            (If
                                (ValueEQ
                                    (ChoiceValue $ choiceId charlie)
                                    (Constant 1)
                                )
                                (Pay
                                    bob
                                    (Account alice)
                                    ada
                                    deposit
                                    Close
                                )
                                (Pay
                                    alice
                                    (Account bob)
                                    ada
                                    deposit
                                    Close
                                )
                            )]
                        1682534671117 Close
                    )]
                1682524671117 Close
            )
```

We first create the *choiceId* helper function that takes in a party and returns a choice ID. Then in our contact we add four variables: three parties and a value. We create a helper function that takes in two parties and returns a case. Now we can change the contract such that Bob can also make the deposit first, not just Alice. And in the main function we can set the names of the roles as we want to. If we compile this code and send this to the simulator, we have two possible actions in the first step. Charles can make a deposit or Simon can make the deposit. This code can be executed from our VSCode container with the following command:

```
/workspace/# cd code/Week07
/workspace/code/Week07# cabal run marlowe
```

This code will print the contract as a JSON object to the console. We see now that using the Haskell editor to write a Marlowe contract has the advantage that we can parameterize our contracts, generalize our code and avoid code duplication. In Blockly you cannot use local definitions so it is not possible to generalize a contract. We could for instance write in Haskell also a contract that takes in an arbitrary list of parties and then each of them has to make a deposit. We see also that Marlowe in contrast to Plutus uses very basic Haskell features. We do not need lenses, template Haskell, monads or type level programming. Marlowe is not always appropriate because it is specifically used for financial contracts but if it is appropriate, it is a very nice option because of all the safety assurances it has and the simpler way of coding.

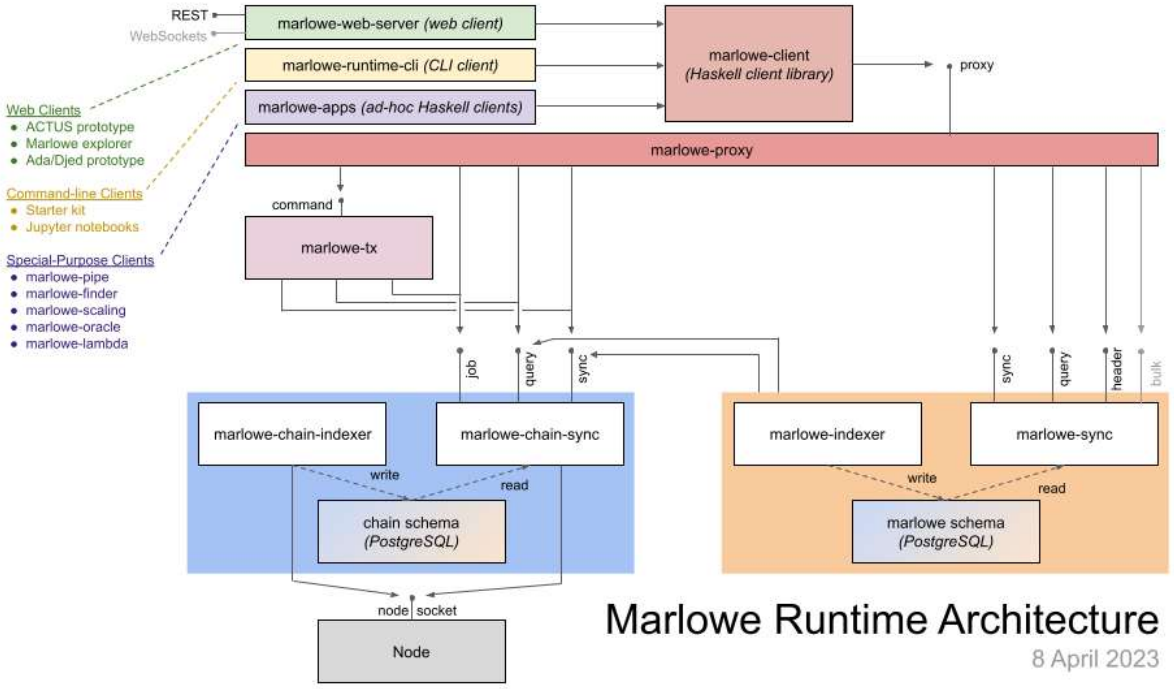
7.2 Homework

Modify the contract from the previous chapter such that Charlie should put down a deposit in the beginning of twice the deposit that Alice and Bob will put down. And if he then does not make a choice when it is his turn Alice and Bob get each half of that deposit.

7.3 Marlowe Starter Kit: Docker

We will now demonstrate how to deploy marlowe-run locally using docker compose so that it can be used with the Marlowe starter kit. The Marlowe starter kit is a series of tutorials that show how to use Marlowe tools and APIs for a variety of simple contracts. In order to run these tutorials, we will need access to a deployment of Marlowe runtime and there are a couple of ways to access that. One is to do it via a cloud service like demeter.run which has a Marlowe runtime extension included. Another way is to use docker and deploy Marlowe runtime locally. In this chapter we will cover how to use docker. The commands presented in this chapter work on a Linux type OS. The instructions can be found at the Marlowe starter kit github repository: https://github.com/input-output-hk/marlowe-starter-kit/blob/main/docker.md

It covers two parts. First are the instructions for deployment. After that once you have the deployment running you need to set some environment variables. The environment variables give you access to the Cardano node, Marlowe proxy service and web access to Marlowe runtime. Below you can see a diagram of the Marlowe runtime architecture.



It is based on a node, a postgre database, a pair of indexing services and a pair of synchronization services that support queries and other operations. There is also a transaction building service. The Marlowe proxy pulls these all together so you do not have to access individual services. Instead, you can use a single endpoint. So, if you are using Marlowe runtime as a client you have several options. One is using a Marlowe rest API that runs on a Marlowe web server. The other option is to use the command line executable that lets you access Marlowe runtime. If you are writing Haskell code you actually have a library where you can access Marlowe runtime. But all these depend on actually having runtime operating so we are going to show you how to use docker compose to start up the Marlowe runtime services. Installation instructions can be found at https://docs.docker.com/compose/install/. After you have installed it, you can clone the Marlowe starter kit to your computer.

```
$ git clone https://github.com/input-output-hk/marlowe-starter-kit.git
```

Then you will need to install nix. Instructions can be found at https://nixos.org/download.html. You will also have to enable Nix flaskes support. You can do this with the following command:

```
mkdir -p ~/.config/nix
```

```
echo "experimental-features = nix-command flakes" >> ~/.config/nix/nix.conf
```

After that cd into the Marlowe starter kit folder and start a nix shell.

```
$ cd marlowe-starter-kit && nix develop
```

This is going to give us access to a lot of Marlowe tools at the command line. We have to decide which network we are going to use. For our example let us pick the pre-production network.

```
$ export NETWORK=preprod
```

Now we can start the docker container with the following command:

```
$ docker-compose up -d
```

This will bring up all the services in the background. You will see some messages displayed that indicate all the services previously shown in the diagram are being started. Exit codes 0 indicate that things are working fine. After the startup process finishes, we can check that all of the services were created and have the status **Up**. To do this we execute the following command:

```
$ docker-compose ps
```

Below is an example output of the above command:

```
               Name                              Command                  State
-------------------------------------------------------------------------------------------
marlowe-starter-kit_chain-indexer_1   /bin/entrypoint                Up
marlowe-starter-kit_chain-sync_1      /bin/entrypoint                Up
marlowe-starter-kit_indexer_1         /bin/entrypoint                Up
marlowe-starter-kit_node_1            entrypoint                     Up (healthy)
marlowe-starter-kit_postgres_1        docker-entrypoint.sh postgres  Up (healthy)
marlowe-starter-kit_proxy_1           /bin/entrypoint                Up
marlowe-starter-kit_sync_1            /bin/entrypoint                Up
marlowe-starter-kit_tx_1              /bin/entrypoint                Up
marlowe-starter-kit_web-server_1      /bin/entrypoint                Up
```

Next, we have to set up a bunch of environment variables. We copy the following code to a terminal window and execute it:

```
# Only required for `marlowe-cli` and `cardano-cli`.
case "$NETWORK" in
  preprod)
    export CARDANO_TESTNET_MAGIC=1
    ;;
  preview)
    export CARDANO_TESTNET_MAGIC=2
    ;;
  *)
```

```
    # Use `mainnet` as the default.
    unset CARDANO_TESTNET_MAGIC
    ;;
esac

# Only required for `marlowe-runtime-cli`.
export MARLOWE_RT_HOST="127.0.0.1"
export MARLOWE_RT_PORT=3700

# Only required for REST API.
export MARLOWE_RT_WEBSERVER_HOST="127.0.0.1"
export MARLOWE_RT_WEBSERVER_PORT=3780
export
MARLOWE_RT_WEBSERVER_URL="http://$MARLOWE_RT_WEBSERVER_HOST:$MARLOWE_RT_WEBSERVER_PORT
"
```

Now we can execute a command to check the sync progress of the Cardano node.

```
cardano-cli query tip --testnet-magic "$CARDANO_TESTNET_MAGIC" | json2yaml
```

Below is an example output of the above command:

```
block: 731649
epoch: 57
era: Babbage
hash: ad9ed62386341677ada848804618b6198c1d4187a0bfbd008500d2ef9595943b
slot: 23307149
syncProgress: '100.00'
```

After we see the node has synced, we can try to discover one of the Marlowe contracts using the Marlowe command line tool.

```
marlowe-runtime-cli log
"f06e4b760f2d9578c8088ea5289ba6276e68ae3cbaf5ad27bcfc77dc413890db#1"
```

Below is an example output of the above command:

```
transaction f06e4b760f2d9578c8088ea5289ba6276e68ae3cbaf5ad27bcfc77dc413890db
(creation)
ContractId:      f06e4b760f2d9578c8088ea5289ba6276e68ae3cbaf5ad27bcfc77dc413890db#1
SlotNo:          11630746
BlockNo:         224384
BlockId:         f729f57b3bd99dd1d55a5a65b8c74f459dadae784dd55536444770dd2c2cdd2e
ScriptAddress:   addr_test1wp4f8ywk4fg672xasahtk4t9k6w3aql943uxz5rt62d4dvqu3c6jv
Marlowe Version: 1

transaction 5a3ed57653b4635c76d2949558f3718e34324a8a1ffc740360ae7d85839de6d9 (close)
ContractId: f06e4b760f2d9578c8088ea5289ba6276e68ae3cbaf5ad27bcfc77dc413890db#1
SlotNo:     16674281
BlockNo:    458964
BlockId:    23adcef9d89afc4354155c62960512c810619d8244239f8c642745b034a58fc2
Inputs:     []
```

If the command is successful, you will see the output above that indicates it found the creation and close parts of the contract. We also have to check the web server for which we can use curl to health check it. The following command will check that one can communicate with the Marlowe runtime web server for which the received response should be 200 OK:

```
curl -sSI "$MARLOWE_RT_WEBSERVER_URL/healthcheck"

HTTP/1.1 200 OK
Date: Thu, 16 Mar 2023 18:12:51 GMT
Server: Warp/3.3.24
Content-Type: application/json;charset=utf-8
```

We can now start up the jupyter notebook with the following command:

```
$ nix run
```

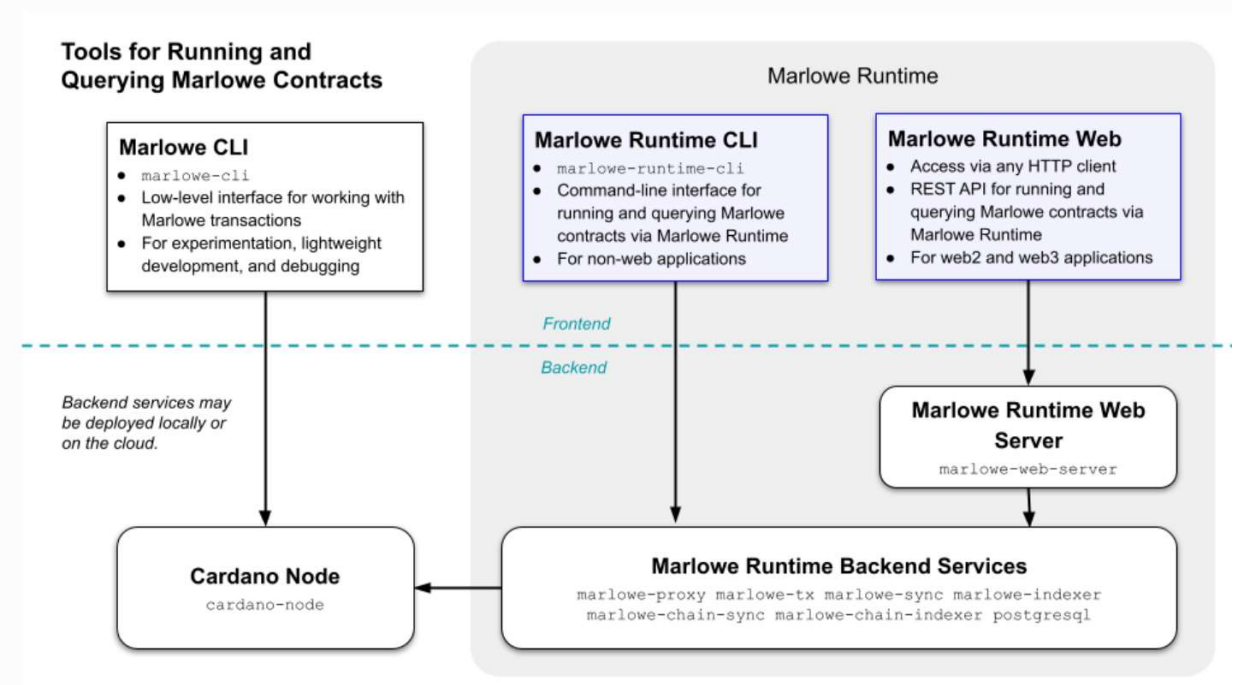
This command starts up a jupyter notebook server that has a bash kernel that contains all the Marlowe tools installed as the Cardano CLI and the Cardano wallet. We can open the notebook in our browser if we type in the address localhost:8891/lab. From there we will have access to all the notebooks from the starter kit. You can also create your own notebooks. There is a special kernel bash with Marlowe tools icon that you can use from the editor.

7.4 Marlowe Starter Kit: Preliminaries

In this chapter we will demonstrate how to create the signing keys and addresses for parties in the tutorials of the Marlowe starter kit. Because these tutorials are run on the blockchain we need to create the cryptographic keys and addresses for them. We will need access to a node which is available on demeter.run or you can run docker locally as described in the previous chapter. In this chapter we are going to use docker and a jupyter notebook. We launch it with the *nix run* command as described in the previous chapter. After it got started we open up the notebook editor in our browser at localhost:8891/lab and open the notebook 00-preliminaries.ipynb. There are several ways of running Marlowe transactions. One way is to use the marlowe-cli that does not use the Marlowe runtime backend. The other two ways are having command line access and web access to the backend. In the diagram below you can see that Marlowe cli just uses the Cardano node. It is a fairly low-level interface. You can use it for experimentation, debugging and things like that. But you do have to do your own UTXO management, so it is a bit tedious. The Marlowe runtime backend services indexes the blockchain, interprets Marlowe transactions, helps you to build transactions and allows you to query Marlowe history. So, this gives you a lot more power. You can access that through the Marlowe runtime cli or you can use any http client and use marlowe's REST API. And that goes

through the Marlowe runtime web server. Right now, we just need the Cardano node. In the chapter where tutorials for Marlowe contracts are presented, we are going to use a variety of these other methods. First, we have to set some environment variables to normalize things and save some typing later. This will find the node socket.

```
if [[ -z "$MARLOWE_RT_PORT" ]]
then

  # Only required for `marlowe-cli` and `cardano-cli`.
  export CARDANO_NODE_SOCKET_PATH="$(docker volume inspect marlowe-starter-kit_shared
| jq -r '.[0].Mountpoint')/node.socket"
  export CARDANO_TESTNET_MAGIC=1 # Note that preprod=1 and preview=2. Do not set this
variable if using mainnet.

fi

# FIXME: This should have been inherited from the parent environment.
if [[ -z "$CARDANO_NODE_SOCKET_PATH" ]]
then
  export CARDANO_NODE_SOCKET_PATH=/ipc/node.socket
fi

# FIXME: This should have been set in the parent environment.
if [[ -z "$CARDANO_TESTNET_MAGIC" ]]
then
  export CARDANO_TESTNET_MAGIC=$CARDANO_NODE_MAGIC
fi

echo "CARDANO_NODE_SOCKET_PATH = $CARDANO_NODE_SOCKET_PATH"
echo "CARDANO_TESTNET_MAGIC = $CARDANO_TESTNET_MAGIC"
```



The main thing is to fund the parties. It is convenient to create what might be called a faucet pair of keys that let you store a whole bunch of test ada for distributing to the parties and the contracts. So, we are going to set the file names for that and then we are going to generate the keys using the Cardano cli. You can go to the Cardano developer portal and learn about all the details of the addresses and keys. At the end we can compute the address of the faucet.

```
FAUCET_SKEY=keys/faucet.skey
FAUCET_VKEY=keys/faucet.vkey

if [[ ! -e "$FAUCET_SKEY" ]]
then
  cardano-cli address key-gen \
    --signing-key-file "$FAUCET_SKEY" \
    --verification-key-file "$FAUCET_VKEY"
fi

FAUCET_ADDR=$(cardano-cli address build --testnet-magic "$CARDANO_TESTNET_MAGIC" --
payment-verification-key-file "$FAUCET_VKEY" )
echo "$FAUCET_ADDR" > keys/faucet.address
echo "FAUCET_ADDR = $FAUCET_ADDR"
```

One of the parties in the tutorial is called the lender which also has keys. We generate them and compute their address. And we do the same for a borrower and a mediator.

```
LENDER_SKEY=keys/lender.skey
LENDER_VKEY=keys/lender.vkey

if [[ ! -e "$LENDER_SKEY" ]]
then
  cardano-cli address key-gen \
    --signing-key-file "$LENDER_SKEY" \
    --verification-key-file "$LENDER_VKEY"
fi

LENDER_ADDR=$(cardano-cli address build --testnet-magic "$CARDANO_TESTNET_MAGIC" --
payment-verification-key-file "$LENDER_VKEY" )
echo "$LENDER_ADDR" > keys/lender.address
echo "LENDER_ADDR = $LENDER_ADDR"
```

```
BORROWER_SKEY=keys/borrower.skey
BORROWER_VKEY=keys/borrower.vkey

if [[ ! -e "$BORROWER_SKEY" ]]
then
  cardano-cli address key-gen \
    --signing-key-file "$BORROWER_SKEY"  \
    --verification-key-file "$BORROWER_VKEY"
fi

BORROWER_ADDR=$(cardano-cli address build --testnet-magic "$CARDANO_TESTNET_MAGIC" --
```

```
payment-verification-key-file "$BORROWER_VKEY" )
echo "$BORROWER_ADDR" > keys/borrower.address
echo "BORROWER_ADDR = $BORROWER_ADDR"
```

```
MEDIATOR_SKEY=keys/mediator.skey
MEDIATOR_VKEY=keys/mediator.vkey

if [[ ! -e "$MEDIATOR_SKEY" ]]
then
  cardano-cli address key-gen \
    --signing-key-file "$MEDIATOR_SKEY"  \
    --verification-key-file "$MEDIATOR_VKEY"
fi

MEDIATOR_ADDR=$(cardano-cli address build --testnet-magic "$CARDANO_TESTNET_MAGIC" --
payment-verification-key-file "$MEDIATOR_VKEY" )
echo "$MEDIATOR_ADDR" > keys/mediator.address
echo "MEDIATOR_ADDR = $MEDIATOR_ADDR"
```

To obtain some test ada we can go to the Cardano faucet and request it for our faucet address: https://docs.cardano.org/cardano-testnet/tools/faucet

We will now fund our lender, borrower and mediator addresses with a 1000 test ada. For this you can use the Daedalus wallet (https://daedaluswallet.io/) any simply send the test ada from the faucet address to the other three addresses. We can also use the Marlowe cli to send funds.

```
# Note that `FAUCET_ADDR` must have already been funded with test ada.

# 1 ada = 1,000,000 lovelace
ADA=1000000

# Send 1000 ada
AMOUNT=$((1000 * ADA))

# Execute the transaction.
marlowe-cli util fund-address \
 --lovelace "$AMOUNT" \
 --out-file /dev/null \
 --source-wallet-credentials "$FAUCET_ADDR":"$FAUCET_SKEY" \
 --submit 600 \
 "$LENDER_ADDR" "$BORROWER_ADDR" "$MEDIATOR_ADDR"
```

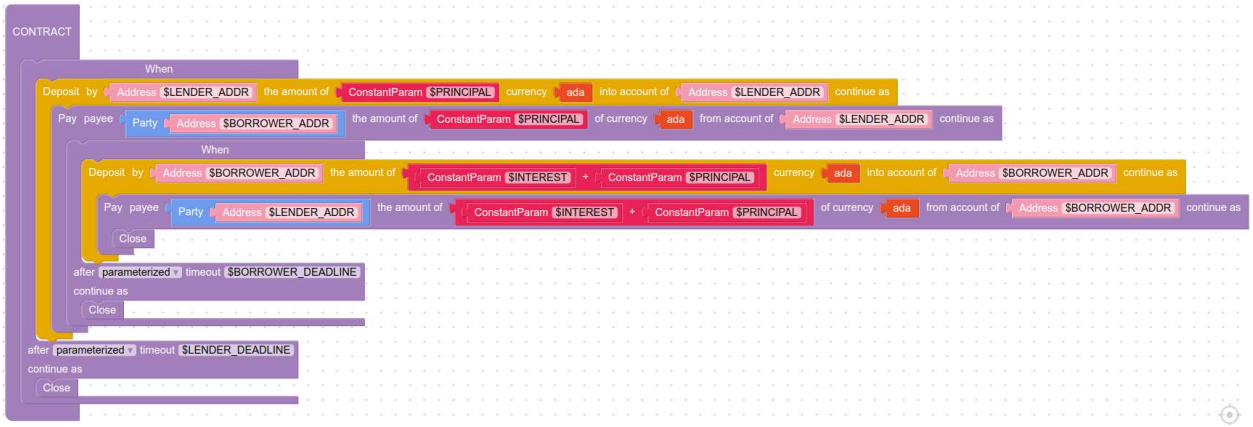
And with the code below we can check if the test ada has arrived at our addresses.

```
echo
echo "Lender @ $LENDER_ADDR"
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$LENDER_ADDR"
echo

echo
```

```
echo "Borrower @ $BORROWER_ADDR"
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$BORROWER_ADDR"
echo

echo
echo "Mediator @ $MEDIATOR_ADDR"
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$MEDIATOR_ADDR"
echo
```

7.5 Marlowe Starter Kit: ZCB using the Marlowe Runtime command-line client

The ZCB stands for zero coupon bond and this tutorial will be about contracts that implement them. For this demonstration we will need Marlowe runtime back-end services. There are a couple of ways to get those running. One is through demeter.run which is cloud hosting that has an extension for Marlowe runtime. Another way is to run things locally with docker. We will provide here instructions for docker. First you need to start up all the Marlowe services as described in the previous chapter. We will again use jupyter notebooks. We can start the editor again by running a nix shell from inside the Marlowe starter kit repo and then execute *nix run*. This builds the jupyter lab server and launches it. We can use the notebook 01-runtime-cli.ipynb. A zero-coupon bond is basically a loan where someone loans principal and then at a later time they get paid back principal and interest. So, in the Marlowe playground there is actually an example you can open there where you have the principal deposit by a lender and the borrower receives it. It is all measured in ada and then later on the borrower pays back the interest and the principal to the lender. It is a very simple contract.



And below you can also see the Marlowe code for this contract.

```
When
    [Case
        (Deposit
            (Address "$LENDER_ADDR")
            (Address "$LENDER_ADDR")
```

```
            (Token "" "")
            (ConstantParam "$PRINCIPAL")
        )
        (Pay
            (Address "$LENDER_ADDR")
            (Party (Address "$BORROWER_ADDR"))
            (Token "" "")
            (ConstantParam "$PRINCIPAL")
            (When
                [Case
                    (Deposit
                        (Address "$BORROWER_ADDR")
                        (Address "$BORROWER_ADDR")
                        (Token "" "")
                        (AddValue
                            (ConstantParam "$INTEREST")
                            (ConstantParam "$PRINCIPAL")
                        )
                    )
                    (Pay
                        (Address "$BORROWER_ADDR")
                        (Party (Address "$LENDER_ADDR"))
                        (Token "" "")
                        (AddValue
                            (ConstantParam "$INTEREST")
                            (ConstantParam "$PRINCIPAL")
                        )
                        Close
                    )]
                (TimeParam "$BORROWER_DEADLINE")
                Close
            )
        )]
    (TimeParam "$LENDER_DEADLINE")
    Close
```

To run this contract, we need to access the node socket and the Marlowe proxy server. Those and embodied in four environment variables:

- CARDANO_NODE_SOCKET_PATH: location of Cardano node's socket.
- CARDANO_TESTNET_MAGIC: testnet magic number.
- MARLOWE_RT_HOST: IP address of the Marlowe Runtime proxy server.
- MARLOWE_RT_PORT: Port number for the Marlowe Runtime proxy server.

Then we also need keys and addresses for the parties. We have a lender and a borrower in the keys folder. They are now already funded each with a 1000 ada. For convenience we also set some other environment variables just to make sure everything is easy to run. Then we can do it independent of whether we are using a cloud service or docker service.

```
if [[ -z "$MARLOWE_RT_PORT" ]]
then

  # Only required for `marlowe-cli` and `cardano-cli`.
```

```
  export CARDANO_NODE_SOCKET_PATH="$(docker volume inspect marlowe-starter-kit_shared
| jq -r '.[0].Mountpoint')/node.socket"
  export CARDANO_TESTNET_MAGIC=1 # Note that preprod=1 and preview=2. Do not set this
variable if using mainnet.

  # Only required for `marlowe-runtime-cli`.
  export MARLOWE_RT_HOST="127.0.0.1"
  export MARLOWE_RT_PORT=3700

fi

# FIXME: This should have been inherited from the parent environment.
if [[ -z "$CARDANO_NODE_SOCKET_PATH" ]]
then
  export CARDANO_NODE_SOCKET_PATH=/ipc/node.socket
fi

# FIXME: This should have been set in the parent environment.
if [[ -z "$CARDANO_TESTNET_MAGIC" ]]
then
  export CARDANO_TESTNET_MAGIC=$CARDANO_NODE_MAGIC
fi

case "$CARDANO_TESTNET_MAGIC" in
  1)
    export "EXPLORER_URL=https://preprod.cardanoscan.io"
    ;;
  2)
    export "EXPLORER_URL=https://preview.cardanoscan.io"
    ;;
  *)
    # Use `mainnet` as the default.
    export "EXPLORER_URL=https://cardanoscan.io"
    ;;
esac

echo "CARDANO_NODE_SOCKET_PATH = $CARDANO_NODE_SOCKET_PATH"
echo "CARDANO_TESTNET_MAGIC = $CARDANO_TESTNET_MAGIC"
echo "MARLOWE_RT_HOST = $MARLOWE_RT_HOST"
echo "MARLOWE_RT_PORT = $MARLOWE_RT_PORT"
```

It prints out the four environment variables that we have listed above. We check the lender and borrower address funds with the following commands:

```
LENDER_SKEY=keys/lender.skey
LENDER_ADDR=$(cat keys/lender.address)
echo "LENDER_ADDR = $LENDER_ADDR"

cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$LENDER_ADDR"
echo "$EXPLORER_URL"/address/"$LENDER_ADDR"
```

```
BORROWER_SKEY=keys/borrower.skey
BORROWER_ADDR=$(cat keys/borrower.address)
```

```
echo "BORROWER_ADDR = $BORROWER_ADDR"

cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$BORROWER_ADDR"
echo "$EXPLORER_URL"/address/"$BORROWER_ADDR"
```

Now we have to design the contract. We are going to use 80 ada of principal and 5 ada of interest. And so, we will again use environment variables for that because it is less error prone.

```
ADA=1000000  # 1 ada = 1,000,000 lovelace

PRINCIPAL=$((80 * ADA))
INTEREST=$((5 * ADA))

echo "PRINCIPAL = $PRINCIPAL lovelace"
echo "INTEREST = $INTEREST lovelace"

MIN_LOVELACE="$((2 * ADA))"
echo "MIN_LOVELACE = $MIN_LOVELACE lovelace"

SECOND=1000 # 1 second = 1000 milliseconds
MINUTE=$((60 * SECOND)) # 1 minute = 60 seconds
HOUR=$((60 * MINUTE)) # 1 hour = 60 minutes

NOW="$((`date -u +%s` * SECOND))"
echo NOW = "$NOW" POSIX milliseconds = "`date -d @$((NOW / SECOND))`"

LENDER_DEADLINE="$((NOW + 1 * HOUR))"
BORROWER_DEADLINE="$((NOW + 3 * HOUR))"
echo LENDER_DEADLINE = "$LENDER_DEADLINE" POSIX milliseconds = "`date -d
@$((LENDER_DEADLINE / SECOND))`"
echo BORROWER_DEADLINE = "$BORROWER_DEADLINE" POSIX milliseconds = "`date -d
@$((BORROWER_DEADLINE / SECOND))`"
```

When you put a Marlowe contract on a blockchain you have to put some minimum ada in it. There is a ledger rule called minimum UTXO so we are going to put a nominal 2 ada in that which will live in the contract the whole time just to keep it on the blockchain. And we are going to have a deadline so we have also set some time parameters. The contract has two deadlines. One is for the lender to deposit the ada that is going to be lent and the other is for the borrower to pay back the principal and the interest. Now we create the JSON file for the contract with the help of the Marlowe cli and we name it zcb-contract.json.

```
marlowe-cli template zcb \
  --minimum-ada "$MIN_LOVELACE" \
  --lender "$LENDER_ADDR" \
  --borrower "$BORROWER_ADDR" \
  --principal "$PRINCIPAL" \
  --interest "$INTEREST" \
  --lending-deadline "$LENDER_DEADLINE" \
  --repayment-deadline "$BORROWER_DEADLINE" \
  --out-contract-file zcb-contract.json \
  --out-state-file zcb-state.json
```

To get the options of the Marlowe cli for the zcb contract you can use the help option.

```
marlowe-cli template zcb --help
```

We can also look at the contract displayed in yaml format if we use the json2yaml command.

```
json2yaml zcb-contract.json
```

Below is a shortened example output of the above command:

```
timeout: 1679606325000
timeout_continuation: close
when:
- case:
    deposits: 80000000
    into_account:
      address: addr_test1vqd3yrtjyx49uld43lvwqaf7z4k03su8gf2x4yr7syzvckgfzm4ck
    of_token:
      currency_symbol: ''
      token_name: ''
    party:
      address: addr_test1vqd3yrtjyx49uld43lvwqaf7z4k03su8gf2x4yr7syzvckgfzm4ck
  then:
    from_account:
      address: addr_test1vqd3yrtjyx49uld43lvwqaf7z4k03su8gf2x4yr7syzvckgfzm4ck
    pay: 80000000
    then:
      timeout: 1679613525000
      timeout_continuation: close
      when:
      - case:
          deposits:
            add: 80000000
            and: 5000000
          into_account:
...
        address: addr_test1vpy4n4peh4suv0y55yptur0066j5kds8r4ncnuzm0vpzfgg0dhz6d
    token:
      currency_symbol: ''
      token_name: ''
```

We see the various Marlowe constructs. Then it is always good to check the safety of the contract. You should always do that if you want to use the contract on the main-net. Here are the steps for checking the safety of a contract:

1. Understand the Marlowe Language (https://marlowe-finance.io/).
2. Understand Cardano's Extended UTxO Model (https://docs.cardano.org/learn/eutxo-explainer).
3. Read and understand the Marlowe Best Practices Guide (https://github.com/input-output-hk/marlowe-cardano/blob/main/marlowe/best-practices.md).

4. Read and understand the Marlowe Security Guide (https://github.com/input-output-hk/marlowe-cardano/blob/main/marlowe/security.md).
5. Use Marlowe Playground (https://play.marlowe-finance.io/) to flag warnings, perform static analysis, and simulate the contract.
6. Use Marlowe CLI's (https://github.com/input-output-hk/marlowe-cardano/blob/main/marlowe-cli/ReadMe.md) *marlowe-cli run analyze* tool to study whether the contract can run on a Cardano network.
7. Run *all execution paths* of the contract on a Cardano testnet (https://docs.cardano.org/cardano-testnet/overview).

Here we will use step 6. First, we have to sort of bundle the contract and its initial state into a Marlowe file. We can do this with the following command.

```
marlowe-cli run initialize \
  --permanently-without-staking \
  --contract-file zcb-contract.json \
  --state-file zcb-state.json \
  --out-file zcb-marlowe.json
```

And then we are going to use the analysis tool. This tool looks at the contract and figures out all possible execution steps like all possible future operations of the contract. And it checks each transaction that might occur in the future. So, this is basically a 100% coverage of what the contract might do. It is checking a bunch of things to make sure you have not made a mistake with the names of parties, the addresses, native tokens and things like that. You can run this analysis with the following command.

```
marlowe-cli run analyze --marlowe-file zcb-marlowe.json
```

The output will look something like the following:

```
Note that path-based analysis ignore the initial state of the contract and instead
start with an empty state.
Starting search for execution paths . . .
 . . . found 3 execution paths.
- Preconditions:
    Duplicate accounts: []
    Duplicate bound values: []
    Duplicate choices: []
    Invalid account parties: []
    Invalid account tokens: []
    Invalid choice parties: []
    Invalid roles currency: false
    Non-positive account balances: []
- Role names:
    Blank role names: false
    Invalid role names: []
```

```
- Tokens:
    Invalid tokens: []
- Maximum value:
    Actual: 88
    Invalid: false
    Maximum: 5000
    Percentage: 1.76
    Unit: byte
- Minimum UTxO:
    Requirement:
      Lovelace: 1120600
- Execution cost:
    Memory:
      Actual: 6588250
      Invalid: false
      Maximum: 14000000
      Percentage: 47.058928
    Steps:
      Actual: 1767311958
      Invalid: false
      Maximum: 1000000000
      Percentage: 17.67311958
- Transaction size:
    Actual: 1630
    Invalid: false
    Maximum: 16384
    Percentage: 9.94873046875
```

We see in the output there are no problems, nothing is invalid. The contract initially has to start out with 1.12 ada. And no matter which path you are going down the contract you will never use more than 47% of the Plutus memory limit or 17% of the Plutus step limit. So, you are fine on the Plutus costs. And it is only 1.5 kB so it is only using about 10% of the budget for storing a contract on the chain. So, we are in good shape there. First, we are going to create the contract. The Marlowe runtime cli command line tool has a create command. You can explore it with the help option that lists all the options for this command.

```
marlowe-runtime-cli create --help
```

We are only going to use a couple of the options and with the following command we are creating the contract on the blockchain and specifying who is the person that is creating it.

```
CONTRACT_ID=$(
marlowe-runtime-cli create \
  --core-file zcb-contract.json \
  --min-utxo "$MIN_LOVELACE" \
  --change-address "$LENDER_ADDR" \
  --manual-sign tx-1.unsigned \
| jq -r 'fromjson | .contractId' \
)
echo "CONTRACT_ID = $CONTRACT_ID"
```

The output will look something like this:

```
CONTRACT_ID = 3bd6f3011ebabd0d55182aa657a7b841faa9f4b823d1fd76a215d917d0c61895#1
```

The command creates an unsigned transaction that gets stored in the tx-1.unsigned file. There are multiple ways how to sign this transaction and submit it to the blockchain:

- *cardano-cli* at the command line
- *cardano-wallet* at the command line or as a REST service
- *cardano-hw-cli* for a hardware wallet at the command line
- a Babbage-compatible CIP-30 wallet in a web browser

For convenience we will use marlowe-cli transaction submit. We have to input the unsigned file to the command, the signer key and a timeout of 600 seconds. That is the time we are willing to wait for a confirmation on the blockchain.

```
TX_1=$(
marlowe-cli transaction submit \
  --tx-body-file tx-1.unsigned \
  --required-signer "$LENDER_SKEY" \
  --timeout 600 \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_1 = $TX_1"
```

One may have to wait a minute or so for the transactions to be confirmed on the blockchain. With the command below we can also print the Cardano explorer link for our transaction.

```
echo "$EXPLORER_URL"/transaction/"$TX_1?tab=utxo"
```

And we can also query the transaction with the Cardano cli.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --tx-in "$TX_1#1"
```

Below is an example output of the above query command.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
3bd6f3011ebabd0d55182aa657a7b841faa9f4b823d1fd76a215d917d0c61895     1        2000000
                                                                             lovelace
                                              + TxOutDatumHash ScriptDataInBabbageEra
                    "a3d7e66932dd99e0a1ca6eb6f3f72d1ac2810f04f62571ed817f9559ad12feb1"
```

With the Marlowe runtime command can fetch a contract from the blockchain and print information about it. You can use the following command:

```
marlowe-runtime-cli log --show-contract "$CONTRACT_ID"
```

Below you can see an example output from this command.

```
transaction 3bd6f3011ebabd0d55182aa657a7b841faa9f4b823d1fd76a215d917d0c61895
(creation)
ContractId:      3bd6f3011ebabd0d55182aa657a7b841faa9f4b823d1fd76a215d917d0c61895#1
SlotNo:          23920017
BlockNo:         755074
BlockId:         3235ba612d6df4e536da88890b39cc0fbb96ff3313643dd999ef31ed699f3af2
ScriptAddress:   addr_test1wqhdyccahvnheppng3fut3phhp3jt5m37zp4529ezz535ms2u9jqv
Marlowe Version: 1

    When [
      (Case
         (Deposit (Address
"addr_test1vqd3yrtjyx49uld43lvwqaf7z4k03su8gf2x4yr7syzvckgfzm4ck") (Address
"addr_test1vqd3yrtjyx49uld43lvwqaf7z4k03su8gf2x4yr7syzvckgfzm4ck")
            (Token "" "")
            (Constant 80000000))
         (Pay (Address
"addr_test1vqd3yrtjyx49uld43lvwqaf7z4k03su8gf2x4yr7syzvckgfzm4ck")
            (Party (Address
"addr_test1vpy4n4peh4suv0y55yptur0066j5kds8r4ncnuzm0vpzfgg0dhz6d"))
            (Token "" "")
            (Constant 80000000)
            (When [
               (Case
                  (Deposit (Address
"addr_test1vpy4n4peh4suv0y55yptur0066j5kds8r4ncnuzm0vpzfgg0dhz6d") (Address
"addr_test1vpy4n4peh4suv0y55yptur0066j5kds8r4ncnuzm0vpzfgg0dhz6d")
                     (Token "" "")
                     (AddValue
                        (Constant 80000000)
                        (Constant 5000000)))
                  (Pay (Address
"addr_test1vpy4n4peh4suv0y55yptur0066j5kds8r4ncnuzm0vpzfgg0dhz6d")
...
                     (AddValue
                        (Constant 80000000)
                        (Constant 5000000)) Close))] 1679613525000 Close)))]
1679606325000 Close
```

It gives us the history of where the transaction was created and then you see the contract is printed out in Marlowe format. Other tools can give you even more details. One of them is Marlowe pipe that does a full dump of the contract in JSON format. We can convert it to yaml and read it so we can verify the creation worked. Below is an example command:

```
echo '{"request" : "get", "contractId" : "'"$CONTRACT_ID"'"}' | marlowe-pipe 2>
/dev/null | json2yaml
```

The next thing we have to do is that the lender has to deposit the principle. We will use the Marlowe deposit command. You can check all the options for this command with:

```
marlowe-runtime-cli deposit --help
```

Our command will have to take in the contract ID, the lender address, the principal and the unsigned transaction.

```
TX_2=$(
marlowe-runtime-cli deposit \
  --contract "$CONTRACT_ID" \
  --from-party "$LENDER_ADDR" \
  --to-party "$LENDER_ADDR" \
  --lovelace "$PRINCIPAL" \
  --change-address "$LENDER_ADDR" \
  --manual-sign tx-2.unsigned \
| jq -r 'fromjson | .txId' \
)
echo "TX_2 = $TX_2"
```

Note that if the transaction would violate the logic of the Marlowe contract, one would receive an error message. For example, if we deposit the incorrect amount or deposit, it to the wrong party's internal account, we would get an `TEApplyNoMatchError` error because the Marlowe validator rejects the transaction. There is also a Marlowe debugging cookbook that lists all the error messages and gives you explains timeout concepts: https://github.com/input-output-hk/marlowe-cardano/blob/main/marlowe/debugging-cookbook.md

A very convenient way is to use the Marlowe playground because it has a simulator. So, you can actually run through the contracts by clicking through and entering data and see the contract operate. Basically, if it runs in the simulator, you can mimic that with the command like so there is a one-to-one correspondence between these actions. After we are sure everything is ok, we can submit the transaction, wait a bit and check the address funds.

```
marlowe-cli transaction submit \
  --tx-body-file tx-2.unsigned \
  --required-signer "$LENDER_SKEY" \
  --timeout 600
```

To print the Cardano explorer link, we can use the following command:

```
echo "$EXPLORER_URL"/transaction/"$TX_2?tab=utxo"
```

If we query the lender address, we see that he has 82 ada less than originally. Two ada were deposited in the contract when it was created and 80 ada were paid to the borrower in the second transaction. The borrower has an additional 80 ada (the loan's principal) now.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$LENDER_ADDR"
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$BORROWER_ADDR"
```

Below is the output of the two commands above.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
bc286bb53f50a7355e2765462546e03d17cabde30420b6bfd1531617386facf4     0       917043702
                                                                            lovelace +
                                                                        TxOutDatumNone

                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
8461a35e612b38d4cb592e4ba1b7f13c2ff2825942d66e7200acc575cd4c8f1c     2      1000000000
                                                                            lovelace +
                                                                        TxOutDatumNone
bc286bb53f50a7355e2765462546e03d17cabde30420b6bfd1531617386facf4     2        80000000
                                                                            lovelace +
                                                                        TxOutDatumNone
```

The Marlowe contract still has the 2 ada from its creation. We can again look at the contract and we will see that now we have two transactions. Marlowe contracts are like an onion where you are peeling away the layers to get to the center. We can either use the log or the pipe command. Here are again both commands.

```
marlowe-runtime-cli log --show-contract "$CONTRACT_ID"

echo '{"request" : "get", "contractId" : "'"$CONTRACT_ID"'"}' | marlowe-pipe 2>
/dev/null | json2yaml
```

Now the borrower has to repay the loan, both the principal and the interest.

```
TX_3=$(
marlowe-runtime-cli deposit \
  --contract "$CONTRACT_ID" \
  --from-party "$BORROWER_ADDR" \
  --to-party "$BORROWER_ADDR" \
  --lovelace "$((PRINCIPAL+INTEREST))" \
  --change-address "$BORROWER_ADDR" \
  --manual-sign tx-3.unsigned \
| jq -r 'fromjson | .txId' \
)
echo "TX_3 = $TX_3"
```

Once again, we can use Marlowe cli to submit the transaction and then wait for confirmation.

```
marlowe-cli transaction submit \
  --tx-body-file tx-3.unsigned \
  --required-signer "$BORROWER_SKEY" \
  --timeout 600
```

We can view the transaction on Cardano explorer.

```
echo "$EXPLORER_URL"/transaction/"$TX_3?tab=utxo"
```

If we query the lenders funds, we see that he has received back the 80 ada of principal and the 2 ada deposited when the contract was created, along with the additional 5 ada of interest, so totally 87 ada. The borrower now has about 5 ada (the loan's interest) less than originally.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$LENDER_ADDR"
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$BORROWER_ADDR"
```

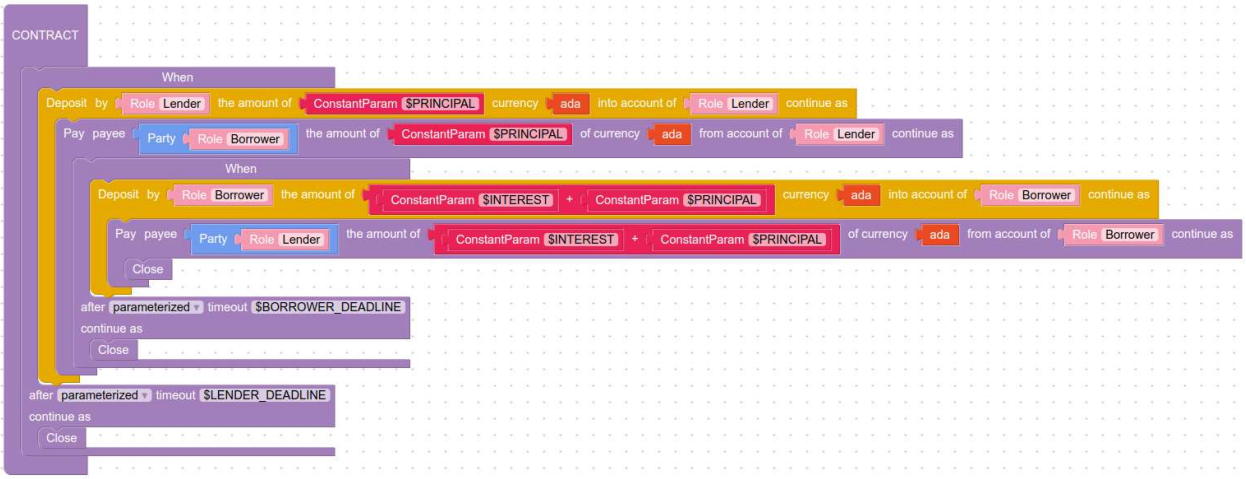
And below is the example output from the above commands.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
bc286bb53f50a7355e2765462546e03d17cabde30420b6bfd1531617386facf4     0       917043702
                                                                                lovelace +
                                                                            TxOutDatumNone

d159062c1321707d12d7abe14cfaca1881a7b1e3c65bb4d637070e4fc0da08c3     1        87000000
                                                                                lovelace +
                                                                            TxOutDatumNone
```

The Marlowe contract has closed, so there is no output to its script address. If we now again take a look at the Marlowe contract, we will see three transactions. At the third transaction there is no more contract code because the contract is done.

7.6 Marlowe Starter Kit: ZCB using the Marlowe Runtime REST API

In this chapter we will use the Marlowe runtime REST API to operate a zero-coupon bond. The code can be found in the 02-runtime-rest.ipynb jupyter notebook. The Marlowe contract in this chapter is very much the same as in the previous one with the exception that we are going to use role tokens here. In the previous chapter we used addresses for the lender and the borrower. In this chapter they are going to have role tokens that are NFTs used to authorize Marlowe transactions. Aside from that everything is the same.




```
When
    [Case
        (Deposit
            (Role "Lender")
            (Role "Lender")
            (Token "" "")
            (ConstantParam "$PRINCIPAL")
        )
        (Pay
            (Role "Lender")
            (Party (Role "Borrower"))
            (Token "" "")
            (ConstantParam "$PRINCIPAL")
            (When
                [Case
                    (Deposit
                        (Role "Borrower")
                        (Role "Borrower")
                        (Token "" "")
                        (AddValue
                            (ConstantParam "$INTEREST")
                            (ConstantParam "$PRINCIPAL")
                        )
                    )
                    (Pay
                        (Role "Borrower")
                        (Party (Role "Lender"))
                        (Token "" "")
                        (AddValue
                            (ConstantParam "$INTEREST")
                            (ConstantParam "$PRINCIPAL")
                        )
                        Close
                    )]
                (TimeParam "$BORROWER_DEADLINE")
                Close
            )
        )]
    (TimeParam "$LENDER_DEADLINE")
```

```
    Close
```

The preliminaries are the same as in the previous chapter. We have to set the 4 environment variables of which two define where we can reach the web server:

- CARDANO_NODE_SOCKET_PATH: location of Cardano node's socket.
- CARDANO_TESTNET_MAGIC: testnet magic number.
- MARLOWE_RT_WEBSERVER_HOST: IP address of the Marlowe Runtime web server.
- MARLOWE_RT_WEBSERVER_PORT: Port number for the Marlowe Runtime web server.

And then we have to create a set of keys for the lender and borrower. We set the environment variables with the following commands:

```
if [[ -z "$MARLOWE_RT_WEBSERVER_PORT" ]]
then

  # Only required for `marlowe-cli` and `cardano-cli`.
  export CARDANO_NODE_SOCKET_PATH="$(docker volume inspect marlowe-starter-kit_shared
| jq -r '.[0].Mountpoint')/node.socket"
  export CARDANO_TESTNET_MAGIC=1 # Note that preprod=1 and preview=2. Do not set this
variable if using mainnet.

  # Only required for Marlowe Runtime REST API.
  export MARLOWE_RT_WEBSERVER_HOST="127.0.0.1"
  export MARLOWE_RT_WEBSERVER_PORT=3780

fi

# FIXME: This should have been inherited from the parent environment.
if [[ -z "$CARDANO_NODE_SOCKET_PATH" ]]
then
  export CARDANO_NODE_SOCKET_PATH=/ipc/node.socket
fi

# FIXME: This should have been set in the parent environment.
if [[ -z "$CARDANO_TESTNET_MAGIC" ]]
then
  export CARDANO_TESTNET_MAGIC=$CARDANO_NODE_MAGIC
fi

case "$CARDANO_TESTNET_MAGIC" in
  1)
    export "EXPLORER_URL=https://preprod.cardanoscan.io"
    ;;
  2)
    export "EXPLORER_URL=https://preview.cardanoscan.io"
    ;;
  *)
    # Use `mainnet` as the default.
    export "EXPLORER_URL=https://cardanoscan.io"
    ;;
esac

MARLOWE_RT_WEBSERVER_URL="http://$MARLOWE_RT_WEBSERVER_HOST":"$MARLOWE_RT_WEBSERVER_PO
```

```
RT"

echo "CARDANO_NODE_SOCKET_PATH = $CARDANO_NODE_SOCKET_PATH"
echo "CARDANO_TESTNET_MAGIC = $CARDANO_TESTNET_MAGIC"
echo "MARLOWE_RT_WEBSERVER_HOST = $MARLOWE_RT_WEBSERVER_HOST"
echo "MARLOWE_RT_WEBSERVER_PORT = $MARLOWE_RT_WEBSERVER_PORT"
echo "MARLOWE_RT_WEBSERVER_URL = $MARLOWE_RT_WEBSERVER_URL"
```

First, we create the variables for the lender and borrower address and key.

```
LENDER_SKEY=keys/lender.skey
LENDER_ADDR=$(cat keys/lender.address)
echo "LENDER_ADDR = $LENDER_ADDR"

BORROWER_SKEY=keys/borrower.skey
BORROWER_ADDR=$(cat keys/borrower.address)
echo "BORROWER_ADDR = $BORROWER_ADDR"
```

Then we check that there are at least 100 ada sitting at both addresses.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$LENDER_ADDR"
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$BORROWER_ADDR"
```

We are again going to have a principal of 80 ada and interest of 5 ada. We start the contract off with 2 ada and then we are going to use the same time constants as in the previous chapter.

```
ADA=1000000  # 1 ada = 1,000,000 lovelace
PRINCIPAL=$((80 * ADA))
INTEREST=$((5 * ADA))
echo "PRINCIPAL = $PRINCIPAL lovelace"
echo "INTEREST = $INTEREST lovelace"

MIN_LOVELACE="$((2 * ADA))"
echo "MIN_LOVELACE = $MIN_LOVELACE lovelace"

SECOND=1000 # 1 second = 1000 milliseconds
MINUTE=$((60 * SECOND)) # 1 minute = 60 seconds
HOUR=$((60 * MINUTE)) # 1 hour = 60 minutes

NOW="$((`date -u +%s` * SECOND))"
echo NOW = "$NOW" POSIX milliseconds = "`date -d @$((NOW / SECOND))`"
```

There are again two deadlines. One is for the lender to deposit the principal and on for the borrower to return the principal and interest.

```
LENDER_DEADLINE="$((NOW + 1 * HOUR))"
BORROWER_DEADLINE="$((NOW + 3 * HOUR))"
echo LENDER_DEADLINE = "$LENDER_DEADLINE" POSIX milliseconds = "`date -d
@$((LENDER_DEADLINE / SECOND))`"
echo BORROWER_DEADLINE = "$BORROWER_DEADLINE" POSIX milliseconds = "`date -d
@$((BORROWER_DEADLINE / SECOND))`"
```

We again create the contract with the Marlowe cli template command.

```
marlowe-cli template zcb \
  --minimum-ada "$MIN_LOVELACE" \
  --lender Lender \
  --borrower Borrower \
  --principal "$PRINCIPAL" \
  --interest "$INTEREST" \
  --lending-deadline "$LENDER_DEADLINE" \
  --repayment-deadline "$BORROWER_DEADLINE" \
  --out-contract-file zcb-contract.json \
  --out-state-file /dev/null
```

You could also design your contract on the Marlowe playground and then download it as a JSON file. If we want to, we can also examine the contract with the json2yaml command.

```
json2yaml zcb-contract.json
```

A shortened example output can be seen below.

```
timeout: 1679608289000
timeout_continuation: close
when:
- case:
    deposits: 80000000
    into_account:
      role_token: Lender
    of_token:
      currency_symbol: ''
      token_name: ''
    party:
      role_token: Lender
  then:
    from_account:
      role_token: Lender
    pay: 80000000
    then:
      timeout: 1679615489000
      timeout_continuation: close
      when:
      - case:
          deposits:
            add: 80000000
            and: 5000000
          into_account:
...
        role_token: Borrower
    token:
      currency_symbol: ''
      token_name: ''
```

We see it is using the field *role_token* in places where there was the *address* field in the previous chapter. We again mention it is good to check the safety of a contract before running

it on main-net. The steps for this are the same as described in the previous chapter. Now we can create the contract. Since we are going to use the REST API, we are going to have JSON payloads in the request. We want the role tokens to be minted in the creation transaction. So, we have to say the role named lender in the contract is actually going to live at this address and so when the role token is minted it is going to be sent to that address. It will be the same with the borrower. It is very important if you have a contract that has roles, to actually mint the role tokens. Otherwise, you are not going to be able to operate the contract. We are going to create the JSON body of the request to build the creation transaction with the following command:

```
yaml2json << EOI > request-1.json
version: v1
contract: `cat zcb-contract.json`
roles:
  Lender: "$LENDER_ADDR"
  Borrower: "$BORROWER_ADDR"
minUTxODeposit: $MIN_LOVELACE
metadata: {}
tags: {}
EOI
cat request-1.json
```

Next, we post the request and view the response.

```
curl "$MARLOWE_RT_WEBSERVER_URL/contracts" \
  -X POST \
  -H 'Content-Type: application/json' \
  -H "X-Change-Address: $LENDER_ADDR" \
  -d @request-1.json \
  -o response-1.json \
  -sS
json2yaml response-1.json
```

The result that gets printed out is quite a bit of binary data contained in the cbor hex field. We can extract the contract ID from that with the following command:

```
CONTRACT_ID="$(jq -r '.resource.contractId' response-1.json)"
echo "CONTRACT_ID = $CONTRACT_ID"
```

And we also need to extract just the transaction body that needs to be signed. We can use the jq tool to run a query on the JSON and pull out the unsigned transaction.

```
jq '.resource.txBody' response-1.json > tx-1.unsigned
```

We again now have several options to sign this transaction as stated in the previous chapter. Here we will use the Marlowe cli for this.

```
TX_1=$(
marlowe-cli transaction submit \
  --tx-body-file tx-1.unsigned \
  --required-signer "$LENDER_SKEY" \
  --timeout 600 \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_1 = $TX_1"
```

We have to provide the signing key and the timeout. We can now query the contract address and the lender address for their funds.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --tx-in "$CONTRACT_ID"
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$LENDER_ADDR"
```

Below you can see the example outputs for the above commands.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
bfb6db4bdf59113c4bdd45d45c7f94070099b813e83b665b5cdc6daeb91a941d     1        2000000
                                                                                lovelace +
                                                                             TxOutDatumHash
                                                                    ScriptDataInBabbageEra
                    "cc80fcdc9dedea89d2f50dd29cf7f84d14bd00e27714310a0cfe5e24e87fd008"

                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
bfb6db4bdf59113c4bdd45d45c7f94070099b813e83b665b5cdc6daeb91a941d     0       995476698
                                                                                lovelace +
                                                                             TxOutDatumNone
bfb6db4bdf59113c4bdd45d45c7f94070099b813e83b665b5cdc6daeb91a941d     3        1034400
                                                                                lovelace +
              1 89506f74616aa4b5a42ec352a56ab7815e547e23e2a32417f400e43c.4c656e646572 +
                                                                             TxOutDatumNone
```

We see that the Marlowe contract contains 2 ada and the lender has beside his ada also one NFT that is the role token. If we would query the borrower address, we would also see that there is one NFT sitting at his address beside the ada funds. The following number from the above NFT “89506f74616aa4b5a42ec352a56ab7815e547e23e2a32417f400e43c” is the policy ID for that NFT and we call it the roles currency for the Marlowe contract. This will be used to authorize the transactions. The Marlowe runtime web server provides a GET endpoint to learn about what is going on with the contract. Here are the commands to query this endpoint.

```
CONTRACT_URL="$MARLOWE_RT_WEBSERVER_URL/`jq -r '.links.contract' response-1.json`"
echo "CONTRACT_URL = $CONTRACT_URL"

curl -sS "$CONTRACT_URL" | json2yaml
```

The last command we produce the following output:

```
links:
  transactions:
contracts/bfb6db4bdf59113c4bdd45d45c7f94070099b813e83b665b5cdc6daeb91a941d%231/transac
tions
resource:
  block:
    blockHeaderHash: 4f1c23e34773b2780bb4507770740fb9ef569d38e08e8c6006964f3d71677529
    blockNo: 755129
    slotNo: 23921744
  continuations: null
  contractId: bfb6db4bdf59113c4bdd45d45c7f94070099b813e83b665b5cdc6daeb91a941d#1
  currentContract:
    timeout: 1679608289000
    timeout_continuation: close
    when:
    - case:
        deposits: 80000000
        into_account:
          role_token: Lender
        of_token:
          currency_symbol: ''
          token_name: ''
        party:
          role_token: Lender
      then:
        from_account:
          role_token: Lender
        pay: 80000000
        then:
          timeout: 1679615489000
          timeout_continuation: close
          when:
          - case:
              deposits:
                add: 80000000
                and: 5000000
              into_account:
                role_token: Borrower
              of_token:
                currency_symbol: ''
                token_name: ''
              party:
                role_token: Borrower
            then:
              from_account:
                role_token: Borrower
              pay:
                add: 80000000
                and: 5000000
              then: close
              to:
                party:
                  role_token: Lender
              token:
                currency_symbol: ''
                token_name: ''
          to:
```

```
        party:
          role_token: Borrower
      token:
        currency_symbol: ''
        token_name: ''
initialContract:
  timeout: 1679608289000
  timeout_continuation: close
  when:
  - case:
      deposits: 80000000
      into_account:
        role_token: Lender
      of_token:
        currency_symbol: ''
        token_name: ''
      party:
        role_token: Lender
    then:
      from_account:
        role_token: Lender
      pay: 80000000
      then:
        timeout: 1679615489000
        timeout_continuation: close
        when:
        - case:
            deposits:
              add: 80000000
              and: 5000000
            into_account:
              role_token: Borrower
            of_token:
              currency_symbol: ''
              token_name: ''
            party:
              role_token: Borrower
          then:
            from_account:
              role_token: Borrower
            pay:
              add: 80000000
              and: 5000000
            then: close
            to:
              party:
                role_token: Lender
            token:
              currency_symbol: ''
              token_name: ''
      to:
        party:
          role_token: Borrower
      token:
        currency_symbol: ''
        token_name: ''
metadata: {}
roleTokenMintingPolicyId: 89506f74616aa4b5a42ec352a56ab7815e547e23e2a32417f400e43c
```

```
  state:
    accounts:
    - - - address: addr_test1vqd3yrtjyx49uld43lvwqaf7z4k03su8gf2x4yr7syzvckgfzm4ck
        - currency_symbol: ''
          token_name: ''
      - 2000000
    boundValues: []
    choices: []
    minTime: 0
  status: confirmed
  tags: {}
  txBody: null
  utxo: bfb6db4bdf59113c4bdd45d45c7f94070099b813e83b665b5cdc6daeb91a941d#1
  version: v1
```

We see information about the contract that tells us what is going on. The next step is the lender is going to deposit the 80 ada. We have to craft a JSON request for this. We can use the Marlowe cli input deposit command for that.

```
marlowe-cli input deposit \
  --deposit-party Lender \
  --deposit-account Lender \
  --deposit-amount "$PRINCIPAL" \
  --out-file input-2.json
json2yaml input-2.json
```

We need to input the party, the account and the amount. We can package this together for the request of the transaction. We give it the Marlowe version number, the inputs, we could attach metadata if we wanted to.

```
yaml2json << EOI > request-2.json
version: v1
inputs: [$(cat input-2.json)]
metadata: {}
tags: {}
EOI
cat request-2.json
```

Next, we post the request and store the response.

```
curl "$CONTRACT_URL/transactions" \
  -X POST \
  -H 'Content-Type: application/json' \
  -H "X-Change-Address: $LENDER_ADDR" \
  -d @request-2.json \
  -o response-2.json \
  -sS
json2yaml response-2.json
```

So, we again extract the actual unsigned transaction and then submit the transaction.

```
jq '.resource.txBody' response-2.json > tx-2.unsigned

TX_2=$(
marlowe-cli transaction submit \
  --tx-body-file tx-2.unsigned \
  --required-signer "$LENDER_SKEY" \
  --timeout 600 \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_2 = $TX_2"
```

If we query the lender's address, we can see that he has approximately 83 ada less than originally. Two ada were deposited in the contract when it was created and 80 ada were paid to the borrower in the second transaction; another one ada was attached to the role token that was sent to the borrower. The lender also holds their own role token.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$LENDER_ADDR"
```

Below is an example output of the above command.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
088395d43bd1aefc7a5b1dc41b472a74f59cca631a61af7c9a212c5c8fa56c14     0       914639783
                                                                              lovelace +
                                                                          TxOutDatumNone
088395d43bd1aefc7a5b1dc41b472a74f59cca631a61af7c9a212c5c8fa56c14     2         1034400
                                                                              lovelace +
             1 89506f74616aa4b5a42ec352a56ab7815e547e23e2a32417f400e43c.4c656e646572 +
                                                                          TxOutDatumNone
```

The Marlowe contract still has the 2 ada from its creation. Marlowe's role-payout address holds the 80 ada on behalf of the borrower. We can query this address with the command:

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --tx-in "$TX_2#3"
```

The output we will get is:

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
088395d43bd1aefc7a5b1dc41b472a74f59cca631a61af7c9a212c5c8fa56c14     3        80000000
                                                                              lovelace +
                                                    TxOutDatumHash ScriptDataInBabbageEra
                      "12548b4001cc30918dc86060c05d1cafe4457ec1e9f0e271d022cd1edddd814f"
```

In the previous chapter when we did this transaction the 80 ada was directly paid to the borrower but when you are using role tokens and roles in Marlowe contracts it does not actually go directly to the party but it goes to the role payout address. So, this is another Plutus

validator that Marlowe uses. It is a very simple validator because it basically holds funds on behalf of someone. We can again view the contract with an HTTP request.

```
curl -sS "$CONTRACT_URL"/transactions/"$TX_2" | json2yaml
```

Now we can withdraw the funds from the role payout validator address in the actual wallet of the borrower. The request is very simple. We just tell the contract ID and that it is the borrower withdrawing funds.

```
yaml2json << EOI > request-3.json
contractId: "$CONTRACT_ID"
role: Borrower
EOI
cat request-3.json
```

Then we can call the withdrawals endpoint in the HTTP post.

```
curl "$MARLOWE_RT_WEBSERVER_URL/withdrawals" \
  -X POST \
  -H 'Content-Type: application/json' \
  -H "X-Change-Address: $BORROWER_ADDR" \
  -d @request-3.json \
  -o response-3.json \
  -sS
json2yaml response-3.json
```

From the outputted response-3.json file we can now extract the unsigned transaction and sign and submit it with help of Marlowe cli.

```
jq '.resource.txBody' response-3.json > tx-3.unsigned

TX_3=$(
marlowe-cli transaction submit \
  --tx-body-file tx-3.unsigned \
  --required-signer "$BORROWER_SKEY" \
  --timeout 600 \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_3 = $TX_3"
```

The key here is that the borrower is redeeming things so they have to sign it. They have their role token in their wallet and that is going to authorize the withdrawal of the money that is held on their behalf. We can now check the borrowers' funds.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$BORROWER_ADDR"
```

The output for the above command can be seen below.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
186e94a9721b90a980171ac8b6c69553bfd18fb72ef7eab5184f921b01ea90ad     0      1079664208
                                                                           lovelace +
                                                                       TxOutDatumNone
186e94a9721b90a980171ac8b6c69553bfd18fb72ef7eab5184f921b01ea90ad     1        1043020
                                                                           lovelace +
         1 89506f74616aa4b5a42ec352a56ab7815e547e23e2a32417f400e43c.426f72726f776572 +
                                                                       TxOutDatumNone
```

We see that the borrower together has around 1080 ada and the NFT for the role token. Now the borrower can repay the loan. We first build the input to deposit the funds to the contract.

```
marlowe-cli input deposit \
  --deposit-party Borrower \
  --deposit-account Borrower \
  --deposit-amount "$((PRINCIPAL+INTEREST))" \
  --out-file input-4.json
json2yaml input-4.json
```

Then we build the request.

```
yaml2json << EOI > request-4.json
version: v1
inputs: [$(cat input-4.json)]
metadata: {}
tags: {}
EOI
cat request-4.json
```

Now post the request to Marlowe Runtime.

```
curl "$CONTRACT_URL/transactions" \
  -X POST \
  -H 'Content-Type: application/json' \
  -H "X-Change-Address: $BORROWER_ADDR" \
  -d @request-4.json \
  -o response-4.json \
  -sS
json2yaml response-4.json
```

Once again, use Marlowe cli to submit the transaction and then wait for confirmation.

```
jq '.resource.txBody' response-4.json > tx-4.unsigned

TX_4=$(
marlowe-cli transaction submit \
  --tx-body-file tx-4.unsigned \
  --required-signer "$BORROWER_SKEY" \
  --timeout 600 \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
```

```
echo "TX_4 = $TX_4"
```

We can check now the borrowers' funds with the following command:

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$BORROWER_ADDR"
```

Below is an example output of the above command.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
d08277e718607803947c16d13d6ba78a464d48667cb08f199fbf3257791f7047     0       994002697
                                                                           lovelace +
                                                                       TxOutDatumNone
d08277e718607803947c16d13d6ba78a464d48667cb08f199fbf3257791f7047     1        1043020
                                                                           lovelace +
         1 89506f74616aa4b5a42ec352a56ab7815e547e23e2a32417f400e43c.426f72726f776572 +
                                                                       TxOutDatumNone
```

We see that it contains around 5 ada less then it originally had. Now we can again look at the contract sitting at the blockchain with the following command:

```
curl -sS "$CONTRACT_URL"/transactions/"$TX_4" | json2yaml
```

After that the lender can construct the transaction with which he withdraws the principal and the interest. We do this with the following commands:

```
yaml2json << EOI > request-5.json
contractId: "$CONTRACT_ID"
role: Lender
EOI
cat request-5.json
```

Below is the command which posts the request to the REST API and stores the result.

```
curl "$MARLOWE_RT_WEBSERVER_URL/withdrawals" \
  -X POST \
  -H 'Content-Type: application/json' \
  -H "X-Change-Address: $LENDER_ADDR" \
  -d @request-5.json \
  -o response-5.json \
  -sS
json2yaml response-5.json
```

Once again, use the marlowe-cli to submit the transaction and then wait for confirmation.

```
jq '.resource.txBody' response-5.json > tx-5.unsigned

TX_5=$(
marlowe-cli transaction submit \
  --tx-body-file tx-5.unsigned \
```

```
  --required-signer "$LENDER_SKEY" \
  --timeout 600 \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_5 = $TX_5"
```

We can now query the lender funds.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$LENDER_ADDR"
```

And here you can see the output of the above command.

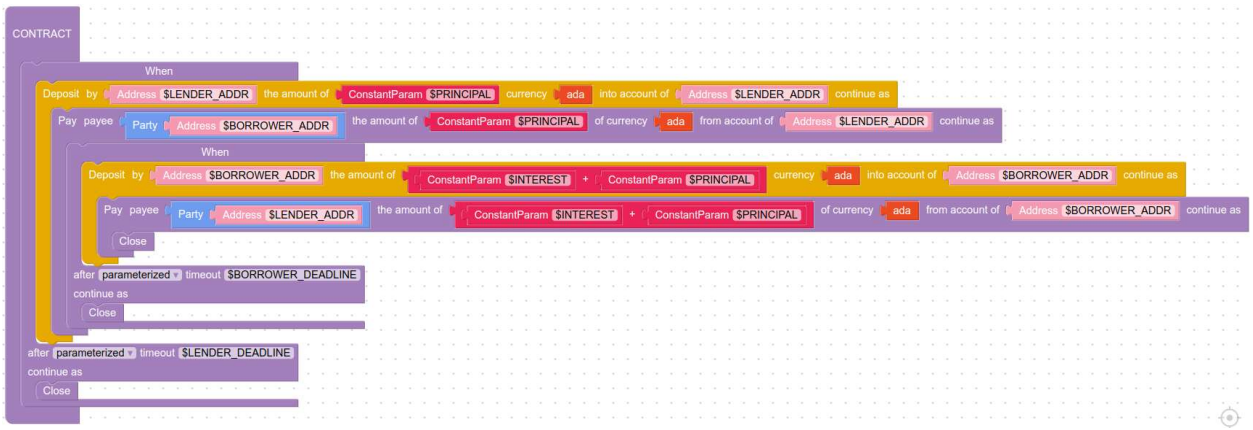
```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
32487203bba96c85e34345e8f88fc60dfb12dac4fb89decf144b86287a1a65e6     0       999308229
                                                                           lovelace +
                                                                       TxOutDatumNone
32487203bba96c85e34345e8f88fc60dfb12dac4fb89decf144b86287a1a65e6     1         1034400
                                                                           lovelace +
             1 89506f74616aa4b5a42ec352a56ab7815e547e23e2a32417f400e43c.4c656e646572 +
                                                                       TxOutDatumNone
d08277e718607803947c16d13d6ba78a464d48667cb08f199fbf3257791f7047     3         2000000
                                                                           lovelace +
                                                                       TxOutDatumNone
```

The lender now has an additional 5 ada (the loan's interest), minus fees, compared to their original balance before the contract started. If you want to see all the things you can do with the REST API it supports openAPI so you can just make the following API call and will get a long printout of all the description of the Marlowe runtime endpoints.

```
curl -sS "$MARLOWE_RT_WEBSERVER_URL/openapi.json" | json2yaml
```

7.7 Marlowe Starter Kit: ZCB using the Marlowe Runtime CLI

In this chapter we will execute a zero-coupon bond contract using the Marlowe cli which is a lower-level tool. The code and instructions in this chapter are taken from the marlowe-cli.ipynb playbook from the Marlowe starter kit repository. The contract in this chapter will be the same one as in chapter 7.5 which was an address-based contract. Below you can see the contract in Blockly code and the Marlowe code of the contract.




```
When
    [Case
        (Deposit
            (Address "$LENDER_ADDR")
            (Address "$LENDER_ADDR")
            (Token "" "")
            (ConstantParam "$PRINCIPAL")
        )
        (Pay
            (Address "$LENDER_ADDR")
            (Party (Address "$BORROWER_ADDR"))
            (Token "" "")
            (ConstantParam "$PRINCIPAL")
            (When
                [Case
                    (Deposit
                        (Address "$BORROWER_ADDR")
                        (Address "$BORROWER_ADDR")
                        (Token "" "")
                        (AddValue
                            (ConstantParam "$INTEREST")
                            (ConstantParam "$PRINCIPAL")
                        )
                    )
                    (Pay
                        (Address "$BORROWER_ADDR")
                        (Party (Address "$LENDER_ADDR"))
                        (Token "" "")
                        (AddValue
                            (ConstantParam "$INTEREST")
                            (ConstantParam "$PRINCIPAL")
                        )
                        Close
                    )]
                (TimeParam "$BORROWER_DEADLINE")
                Close
            )
        )]
    (TimeParam "$LENDER_DEADLINE")
    Close
```

In this case we do not need Marlowe runtime, we just need a connection to the node socket. The keys and addresses for the borrower and lender we create as in chapter 7.4. If we're using demeter.run and added the Cardano Marlowe runtime extension, then we already have access to Cardano node and Marlowe runtime. The following commands will set the required environment variables to use a local docker deployment on the default ports. It will also set some supplementary environment variables.

```
if [[ -z "$CARDANO_NODE_SOCKET_PATH" ]]
then

  # Only required for `marlowe-cli` and `cardano-cli`.
  export CARDANO_NODE_SOCKET_PATH="$(docker volume inspect marlowe-starter-kit_shared
| jq -r '.[0].Mountpoint')/node.socket"
  export CARDANO_TESTNET_MAGIC=1 # Note that preprod=1 and preview=2. Do not set this
variable if using mainnet.

fi

# FIXME: This should have been inherited from the parent environment.
if [[ -z "$CARDANO_NODE_SOCKET_PATH" ]]
then
  export CARDANO_NODE_SOCKET_PATH=/ipc/node.socket
fi

# FIXME: This should have been set in the parent environment.
if [[ -z "$CARDANO_TESTNET_MAGIC" ]]
then
  export CARDANO_TESTNET_MAGIC=$CARDANO_NODE_MAGIC
fi

case "$CARDANO_TESTNET_MAGIC" in
  1)
    export "EXPLORER_URL=https://preprod.cardanoscan.io"
    ;;
  2)
    export "EXPLORER_URL=https://preview.cardanoscan.io"
    ;;
  *)
    # Use `mainnet` as the default.
    export "EXPLORER_URL=https://cardanoscan.io"
    ;;
esac

echo "CARDANO_NODE_SOCKET_PATH = $CARDANO_NODE_SOCKET_PATH"
echo "CARDANO_TESTNET_MAGIC = $CARDANO_TESTNET_MAGIC"
```

We define the lender address and key variables with the following command.

```
LENDER_SKEY=keys/lender.skey
LENDER_ADDR=$(cat keys/lender.address)
echo "LENDER_ADDR = $LENDER_ADDR"
```

We also check that the lender has at least one hundred ada.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$LENDER_ADDR"
```

We can view the address on a Cardano explorer. It sometimes takes thirty seconds or so for the transaction to be visible in an explorer.

```
echo "$EXPLORER_URL"/address/"$LENDER_ADDR"
```

We do the same for the borrower. First, we define the address and key variables.

```
BORROWER_SKEY=keys/borrower.skey
BORROWER_ADDR=$(cat keys/borrower.address)
echo "BORROWER_ADDR = $BORROWER_ADDR"
```

Then we also check the funds for the borrower.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$BORROWER_ADDR"
```

And again, we can view the address on a Cardano explorer.

```
echo "$EXPLORER_URL"/address/"$BORROWER_ADDR"
```

The contract design will be the same as in the previous chapter. It is going to have 80 ada of principal and interest of 5 ada. We set these variables with following code:

```
ADA=1000000  # 1 ada = 1,000,000 lovelace

PRINCIPAL=$((80 * ADA))
INTEREST=$((5 * ADA))

echo "PRINCIPAL = $PRINCIPAL lovelace"
echo "INTEREST = $INTEREST lovelace"
```

On the Cardano blockchain, the protocol parameters require that each UTXO contain at least some ada. Here we will start the contract with 2 ada.

```
MIN_LOVELACE="$((2 * ADA))"
echo "MIN_LOVELACE = $MIN_LOVELACE lovelace"
```

Next, we define the current time, measured in POSIX milliseconds.

```
SECOND=1000 # 1 second = 1000 milliseconds
MINUTE=$((60 * SECOND)) # 1 minute = 60 seconds
HOUR=$((60 * MINUTE)) # 1 hour = 60 minutes

NOW="$((`date -u +%s` * SECOND))"
```

```
echo NOW = "$NOW" POSIX milliseconds = "`date -d @$((NOW / SECOND))`"
```

The contract has a lending deadline and a repayment deadline. For convenience in this example, set the deadlines to the near future.

```
LENDER_DEADLINE="$((NOW + 1 * HOUR))"
BORROWER_DEADLINE="$((NOW + 3 * HOUR))"
echo LENDER_DEADLINE = "$LENDER_DEADLINE" POSIX milliseconds = "`date -d
@$((LENDER_DEADLINE / SECOND))`"
echo BORROWER_DEADLINE = "$BORROWER_DEADLINE" POSIX milliseconds = "`date -d
@$((BORROWER_DEADLINE / SECOND))`"
```

Now we create the JSON file for the contract zcb-contract.json.

```
marlowe-cli template zcb \
  --minimum-ada "$MIN_LOVELACE" \
  --lender "$LENDER_ADDR" \
  --borrower "$BORROWER_ADDR" \
  --principal "$PRINCIPAL" \
  --interest "$INTEREST" \
  --lending-deadline "$LENDER_DEADLINE" \
  --repayment-deadline "$BORROWER_DEADLINE" \
  --out-contract-file zcb-contract.json \
  --out-state-file zcb-state.json
```

We can view the contract in yaml format as we did in the previous chapter with the command:

```
json2yaml zcb-contract.json
```

As mentioned in the previous chapters it is always good to check the viability of the contract. We will build the creation information for the contract with the marlowe-cli tool that has a run initialize option. You can see the various command options if you run it with the --help option.

```
marlowe-cli run initialize --help
```

There is also an option to stake the ada at the Marlowe validator. If you are using a reference script there is an option to find that reference. If the contract would have roles, you could also specify them with this command. To build the JSON file we run the following command:

```
marlowe-cli run initialize \
  --permanently-without-staking \
  --contract-file zcb-contract.json \
  --state-file zcb-state.json \
  --out-file marlowe-1.json
```

This command basically gathers information about the contract file we have created and the initial state file. We now use the marlowe-cli run auto-execute command to construct and submit the creation transaction.

```
TX_1=$(
marlowe-cli run auto-execute \
  --marlowe-out-file marlowe-1.json \
  --change-address "$LENDER_ADDR" \
  --required-signer "$LENDER_SKEY" \
  --out-file tx-1.signed \
  --submit 600 \
  --print-stats \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_1 = $TX_1"
```

It is taking in the information about the Marlowe state which is embodied in the Plutus datum before and after the transaction. And it is setting up a transaction that makes that transition from before to after. We get a description of the command options if we run this command with the --help option. Below is an example output from the above command.

```
Fee: Lovelace 212185
Size: 941 / 16384 = 5%
Execution units:
  Memory: 0 / 14000000 = 0%
  Steps: 0 / 10000000000 = 0%
TX_1 = f36feb37473ec171d9a79553b9a7f729b78076e97be20f54aaca88094a1752d5
```

We can view the transaction again on the Cardano explorer.

```
echo "$EXPLORER_URL"/transaction/"$TX_1?tab=utxo"
```

Also, we can examine the contract's UTXO using the cardano-cli. We will make the lender deposit his 80 ada of principal into the contract using the marlowe-cli run prepare command.
We can get a description of the command line option for the marlowe-cli run prepare command with the --help option.

```
marlowe-cli run prepare --help
```

This command lets you basically take action on a Marlowe contract. With it you can make any number of deposits, choices and notifications. You could make several deposits if you wanted. You will have to give it some time constraints on when the transaction has to be submitted. So, for this contract the lender is providing the funding and receiving the change from this transaction, so we provide his address. We run the following command:

```
marlowe-cli run prepare \
  --deposit-account "$LENDER_ADDR" \
  --deposit-party "$LENDER_ADDR" \
  --deposit-amount "$PRINCIPAL" \
  --invalid-before "$((`date -u +%s` * SECOND - 1 * MINUTE))" \
  --invalid-hereafter "$((`date -u +%s` * SECOND + 5 * MINUTE))" \
  --marlowe-file marlowe-1.json \
  --out-file marlowe-2.json
```

We give it a time window of about 6 minutes so that the node has time to confirm things. And the Marlowe file we created originally for the first transaction was marlowe-1.json so we are starting from that and we are outputting the result in the marlowe-2.json file. These files basically track what is going on step by step in the contract. The Marlowe interpreter is a state machine and these are the states and these are the information files for those transitions. Next, we can build and submit the transaction. This time it is a bit different because we are not in the creation situation where there was no previous state. So, we start with that prior state.

```
TX_2=$(
marlowe-cli run auto-execute \
  --tx-in-marlowe "$TX_1#1" \
  --marlowe-in-file marlowe-1.json \
  --marlowe-out-file marlowe-2.json \
  --change-address "$LENDER_ADDR" \
  --required-signer "$LENDER_SKEY" \
  --out-file tx-2.signed \
  --submit 600 \
  --print-stats \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_2 = $TX_2"
```

The lender is receiving the change from the transaction and is also signing the transaction. And we also have to input the previous Marlowe transaction.

```
Fee: Lovelace 758511
Size: 1570 / 16384 = 9%
Execution units:
  Memory: 6730260 / 14000000 = 48%
  Steps: 1806177146 / 10000000000 = 18%
TX_2 = 017c068bf76aba6ef57b6b5aab592cd9d861c4903e100794ca7451d16c701a02
```

In the example output we see now that the transaction is a little bit bigger and more costly. We again compute the transaction link for Cardano scan.

```
echo "$EXPLORER_URL"/transaction/"$TX_2?tab=utxo"
```

If we check the funds from the lender, we will see he has 82 ada less than originally.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$LENDER_ADDR"
```

Below is an example output of the above command.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
017c068bf76aba6ef57b6b5aab592cd9d861c4903e100794ca7451d16c701a02     0       917029304
                                                                              lovelace +
                                                                          TxOutDatumNone
```

And the borrower will have an additional 80 ada now.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$BORROWER_ADDR"
```

Below is an example output of the above command.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
017c068bf76aba6ef57b6b5aab592cd9d861c4903e100794ca7451d16c701a02     2        80000000
                                                                              lovelace +
                                                                          TxOutDatumNone
ffb18377900e003285f9cf357f96b1691eac3e32a89916e6c49a35bd9a11cdf2     2      1000000000
                                                                              lovelace +
                                                                          TxOutDatumNone
```

We do not have a tool to view the contract without using Marlowe runtime. At the end now the borrower has to repay the loan plus interest. The code for the transaction will look very similar as before, it is just another deposit and we are depositing principal and interest.

```
marlowe-cli run prepare \
  --deposit-account "$BORROWER_ADDR" \
  --deposit-party "$BORROWER_ADDR" \
  --deposit-amount "$((PRINCIPAL+INTEREST))" \
  --invalid-before "$((`date -u +%s` * SECOND - 1 * MINUTE))" \
  --invalid-hereafter "$((`date -u +%s` * SECOND + 5 * MINUTE))" \
  --marlowe-file marlowe-2.json \
  --out-file marlowe-3.json
```

With the above transaction command, also the lender gets back its initial deposit of 2 ada that he put in the Marlowe contract. Once again, we use the marlowe-cli run auto-execute to build and submit the transaction and then wait for confirmation.

```
TX_3=$(
marlowe-cli run auto-execute \
  --tx-in-marlowe "$TX_2#1" \
  --marlowe-in-file marlowe-2.json \
  --marlowe-out-file marlowe-3.json \
  --change-address "$BORROWER_ADDR" \
  --required-signer "$BORROWER_SKEY" \
```

```
  --out-file tx-3.signed \
  --submit 600 \
  --print-stats \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_3 = $TX_3"
```

We again output the Cardano scan link of the transaction.

```
echo "$EXPLORER_URL"/transaction/"$TX_3?tab=utxo"
```

And then also check the final balances of the lender and borrower.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$LENDER_ADDR"
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$BORROWER_ADDR"
```

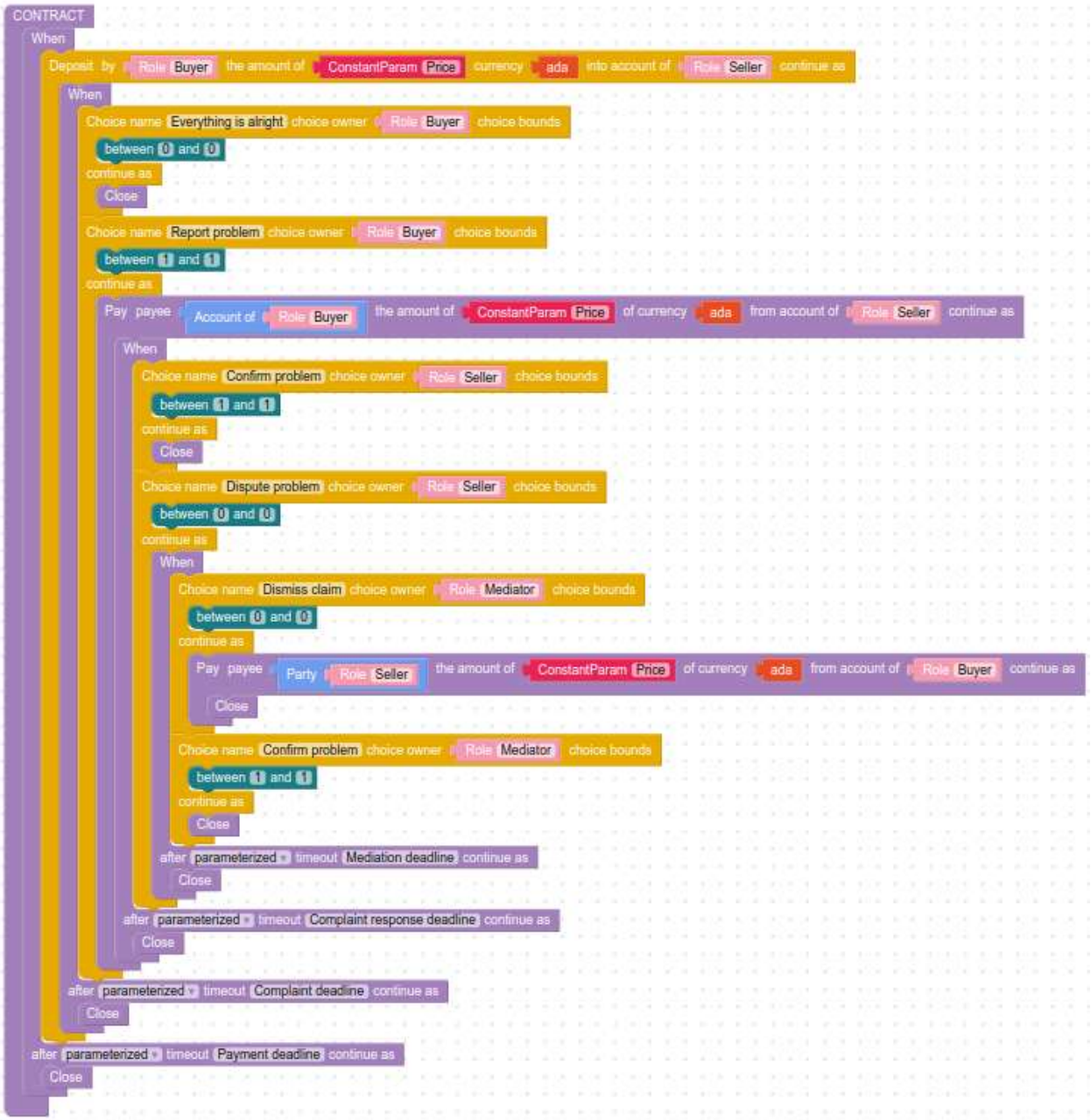
Below you can see an example output of the above commands.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
017c068bf76aba6ef57b6b5aab592cd9d861c4903e100794ca7451d16c701a02     0       917029304
                                                                           lovelace +
                                                                       TxOutDatumNone
b2486f82eaccef641a3b44de4c07146304238b25b0678ab1bfb70e3bc244c338     1        87000000
                                                                           lovelace +
                                                                       TxOutDatumNone

                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
017c068bf76aba6ef57b6b5aab592cd9d861c4903e100794ca7451d16c701a02     2        80000000
                                                                           lovelace +
                                                                       TxOutDatumNone
b2486f82eaccef641a3b44de4c07146304238b25b0678ab1bfb70e3bc244c338     0       914430264
                                                                           lovelace +
                                                                       TxOutDatumNone
```

We see that the 87 ada has been received by the lender and the borrower has 85 ada less than in the beginning. And this transaction also closes the contract. This is running without Marlowe runtime so you have to keep track of all the UTXOs by yourself.

7.8 Marlowe Starter Kit: Escrow using the Marlowe Runtime's REST API

In this chapter we will demonstrate how to use an escrow contract using the Marlowe runtime. The contract has the following structure in Blockly.

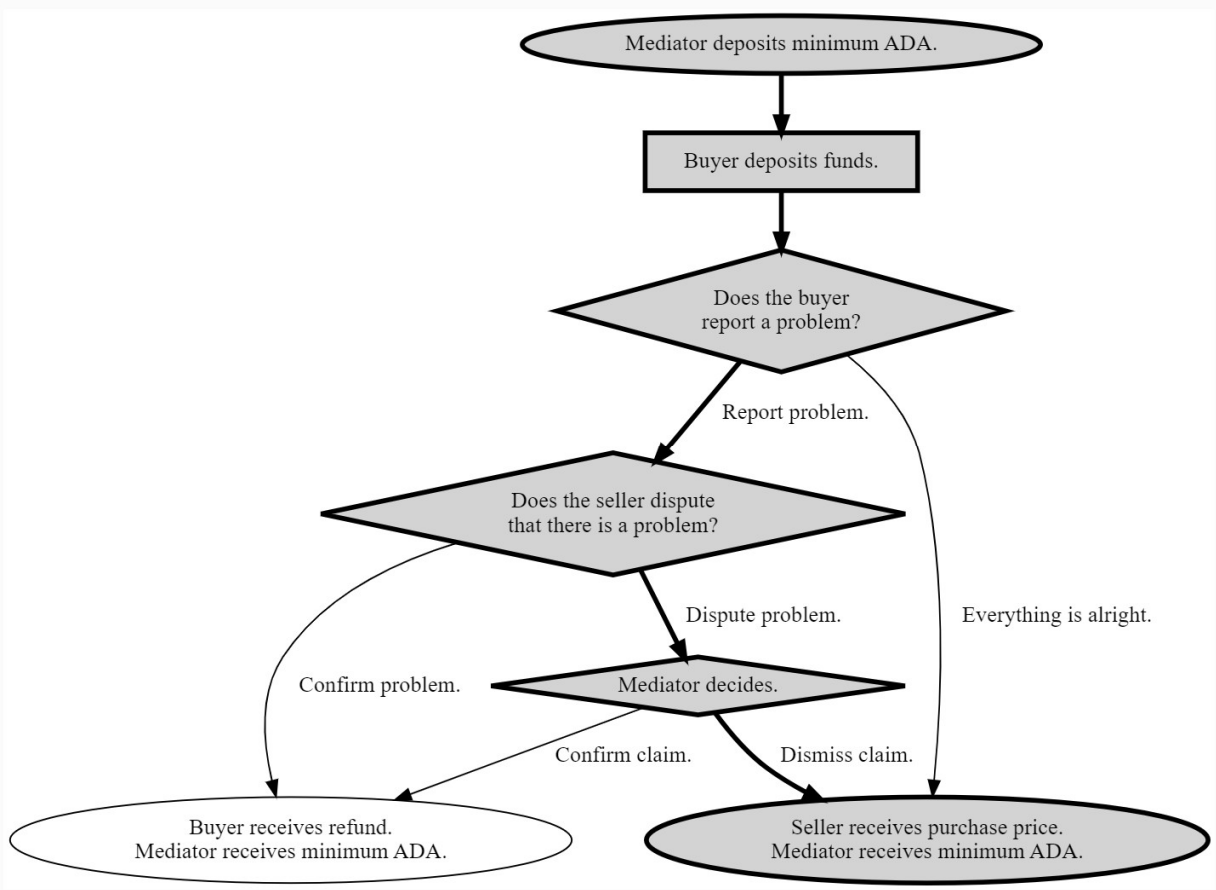


You can view the Marlowe code for this contract below.

```
When [
  (Case
     (Deposit Role "Seller" Role "Buyer"
        (Token "" "")
        (ConstantParam "Price"))
     (When [
           (Case
              (Choice
                 (ChoiceId "Everything is alright" Role "Buyer") [
                 (Bound 0 0)]) Close)
           ,
```

```
            (Case
               (Choice
                  (ChoiceId "Report problem" Role "Buyer") [
                  (Bound 1 1)])
               (Pay Role "Seller"
                  (Account Role "Buyer")
                  (Token "" "")
                  (ConstantParam "Price")
                  (When [
                        (Case
                           (Choice
                              (ChoiceId "Confirm problem" Role "Seller") [
                              (Bound 1 1)]) Close)
                        ,
                        (Case
                           (Choice
                              (ChoiceId "Dispute problem" Role "Seller") [
                              (Bound 0 0)])
                           (When [
                                 (Case
                                    (Choice
                                       (ChoiceId "Dismiss claim" Role "Mediator") [
                                       (Bound 0 0)])
                                    (Pay Role "Buyer"
                                       (Party Role "Seller")
                                       (Token "" "")
                                       (ConstantParam "Price") Close))
                                 ,
                                 (Case
                                    (Choice
                                       (ChoiceId "Confirm problem" Role "Mediator") [
                                       (Bound 1 1)]) Close)]
                                       (TimeParam "Mediation deadline") Close))]
                                       (TimeParam "Complaint response deadline")
                                                                          Close)))]
                                       (TimeParam "Complaint deadline") Close))]
                                       (TimeParam "Payment deadline") Close
```

You can also find this contract on the Marlowe playground (https://play.marlowe-finance.io/). The contract has a buyer, a seller and a mediator. The seller is buying something, for example a bicycle, so he is depositing the funds for the price of the bicycle. He can then check if everything is ok with the received product. If everything is alright the seller gets his funds. But if he complains there is some logic in the contract to try to reach an agreement and if they cannot reach it a mediator steps in. In case the seller agrees with the buyer about the problem the buyer receives a refund. If he does not agree then a mediator decides whether the buyer should get his funds back or not. In the image below you can see the paths of the contract. The path we are going to demonstrate in this chapter is the one that is colored gray.



We are going to use the REST API for the Marlowe runtime. We are going to have three parties. You can use chapter 7.4 to create the address and keys for those parties. We are going to have a node socket available and a Marlowe runtime web server. The following commands will set the required environment variables to use a local docker deployment on the default ports. It will also set some supplementary environment variables.

```
if [[ -z "$MARLOWE_RT_WEBSERVER_PORT" ]]
then

  # Only required for `marlowe-cli` and `cardano-cli`.
  export CARDANO_NODE_SOCKET_PATH="$(docker volume inspect marlowe-starter-kit_shared
| jq -r '.[0].Mountpoint')/node.socket"
  export CARDANO_TESTNET_MAGIC=1 # Note that preprod=1 and preview=2. Do not set this
variable if using mainnet.

  # Only required for Marlowe Runtime REST API.
  export MARLOWE_RT_WEBSERVER_HOST="127.0.0.1"
  export MARLOWE_RT_WEBSERVER_PORT=3780

fi

# FIXME: This should have been inherited from the parent environment.
if [[ -z "$CARDANO_NODE_SOCKET_PATH" ]]
then
```

```
  export CARDANO_NODE_SOCKET_PATH=/ipc/node.socket
fi

# FIXME: This should have been set in the parent environment.
if [[ -z "$CARDANO_TESTNET_MAGIC" ]]
then
  export CARDANO_TESTNET_MAGIC=$CARDANO_NODE_MAGIC
fi

case "$CARDANO_TESTNET_MAGIC" in
  1)
    export "EXPLORER_URL=https://preprod.cardanoscan.io"
    ;;
  2)
    export "EXPLORER_URL=https://preview.cardanoscan.io"
    ;;
  *)
    # Use `mainnet` as the default.
    export "EXPLORER_URL=https://cardanoscan.io"
    ;;
esac

MARLOWE_RT_WEBSERVER_URL="http://$MARLOWE_RT_WEBSERVER_HOST":"$MARLOWE_RT_WEBSERVER_PO
RT"

echo "CARDANO_NODE_SOCKET_PATH = $CARDANO_NODE_SOCKET_PATH"
echo "CARDANO_TESTNET_MAGIC = $CARDANO_TESTNET_MAGIC"
echo "MARLOWE_RT_WEBSERVER_HOST = $MARLOWE_RT_WEBSERVER_HOST"
echo "MARLOWE_RT_WEBSERVER_PORT = $MARLOWE_RT_WEBSERVER_PORT"
echo "MARLOWE_RT_WEBSERVER_URL = $MARLOWE_RT_WEBSERVER_URL"
```

We will check now that an address and key has been created for the seller, buyer and mediator.

```
SELLER_SKEY=keys/lender.skey
SELLER_ADDR=$(cat keys/lender.address)
echo "SELLER_ADDR = $SELLER_ADDR"

BUYER_SKEY=keys/borrower.skey
BUYER_ADDR=$(cat keys/borrower.address)
echo "BUYER_ADDR = $BUYER_ADDR"

MEDIATOR_SKEY=keys/mediator.skey
MEDIATOR_ADDR=$(cat keys/mediator.address)
echo "MEDIATOR_ADDR = $MEDIATOR_ADDR"
```

We also check that those three parties have at least one hundred ada in their accounts.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$SELLER_ADDR"
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$BUYER_ADDR"
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$MEDIATOR_ADDR"
```

Now we can create the contract via the command line. First, we set some more environment variables. We set the purchase price to 75 ada, the minimum amount for the contract deposit to 2 ada and also define some time variables.

```
ADA=1000000  # 1 ada = 1,000,000 lovelace
PRICE=$((75 * ADA))
echo "PRICE = $PRICE lovelace"

MIN_LOVELACE="$((2 * ADA))"
echo "MIN_LOVELACE = $MIN_LOVELACE lovelace"

SECOND=1000 # 1 second = 1000 milliseconds
MINUTE=$((60 * SECOND)) # 1 minute = 60 seconds
HOUR=$((60 * MINUTE)) # 1 hour = 60 minutes
```

In the Marlowe playground when you opened up the escrow example in order to download the contract you would need to click on the **Send to simulator** button. And then you get a form where you can fill in the parameters of the contract. After that you can click on the **Download as JSON** button and you will get the JSON file we will be working with in this chapter. Here we will create the contract at the command line. We define the current time, measured in POSIX milliseconds. The contract also has four deadlines. For convenience in this example, we set the deadlines to the near future.

```
NOW="$((`date -u +%s` * SECOND))"
echo NOW = "$NOW" POSIX milliseconds = "`date -d @$((NOW / SECOND))`"

PAYMENT_DEADLINE=$((NOW+1*HOUR))    # The payment deadline, one hour from now.
COMPLAINT_DEADLINE=$((NOW+2*HOUR))  # The complaint deadline, two hours from now.
DISPUTE_DEADLINE=$((NOW+3*HOUR))    # The dispute deadline, three hours from now.
MEDIATION_DEADLINE=$((NOW+4*HOUR))  # The mediation deadline, four hours from now.

echo PAYMENT_DEADLINE = "$PAYMENT_DEADLINE" POSIX milliseconds = "`date -d
@$((PAYMENT_DEADLINE / SECOND))`"
echo COMPLAINT_DEADLINE = "$COMPLAINT_DEADLINE" POSIX milliseconds = "`date -d
@$((COMPLAINT_DEADLINE / SECOND))`"
echo DISPUTE_DEADLINE = "$DISPUTE_DEADLINE" POSIX milliseconds = "`date -d
@$((DISPUTE_DEADLINE / SECOND))`"
echo MEDIATION_DEADLINE = "$MEDIATION_DEADLINE" POSIX milliseconds = "`date -d
@$((MEDIATION_DEADLINE / SECOND))`"
```

The deadlines are the initial payment deadline by the buyer, the deadline for him to complain, the deadline for the seller which can dispute and the mediator deadline to make his choice. Now we can create the JSON file escrow-contract.json for the contract using the marlowe-cli.

```
marlowe-cli template escrow \
  --minimum-ada "$MIN_LOVELACE" \
  --price "$PRICE" \
  --seller Seller \
  --buyer Buyer \
  --mediator Mediator \
  --payment-deadline "$PAYMENT_DEADLINE" \
  --complaint-deadline "$COMPLAINT_DEADLINE" \
  --dispute-deadline "$DISPUTE_DEADLINE" \
  --mediation-deadline "$MEDIATION_DEADLINE" \
```

```
  --out-contract-file escrow-contract.json \
  --out-state-file /dev/null
```

If you want to know more about the parameters in the command you can use the --help option.

```
marlowe-cli template escrow --help
```

We can now view the contract in yaml format.

```
json2yaml escrow-contract.json
```

Also, here we state as in the previous chapters that it is important to check the safety of the contract. This contract has 4 possible paths if timeouts are not considered which is more complex than the zero-coupon bond contract that had only 1 path. So, it is a good idea to test it on the playground in the simulator or on the Cardano testnet to check that the contract logic actually matches your intentions. Once we have the JSON file the mediator can create the contract and deposit the 2 ada to it. We will use an HTTP post request to the contract's endpoint of Marlowe runtime to do that. We are going bundle the body of the request with the following command where we use Marlowe version 1:

```
yaml2json << EOI > request-1.json
version: v1
contract: `cat escrow-contract.json`
roles:
  Seller: "$SELLER_ADDR"
  Buyer: "$BUYER_ADDR"
  Mediator: "$MEDIATOR_ADDR"
minUTxODeposit: $MIN_LOVELACE
metadata: {}
tags: {}
EOI
cat request-1.json
```

We also defined the participants names and addresses for which role tokens will be minted. We can now use the output request-1.json file and post the request.

```
curl "$MARLOWE_RT_WEBSERVER_URL/contracts" \
  -X POST \
  -H 'Content-Type: application/json' \
  -H "X-Change-Address: $MEDIATOR_ADDR" \
  -d @request-1.json \
  -o response-1.json \
  -sS
json2yaml response-1.json
```

We now extract the contract ID that will be used throughout the life of the contract to tell Marlowe runtime which contract we are operating on.

```
CONTRACT_ID="$(jq -r '.resource.contractId' response-1.json)"
echo "CONTRACT_ID = $CONTRACT_ID"
```

Out of that response body we extract just the transaction and store it in the tx-1.unsigned file.

```
jq '.resource.txBody' response-1.json > tx-1.unsigned
```

Now we have to sign and submit this transaction to a node. There are five ways to do this:

- cardano-cli at the command line
- cardano-wallet at the command line or as a REST service
- cardano-hw-cli for a hardware wallet at the command line
- a Babbage-compatible CIP-30 wallet in a web browser
- marlowe-cli at the command line

For convenience, here we use the marlowe-cli transaction submit command. One may have to wait a minute or so for the transactions to be confirmed on the blockchain.

```
TX_1=$(
marlowe-cli transaction submit \
  --tx-body-file tx-1.unsigned \
  --required-signer "$MEDIATOR_SKEY" \
  --timeout 600 \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_1 = $TX_1"
```

We see that we have provided the unsigned transaction, the signing key and a timeout of 10 minutes to the above command. We can view the transaction on the Cardano explorer.

```
echo "$EXPLORER_URL"/transaction/"$TX_1?tab=utxo"
```

Also, we can check the funds of all three parties and the Marlowe contract.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$SELLER_ADDR"
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$BUYER_ADDR"
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$MEDIATOR_ADDR"
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --tx-in "$CONTRACT_ID"
```

From the output we will see that beside the funds of the users there are also NFT role tokens sitting at their addresses. We can now use Marlowe runtime to take a peek at the contract.

```
CONTRACT_URL="$MARLOWE_RT_WEBSERVER_URL/`jq -r '.links.contract' response-1.json`"
echo "CONTRACT_URL = $CONTRACT_URL"

curl -sS "$CONTRACT_URL" | json2yaml
```

After we have created the contract on the testnet the buyer can deposit the funds into the seller's account. We have to create the JSON request. The marlowe-cli input deposit tool conveniently formats the correct JSON for a deposit.

```
marlowe-cli input deposit \
  --deposit-party Buyer \
  --deposit-account Seller \
  --deposit-amount "$PRICE" \
  --out-file input-2.json
json2yaml input-2.json
```

In the above command we see that we say the buyer is depositing the price at the seller's account. Now we can bundle that input into a request by supplying a Marlowe version and then if we wanted metadata in the transaction, we could add it. We could also add tags which are handy if we would be building a web 2 or web 3 application. If you use the tags with special queries, you can fetch information about the contracts. We use the following command:

```
yaml2json << EOI > request-2.json
version: v1
inputs: [$(cat input-2.json)]
metadata: {}
tags: {}
EOI
cat request-2.json
```

Next, we post the request and store the response.

```
curl "$CONTRACT_URL/transactions" \
  -X POST \
  -H 'Content-Type: application/json' \
  -H "X-Change-Address: $BUYER_ADDR" \
  -d @request-2.json \
  -o response-2.json \
  -sS
json2yaml response-2.json
```

The buyer is the one which did the signing and gets the change. We use the jq command to extract the unsinged part of the transaction.

```
jq '.resource.txBody' response-2.json > tx-2.unsigned
```

Then we use marlowe-cli to submit the transaction and then wait for confirmation.

```
TX_2=$(
marlowe-cli transaction submit \
  --tx-body-file tx-2.unsigned \
  --required-signer "$BUYER_SKEY" \
  --timeout 600 \
```

```
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_2 = $TX_2"
```

After some seconds we can view the transaction on the Cardano explorer.

```
echo "$EXPLORER_URL"/transaction/"$TX_2?tab=utxo"
```

If we query now the address of the Marlowe contract we will see it contains 77 ada, which is the price plus the initial deposit of 2 ada.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --tx-in "$TX_2#1"
```

Below is an example output of the above command:

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
5c45ab6080a4b5bf47474e0e3eac6ff54a93737fa25ce9c0997b30a51fc4d526     1        77000000
                                                                                lovelace +
                                                  TxOutDatumHash ScriptDataInBabbageEra
                    "cec4e93035eef608715e63a0bb412ef897093f3119f108139d91323481e86e0a"
```

With the command below we can view the progress of the contract on the blockchain.

```
curl -sS "$CONTRACT_URL"/transactions/"$TX_2" | json2yaml
```

In the output at the bottom, we can see that the contract is holding 2 ada at the benefit of the contract creator and 75 ada for the seller. Now the buyer has to make a choice and needs to say whether everything is ok or report a problem. We can use the marlowe-cli input choose command to make a choice. According to the contract number 1 represents the choice where the buyer says there is a problem.

```
marlowe-cli input choose \
  --choice-name "Report problem" \
  --choice-party Buyer \
  --choice-number 1 \
  --out-file input-3.json
json2yaml input-3.json
```

We can then bundle this in a request.

```
yaml2json << EOI > request-3.json
version: v1
inputs: [$(cat input-3.json)]
metadata: {}
tags: {}
EOI
```

```
cat request-3.json
```

Next, we post the request and store the response.

```
curl "$CONTRACT_URL/transactions" \
  -X POST \
  -H 'Content-Type: application/json' \
  -H "X-Change-Address: $BUYER_ADDR" \
  -d @request-3.json \
  -o response-3.json \
  -sS
json2yaml response-3.json
```

We pull out the unsigned transaction.

```
jq '.resource.txBody' response-3.json > tx-3.unsigned
```

And we use the marlowe-cli to submit the transaction and then wait for confirmation.

```
TX_3=$(
marlowe-cli transaction submit \
  --tx-body-file tx-3.unsigned \
  --required-signer "$BUYER_SKEY" \
  --timeout 600 \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_3 = $TX_3"
```

We can view the transaction in the Cardano explorer.

```
echo "$EXPLORER_URL"/transaction/"$TX_3?tab=utxo"
```

If we query the buyers address, we see he has still approximately 75 ada less than originally.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$BUYER_ADDR"
```

And if we query the Marlowe contract we will see it has still 77 ada.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --tx-in "$TX_3#1"
```

An example output of the above command:

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
3a7ffde61ba8e1a2281e1a165f42901b302cc3eb2ad122e7f7396f1807e85a7d     1        77000000
                                                                                lovelace +
                                                  TxOutDatumHash ScriptDataInBabbageEra
                     "859a9bb208ab9beed6c6d75f9c5da732f41f52fe9748d1d79931daaa28c5e989"
```

Marlowe has an internal state of accounts, bound values and choices that some transactions update. If we query the contract, we will see that the datum has changed because the state of the contract changed. We can use the following command for that:

```
curl -sS "$CONTRACT_URL"/transactions/"$TX_3" | json2yaml
```

Now we will say that the seller does not agree with the buyer and he does not think there is a problem. So, we are going to create input to represent that and we are basically saying there is a dispute for which the choice number zero can be used.

```
marlowe-cli input choose \
  --choice-name "Dispute problem" \
  --choice-party Seller \
  --choice-number 0 \
  --out-file input-4.json
json2yaml input-4.json
```

We bundle this into our fourth request.

```
yaml2json << EOI > request-6.json
contractId: "$CONTRACT_ID"
role: Seller
EOI
cat request-6.json
```

Then we post the request and store the response.

```
yaml2json << EOI > request-4.json
version: v1
inputs: [$(cat input-4.json)]
metadata: {}
tags: {}
EOI
cat request-4.json
```

The seller is doing the operation so he is in the change address header. We can now post the request and store the response.

```
curl "$CONTRACT_URL/transactions" \
  -X POST \
  -H 'Content-Type: application/json' \
  -H "X-Change-Address: $SELLER_ADDR" \
  -d @request-4.json \
  -o response-4.json \
  -sS
json2yaml response-4.json
```

After that we again extract the unsigned transaction and submit it with the marlowe-cli.

```
jq '.resource.txBody' response-4.json > tx-4.unsigned

TX_4=$(
marlowe-cli transaction submit \
  --tx-body-file tx-4.unsigned \
  --required-signer "$SELLER_SKEY" \
  --timeout 600 \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_4 = $TX_4"
```

We can view the transaction on the Cardano explorer.

```
echo "$EXPLORER_URL"/transaction/"$TX_4?tab=utxo"
```

If we query the Marlowe contract, we see that it still has the 77 ada.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --tx-in "$TX_4#1"
```

An example output of the above command:

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
87dba73e0145b13004602c17eb71c3a9a3a9d26760179a41df9f0c0b71f84894     1        77000000
                                                                              lovelace +
                                                TxOutDatumHash ScriptDataInBabbageEra
                    "b9755884ecbd03f8a11bb6b8107dafa2c21d42eeff45d46a7989e162714ee6fd"
```

Notice however that the *TxHash* has changed and also the datum hash got changed compared to the previous query because we updated the state of the contract. We can again view the full progress of the contract with the command below:

```
curl -sS "$CONTRACT_URL"/transactions/"$TX_4" | json2yaml
```

Now the mediator has to step in because the buyer and the seller do not agree. In our case the mediator rules in favor of the seller which is choice number zero.

```
marlowe-cli input choose \
  --choice-name "Dismiss claim" \
  --choice-party Mediator \
  --choice-number 0 \
  --out-file input-5.json
json2yaml input-5.json
```

Again, we create the request file.

```
yaml2json << EOI > request-5.json
version: v1
```

```
inputs: [$(cat input-5.json)]
metadata: {}
tags: {}
EOI
cat request-5.json
```

Then we post the request and store the response.

```
curl "$CONTRACT_URL/transactions" \
  -X POST \
  -H 'Content-Type: application/json' \
  -H "X-Change-Address: $MEDIATOR_ADDR" \
  -d @request-5.json \
  -o response-5.json \
  -sS
json2yaml response-5.json
```

Once again, use the marlowe-cli to submit the transaction and then wait for confirmation.

```
jq '.resource.txBody' response-5.json > tx-5.unsigned

TX_5=$(
marlowe-cli transaction submit \
  --tx-body-file tx-5.unsigned \
  --required-signer "$MEDIATOR_SKEY" \
  --timeout 600 \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_5 = $TX_5"
```

Then we retrieve the link for the Cardano explorer.

```
echo "$EXPLORER_URL"/transaction/"$TX_5?tab=utxo"
```

If we query the Marlowe contract now we see it is closed, but the role-payout address has the 75 ada for the benefit of the seller.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --tx-in "$TX_5#2"
```

An example output of the above command:

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
e247a0d8cc28973ca01736c56f1c51725210e3b8584687d32f6929c800a8a66b     2        75000000
                                                                                lovelace +
                                                    TxOutDatumHash ScriptDataInBabbageEra
                     "3d3d354ce405bd517f65eb8b2905e8da5ab74845fabddd149d8fc8f173bdce3f"
```

And we see the transaction hash and the datum also get updated. The datum now encodes the information that the funds left at the address belong to the seller.

We can see again the progress of the contract if we fetch information from the blockchain.

```
curl -sS "$CONTRACT_URL"/transactions/"$TX_5" | json2yaml
```

We will see the last choice and that the contract is closed. Now 75 ada is held at marlowe's role payout address for the benefit of the seller. The seller can withdraw these funds at any time. The contract ID and role name are included in the request body for a withdrawal.

```
yaml2json << EOI > request-6.json
contractId: "$CONTRACT_ID"
role: Seller
EOI
cat request-6.json
```

We again post the request and store the response.

```
curl "$MARLOWE_RT_WEBSERVER_URL/withdrawals" \
  -X POST \
  -H 'Content-Type: application/json' \
  -H "X-Change-Address: $SELLER_ADDR" \
  -d @request-6.json \
  -o response-6.json \
  -sS
json2yaml response-6.json
```

And then we use the marlowe-cli to submit the transaction and wait for confirmation.

```
jq '.resource.txBody' response-6.json > tx-6.unsigned

TX_6=$(
marlowe-cli transaction submit \
  --tx-body-file tx-6.unsigned \
  --required-signer "$SELLER_SKEY" \
  --timeout 600 \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_6 = $TX_6"
```

We can view the transaction on Cardano explorer.

```
echo "$EXPLORER_URL"/transaction/"$TX_6?tab=utxo"
```

If we query the seller’s address, we see that he has around 75 ada more than in the beginning.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$SELLER_ADDR"
```

Below is an example output for the above command.

```
                           TxHash                                 TxIx        Amount
```

```
--------------------------------------------------------------------------------------
eb848199e21597df56a45d744f29a4f3a41425c876380f4c58542a296e369cee     0      1073964776
                                                                                lovelace +
                                                                            TxOutDatumNone
eb848199e21597df56a45d744f29a4f3a41425c876380f4c58542a296e369cee     1        1034400
                                                                                lovelace +
             1 03e6a02b3cb1db721248de96664a7f92318b17bd5f54657bd8031072.53656c6c6572 +
                                                                            TxOutDatumNone
```

We can now also look at this last withdrawal transaction with the following command:

```
curl -sS "$MARLOWE_RT_WEBSERVER_URL"/withdrawals/"$TX_6" | json2yaml
```

Below is an example output of this command:

```
block:
  blockHeaderHash: ebe3a661cd9b4799bfcecbcf3ae35fbc4d2ee24bfc7aa67c0a5fb6b250216562
  blockNo: 757411
  slotNo: 23980393
payouts:
- contractId: b224b1873d986a5e34d66e6d86f47aa51aed7d3535b933e31fc9d8f98e742b7c#1
  payout: e247a0d8cc28973ca01736c56f1c51725210e3b8584687d32f6929c800a8a66b#2
  role: Seller
  roleTokenMintingPolicyId: 03e6a02b3cb1db721248de96664a7f92318b17bd5f54657bd8031072
status: confirmed
withdrawalId: eb848199e21597df56a45d744f29a4f3a41425c876380f4c58542a296e369cee
```

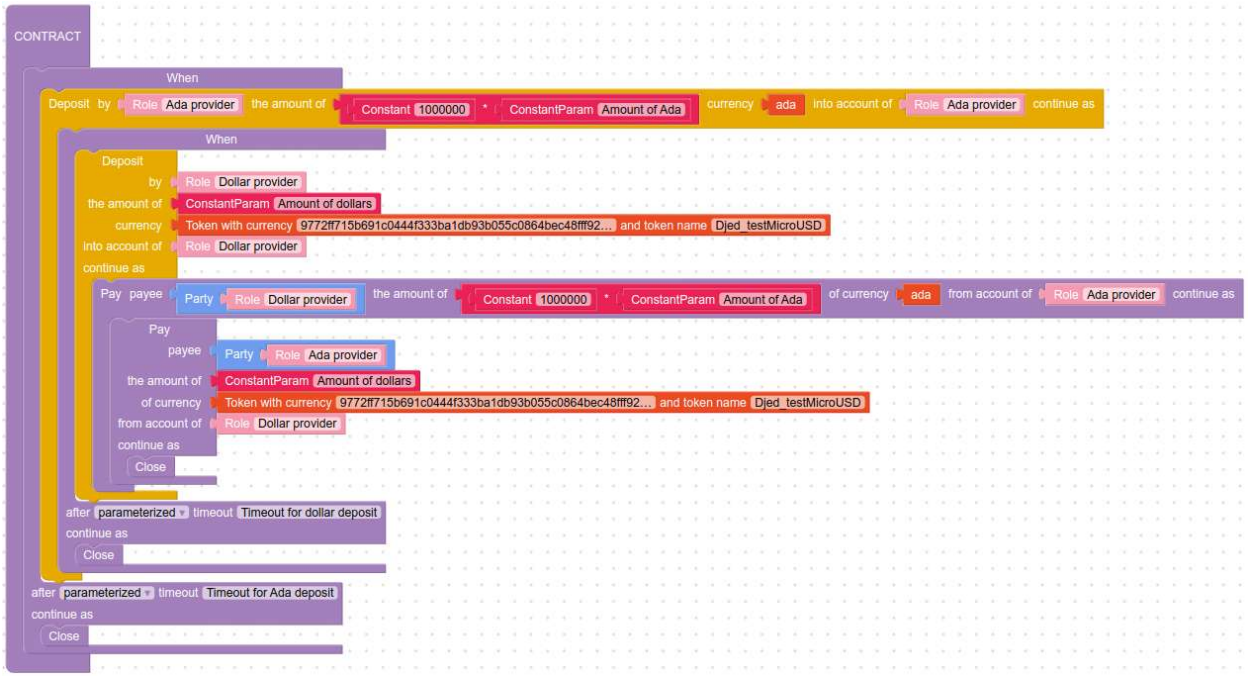
7.9 Marlowe Starter Kit: Swap contract using the Marlowe Runtime's REST API

In this chapter we are going to demonstrate the swap contract example. Here we are going to swap ada for djed which is a stable coin on the Cardano blockchain. The code for this example is contained in the 05-swap-rest.ipynb from the Marlowe starter kit github project. The contract has an exchange rate built into it. Below you can see the Marlowe code for it.

```
When
    [Case
        (Deposit
            (Role "Ada provider")
            (Role "Ada provider")
            (Token "" "")
            (MulValue
                (Constant 1000000)
                (ConstantParam "Amount of Ada")
            )
        )
        (When
            [Case
                (Deposit
                    (Role "Dollar provider")
```

```
                    (Role "Dollar provider")
                    (Token   "9772ff715b691c0444f333ba1db93b055c0864bec48fff92d1f2a7fe"
"Djed_testMicroUSD")
                    (ConstantParam "Amount of dollars")
                )
                (Pay
                    (Role "Ada provider")
                    (Party (Role "Dollar provider"))
                    (Token "" "")
                    (MulValue
                        (Constant 1000000)
                        (ConstantParam "Amount of Ada")
                    )
                    (Pay
                        (Role "Dollar provider")
                        (Party (Role "Ada provider"))
                        (Token
"9772ff715b691c0444f333ba1db93b055c0864bec48fff92d1f2a7fe" "Djed_testMicroUSD")
                        (ConstantParam "Amount of dollars")
                        Close
                    )
                )]
            (TimeParam "Timeout for dollar deposit")
            Close
        )]
    (TimeParam "Timeout for Ada deposit")
    Close
```

In Blockly the contract looks like this:



We again need to define environment variables for web server data and create addresses for contract parties. The instructions for this can be found in chapter 7.4. Next, we have to set up some more environment variables we will use in this chapter. You can see the code below.

```
if [[ -z "$MARLOWE_RT_WEBSERVER_PORT" ]]
then

  # Only required for `marlowe-cli` and `cardano-cli`.
  export CARDANO_NODE_SOCKET_PATH="$(docker volume inspect marlowe-starter-kit_shared
| jq -r '.[0].Mountpoint')/node.socket"
  export CARDANO_TESTNET_MAGIC=1 # Note that preprod=1 and preview=2. Do not set this
variable if using mainnet.

  # Only required for Marlowe Runtime REST API.
  export MARLOWE_RT_WEBSERVER_HOST="127.0.0.1"
  export MARLOWE_RT_WEBSERVER_PORT=3780

fi

# FIXME: This should have been inherited from the parent environment.
if [[ -z "$CARDANO_NODE_SOCKET_PATH" ]]
then
  export CARDANO_NODE_SOCKET_PATH=/ipc/node.socket
fi

# FIXME: This should have been set in the parent environment.
if [[ -z "$CARDANO_TESTNET_MAGIC" ]]
then
  export CARDANO_TESTNET_MAGIC=$CARDANO_NODE_MAGIC
fi

case "$CARDANO_TESTNET_MAGIC" in
  1)
    export "EXPLORER_URL=https://preprod.cardanoscan.io"
    ;;
  2)
    export "EXPLORER_URL=https://preview.cardanoscan.io"
    ;;
  *)
    # Use `mainnet` as the default.
    export "EXPLORER_URL=https://cardanoscan.io"
    ;;
esac

MARLOWE_RT_WEBSERVER_URL="http://$MARLOWE_RT_WEBSERVER_HOST":"$MARLOWE_RT_WEBSERVER_PO
RT"

echo "CARDANO_NODE_SOCKET_PATH = $CARDANO_NODE_SOCKET_PATH"
echo "CARDANO_TESTNET_MAGIC = $CARDANO_TESTNET_MAGIC"
echo "MARLOWE_RT_WEBSERVER_HOST = $MARLOWE_RT_WEBSERVER_HOST"
echo "MARLOWE_RT_WEBSERVER_PORT = $MARLOWE_RT_WEBSERVER_PORT"
echo "MARLOWE_RT_WEBSERVER_URL = $MARLOWE_RT_WEBSERVER_URL"
```

The output of the commands above will display the node socket path, the testnet magic and some Marlowe web server data. Next, we define the variables for the ada and dollar providers.

```
ADA_PROVIDER_SKEY=keys/lender.skey
ADA_PROVIDER_ADDR=$(cat keys/lender.address)
echo "ADA_PROVIDER_ADDR = $ADA_PROVIDER_ADDR"
```

```
USD_PROVIDER_SKEY=keys/borrower.skey
USD_PROVIDER_ADDR=$(cat keys/borrower.address)
echo "USD_PROVIDER_ADDR = $USD_PROVIDER_ADDR"
```

We can now query the funds at those addresses and check that they at least have a 100 ada.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$ADA_PROVIDER_ADDR"
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$USD_PROVIDER_ADDR"
```

Then we design the contract. We are going to swap 294 ada for 100 djed tokens. So we set some more environment variables that we will need to create the contract.

```
ADA=1000000  # 1 ada = 1,000,000 lovelace
LOVELACE_AMOUNT=$((294 * ADA))
MICROUSD_AMOUNT=$((100 * 1000000))
echo "LOVELACE_AMOUNT = $LOVELACE_AMOUNT lovelace"
echo "MICROUSD_AMOUNT = $MICROUSD_AMOUNT µUSD"

MIN_LOVELACE="$((2 * ADA))"
echo "MIN_LOVELACE = $MIN_LOVELACE lovelace"

SECOND=1000 # 1 second = 1000 milliseconds
MINUTE=$((60 * SECOND)) # 1 minute = 60 seconds
HOUR=$((60 * MINUTE)) # 1 hour = 60 minutes
```

This time we will design the contract using Marlowe playground. Here are the steps:

1. Got to https://play.marlowe-finance.io/ in a web browser.
2. Click on the **Open an Example** button.
3. Click on the **Marlowe** or **Blockly** button under the Swap section.
4. Edit the contract so that the dollar token has the policy ID and name for the dollar token. In this example we use test djed, which has the token name **Djed_testMicroUSD** and policy ID 9772ff715b691c0444f333ba1db93b055c0864bec48fff92d1f2a7fe.
5. Click on the **Send to Simulator** button.
6. Set the **Timeout for ada deposit** to one hour into the future.
7. Set the **Timeout for dollar deposit** to two hours into the future.
8. Set the **Amount of Ada** to 294.
9. Set the **Amount of dollars** to 100,000,000, since the units of measure are millionths.
10. Click on the **Download as JSON** button, set the file name to swap-contract.json, and store the file in the marlowe-starter-kit folder.

We can now examine the contract as a yaml file.

```
json2yaml swap-contract.json
```

We can see the data that we have inputted on the Marlowe playground. We again state here that it is a good practice to check the safety of your contract. The options you can use for that were outlined in chapter 7.5. We now have to create the contract. We are using the HTTP post endpoint contracts in Marlowe runtime. We provide the contract, the role names and the minimal deposit when we create the JSON body of the request to build the transaction.

```
yaml2json << EOI > request-1.json
version: v1
contract: `cat swap-contract.json`
roles:
  Ada provider: "$ADA_PROVIDER_ADDR"
  Dollar provider: "$USD_PROVIDER_ADDR"
minUTxODeposit: $MIN_LOVELACE
metadata: {}
tags: {}
EOI
cat request-1.json
```

Then we post the request and view the response.

```
curl "$MARLOWE_RT_WEBSERVER_URL/contracts" \
  -X POST \
  -H 'Content-Type: application/json' \
  -H "X-Change-Address: $ADA_PROVIDER_ADDR" \
  -d @request-1.json \
  -o response-1.json \
  -sS
json2yaml response-1.json
```

We can now extract the contract ID.

```
CONTRACT_ID="$(jq -r '.resource.contractId' response-1.json)"
echo "CONTRACT_ID = $CONTRACT_ID"
```

The CBOR serialization (in text-envelope format) is also embedded in the response.

```
jq '.resource.txBody' response-1.json > tx-1.unsigned
```

We will use the *marlowe-cli transaction submit* command to submit our transaction.

```
TX_1=$(
marlowe-cli transaction submit \
  --tx-body-file tx-1.unsigned \
  --required-signer "$ADA_PROVIDER_SKEY" \
  --timeout 600 \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_1 = $TX_1"
```

We can now check the transaction on the Cardano explorer.

```
echo "$EXPLORER_URL"/transaction/"$TX_1?tab=utxo"
```

The Marlowe contract now contains 2 ada. If we check the address of the ada and dollar provider we will see that they have received their role tokens.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$ADA_PROVIDER_ADDR"
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$USD_PROVIDER_ADDR"
```

We can look at the contract using Marlowe runtime. First, we compute the URL and then fetch the data from the blockchain with the curl command.

```
CONTRACT_URL="$MARLOWE_RT_WEBSERVER_URL/`jq -r '.links.contract' response-1.json`"
echo "CONTRACT_URL = $CONTRACT_URL"

curl -sS "$CONTRACT_URL" | json2yaml
```

Now the ada provider is going to make his deposit. We are going to use the marlowe-cli input command to create the JSON file.

```
marlowe-cli input deposit \
  --deposit-party 'Ada provider' \
  --deposit-account 'Ada provider' \
  --deposit-amount "$LOVELACE_AMOUNT" \
  --out-file input-2.json
json2yaml input-2.json
```

When creating the request file, we are using Marlowe version 1 and the input-2.json file.

```
yaml2json << EOI > request-2.json
version: v1
inputs: [$(cat input-2.json)]
metadata: {}
tags: {}
EOI
cat request-2.json
```

Now we can post the request and store the response.

```
curl "$CONTRACT_URL/transactions" \
  -X POST \
  -H 'Content-Type: application/json' \
  -H "X-Change-Address: $ADA_PROVIDER_ADDR" \
  -d @request-2.json \
  -o response-2.json \
  -sS
json2yaml response-2.json
```

Once again, use the marlowe-cli to submit the transaction and then wait for confirmation.

```
jq '.resource.txBody' response-2.json > tx-2.unsigned

TX_2=$(
marlowe-cli transaction submit \
  --tx-body-file tx-2.unsigned \
  --required-signer "$ADA_PROVIDER_SKEY" \
  --timeout 600 \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_2 = $TX_2"
```

We can view the transaction in the Cardano explorer.

```
echo "$EXPLORER_URL"/transaction/"$TX_2?tab=utxo"
```

If we query the funds of the Marlowe contract we see it still has the 2 ada from its creation and an additional 294 ada.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --tx-in "$TX_2#1"
```

An example output of the above command can be seen below.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
84e284b1ad405c2b32e4eb4d93c31ec15b10470737716f3461ed0623e8572e58     1       296000000
                                                                              lovelace +
                                                    TxOutDatumHash ScriptDataInBabbageEra
                    "c945bedf448b262fe3e5c36cf3033a4a4914456277f0de38432c4f28bd352ed3"
```

We can view the state of the contract using Marlowe runtime.

```
curl -sS "$CONTRACT_URL"/transactions/"$TX_2" | json2yaml
```

Now we can deposit the 100 djed into the contract. Here the deposit will be a little different. We have to give it the type of token that is being deposited and there is a notation for that. The deposit is represented as JSON input to the contract. The *marlowe-cli input deposit* command conveniently formats the correct JSON for a deposit.

```
marlowe-cli input deposit \
  --deposit-party 'Dollar provider' \
  --deposit-account 'Dollar provider' \
  --deposit-token
9772ff715b691c0444f333ba1db93b055c0864bec48fff92d1f2a7fe.Djed_testMicroUSD \
  --deposit-amount "$MICROUSD_AMOUNT" \
  --out-file input-3.json
json2yaml input-3.json
```

Then we have to put together the request.

```
yaml2json << EOI > request-3.json
version: v1
inputs: [$(cat input-3.json)]
metadata: {}
tags: {}
EOI
cat request-3.json
```

After that we post the request and store the response.

```
curl "$CONTRACT_URL/transactions" \
  -X POST \
  -H 'Content-Type: application/json' \
  -H "X-Change-Address: $USD_PROVIDER_ADDR" \
  -d @request-3.json \
  -o response-3.json \
  -sS
json2yaml response-3.json
```

Once again, we use the marlowe-cli to submit the transaction and then wait for confirmation.

```
jq '.resource.txBody' response-3.json > tx-3.unsigned

TX_3=$(
marlowe-cli transaction submit \
  --tx-body-file tx-3.unsigned \
  --required-signer "$USD_PROVIDER_SKEY" \
  --timeout 600 \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_3 = $TX_3"
```

We can now view the transaction on the Cardano scan website.

```
echo "$EXPLORER_URL"/transaction/"$TX_3?tab=utxo"
```

The Marlowe contract has closed, but the roll-payout address holds 100 djed for the benefit of the ada provider. We can query this information with the following command:

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --tx-in "$TX_3#2"
```

Below you can see an example output of the command above.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
05b05966afb5d346d3f2a03470df632b5b697265f637d2adb5fcb463bd127f5e     2        1258520
                                                                                lovelace +
                                                                                 100000000
```

```
 9772ff715b691c0444f333ba1db93b055c0864bec48fff92d1f2a7fe.446a65645f746573744d6963726f5
                                                55344 + TxOutDatumHash ScriptDataInBabbageEra
                     "758d98b7ec8818a2cd32baa416d6a337bcb837a2393815744e3f4935bebedd3e"
```

The roll-payout address also holds 294 ada for the benefit of the dollar provider.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --tx-in "$TX_3#3"
```

Below you can see an example output of the command above.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
05b05966afb5d346d3f2a03470df632b5b697265f637d2adb5fcb463bd127f5e     3       294000000
                                                                                lovelace +
                                                    TxOutDatumHash ScriptDataInBabbageEra
                     "366c6e0ba1da1c0fe3488c49fba1f8dcdd6f4dcc723bbd08bc62caf12633facc"
```

The 2 ada that was held by the contract got refunded to the creator of the contract. We can check again the state of the contract on the blockchain.

```
curl -sS "$CONTRACT_URL"/transactions/"$TX_3" | json2yaml
```

We now first create a request from the ada provider that he can withdraw his funds. We only have to say what the contract ID is and what the role is.

```
yaml2json << EOI > request-4.json
contractId: "$CONTRACT_ID"
role: "Ada provider"
EOI
cat request-4.json
```

Then we post the request and store the response.

```
curl "$MARLOWE_RT_WEBSERVER_URL/withdrawals" \
  -X POST \
  -H 'Content-Type: application/json' \
  -H "X-Change-Address: $ADA_PROVIDER_ADDR" \
  -d @request-4.json \
  -o response-4.json \
  -sS
json2yaml response-4.json
```

We can now submit the transaction and wait for confirmation.

```
jq '.resource.txBody' response-4.json > tx-4.unsigned

TX_4=$(
marlowe-cli transaction submit \
  --tx-body-file tx-4.unsigned \
```

```
  --required-signer "$ADA_PROVIDER_SKEY" \
  --timeout 600 \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_4 = $TX_4"
```

We can again retrieve the link for Cardano scan.

```
echo "$EXPLORER_URL"/transaction/"$TX_4?tab=utxo"
```

And we can check the funds for the ada provider.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$ADA_PROVIDER_ADDR"
```

Below is an example output of the command above.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
05b05966afb5d346d3f2a03470df632b5b697265f637d2adb5fcb463bd127f5e     4        2000000
                                                                              lovelace +
                                                                          TxOutDatumNone
cdd5691e3085d6dd50ac45911b2a6bbd372a232a05a68851263279a568973a37     0       700481539
                                                                              lovelace +
                                                                          TxOutDatumNone
cdd5691e3085d6dd50ac45911b2a6bbd372a232a05a68851263279a568973a37     1        1060260
                                                                              lovelace +
 1 bb7053ce07dda2292f779c92ae789b4c4a99376f5f6d92b5c4a9f6a4.4164612070726f7669646572 +
                                                                          TxOutDatumNone
cdd5691e3085d6dd50ac45911b2a6bbd372a232a05a68851263279a568973a37     2        1099050
                                                                              lovelace +
                                                                           100000000
 9772ff715b691c0444f333ba1db93b055c0864bec48fff92d1f2a7fe.446a65645f746573744d6963726f5
                                                                             55344 +
                                                                          TxOutDatumNone
```

We see that the ada provider has now a 100 djed. We can fetch the withdrawal information from the blockchain with the following command:

```
curl -sS "$MARLOWE_RT_WEBSERVER_URL"/withdrawals/"$TX_4" | json2yaml
```

For the dollar provider we follow the same procedure. First, we create the request file.

```
yaml2json << EOI > request-5.json
contractId: "$CONTRACT_ID"
role: "Dollar provider"
EOI
cat request-5.json
```

Then we post the request and store the response.

```
curl "$MARLOWE_RT_WEBSERVER_URL/withdrawals" \
  -X POST \
  -H 'Content-Type: application/json' \
  -H "X-Change-Address: $USD_PROVIDER_ADDR" \
  -d @request-5.json \
  -o response-5.json \
  -sS
json2yaml response-5.json
```

And we use the marlowe-cli to submit the transaction and wait for confirmation.

```
jq '.resource.txBody' response-5.json > tx-5.unsigned

TX_5=$(
marlowe-cli transaction submit \
  --tx-body-file tx-5.unsigned \
  --required-signer "$USD_PROVIDER_SKEY" \
  --timeout 600 \
| sed -e 's/^TxId "\(.*\)"$/\1/' \
)
echo "TX_5 = $TX_5"
```

We retrieve the link for Cardano scan.

```
echo "$EXPLORER_URL"/transaction/"$TX_5?tab=utxo"
```

Then we can check the funds for the dollar provider.

```
cardano-cli query utxo --testnet-magic "$CARDANO_TESTNET_MAGIC" --address "$USD_PROVIDER_ADDR"
```

Below is an example output of the command above.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
315f54a7c39c96f1bcc521954edc5e33b378d3aebfc71f2a1168d358adef5867     0      1293498960
                                                                              lovelace +
                                                                          TxOutDatumNone
315f54a7c39c96f1bcc521954edc5e33b378d3aebfc71f2a1168d358adef5867     1        1073190
                                                                              lovelace +
  1 bb7053ce07dda2292f779c92ae789b4c4a99376f5f6d92b5c4a9f6a4.446f6c6c61722070726f7669646572
                                                                                      +
                                                                          TxOutDatumNone
```

We see that the dollar provider now has around 294 ada more. We can again fetch the withdrawal information from the blockchain with the following command:

```
curl -sS "$MARLOWE_RT_WEBSERVER_URL"/withdrawals/"$TX_5" | json2yaml
```

8 Staking

In this lecture we will talk about staking and how it relates to Plutus. In the previous lectures we have talked about the script purpose of the validator scripts and we have concentrated on two most common purposes. They were spending, which is responsible for unlocking a script UTXO and minting which is used when you mint or burn a Cardano native token that has a Plutus script as its minting policy. The other two possible script purposes are rewarding and certifying. In this chapter we will talk about rewarding. We will talk about how we can use Plutus to somehow restrict or influence under which circumstances staking rewards can be withdrawn. Staking became possible after the Alonzo hard fork when Plutus was deployed to Cardano's mainnet and testnet. Before that we only had the playground and the emulator and those did not support staking. So, even though Plutus always in principle supported it was not really possible to try it out. But after Alonzo if you for example want to play with staking on the testnet you have the problem that things take long. The rhythm of staking depends on so-called epochs, which is 5 days so if you wanted to try things out, you would normally have to wait for at least five days or even longer depending on what you wanted to do. And that of-course is tedious. There is an alternative namely to spin up a private testnet. There you have full control over the parameters and you can make slot lengths or epochs much shorter so that it becomes feasible to play around with staking in a reasonable amount of time. This is what we will use in this chapter. In the first chapter we will look at how to access the private testnet.

8.1 The Private Testnet

If you are using the docker container together with VSCode as presented in chapter 1.1 the libraries you need to run the private testnet are already added to the container. Also, the folder *cardano-private-testnet-setup* is included in the Plutus Pioneer program GitHub repository as a git sub-tree. The official repository for the private Cardano testnet can be found at:
https://github.com/woofpool/cardano-private-testnet-setup

The Babbage branch from the repository above was added to our pioneer GitHub repository because we want to use Plutus version 2 which requires that private testnet to run on Babbage. And the version of the main branch as of time of writing still runs in the Alonzo era. This repository also contains nice explanations and a lot of options on how you can configure it. We will use it in default configuration which will spin up three nodes, create three users, set up three stake pools and provide initial funds to the users. In order for this to work you need a Cardano node which is already installed in the docker container. You can type in the following command to check the version of the Cardano node:

```
cardano-node --version
```

To spin up the private testnet you have to run the following commands:

```
workspace# cd cardano-private-testnet-setup/
workspace/cardano-private-testnet-setup# ./scripts/automate.sh
```

You may encounter the following issue below:

```
/workspace/cardano-private-testnet-setup# ./scripts/automate.sh
bash: ./automate.sh: /bin/sh^M: bad interpreter: No such file or directory
```

In this case run the command below for all the files in the cardano-private-testnet-setup/scripts folder. In this example we do it only for the *automate.sh* file.

```
/workspace/cardano-private-testnet-setup/scripts# sed -i -e 's/\r$//' automate.sh
```

It will take a couple of seconds for the testnet to spin up. When it does you should see the message "*Congrats! Your network is ready for use!"*. The node socket will be created at *private-testnet/node-spo1/node.socket*. The main advantage of the testnet for us is how fast it can progress. An epoch can pass in only a couple of minutes. One disadvantage of this is if you know something about Cardano, how it works, how you run a node or how you operate the stake pool there is this concept of KES keys that have to be renewed in regular intervals. For the normal Cardano mainnet that happens every couple of weeks. So, everybody that operates a stake pool has to frequently every couple of weeks generate new KES keys and replace the old ones. Because this private testnet is so fast this is not necessary only after a number of weeks but happens approximately after an hour. This means if you take a long time trying things out following the instructions from this chapter you might run into the problem that suddenly things stop working because your KES keys are outdated. The easiest thing in that case is to stop the testnet and start over again. Of course, you can also try to renew your keys. We can stop the testnet by pressing CTRL+C. Sometimes it can happen that some of the nodes will still be running. You can in that case check the running processes with top and if you see a cardano-node is listed you can use its PID to stop the process. Once our testnet is running we can open up a new terminal in VSCode that will also be attached to the docker container and do some interactions with the testnet. In the code/Week08/ folder of the Plutus Pioneer repository there is a script folder that contains bash scripts which let us interact with the testnet. With the *query-tip.sh* script we can see the testnet parameters and the sync progress.

```
/workspace/code/Week08/scripts# ./query-tip.sh
{
    "block": 199,
```

```
    "epoch": 3,
    "era": "Babbage",
    "hash": "d7c1254160e5f1ed5b2cfd63fd9fe0c0fa7dde635869d3245ec216f2ae8e1fe3",
    "slot": 1773,
    "syncProgress": "100.00"
}
```

From the output above we see the testnet is running and is 100% synced. We are currently in block 199. If we look into the folder *cardano-private-testnet-setup/private-testnet* we see various subfolders and one of them is addresses. There we will find all payment addresses for the three test users and also their staking address. In the folder *stake-delegator-keys* we have the corresponding public and private keys. For each of the three users we have public and private payment and public and private staking keys. First, we check how many funds user one has. In the *code/Week08/scripts* folder we use the *query-utxo-user1.sh* script.

```
/workspace/code/Week08/scripts# cat query-utxo-user1.sh
#!/bin/bash
export CARDANO_NODE_SOCKET_PATH=/workspace/cardano-private-testnet-setup/private-
testnet/node-spo1/node.sock
cardano-cli query utxo \
    --testnet-magic 42 \
    --address $(cat /workspace/code/Week08/tmp/user1-script.addr)
```

After we execute the script, we get the following output.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
08ff7e184717c33b632c39912d2f88a44d982110100cc167b98c6af550cfb126     0    300000000000
                                                                           lovelace +
                                                                       TxOutDatumNone
```

User one has 300.000 ada at his address. The script *query-stake-pools.sh* lists the stake pools we are running together with their IDs. You can see the script code below.

```
/workspace/code/Week08/scripts# cat query-stake-pools.sh

#!/bin/bash
export CARDANO_NODE_SOCKET_PATH=/workspace/cardano-private-testnet-setup/private-
testnet/node-spo1/node.sock
cardano-cli query stake-pools --testnet-magic 42
```

And here is an example output of the script.

```
pool1xlgl5h9rzm8yztpnffkcy0z2293kenmpnx3e353wc9d7ummm6h2
pool1w69fkcxtghp4yhy4v9na3ruklpyke2adgcdgz6qmvpk5s5ge80u
pool10aaskwk6gjctm785cc9n59mua7thfgu7kr340gfms5y8qrztylq
```

Then we can also query stake address information. The following script does that for user one.

```
/workspace/code/Week08/scripts# cat query-stake-address-info-user1.sh

#!/bin/bash
export CARDANO_NODE_SOCKET_PATH=/workspace/cardano-private-testnet-setup/private-
testnet/node-spo1/node.sock
cardano-cli query stake-address-info \
    --testnet-magic 42 \
    --address $(cat /workspace/cardano-private-testnet-setup/private-
                    testnet/addresses/staking1.addr)
```

An example output of the script can be seen below.

```
[
    {
        "address": "stake_test1upsae5v7mqyl3uqnhgv3g2282gwg0w07p06n238acsw5jgqrzk8l4",
        "delegation": "pool10aaskwk6gjctm785cc9n59mua7thfgu7kr340gfms5y8qrztylq",
        "rewardAccountBalance": 42271389694
    }
]
```

We see that user one is delegating to a pool. We see since the testnet is running quite a lot of rewards have already accumulated - more than 550.000 ada. Let us try to withdraw these rewards. We can do this with the *withdraw-user1.sh* script.

```
#!/bin/bash

txin=$1
amt=$(/workspace/code/Week08/scripts/query-stake-address-info-user1.sh                |
jq .[0].rewardAccountBalance)
body=/workspace/code/Week08/tmp/tx.txbody
signed=/workspace/code/Week08/tmp/tx.tx

echo "txin = $1"
echo "amt = $amt"

export      CARDANO_NODE_SOCKET_PATH=/workspace/cardano-private-testnet-setup/private-
testnet/node-spo1/node.sock

cardano-cli transaction build \
    --babbage-era \
    --testnet-magic 42 \
    --change-address $(cat /workspace/cardano-private-testnet-setup/private-
                           testnet/addresses/payment1.addr) \
    --out-file $body \
    --tx-in "$txin" \
    --withdrawal "$(cat /workspace/cardano-private-testnet-setup/private-
                         testnet/addresses/staking1.addr)+$amt" \

cardano-cli transaction sign \
    --testnet-magic 42 \
    --tx-body-file $body \
    --out-file $signed \
    --signing-key-file /workspace/cardano-private-testnet-setup/private-testnet/stake-
```

```
                        delegator-keys/payment1.skey \
    --signing-key-file /workspace/cardano-private-testnet-setup/private-testnet/stake-
                        delegator-keys/staking1.skey

cardano-cli transaction submit \
    --testnet-magic 42 \
    --tx-file $signed
```

It takes in one command line argument, the UTXO that we spend. Next, there is a peculiarity in Cardano regarding withdrawal of staking rewards. Namely you can withdraw all of them at once. We saw just how we can manually look up how many rewards have accumulated. Because the private testnet is running so fast this number can change quite rapidly. So, if we first look up the rewards and then manually construct the transaction to withdraw them by the time we want to submit the transaction the number could already have changed and the transaction would fail. Therefore, we do the lookup of the funds in the script itself. This gets computed in the variable *amt*. We use the Linux *jq* tool to extract the *rewardAccountBalance* number. The *body* and *signed* variables are file paths where we save the unsigned and signed transaction to. Then we build the transaction which is in the Babbage era with testnet magic 42. As change address we specify user one payment address. In the last of building the transaction we specify the withdrawal that contains the staking address and the amount that we looked up in the beginning. Then we sign the transaction where we need to add two signatures. First, we are spending one of the UTXOs from user one that is sitting at the public key address and the owner of that key needs to sign the transaction. Then we do a withdrawal from a staking address that is given by a public key pair. And the rule of that is if you do anything staking related with the staking address that is based on a key pair you have to sign the transaction with the corresponding staking private key. And in the end, we submit the transaction. We can try this out with the following command.

```
/workspace/code/Week08/scripts# ./withdraw-user1.sh
37b2184df4044d3435c91ac1bb76eba78043c9bf308cf68ade9f283dc8a1ff60#0
```

Below you can see an example output of the above command.

```
txin = 37b2184df4044d3435c91ac1bb76eba78043c9bf308cf68ade9f283dc8a1ff60#0
amt = 0
Estimated transaction fee: Lovelace 171177
Transaction successfully submitted.
```

Now we have to wait a bit for it to go through and then we should see that user one has more funds than before. If we query after some time user's 1 address the initial amount of 300.000 ada should increase by the accumulated rewards.

```
                           TxHash                                 TxIx        Amount
```

```
--------------------------------------------------------------------------------------
4184b0b1b8363b81b30c58bfb5f3972dfce9a2dcf886998e37929964f3b3619b    0     342271389694
                                                                             lovelace +
                                                                        TxOutDatumNone
```

This concludes this chapter and we note that nothing we have shown so far has anything to do with Plutus. In the next chapter we look at how we can combine Plutus with staking. In particular instead of using a stake address that is based on a public-private key pair we will create a stake address that is based on a Plutus script. Then instead of providing the signing key for staking we will provide the Plutus script with an arbitrary logic which will then govern under which conditions we then can for example do a withdrawal of rewards.

8.2 Plutus & Staking

In the `ScriptContext` data type that a Plutus script receives as input we find also information about the script purpose. Until now we have looked only at the minting and spending purpose. Instead of using a public-private key pair to create a stake address we can instead use a Plutus script and then the hash of the Plutus script gives a stake script address. If we want to withdraw our rewards from a normal stake address, we have to provide the signing key for the stake address. If we use a script stake address instead then the corresponding script will be executed. It will receive a redeemer and the script context and again we can use arbitrary logic to determine whether this transaction is allowed to actually withdraw those rewards. Similarly, for certifying there are various certificates that we can attach to a stake address. In particular important for staking there are registration, delegation and deregistration certificates. If we newly create a stake address, we first have to register it by creating a transaction that contains a registration certificate for this stake address. Then if we want to delegate to a pool or change an existing validation, we have to use a transaction that contains a delegation certificate. That certificate then contains the pool we want to delegate to. Finally, we can also unregister a stake address again and get the original deposit back that we had to pay when we registered the stake address. So, for delegation for example then the corresponding script will be executed and can contain arbitrary logic to determine whether this delegation for example is legal or not. In this chapter we want to concentrate on the rewarding purpose. If we look at the `TxInfo` type again we remember that we covered some fields of it in previous chapters. Staking related fields are the `txInfoDCert` and the `txInfoWdrl` field. The first one contains a list of certificates that are attached to the transaction. And the second one that is used for withdrawals is a map from staking credential to integer. In this chapter we will look at the withdrawals field to see how Plutus influences withdrawals. When a transaction withdraws rewards in the map the keys that are staking addresses are used in the transaction to withdraw from them the rewards. And

the integer that corresponds to a staking address specifies how many lovelace we want to withdraw. When this staking credential is given by a Plutus script then when we try to do a non-zero withdrawal the corresponding Plutus script will be executed. And again, as before as for validators and for minting if the script runs without errors the withdrawal is OK and if it produces an error the transaction becomes invalid. Let us look at the *staking.hs* example of a Plutus staking script that can be found in week's 8 folder of the Plutus pioneer program.

```
{-# LANGUAGE DataKinds             #-}
{-# LANGUAGE FlexibleContexts      #-}
{-# LANGUAGE MultiParamTypeClasses #-}
{-# LANGUAGE NoImplicitPrelude     #-}
{-# LANGUAGE OverloadedStrings     #-}
{-# LANGUAGE ScopedTypeVariables   #-}
{-# LANGUAGE TemplateHaskell       #-}
{-# LANGUAGE TypeApplications      #-}
{-# LANGUAGE TypeFamilies          #-}
{-# LANGUAGE TypeOperators         #-}

module Staking
    ( stakeValidator
    , saveStakeValidator
    ) where

import           Plutus.V1.Ledger.Value (valueOf)
import           Plutus.V2.Ledger.Api   (Address, BuiltinData,
                                         ScriptContext (scriptContextPurpose,
                                                        scriptContextTxInfo),
                                         ScriptPurpose (Certifying, Rewarding),
                                         StakeValidator, StakingCredential,
                                         TxInfo (txInfoOutputs, txInfoWdrl),
                                         TxOut (txOutAddress, txOutValue),
                                         adaSymbol, adaToken,
                                         mkStakeValidatorScript)
import qualified PlutusTx
import qualified PlutusTx.AssocMap      as PlutusTx
import           PlutusTx.Prelude       (AdditiveSemigroup ((+)), Bool (..),
                                         Eq ((==)), Integer,
                                         Maybe (Just, Nothing),
                                         MultiplicativeSemigroup ((*)),
                                         Ord ((>=)), Semigroup ((<>)), foldl,
                                         otherwise, traceError, traceIfFalse,
                                         ($), (.))
import           Prelude                (IO, String, ioError)
import           System.IO.Error        (userError)
```

```haskell
import           Utilities              (tryReadAddress, wrapStakeValidator,
                                         writeStakeValidatorToFile)

{-# INLINABLE mkStakeValidator #-}
mkStakeValidator :: Address -> () -> ScriptContext -> Bool
mkStakeValidator addr () ctx = case scriptContextPurpose ctx of
    Certifying _   -> True
    Rewarding cred -> traceIfFalse "insufficient reward sharing" $
                      2 * paidToAddress >= amount cred
    _              -> False
  where
    info :: TxInfo
    info = scriptContextTxInfo ctx

    amount :: StakingCredential -> Integer
    amount cred = case PlutusTx.lookup cred $ txInfoWdrl info of
        Just amt -> amt
        Nothing  -> traceError "withdrawal not found"

    paidToAddress :: Integer
    paidToAddress = foldl f 0 $ txInfoOutputs info
      where
        f :: Integer -> TxOut -> Integer
        f n o
            | txOutAddress o == addr = n + valueOf (txOutValue o)
                                           adaSymbol adaToken
            | otherwise              = n

{-# INLINABLE mkWrappedStakeValidator #-}
mkWrappedStakeValidator :: Address -> BuiltinData -> BuiltinData -> ()
mkWrappedStakeValidator = wrapStakeValidator . mkStakeValidator

stakeValidator :: Address -> StakeValidator
stakeValidator addr = mkStakeValidatorScript $
    $$(PlutusTx.compile [|| mkWrappedStakeValidator ||])
    `PlutusTx.applyCode`
    PlutusTx.liftCode addr

-------------------------------------------------------------------------------
------------------------------ HELPER FUNCTIONS -------------------------------

saveStakeValidator :: String -> IO ()
saveStakeValidator bech32 = do
    case tryReadAddress bech32 of
        Nothing   -> ioError $ userError $ "Invalid address: " <> bech32
```

```
        Just addr -> writeStakeValidatorToFile "./assets/staking.plutus" $
                     stakeValidator addr
```

The idea is that we want to define a stake validator that controls withdrawals of rewards and it is parameterized by an address. It then allows withdrawals from the corresponding staking credentials given by the script if at least half of the withdrawn rewards go to the specified address. We use a typed version for our validator. The actual thing that runs will be an untyped version where the input parameters are of type `BuiltinData` and the return type is unit. In the *`mkStakeValidator`* function the first argument that is the address is a parameter of the script. The next argument is the redeemer, because validation scripts for staking similar as minting policies do not contain a datum. Since we do not need it, we can use unit for the redeemer type. Then follows the script context and in the end, we return a Bool. In the validation logic we look at the script purpose and if it is certifying we return true. However, if it is rewarding then we check if at least half of the withdrawn rewards are given to the specified address that we used as an additional parameter to the validator script. We use the *`paidToAddress`* helper function which will give us the amount of lovelace sent to that specified address. And we check that twice that amount is greater or equal to the withdrawn amount which we compute with the helper function *`amount`* that takes in the staking credentials. How does the *`amount`* function work? Remember we looked at the `txInfoWdrl` field of the transaction information which was a map from staking credentials to integers. We know the staking credential that is given by the script purpose. That is the staking credential given by the validation script we wrote. So, that is a bit circular again, but the same happens for minting policies where we also want to talk sometimes about the currency symbol of the minting policy we are just defining. And this is similar here. We are just defining the staking credentials but it is given to us in the script purpose. With the `txInfoWdrl` constructor we get a map of withdrawals and just look up our credentials. If it is not in the map then we would throw an error which cannot really happen because the script will only run if we get to this point that we do a withdrawal for this credential. We will find this integer and that is the amount we return. This takes care of the right-hand side of the inequation in the validator script which is the number of lovelace we are withdrawing as rewards. Then we have to figure out how much we are paying to the address that is a parameter to the script. We just loop over all the outputs of the transaction and for each of these outputs we check whether it goes to the address in question we are interested in. We have an accumulator variable in the helper function *`f`* defined inside the *`paidToAddress`* function because there could be several outputs to this address. And we are just interested in the total amount that is the sum. If the rewards do not go to the right address the sum does not change and if they do go then we just extract from the output value using the function `valueOf` that takes in a currency symbol and token name. We apply that to the ada symbol and token name and get the amount of lovelace that goes to this address in this output. This gives us the

left-hand side of the inequation in the validator script. Then we check the inequality condition and if it is the case that we send more than half of the amount we are withdrawing to the given address the validation logic passes. If not, we raise an error with the message "`insufficient reward sharing`". After we have defined our validator, we have to convert it into an untyped version. In the Utilities module defined in the Plutus pioneer GitHub repository, we define the `wrapStakeValidator` helper function that does this conversion for us. It is identical to the function we used for wrapping a minting policy which also does not take in a datum. We just used another name for it such that we indicate for what purpose we are using it. Once we have converted it to an untyped version we use the usual template Haskell features to compile the stake validator and we also apply the address parameter. The exception is this time we use the *`mkStakeValidatorScript`* function that is defined in the `Plutus.V2.Ledger.Api` module. If we now want to use this from the cardano-cli we need a way to execute this and serialize the resulting stake validator to disk. From the Utilities module in the *Serialise.hs* file we already used serialization functions for normal validators and policies. Now we will use the function *`writeStakeValidatorToFile`* that is also defined in this file. In order to actually apply this, we need a Plutus address. Normally Plutus addresses are in the form of a string in the bech32 representation. For example, if we use this private testnet setup then we have the addresses of the users in the form of such a string. It is not completely trivial to take such a script and convert it to a Plutus address. For this we use the Conversion module that is also defined as a part of the Utilities module. From it we use the following function displayed below to convert a string to an actual Plutus address type.

```
credentialLedgerToPlutus :: Ledger.Credential a StandardCrypto ->
                            Plutus.Credential
credentialLedgerToPlutus (ScriptHashObj (ScriptHash h)) =
  Plutus.ScriptCredential $ Plutus.ValidatorHash $ toBuiltin $ hashToBytes h
credentialLedgerToPlutus (KeyHashObj (KeyHash h))       =
  Plutus.PubKeyCredential $ Plutus.PubKeyHash $ toBuiltin $ hashToBytes h

stakeReferenceLedgerToPlutus :: Ledger.StakeReference StandardCrypto ->
                                Maybe Plutus.StakingCredential
stakeReferenceLedgerToPlutus (StakeRefBase x) =
    Just $ StakingHash $ credentialLedgerToPlutus x
stakeReferenceLedgerToPlutus (StakeRefPtr (Ptr (Api.SlotNo x) (TxIx y)
                              (CertIx z))) =
    Just $ StakingPtr (fromIntegral x) (fromIntegral y) (fromIntegral z)
stakeReferenceLedgerToPlutus StakeRefNull =
    Nothing

tryReadAddress :: String -> Maybe Plutus.Address
tryReadAddress x = case Api.deserialiseAddress Api.AsAddressAny $ pack x of
```

```
    Nothing                                                -> Nothing
    Just (Api.AddressByron _)                              -> Nothing
    Just (Api.AddressShelley (ShelleyAddress _ p s)) -> Just Plutus.Address
        { Plutus.addressCredential        = credentialLedgerToPlutus p
        , Plutus.addressStakingCredential = stakeReferenceLedgerToPlutus s
        }
```

We see we have to consider all sorts of cases. In Plutus there are various types of addresses. Some are defined in the Cardano API which is what the cardano-cli uses under the hood. That in turn uses more fundamental types from Cardano ledger modules and Plutus does its own thing. Now if you actually want to use it you have to convert between those. So, what we do in the *tryReadAddress* function is that we first deserialize the given string to a Cardano API address. Then there are various cases. We are interested in the case it is a Shelley era address which has a payment and a staking part which we handle separately with *credentialLedgerToPlutus* and *stakeReferenceLedgerToPlutus* helper functions. The first converts the payment part and the second one converts the staking part. Both functions also cover several cases. In the payment part it can be a script address or a public key address. In the staking part it cannot be there in which case it is an address that does not use staking. Or it can again be a script or public key address. There is also an additional case which is a so-called pointer credential where you can point to a specific point on the chain. These three functions can be used to convert a string representation of an address to a Plutus address. The *saveStakeValidator* function tries to parse the given string as a Plutus address and in case it does not succeed it raises an error. If it succeeds we use the serialization function to write the resulting validator to the *staking.plutus* file in the assets/ folder.

8.3 Trying it on the Testnet

In this chapter we can try to use the stake validator defined in the previous chapter on the private testnet. First, we need to decide what address we use as a parameter. From the three predefined users that the testnet provides we take the second one. Its address that is stored in the *payment2.addr* file from the *private-testnet/addresses/* is displayed below.

```
addr_test1qzxkl9zxmyptd4ua8zw2gcwr5qyj9djdr7ucu97gs8sh5uwm9kpwsxt2ksuf5yqnxqjeq8k59nar
9nacqgj793xgwa6q0j2rhv
```

We can start the repl in the Week08 folder and call the function we defined to serialize the stake validator which we provide the payment address from user two.

```
/workspace/code/Week08# cabal repl
Prelude Staking> saveStakeValidator
"addr_test1qzxkl9zxmyptd4ua8zw2gcwr5qyj9djdr7ucu97gs8sh5uwm9kpwsxt2ksuf5yqnxqjeq8k59na
```

```
r9nacqgj793xgwa6q0j2rhv"
Serialized script to: ./assets/staking.plutus
```

We now use this script and try it out on the testnet. First, we have to build a stake address from this script. This will be the first example of a stake address that we see that is not based on a key pair but on the script that we have just defined. Then we need to register this stake address and we need to delegate to a pool. We have the *register-and-delegate.sh* script for this.

```
#!/bin/bash

tmp=/workspace/code/Week08/tmp
txin=$1
echo "txin: $txin"

script=/workspace/code/Week08/assets/staking.plutus
script_stake_addr=$tmp/user1-script-stake.addr
script_payment_addr=$tmp/user1-script.addr
registration=$tmp/registration.cert
delegation=$tmp/delegation.cert
pp=$tmp/protocol-params.json
body=$tmp/tx.txbody
signed=$tmp/tx.tx

export CARDANO_NODE_SOCKET_PATH=/workspace/cardano-private-testnet-setup/private-
                                testnet/node-spo1/node.sock

cardano-cli stake-address build \
    --testnet-magic 42 \
    --stake-script-file $script \
    --out-file $script_stake_addr

echo "stake address: $(cat $script_stake_addr)"

cardano-cli address build \
    --testnet-magic 42 \
    --payment-verification-key-file=/workspace/cardano-private-testnet-setup/private-
                                    testnet/stake-delegator-keys/payment1.vkey \
    --stake-script-file=$script \
    --out-file $script_payment_addr

echo "payment address: $(cat $script_payment_addr)"


cardano-cli stake-address registration-certificate \
    --stake-script-file $script \
    --out-file $registration

cardano-cli stake-address delegation-certificate \
    --stake-script-file $script \
    --stake-pool-id=$(/workspace/code/Week08/scripts/query-stake-pools.sh | head -n 1)
    \
    --out-file $delegation

cardano-cli query protocol-parameters \
    --testnet-magic 42 \
```

```
    --out-file $pp

cardano-cli transaction build \
    --babbage-era \
    --testnet-magic 42 \
    --change-address $(cat $script_payment_addr) \
    --out-file $body \
    --tx-in $txin \
    --tx-in-collateral $txin \
    --certificate-file $registration \
    --certificate-file $delegation \
    --certificate-script-file $script \
    --certificate-redeemer-file /workspace/code/Week08/assets/unit.json \
    --protocol-params-file $pp

cardano-cli transaction sign \
    --testnet-magic 42 \
    --tx-body-file $body \
    --out-file $signed \
    --signing-key-file /workspace/cardano-private-testnet-setup/private-testnet/stake-
                       delegator-keys/payment1.skey

cardano-cli transaction submit \
    --testnet-magic 42 \
    --tx-file $signed
```

Because we need to craft a transaction and submit it, we need an input which we store to the *txin* parameter. After that we define several file paths. The first one is the serialized script, then where we want to write the stake address to. We also want to use that stake address to build a payment address. And we want to have the payment part being that of user one but then the stake part being our new stake address. After that we define the registration and delegation certificate files. In the end, we define the protocol parameters, transaction body and the signed transaction files. And we also have to set the node socket file path variable. Now the first step is to build the stake address. For that we have the cardano-cli command *stake-address build*. It takes in a testnet magic, the script we created and a file path where to save the stake address. Then we want to build a payment address from that. We again input the testnet magic, the payment verification key of user one, the staking script file and in the end the output file path. After that we create a registration certificate for this script-based stake address and a delegation certificate. To do this we must have a pool ID that we want to delegate to. We use the script *query-stake-pools.sh* and just take the first one. We query the protocol parameters. And then we can build our transaction. As change address we use the newly created address. We also have other choices for that but by doing this we make sure that this newly created address is actually funded. The funds will come from the old address of user one. The one that just used the ordinary stake address based on a key pair. In principle all the funds except for the costs of this transaction will go to this new address where user one still has the spending power but the stake component has now been replaced by our script. When building the transaction,

we use the specified input that we also use as collateral which is needed because Plutus scripts are involved. Scripts have to be executed. The logic of our script was that for the certificates part it always returns true. But nevertheless, it has to execute this so we have to provide collateral. Then we attach the registration and delegation certificates. We have to provide the script as witness. Earlier when we withdrew the rewards for user one with this old stake address because it was a key based address as witness, we needed to sign the transactions with the corresponding staking key. Now this is replaced by providing the certificate. We input the redeemer JSON file that represents a unit. And in the end, we add the protocol parameters file. After we have built our transaction, we have to sign it. This has only to be done with the payment key of user one. Finally, we submit the transaction. In order to use that script, we have to look up the UTXO that we can use to spend. We use the *query-utxo-user1.sh* script for that.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
4184b0b1b8363b81b30c58bfb5f3972dfce9a2dcf886998e37929964f3b3619b    0     342271389694
                                                                          lovelace +
                                                                     TxOutDatumNone
```

When we use the bash script, we input the above transaction hash and index to it.

```
/workspace/code/Week08# cd scripts/
/workspace/code/Week08/scripts# ./register-and-delegate.sh
4184b0b1b8363b81b30c58bfb5f3972dfce9a2dcf886998e37929964f3b3619b#0
```

An example output of the above command can be seen below.

```
txin: b1d51a046b9cd92ce504382fdd83ea4436d265ff97cb0a6eefb1dce6f8096550#0
stake address: stake_test17q7pv2ap3scy9s9qurfjgwg6mmvsva87x5r8pydw9a66jqcuqdj7g
payment                                                                    address:
addr_test1ypnuk08vjnrjq66frsj0cnk0fgexkf4d66d2tdvd2vgedqeuzc46rrpsgtq2pcxnysu34hkeqe60
udgxwzg6utm44ypswpjxdv
Estimated transaction fee: Lovelace 345564
Transaction successfully submitted.
```

We see the log messages. We have the UTXO that we have spent. The newly created stake address is now based on our script. We also created the payment address and we submitted the transaction. Now there should not be any funds left at the initial address and all the funds should have moved to the new payment address we created. So, if we query the funds for user one there will be none. And with the script *query-utxo-user1-script.sh* we can check the funds for the new address we have just created. The script can be seen below.

```
#!/bin/bash
export CARDANO_NODE_SOCKET_PATH=/workspace/cardano-private-testnet-setup/private-
                                testnet/node-spo1/node.sock
cardano-cli query utxo \
```

```
    --testnet-magic 42 \
    --address $(cat /workspace/code/Week08/tmp/user1-script.addr)
```

And if we execute it, we see that almost all of the 300.000 ada went to this address. The difference is due to the registration deposit and transaction fees.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
7c018e5e39649a52b9e1f9527e9ac8dcc70b1da818629dc30e6322cbea6f4e44   0      299999654436
                                                                             lovelace +
                                                                          TxOutDatumNone
```

Now we can try to withdraw rewards from this new stake address. By now some should have accumulated already. We will do this with the *withdraw-user1-script.sh*.

```
#!/bin/bash

txin=$1
tmp=/workspace/code/Week08/tmp
amt1=$(/workspace/code/Week08/scripts/query-stake-address-info-user1-script.sh
     | jq .[0].rewardAccountBalance)
amt2=$(expr $amt1 / 2 + 1)
pp=$tmp/protocol-params.json
body=$tmp/tx.txbody
signed=$tmp/tx.tx

echo "txin = $1"
echo "amt1 = $amt1"
echo "amt2 = $amt2"

export CARDANO_NODE_SOCKET_PATH=/workspace/cardano-private-testnet-setup/private-
                                testnet/node-spo1/node.sock

cardano-cli query protocol-parameters \
    --testnet-magic 42 \
    --out-file $pp

cardano-cli transaction build \
    --babbage-era \
    --testnet-magic 42 \
    --change-address $(cat $tmp/user1-script.addr) \
    --out-file $body \
    --tx-in $txin \
    --tx-in-collateral $txin \
    --tx-out "$(cat /workspace/cardano-private-testnet-setup/private-
                    testnet/addresses/payment2.addr)+$amt2 lovelace" \
    --withdrawal "$(cat $tmp/user1-script-stake.addr)+$amt1" \
    --withdrawal-script-file /workspace/code/Week08/assets/staking.plutus \
    --withdrawal-redeemer-file /workspace/code/Week08/assets/unit.json \
    --protocol-params-file $pp

cardano-cli transaction sign \
    --testnet-magic 42 \
    --tx-body-file $body \
```

```
    --out-file $signed \
    --signing-key-file /workspace/cardano-private-testnet-setup/private-testnet/stake-
                       delegator-keys/payment1.skey

cardano-cli transaction submit \
    --testnet-magic 42 \
    --tx-file $signed
```

The idea is that user one tries to withdraw the rewards from the newly created stake address. First, we set the *tmp* folder. The amount is as before. Before we used the script-based stake address, so we query the accumulated rewards with the *query-utxo-user1-script.sh* script. It is as before except now the address is the new address we created. We extract the amount using the *jq* tool. We also have a second amount. Remember the script requires that we pay at least half of the withdrawn rewards to the specified address which is the address of user two in our case. We use the Linux expression command to calculate this amount. We add one in case if amount one divided by two is an odd number which would be then rounded down. Then we also set the node socket file path. We query the protocol parameters and save them to a file. After that we construct the transaction. We specify the era and the testnet magic. As change address we use the new address where we also take the original funds from. Then we specify to which file to write. We use the specified UTXO as input and use that also as collateral. Now we pay user two the half amount we have calculated before. And we do our withdrawal. So, we say from which stake address which is the new stake address we have computed with the full amount of available rewards. And we need to specify the script file in this case. For the redeemer we input a unit representation. At the end we attach the protocol parameters file. When we sign this transaction only the payment part needs a signature because the staking part is handled by the script. Finally, we submit the transaction. Now we can use the UTXO that the *query-utxo-user1-script.sh* script returns as input. We can now run the following command:

```
/workspace/code/Week08/scripts# ./withdraw-user1-script.sh
7c018e5e39649a52b9e1f9527e9ac8dcc70b1da818629dc30e6322cbea6f4e44#0
```

If we delete the line where we pay half of the amount to user two in the transaction build command the script will return an error with the message "*insufficient reward sharing*". That error comes from the Plutus code. If we submit the transaction with the line that pays half of the rewards to user two the transaction passes successfully. We can then query the funds for the user two and see that he has beside the initial UTXO also a second one containing rewards.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
9f54c1a9319b02a2da7110bb90946845dbe8e5fce0f770a4e8ee2955eed223cb     0    300000000000
                                                                           lovelace +
                                                                       TxOutDatumNone
902aba5b2b72df2423ee33d4af294e21e0903c31e34917a4de8570b3e8d08c7b     0      5070281709
```

```
                                                                                    lovelace +
                                                                                TxOutDatumNone
```

And if we query the funds for user 1 with the *query-utxo-user1-script.sh* script he has now more then the original 300.000 ada because he also got the other half of the rewards.

```
                           TxHash                                 TxIx        Amount
--------------------------------------------------------------------------------------
902aba5b2b72df2423ee33d4af294e21e0903c31e34917a4de8570b3e8d08c7b     0    305069582064
                                                                              lovelace +
                                                                          TxOutDatumNone
```

8.4 Homework

In the previous chapter where we showed an example of a stake validator, we have to note that it is not very realistic. The reason for this is that anybody can withdraw rewards. There is no restriction except that half of the rewards have to go to the address of user two. Apart from that any arbitrary person can withdraw rewards. In this homework you have to improve this and make it a bit more realistic. You should extend the validation logic by introducing a second parameter which will be a public key hash. And then you should add the condition that this public key has to sign the transaction if the transaction is done involving the stake key. So, for registration, delegation and withdrawals. And the address as before in case of withdrawal must receive half of the rewards. This makes the example much more realistic. The public key hash could belong to the payment key of user one. Then only user one would be allowed to do the withdrawals or change delegation or set up delegation in the first place. This time there is no test suite given for this homework so you will have to test it manually on the private testnet.

```
{-# LANGUAGE DataKinds              #-}
{-# LANGUAGE FlexibleContexts       #-}
{-# LANGUAGE MultiParamTypeClasses #-}
{-# LANGUAGE NoImplicitPrelude      #-}
{-# LANGUAGE OverloadedStrings      #-}
{-# LANGUAGE ScopedTypeVariables    #-}
{-# LANGUAGE TemplateHaskell        #-}
{-# LANGUAGE TypeApplications       #-}
{-# LANGUAGE TypeFamilies           #-}
{-# LANGUAGE TypeOperators          #-}

module Homework
    ( stakeValidator'
    , saveStakeValidator'
    ) where

import           Plutus.V2.Ledger.Api (Address, BuiltinData, PubKeyHash,
                                       ScriptContext, StakeValidator,
```

```
                                    mkStakeValidatorScript)
import qualified PlutusTx
import           PlutusTx.Prelude     (Bool (..), ($), (.))
import           Prelude              (IO, String, undefined)
import           Utilities            (wrapStakeValidator)

-- | A staking validator with two parameters, a pubkey hash and an address.
-- | The validator should work as follows:
--   1.) The given pubkey hash needs to sign all transactions involving this
--       validator.
--   2.) The given address needs to receive at least half of all withdrawn
--       rewards.
{-# INLINABLE mkStakeValidator' #-}
mkStakeValidator' :: PubKeyHash -> Address -> () -> ScriptContext -> Bool
mkStakeValidator' _pkh _addr () _ctx = undefined

{-# INLINABLE mkWrappedStakeValidator' #-}
mkWrappedStakeValidator' :: PubKeyHash -> Address -> BuiltinData ->
                            BuiltinData -> ()
mkWrappedStakeValidator' pkh = wrapStakeValidator . mkStakeValidator' pkh

stakeValidator' :: PubKeyHash -> Address -> StakeValidator
stakeValidator' pkh addr = mkStakeValidatorScript $
    $$(PlutusTx.compile [|| mkWrappedStakeValidator' ||])
        `PlutusTx.applyCode` PlutusTx.liftCode pkh
        `PlutusTx.applyCode` PlutusTx.liftCode addr

-------------------------------------------------------------------------------
------------------------------------ HELPER FUNCTIONS --------------------------

saveStakeValidator' :: String -> String -> IO ()
saveStakeValidator' _pkh _bech32 = undefined
```

9 Resources

[1] Cardano Documentation

https://docs.cardano.org/plutus/learn-about-plutus

https://web.archive.org/web/20220704234049/https://docs.cardano.org/plutus/eutxo-explainer